

Data Analysis

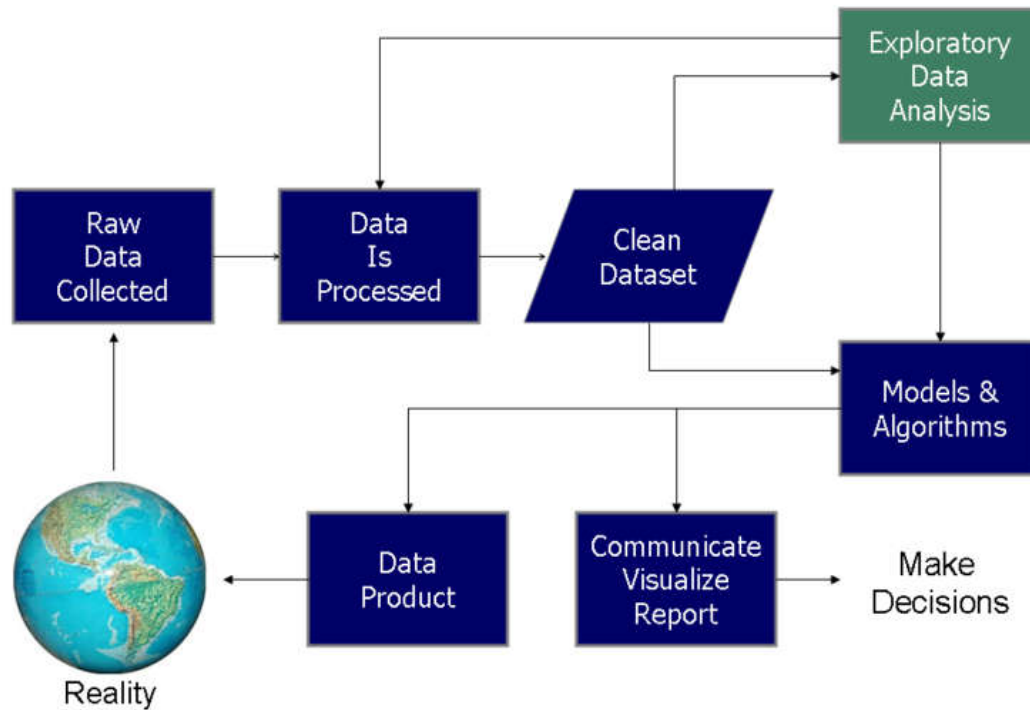
Lecture 1: Introduction

Dr. Nataliya K. Sakhnenko
Samsung R&D Institute, Ukraine

September 26, 2020,
Kiev Polytechnic Institute,
Institute of Physics and Technology

Introduction

Data Science Process



Data analysis is a process of inspecting, cleaning, transforming, and modeling data with the goal of discovering useful information, informing conclusions, and supporting decision-making.



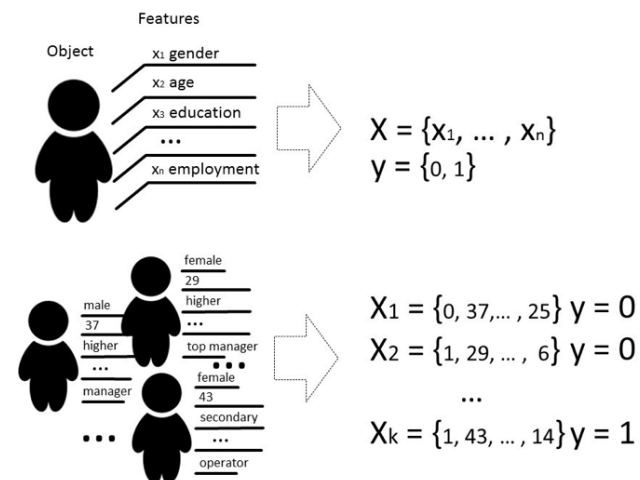
Data science process flowchart from *Doing Data Science*, by Schutt & O'Neil (2013)

<https://en.wikipedia.org/wiki/Data>

Features

Feature is an individual measurable property or characteristic of a phenomenon being observed.

- Let X be object
- $f: X \rightarrow D$
- feature vector $(f_1(x), f_2(x), \dots, f_n(x))$
- $\begin{pmatrix} f_1(x_1) & f_2(x_1) & \dots & f_n(x_1) \\ \dots & \dots & \dots & \dots \\ f_1(x_m) & f_2(x_m) & \dots & f_n(x_m) \end{pmatrix}$ is feature set
- $X_{m \times n}$, m is number of examples(objects), n is number of features



Possible Issues with Data:

- Insufficient quantity of data
- Nonrepresentative data
- Poor-quality data
- Irrelevant features

Machine Learning

- Tom Mitchel : "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E ."

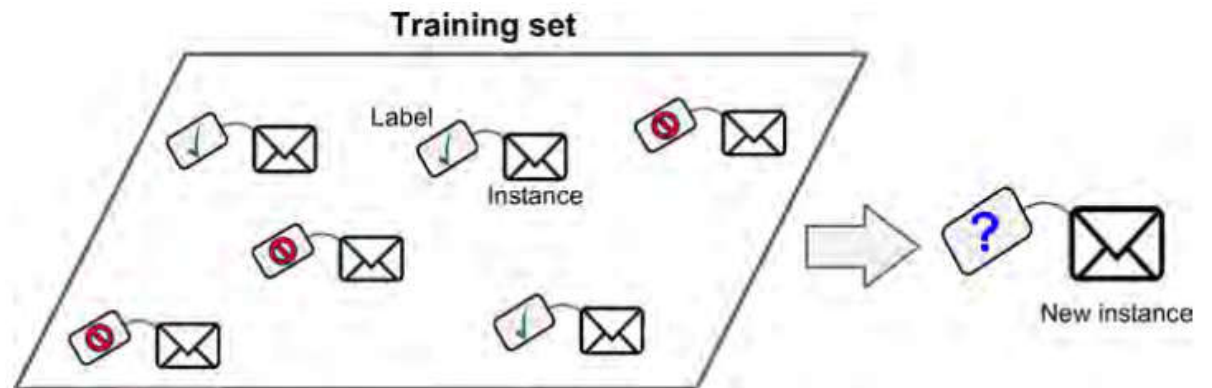
DATA +
FEATURES +
ALGORITHM

https://vas3k.com/blog/machine_learning/

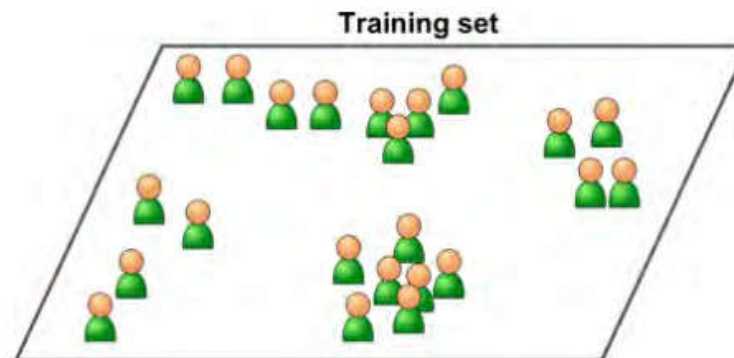


Supervised and unsupervised learning

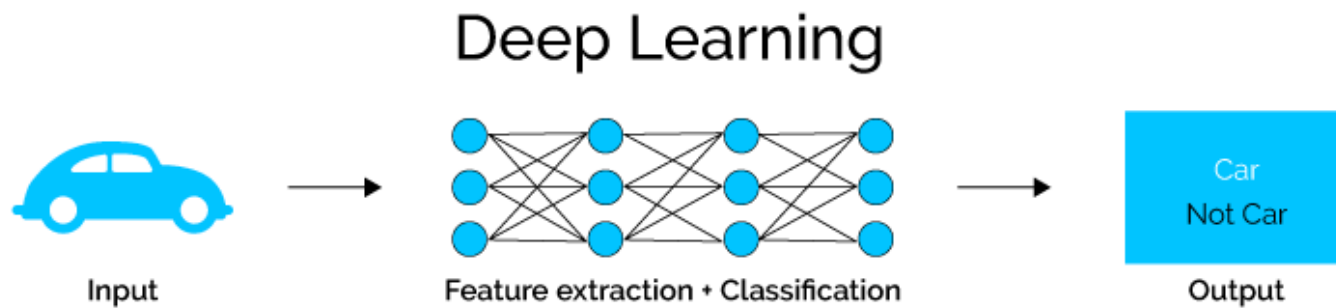
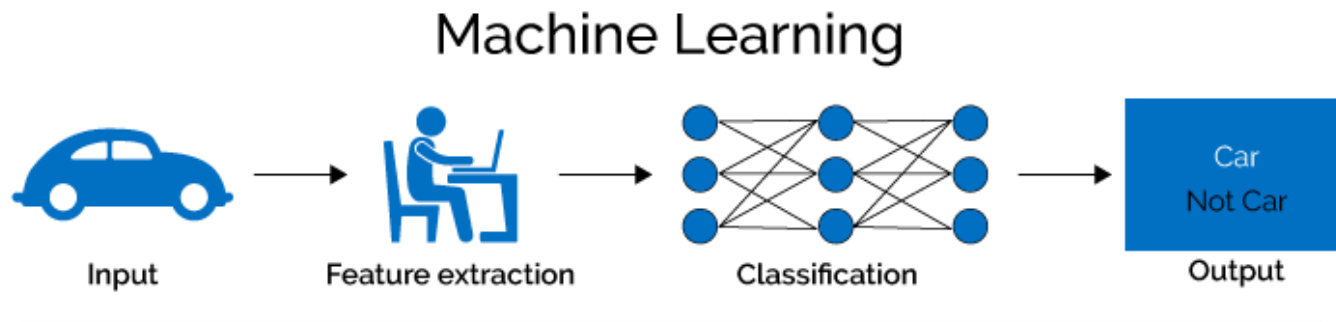
In supervised learning the training data includes the desired solutions, called labels



In unsupervised learning the training data is unlabeled



Basic ML algorithms vs Deep Learning



Deep Learning Applications

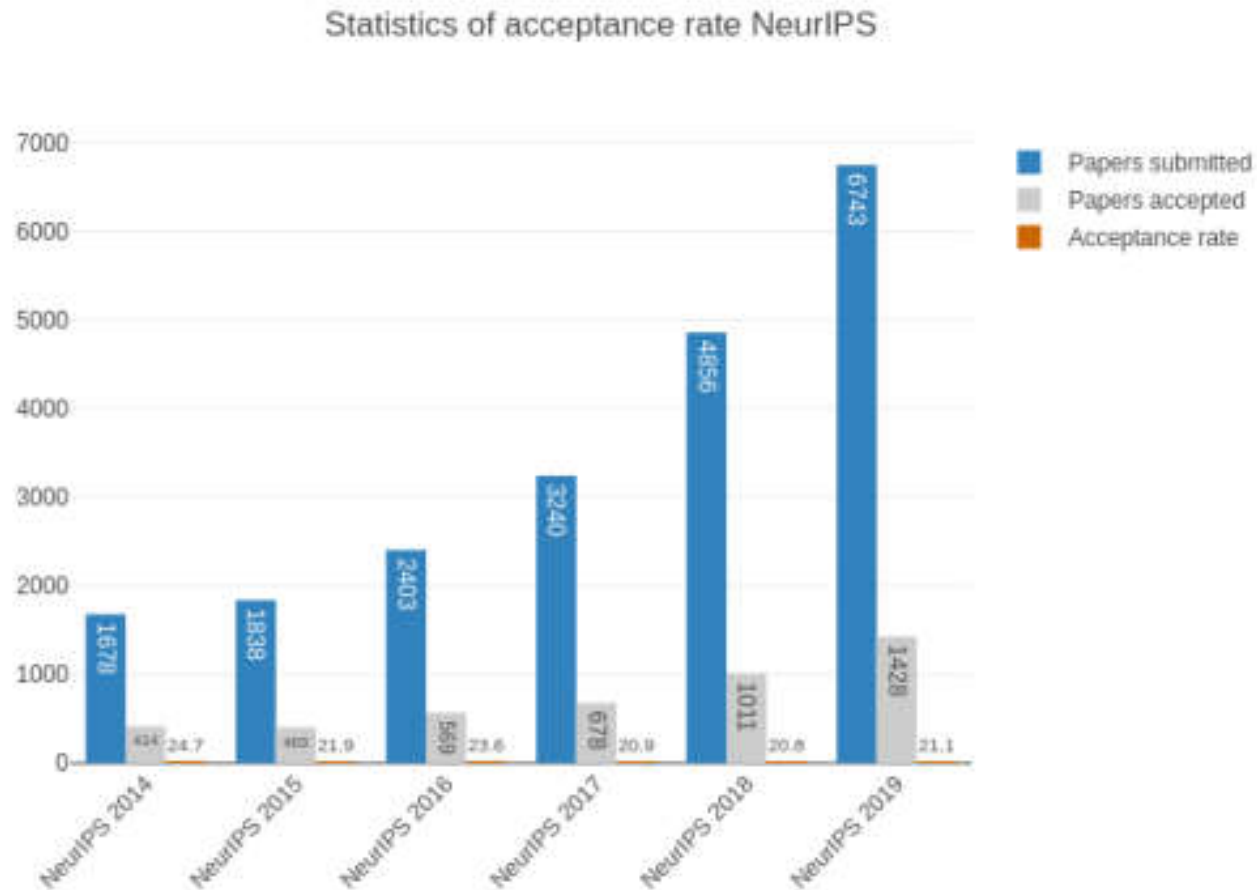
Vision

- Object detection
- Object segmentation
- *Object generation?*

Language

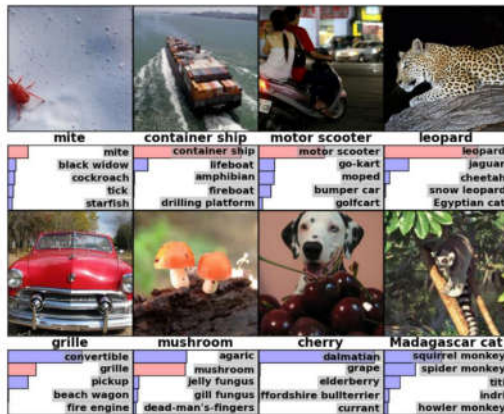
- Parsing
- Machine translation
- Question answering
- General language understanding evaluation (GLUE)

Deep Learning Research Community is Growing



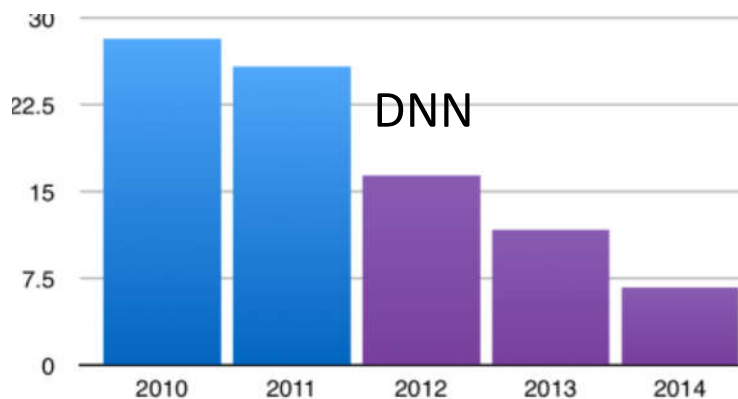
https://lexfridman.com/files/slides/2020_01_06_deep_learning_state_of_the_art.pdf

Deep Learning in CV

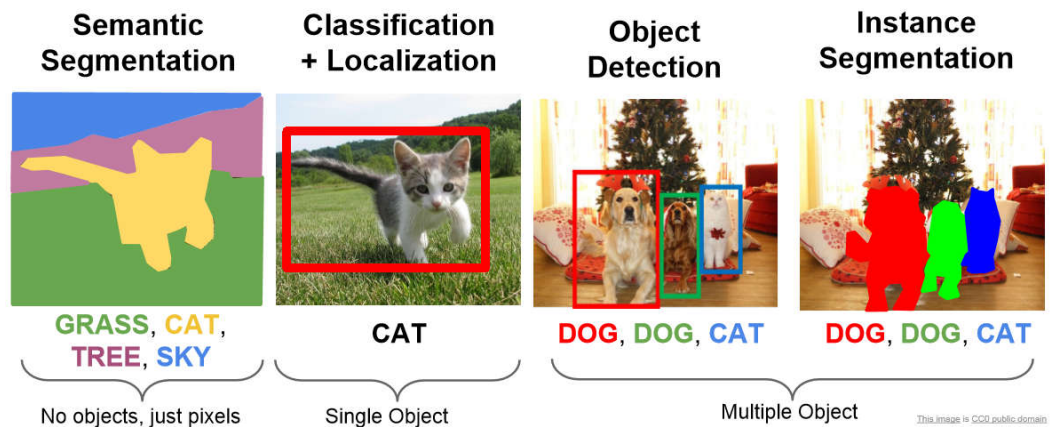


Krizhevsky A., Sutskever I., Hinton G.E. <http://www.cs.toronto.edu/~fritz/obsps/imagenet.pdf>

ImageNet Classification Error



Computer Vision Tasks



CS231n

The ImageNet project is a large visual database (more than 14 million images) designed for use in visual object recognition software research.

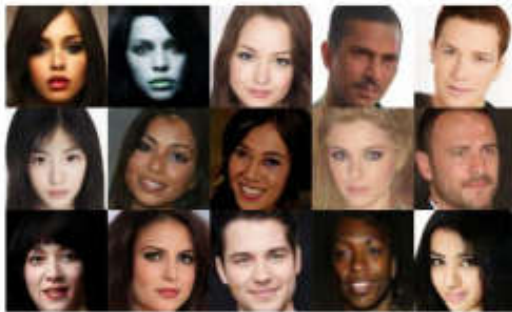
Deep Generative Modelling (GANs)

2017: Year of the GAN

Better training and generation



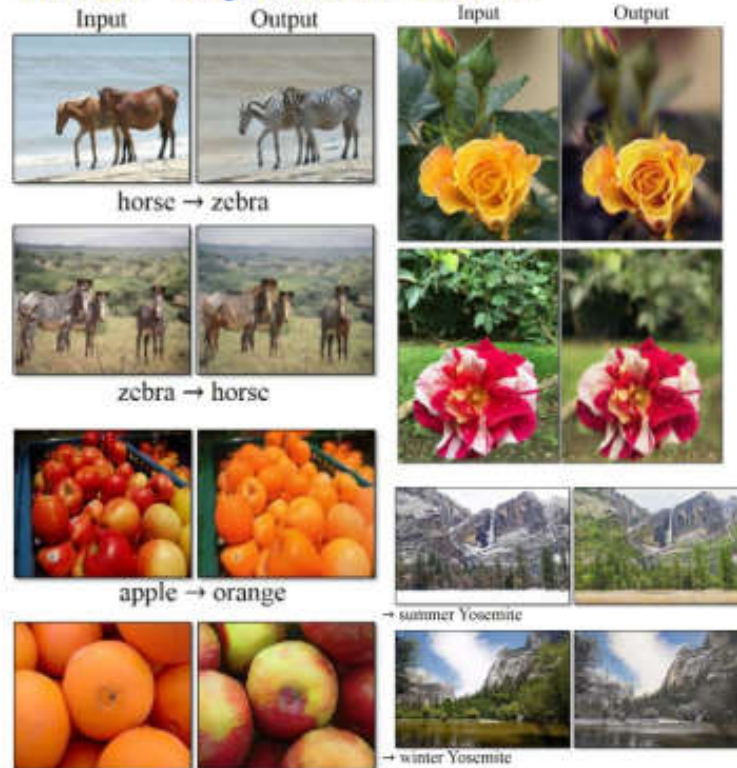
LSGAN. Mao et al. 2017.



RFGAN Bertholet et al 2017

CS231n

Source->Target domain transfer



CycleGAN. Zhu et al. 2017.

Text -> Image Synthesis

this small bird has a pink breast and crown, and black primaries and secondaries.

this magnificent fellow is almost all black with a red crest, and white cheek patch.



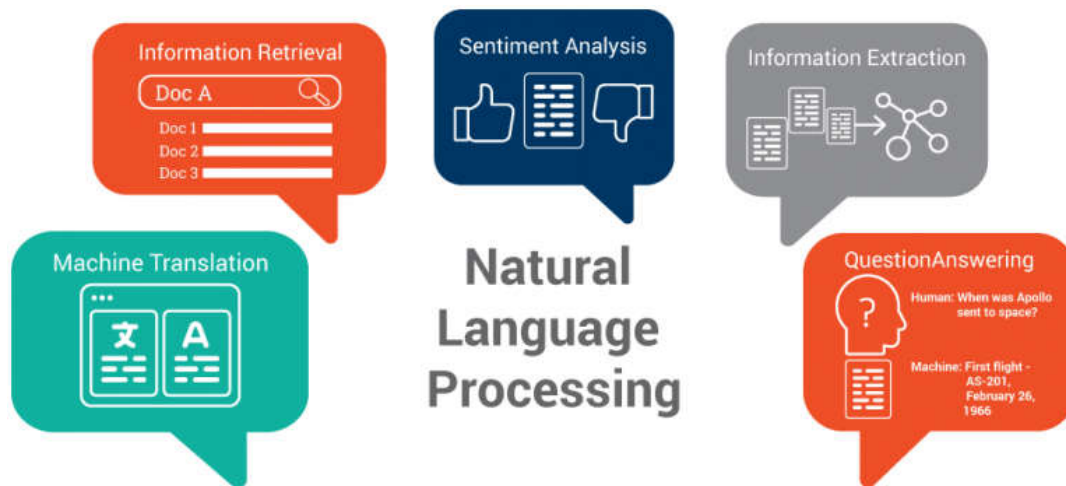
Reed et al. 2017.

Many GAN applications

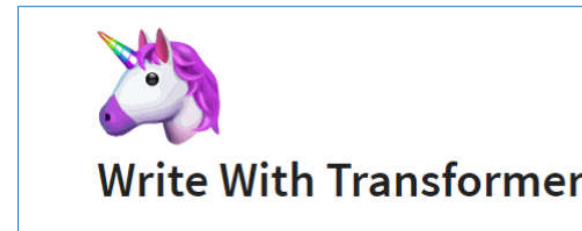


Pix2pix. Isola 2017. Many examples at <https://phillipi.github.io/pix2pix/>

Deep Learning in NLP



Syntax	Semantics
Part-of-speech tagging	Language models
Morphological segmentation	Information retrieval
Stemming and lemmatization	Machine translation
Sentence boundary disambiguation	Sentiment analysis
Named entity recognition	Question answering
Syntactic parsing	Text generation



Attention Is All You Need, 2017

The meaning of life is not what I think it is, it's what I do to make it.

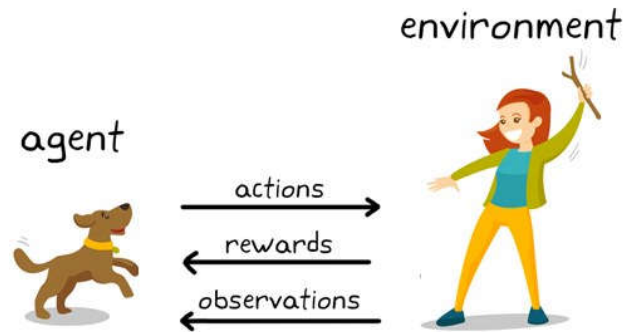
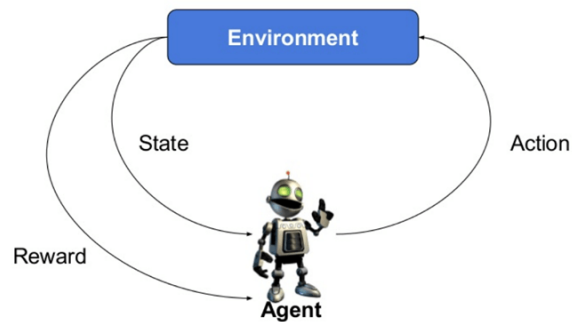
The limits of deep learning are still in the process of being figured out.

Transformer Based Language Models

- BERT ([Google](#))
- XLNet ([Google/CMU](#))
- RoBERTa ([Facebook](#))
- DistilBERT ([HuggingFace](#))
- CTRL ([Salesforce](#))
- GPT-2 ([OpenAI](#))
- ALBERT ([Google](#))
- Megatron ([NVIDIA](#))

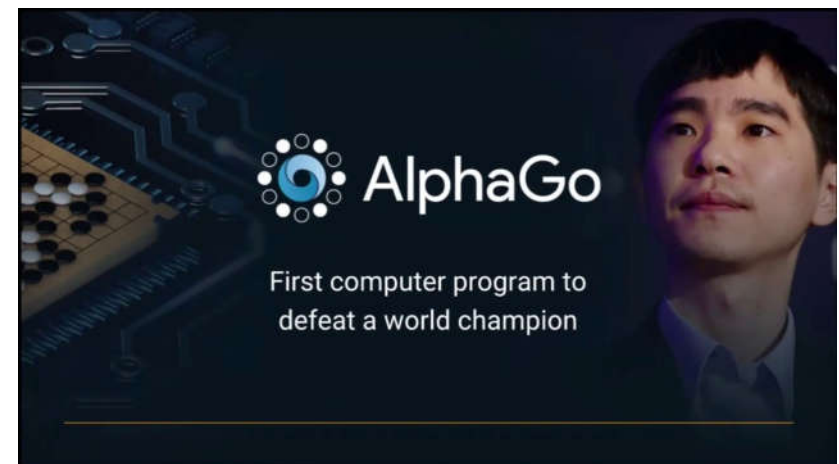
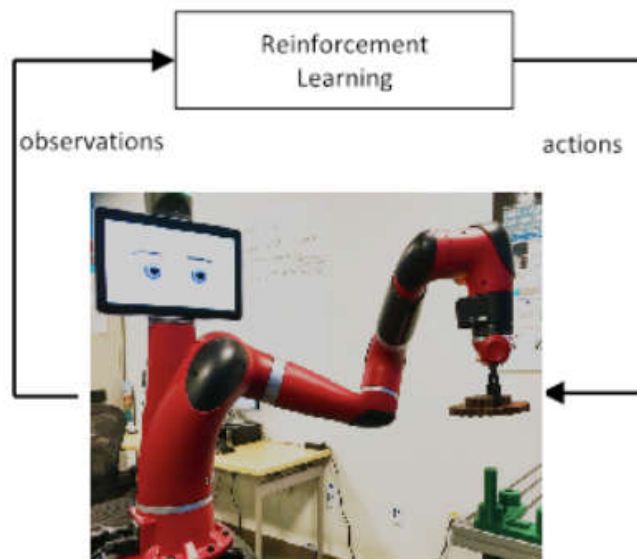
Reinforcement Learning

Typical RL scenario



RL applications:

- Robotics
- Gaming
- Finance
- Health Care



Roadmap

Basic Machine Learning		Deep Learning
Supervised	Unsupervised	☐ MLP ☐ CNN ☐ RNN ☐ Generative models: GANs, VAEs, NFs ☐ Attention and Transformers ☐ Intro to RL
☐ Regression (linear and polynomial) ☐ Classification ✓ Logistic regression, KNN, Decision Tree, SVM, NB ✓ Ensemble methods: Random Forest, AdaBoost	☐ Clustering (k-means, EM) ☐ Dimensionality reduction (PCA, t-SNE)	

Let's start

Supervised Learning

$T : X \rightarrow Y$, T is target function

Given set of training examples: $\{(x_1, y_1), (x_2, y_2), \dots (x_m, y_m)\}$

x_i is feature vector of i^{th} example, y_i is its label

Learning algorithm seeks a function (algorithm) $a \approx T$

If $Y = \mathbb{R}$, it is **regression** problem

If $Y = \{1, 2, \dots, M\}$, it is **classification** problem

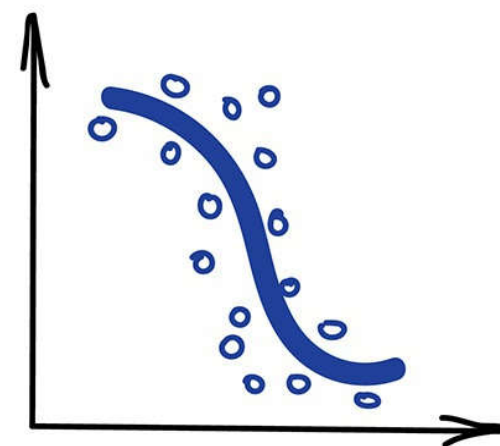
Regression:

- Stock price forecasts
- Demand and sales volume analysis
- Medical diagnosis
- Any number-time correlations

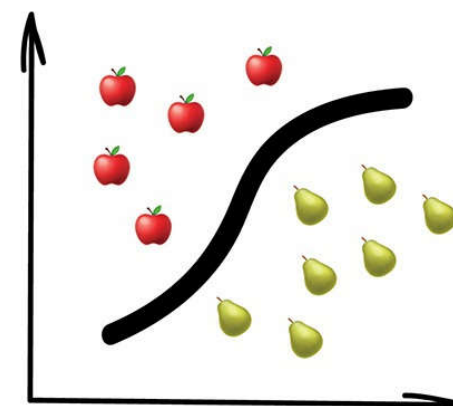
Classification:

- Spam filtering
- Language detection
- Sentiment analysis
- Recognition of handwritten characters and numbers
- Fraud detection

https://vas3k.com/blog/machine_learning/



Regression



Classification

Regression

Linear Regression

$$T : X \rightarrow Y, Y \in \mathbb{R}$$

$x_j^{(i)}$ is value of feature j in i^{th} train example

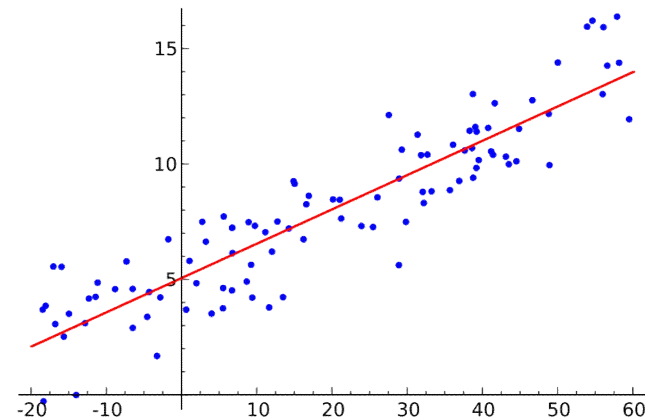
set $x_0^{(i)} = 1$ for all values of i

$$X = \begin{pmatrix} x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ x_0^{(2)} & x_1^{(2)} & \dots & x_n^{(2)} \\ \dots & \dots & \dots & \dots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

Hypothesis function:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots \theta_n x_n = \theta^T x$$

$$h_{\theta}(x) \approx T$$



MSE cost function for linear regression model

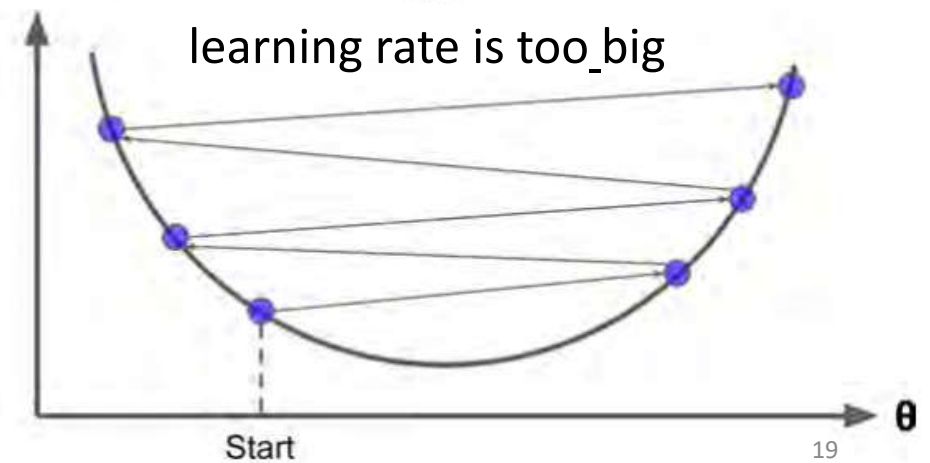
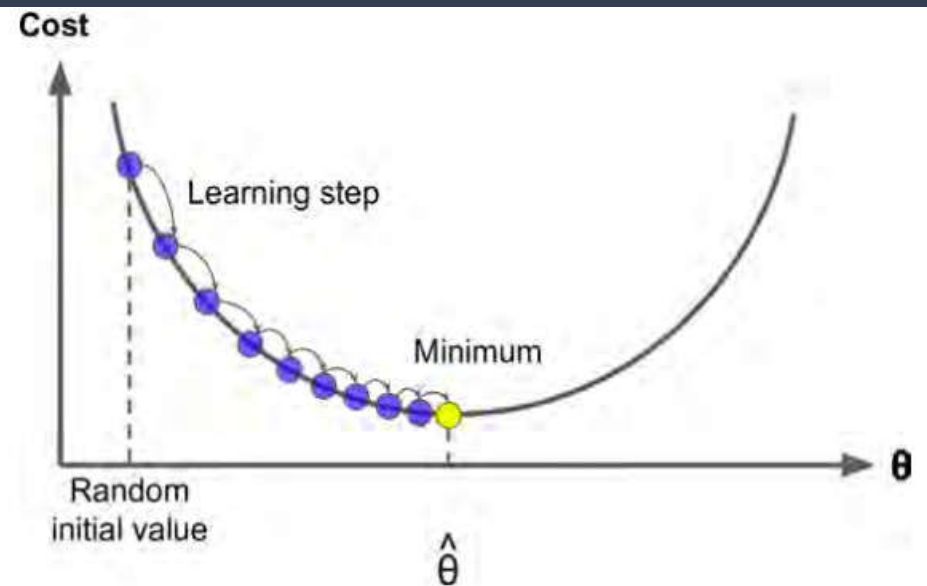
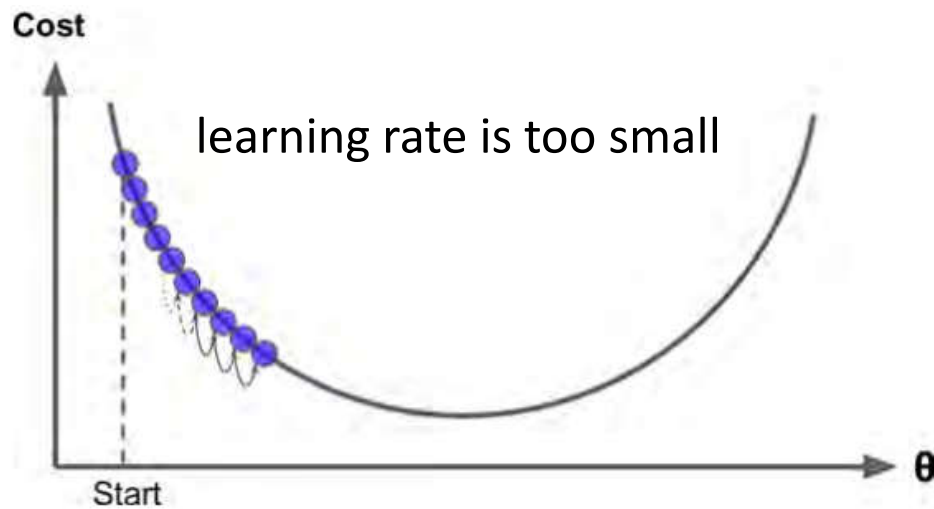
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad (1)$$

Find: $\arg \min J(\theta)$

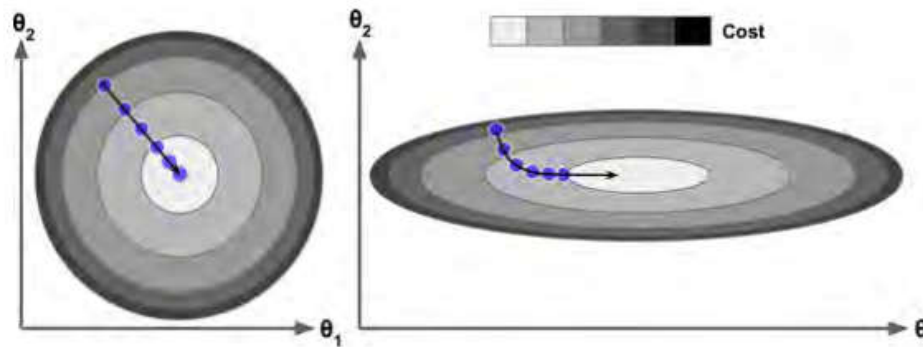
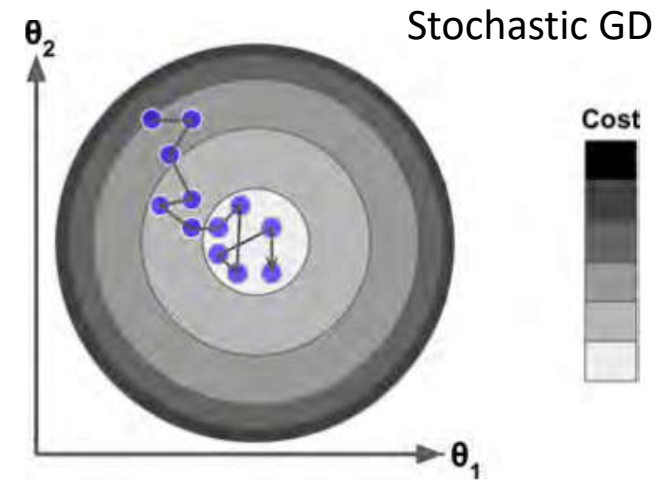
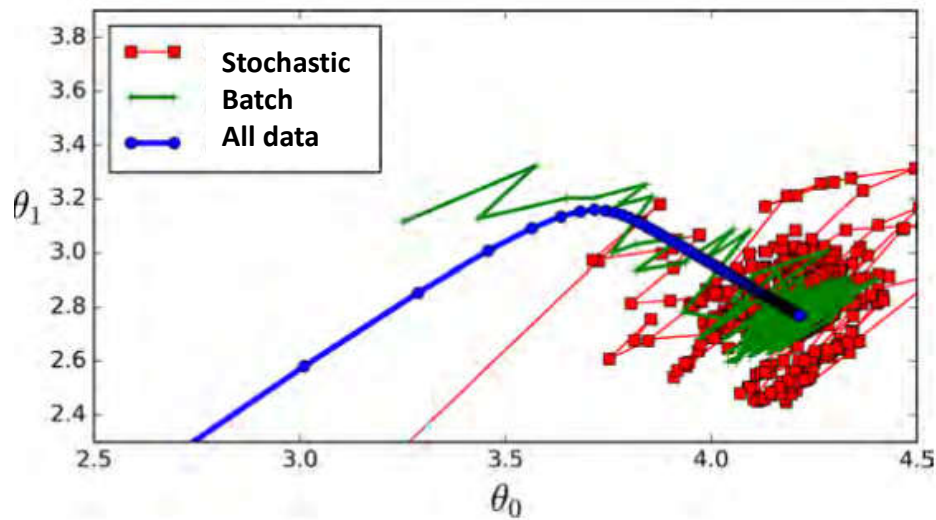
Gradient descent

$$\theta := \theta - \alpha \cdot \nabla J(\theta)$$

*Hyperparameter
 α is learning rate*



Stochastic vs Batch gradient descent



Feature scaling is
needed for
gradient descent

Gradient descent with and without feature scaling

Normal equation

Find optimum without iteration

$$J(\theta) = \frac{1}{2m} \|X\theta - Y\|^2 \text{ is (1) in matrix form,}$$

$$\frac{\partial J}{\partial \theta} = \frac{1}{m} X^T \|X\theta - Y\| \Rightarrow X^T \|X\theta - Y\| = 0 \Rightarrow X^T X\theta = X^T Y$$

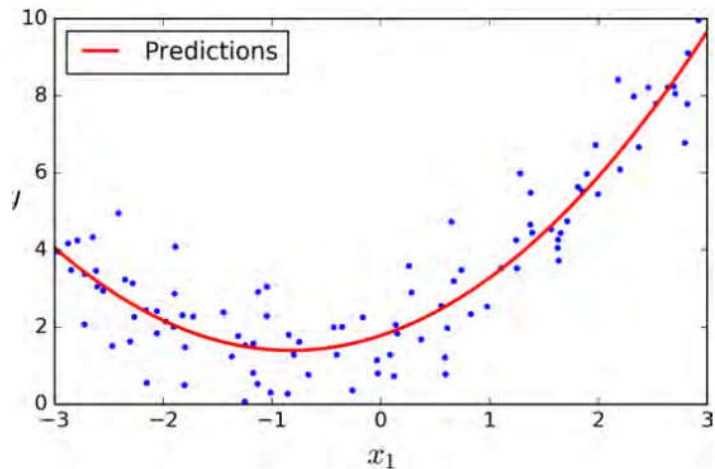
$$(X^T X)^{-1} (X^T X)\theta = (X^T X)^{-1} X^T Y$$

$$\theta = (X^T X)^{-1} X^T Y$$

No need to do
feature scaling
with normal
equation

Gradient Descent	Normal Equation
Need to choose learning rate	No need to choose learning rate
Needs many iterations	No need to iterate
Works well when n is large	Slow if n is very large

Polynomial regression

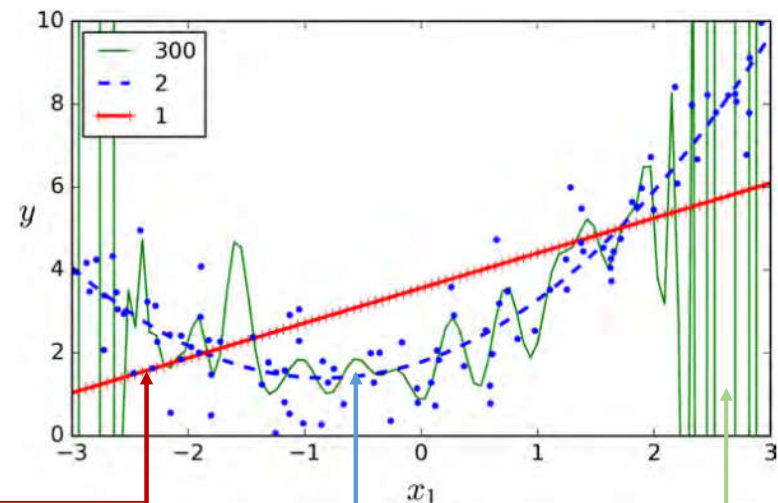


How to fit nonlinear data using linear model? Add powers of each feature and use linear regression.

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2$$

What if we use

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \dots + \theta_{300} x_{300}^2 \quad ?$$



underfitting

correct

overfitting

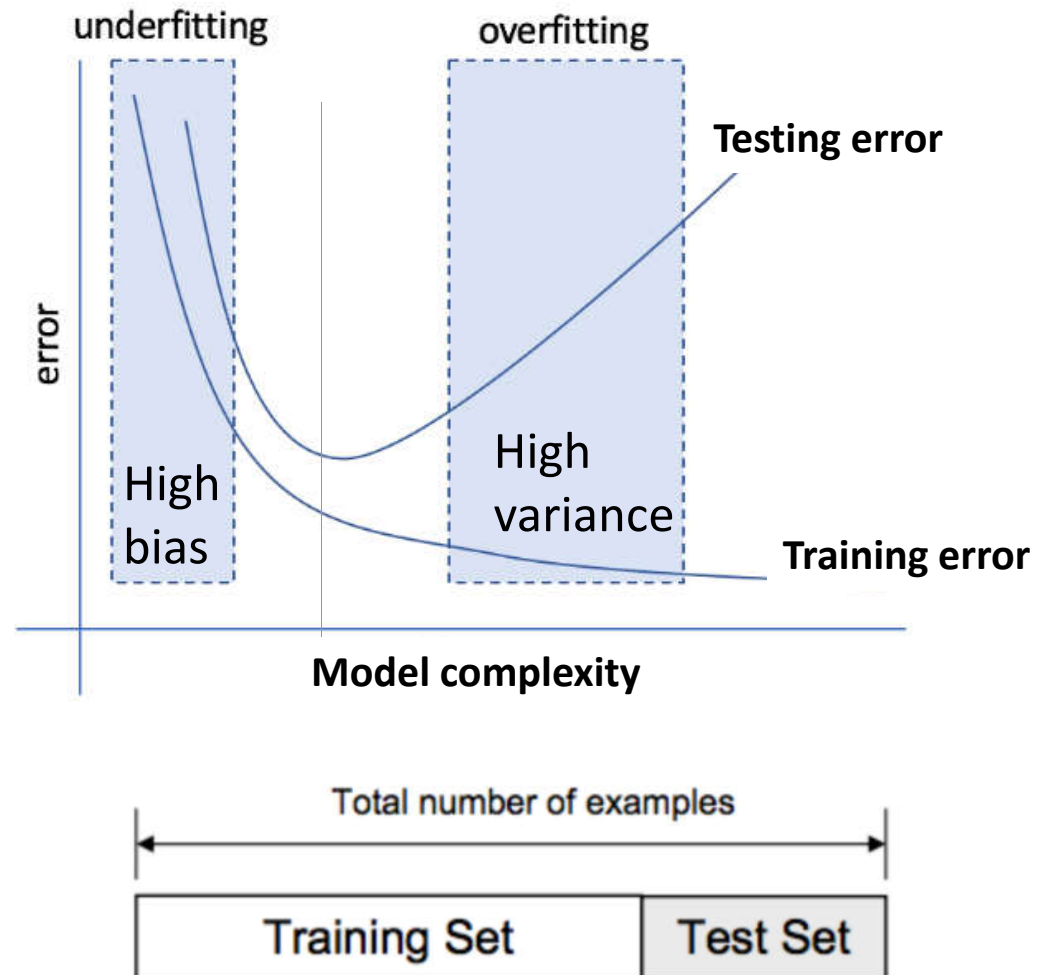
Bias/Variance tradeoff

Bias

This part of error is due to wrong assumptions, such as the data is linear when its actually quadratic (underfitting)

Variance

This part of error is from sensitivity to small fluctuations in the training set. A model with many degrees of freedom (as polynomial model) is likely to have high variance (overfitting)



Regularization

A good way to reduce overfitting is to regularize the model (to constrain it)

L_2 regularization

$$J(\theta) = MSE(\theta) + \frac{\lambda}{2} \sum_{i=1}^n \theta_i^2$$

L_1 regularization

$$J(\theta) = MSE(\theta) + \frac{\lambda}{2} \sum_{i=1}^n |\theta_i|$$

λ is regularization parameter (hyperparameter of model)

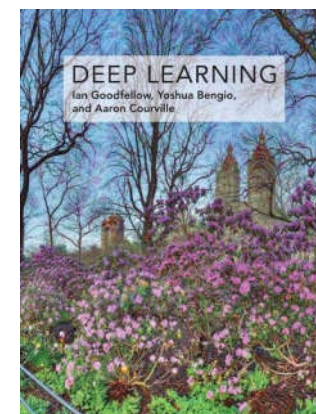
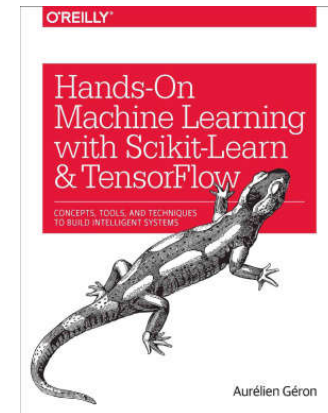
References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow', Ch.1-2
- I.Goodfellow 'Deep Learning', Ch.5.1-5.4

Coursera, Machine Learning, Andrew Ng (week 1-2)

<https://www.coursera.org/learn/machine-learning/home/welcome>



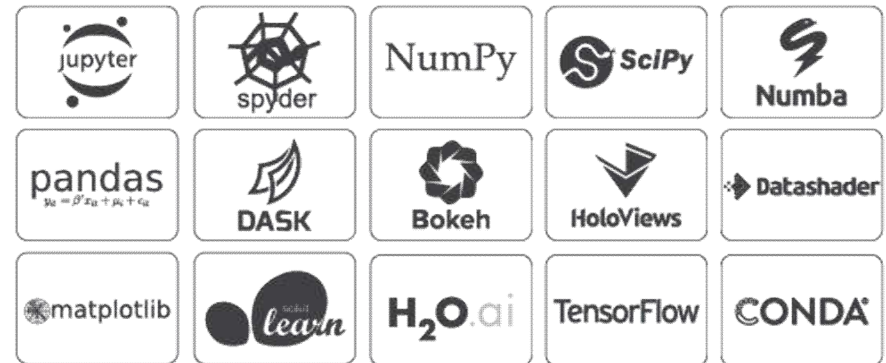
<https://github.com/yanshengjia/ml-road/tree/master/resources>

Appendix

TODO

Download Anaconda from:

<https://www.anaconda.com/>



Data Analysis

Lecture 2: Classification

Dr. Nataliya K. Sakhnenko
Samsung R&D Institute, Ukraine

October 3, 2020,
Kiev Polytechnic Institute,
Institute of Physics and Technology

Recap: Supervised Learning

$T : X \rightarrow Y$, T is target function

Given set of training examples: $\{(x_1, y_1), (x_2, y_2), \dots (x_m, y_m)\}$

x_i is feature vector of i^{th} example, y_i is its label

Learning algorithm seeks a function (algorithm) $a \approx T$

If $Y = \mathbb{R}$, it is **regression** problem

If $Y = \{1, 2, \dots M\}$, it is **classification** problem

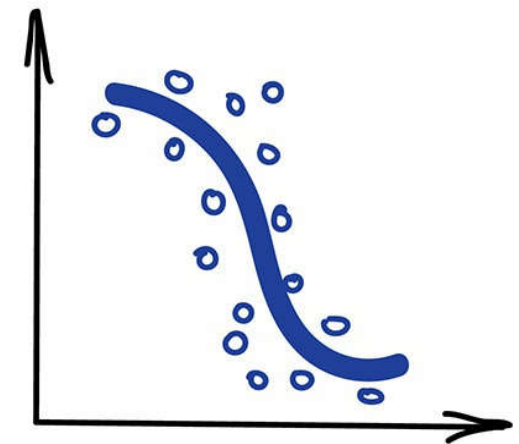
Regression:

- Stock price forecasts
- Demand and sales volume analysis
- Medical diagnosis
- Any number-time correlations

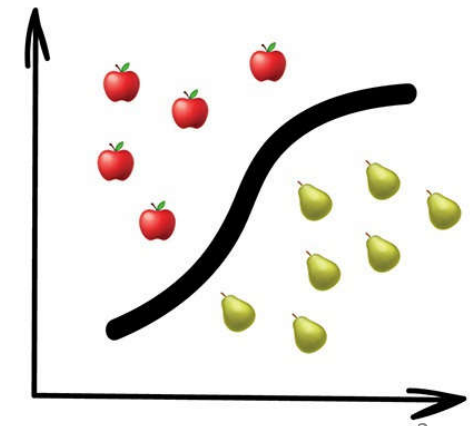
Classification:

- Spam filtering
- Language detection
- Sentiment analysis
- Recognition of handwritten characters and numbers
- Fraud detection

<https://vas3k.com/blog/machine-learning/>



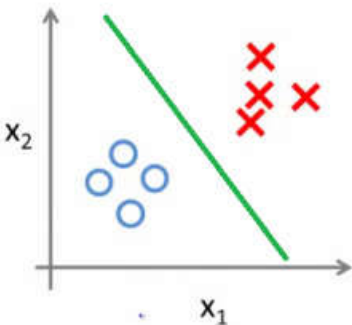
Regression



Classification

Classification Performance Measures

Binary Classifier



Confusion matrix

		Predicted Class	
		No	Yes
Observed Class	No	TN	FP
	Yes	FN	TP

TN

True Negative

FP

False Positive

FN

False Negative

TP

True Positive

$$T : X \rightarrow Y, \quad Y = \{0,1\}$$

$$N = 0, \quad P = 1$$

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

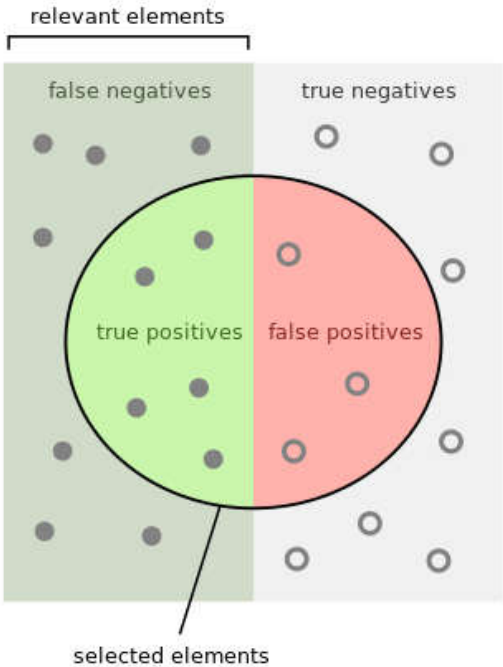
$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

Type I error

(false positive)

Type II error

(false negative)



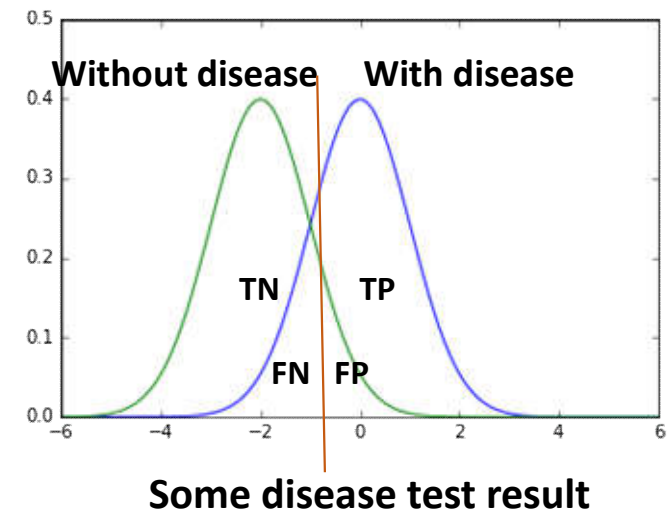
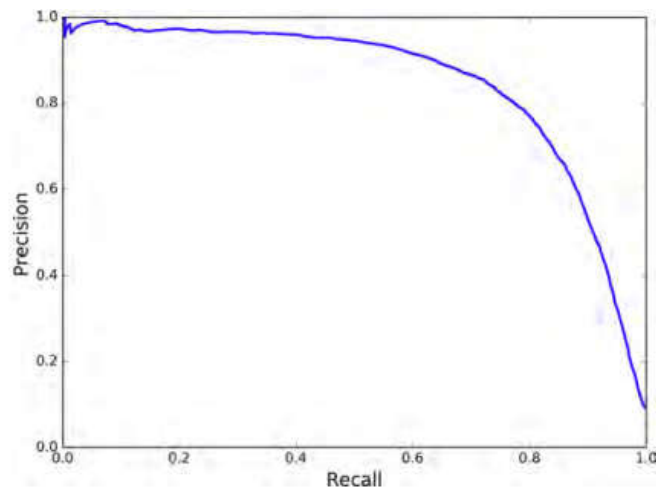
How many selected items are relevant?

Precision =

How many relevant items are selected?

Recall =

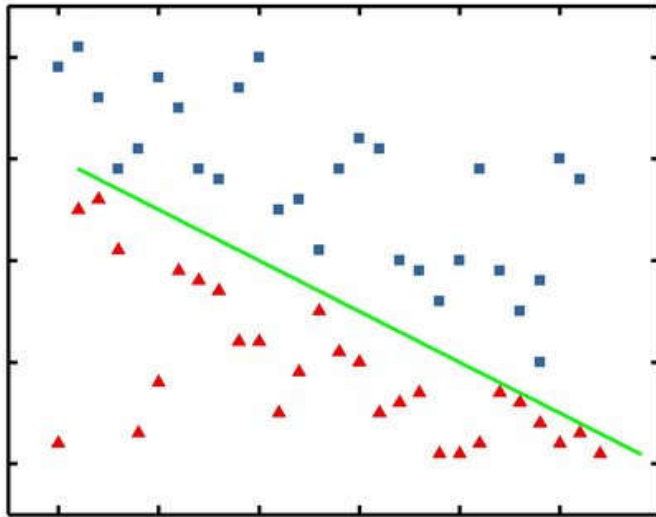
Precision Recall Trade-off



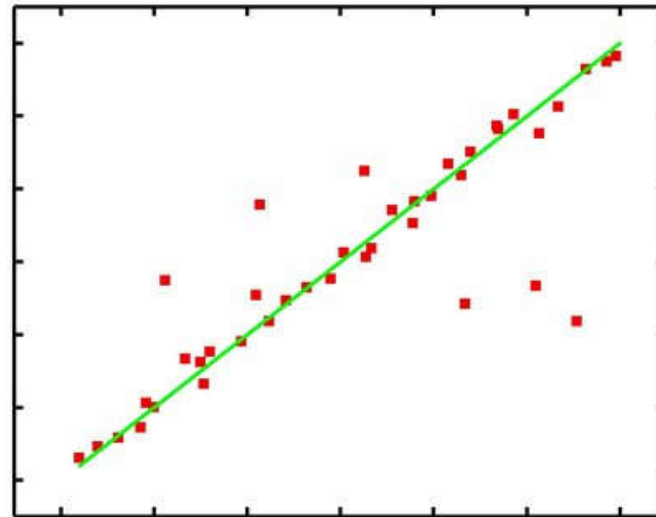
- **Low precision, high recall**: predict almost everything as positive
- **High precision, low recall**: predict positive when very sure
- If we want to find an optimal blend of precision and recall we can combine the two metrics using what is called the **F1 score**

Linear Classifier: Logistic Regression

Binary classification



(a) Logistic Regression



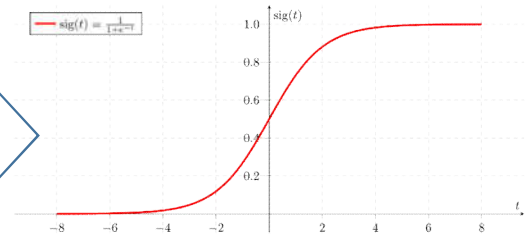
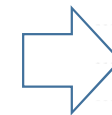
(b) Linear Regression

$$T: X \rightarrow Y, Y = \{0, 1\}$$

$$\hat{p} = h_{\theta}(x) = \sigma(\theta^T x)$$

Logistic regression model
estimates probability

$$\sigma(z) = \frac{1}{1 + e^{-z}} \text{ is sigmoid}$$



Cost function

For one sample: $\text{err} = \begin{cases} -\log(\hat{p}), & \text{if } y = 1 \\ -\log(1 - \hat{p}), & \text{if } y = 0 \end{cases}$

Logistic regression cost function over the whole training set (**log loss**)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

Goal: $\arg \min J(\theta)$

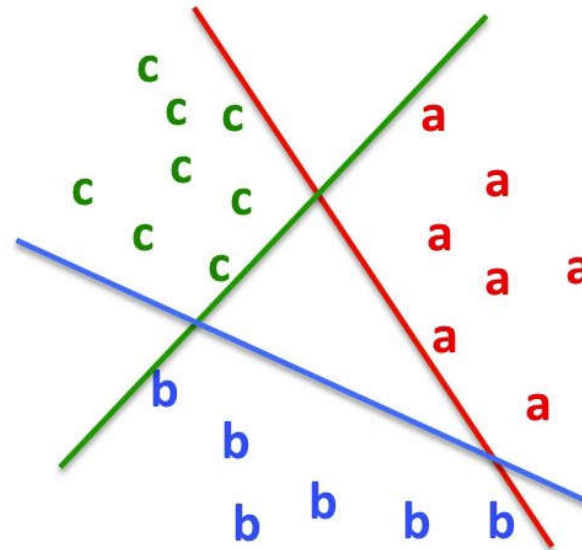
This function is convex, so **gradient descent** is guaranteed the global minimum

K classes: binary classification

One-vs-all classification is a method which involves training K distinct binary classifiers, each designed for recognizing a particular class.

One-vs-All (one-vs-rest):

- 'a' vs 'not a'
- 'b' vs 'not b'
- 'c' vs 'not c'



K classes: soft-max regression

$$T: X \rightarrow Y, Y = \{1, 2, \dots, K\}$$

$$h_{\theta}(x) = \begin{Bmatrix} P(y=1 | x; \theta) \\ P(y=2 | x; \theta) \\ \dots \\ P(y=K | x; \theta) \end{Bmatrix} = \frac{1}{\sum_{i=1}^K \exp(\theta_i^T x)} \begin{Bmatrix} \exp(\theta_1^T x) \\ \exp(\theta_2^T x) \\ \dots \\ \exp(\theta_K^T x) \end{Bmatrix}$$

The logistic regression model can be generalized to support multiple classes directly

One-hot encoding

$$\begin{bmatrix} 1 \\ 0 \\ \dots \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ \dots \\ 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 \\ 0 \\ \dots \\ 1 \end{bmatrix}, Y \in R^K$$

1, 2, ... K

Note that each class has its own parameter vector θ_k

K classes

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\theta}(x^{(i)}))$$

Cross entropy
cost function

$y_k^{(i)}$ is equal to 1 if the target class for the i^{th} instance is k , otherwise, it is equal to 0

When $K = 2$, this cost function is equivalent to log loss

Goal: $\arg \min J(\theta)$

Cross entropy (term originated from information theory) between two probability distributions p and q :

$$H(p, q) = -\sum_x p(x) \log q(x)$$

Decision Tree

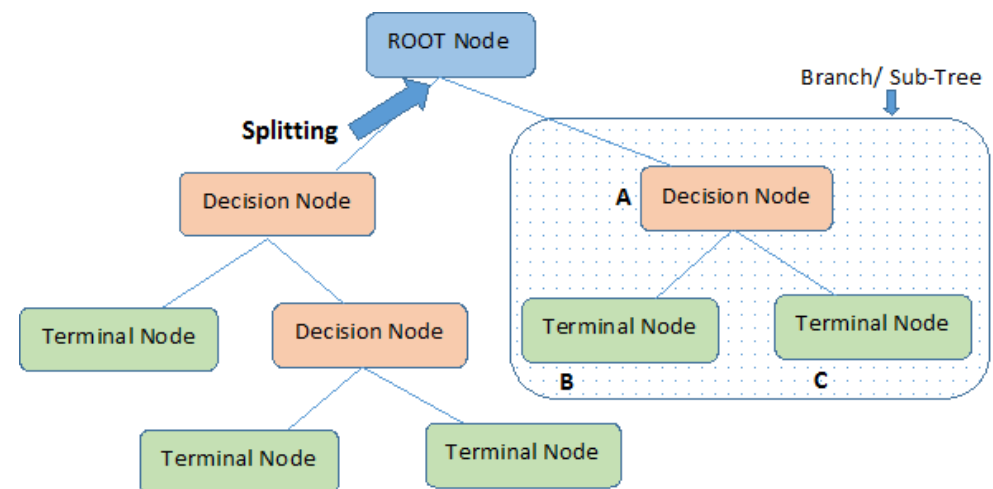
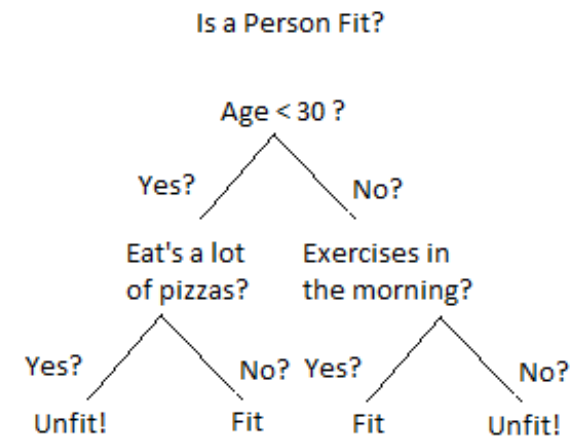
Decision Tree

Decision trees represent *rules*

Decision tree is a classifier in the form of a tree structure:

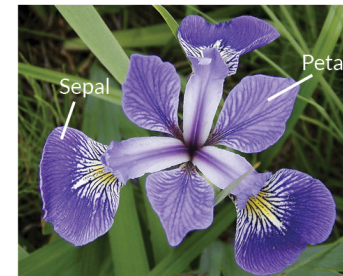
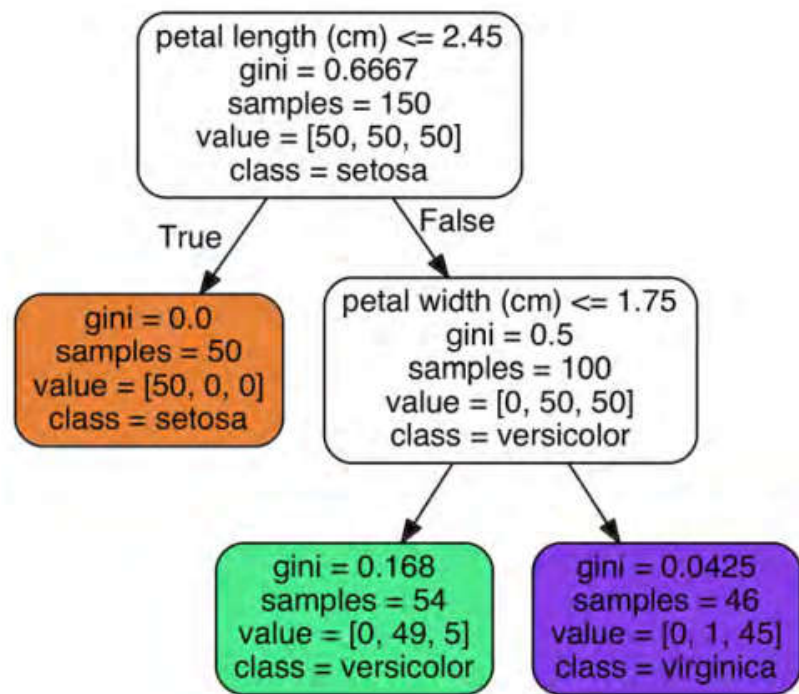
- **Root Node:** It represents entire population or sample and this further gets divided into two or more homogeneous sets.
- **Splitting:** It is a process of dividing a node into two sub-nodes.
- **Decision Node:** When a sub-node splits into further sub-nodes, then it is called decision node.
- **Leaf/ Terminal Node:** Nodes do not split is called Leaf or Terminal node.

https://www.analyticsvidhya.com/blog/2016/04/complete-tutorial-tree-based-modeling-scratch-in-python/?utm_source=blog&utm_medium=decoding-black-box-step-by-step-guide-interpretable-machine-learning-models-python



Note:- A is parent node of B and C.

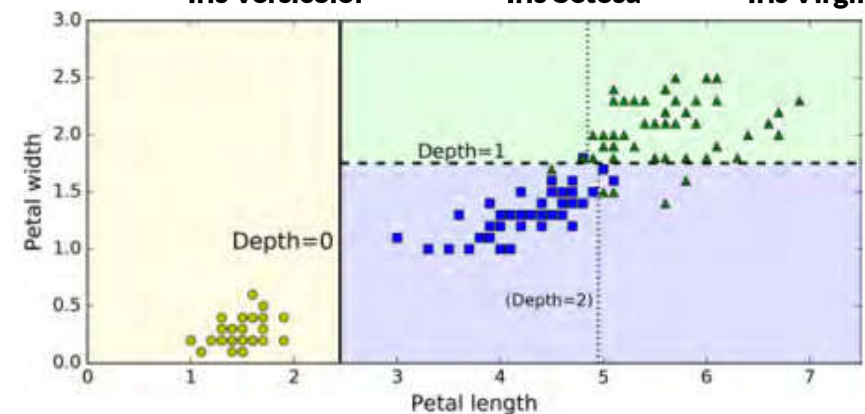
Gini Criterion



Iris Versicolor

Iris Setosa

Iris Virginica



The *Iris* flower data set, 1936, Ronald Fisher

The data set consists of 50 samples from each of three species of *Iris* (*Iris setosa*, *Iris virginica* and *Iris versicolor*). Four features were measured from each sample: the length and the width of the sepal and petals, in centimeters.

Gini impurity: $G_i = 1 - \sum_{k=1}^n p_{i,k}^2$

$p_{i,k}$ is the ratio of class k instances among the training in the i^{th} node

CART Training

Classification and regression tree (CART)

Produces binary trees only
(nonleaf nodes always have two children)

CART cost function for classification

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

$G_{\text{left}} / G_{\text{right}}$ measures impurity of left/right subset

$m_{\text{left}} / m_{\text{right}}$ is the number of instances in left/right subset

- The algorithm first splits the training set in two subsets using a single feature k and threshold t_k . It searches for the pair (k, t_k) that produces the purest subsets.
- CART is a greedy algorithm: it greedily searches for an optimal split at the top level, then repeats the process at each level. It does not check whether or not the split will lead to the lowest possible impurity several levels down.

Note: ID3 algorithm can produce Decision Trees with nodes that have more than two children

Entropy Criterion

Entropy:

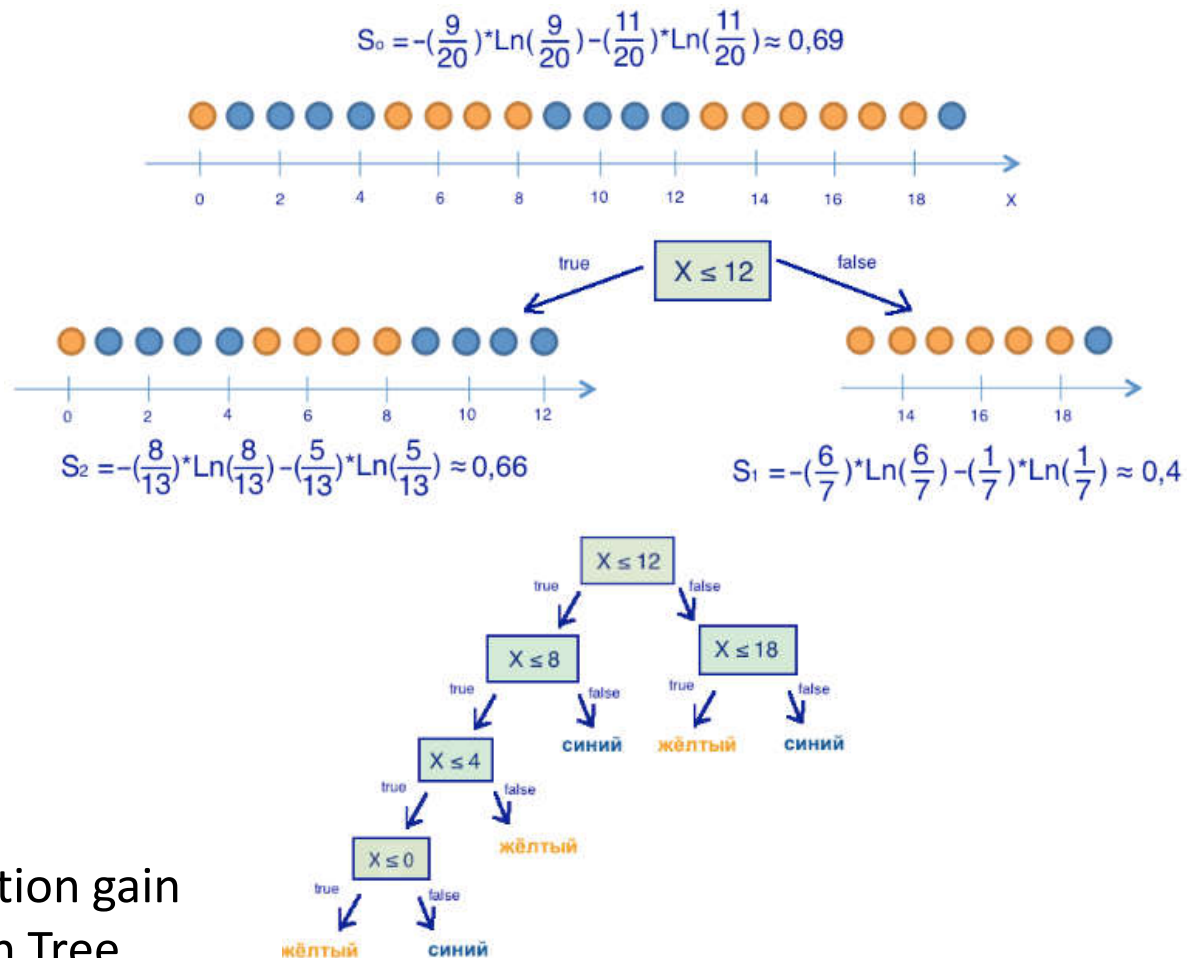
$$S_i = - \sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^n p_{i,k} \log(p_{i,k})$$

$p_{i,k}$ is the ratio of class k instances among the training in the i^{th} node

Information Gain:

$$IG = S_{\text{parent}} - \frac{m_{\text{left}}}{m} S_{\text{left}} - \frac{m_{\text{right}}}{m} S_{\text{right}}$$

Algorithm chooses the highest Information gain pairs (k, t_k) to split/construct a Decision Tree.

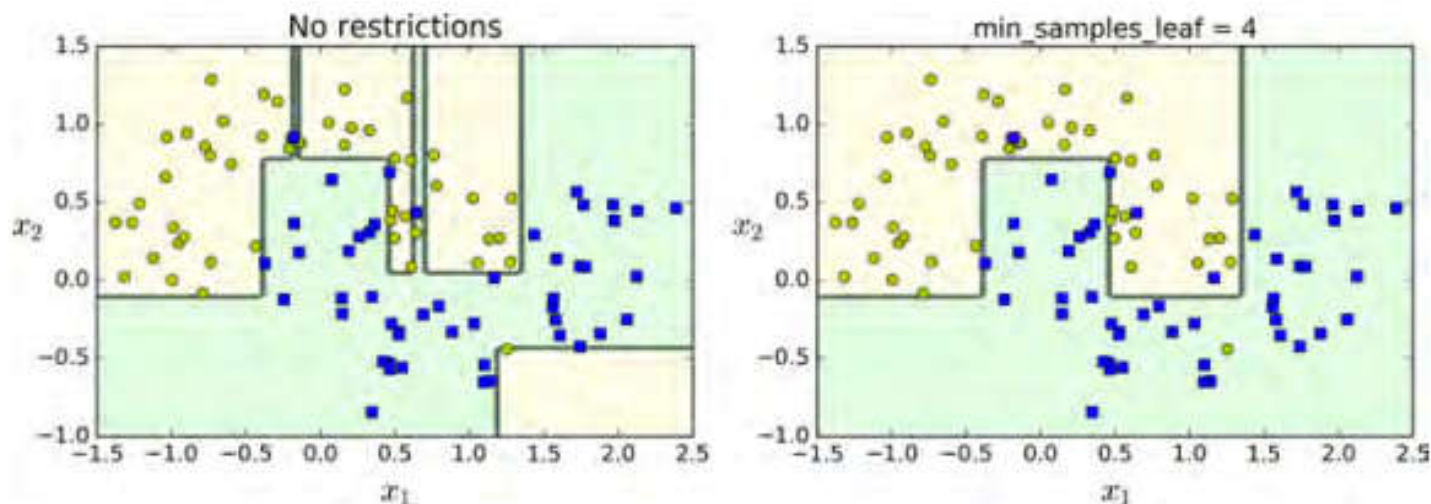


Regularization

If left unconstrained, DT will fit train data very closely, most likely overfitting it.

DT regularization techniques :

- restrict max depth
- restrict min samples split
- pruning (deleting unnecessary nodes)

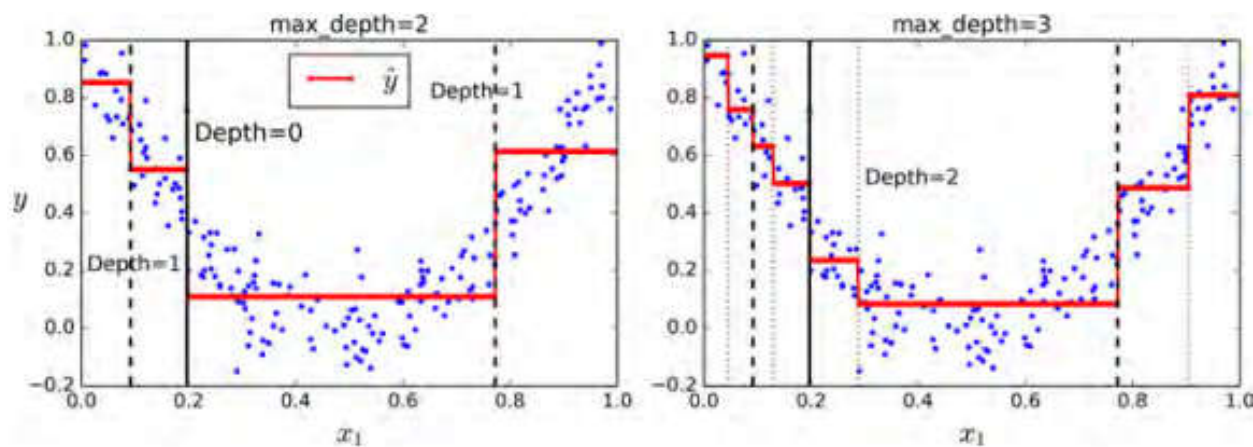
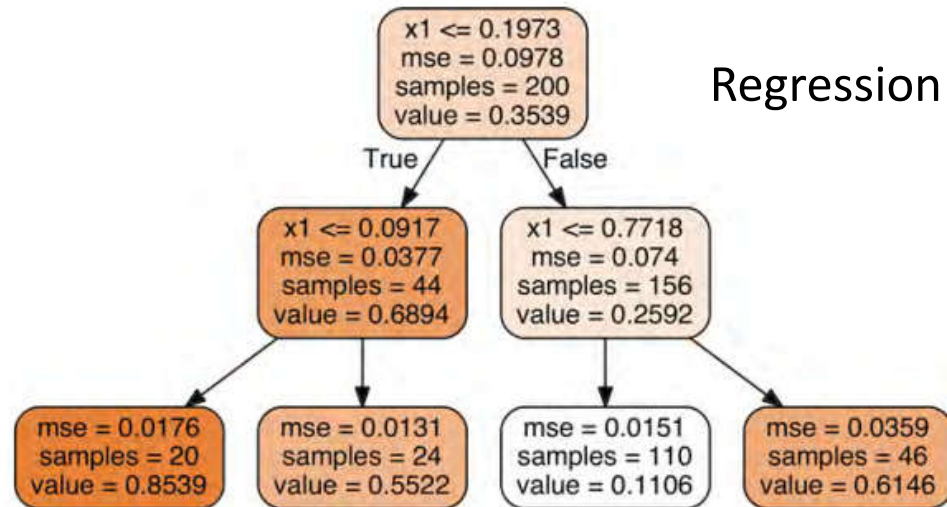


Regression

CART cost function for regression

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}$$

Regression Tree



kNN

kNN (k nearest neighbors)

- instance-based learning algorithm
- k -NN assumes that all instances are points in some n -dimensional space and defines neighbors in terms of distance
- k is the number of neighbors considered

Basic idea:

- K nearest neighbors vote for the class
- In practice, k is usually chosen to be odd, so as to avoid ties
- The $k = 1$ rule is generally called the nearest-neighbor classification rule

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

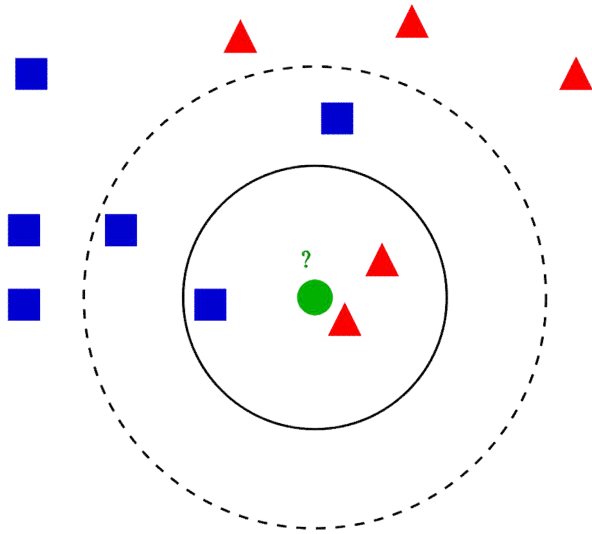
Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

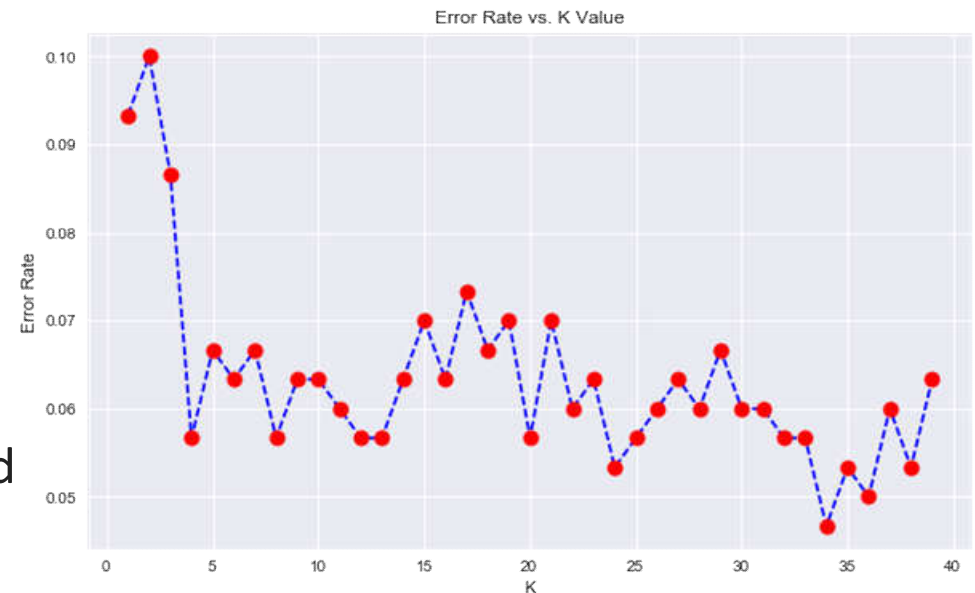
kNN



The test sample (green dot) should be classified either to blue squares or to red triangles.

- If $k = 3$ (solid line circle) it is assigned to the red triangles
- If $k = 5$ (dashed line circle) it is assigned to the blue squares

https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm



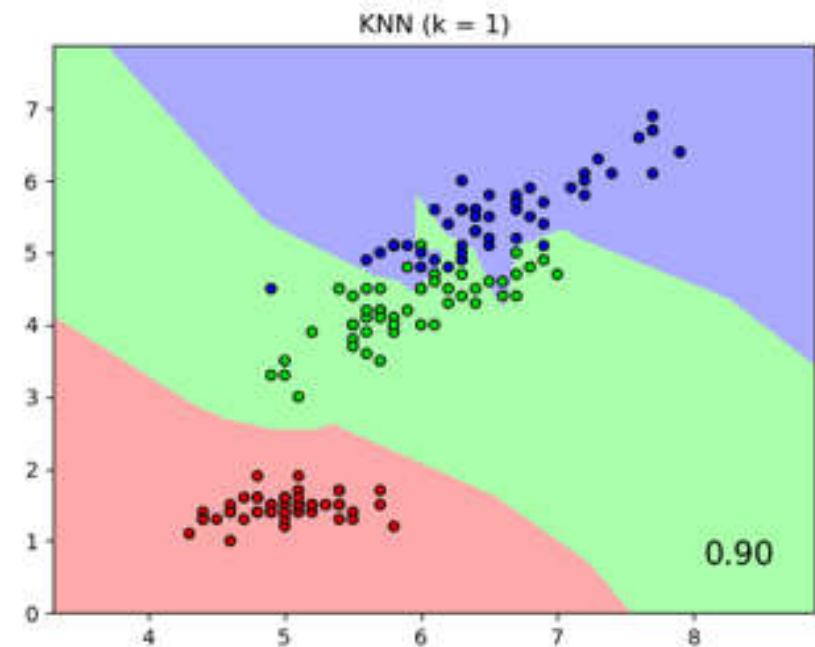
Selection of K

$$\arg \max_{y \in Y} \sum_{i=1}^K [y^i]$$

Weighted kNN

Weight the contribution of each of the k-neighbors with their distance

$$\arg \max_{y \in Y} \sum_{i=1}^K [y^i] \omega_i$$
$$\omega_i = \frac{1}{d(x_i, x_{test})}$$



kNN for regression

Store the input data

$$\begin{bmatrix} x_{1,0} & x_{1,1} & \dots & x_{1,n} \\ x_{2,0} & x_{2,1} & \dots & x_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,0} & x_{m,1} & \dots & x_{m,n} \end{bmatrix} \text{ and } \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{bmatrix}$$



When asked to predict a new value (a query)

$$y_i = ?$$



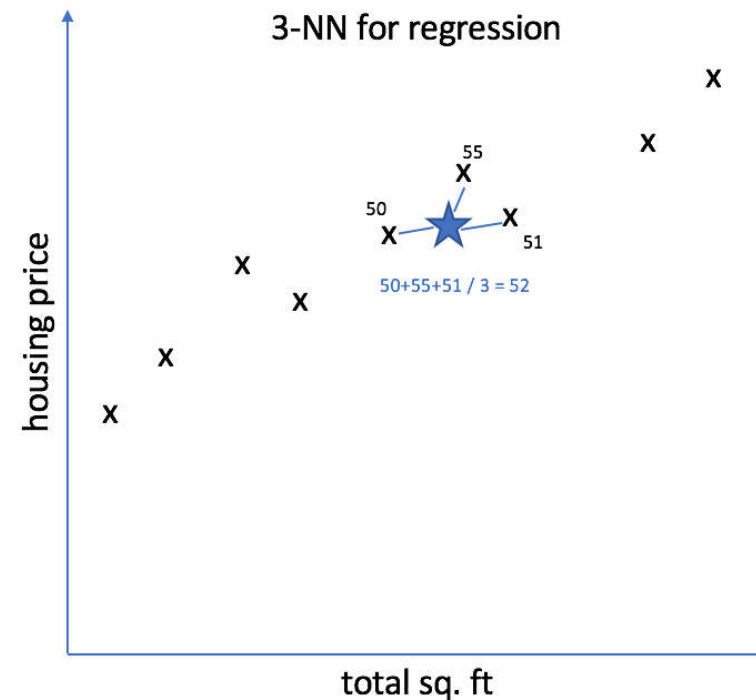
Search for similar data points previously stored

$$\begin{bmatrix} x_{4,1} & x_{4,2} & \dots & x_{4,n} \\ x_{9,1} & x_{9,1} & \dots & x_{9,n} \\ x_{15,1} & x_{15,1} & \dots & x_{15,n} \end{bmatrix} \text{ and } \begin{bmatrix} y_4 \\ y_9 \\ y_{15} \end{bmatrix}$$



And use them to generate your prediction

$$y_i = \frac{y_4 + y_9 + y_{15}}{3}$$



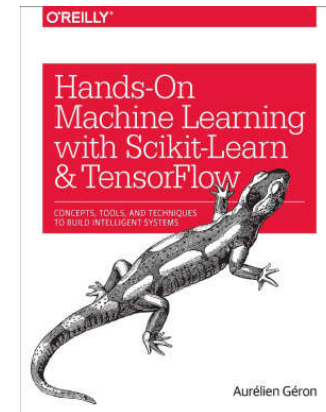
References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow',

Coursera, Machine Learning, Andrew Ng

<https://www.coursera.org/learn/machine-learning/home/welcome>



<https://github.com/yanshengjia/ml-road/tree/master/resources>

Data Analysis

Lecture 3: Classification

Dr. Nataliya K. Sakhnenko
Samsung R&D Institute, Ukraine

October 10, 2020,
Kiev Polytechnic Institute,
Institute of Physics and Technology

Sklearn lib

Scikit-learn

- Simple and efficient tools for data mining and data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable



Data in scikit-learn stored as a 2D array [n_samples, n_features]

Scikit-learn's interface

Linear Models

```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
model.fit(X_train, Y_train)  
model.predict(X_new)
```

Ordinary least squares
Linear Regression

Stochastic gradient descent

```
from sklearn.linear_model import SGDRegressor(loss='squared_loss',  
                                              penalty='l2', alpha=0.0001)
```

```
from sklearn.linear_model import LogisticRegression()
```

Scikit-learn

kNN

```
sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, weights='uniform', p=2,  
                                       metric='minkowski')
```

```
sklearn.neighbors.KNeighborsRegressor()
```

Decision Tree

```
sklearn.tree.DecisionTreeClassifier(criterion='gini', max_depth=None,  
                                    min_samples_split=2, class_weight=None)
```

```
sklearn.tree.DecisionTreeRegressor()
```

Scikit-learn

Model Evaluation

```
from sklearn.metrics import classification_report  
from sklearn.metrics import confusion_matrix  
  
print(classification_report(y_labels, y_predictions))  
print(confusion_matrix(y_labels, y_predictions))
```

Train Test split

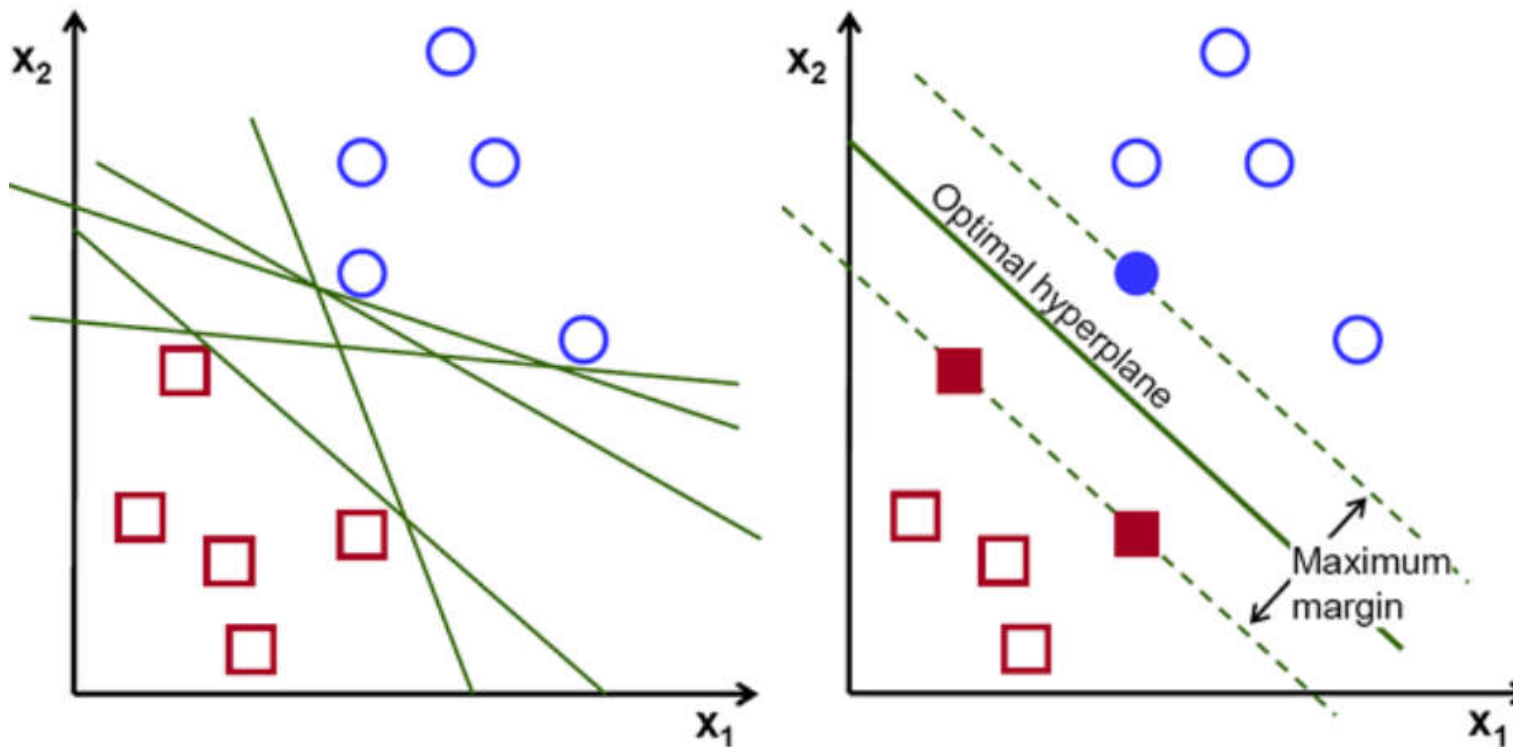
```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.30)
```

Roadmap

Basic Machine Learning		Deep Learning
Supervised	Unsupervised	☐ MLP ☐ CNN ☐ RNN ☐ Generative models: GANs, VAEs, NFs ☐ Attention and Transformers ☐ Intro to RL
☐ Regression (linear and polynomial) ☐ Classification ✓ Logistic regression, KNN, Decision Tree, SVM, NB ✓ Ensemble methods: Random Forest, AdaBoost	☐ Clustering (k-means, EM) ☐ Dimensionality reduction (PCA, t-SNE)	

Support Vector Machine

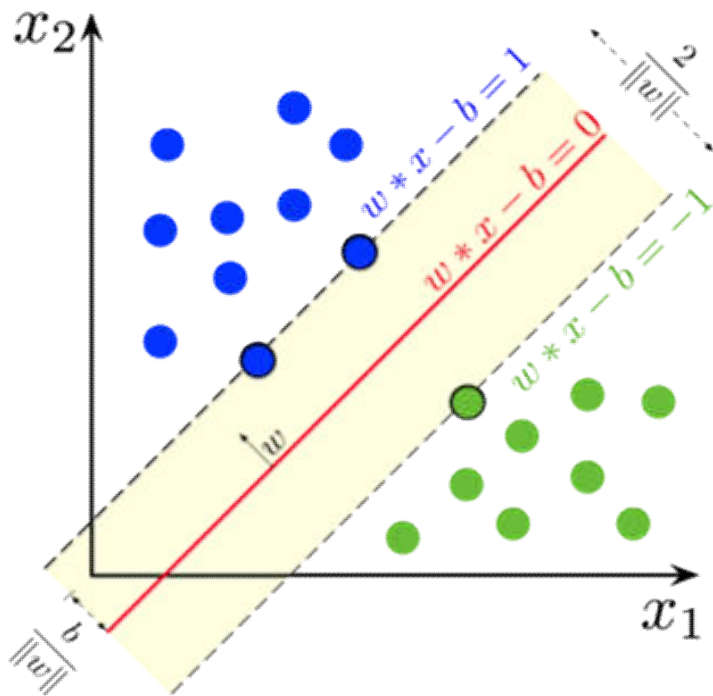
Linear SVM Classification



The left plot shows the decision boundaries of five possible linear classifiers. The solid line in the right plot represents the decision boundary of SVM classifier; this boundary not only separates the two classes but also stays as far away from the closest training instances as possible

Linear SVM: Hard Margin

$$Y = \{-1, 1\}$$



We want decision algorithm in the form:

$$a(x) = \text{sign}(w^T \cdot x + b)$$

Margin:

$$\frac{w^T}{|w|} \cdot (x_+ - x_-) = \frac{w^T \cdot x_+ - w^T \cdot x_-}{|w|} = \frac{2}{|w|}$$

we can formulate the *quadratic optimization problem*

$$\underset{w, b}{\text{minimize}} \quad \frac{1}{2} w^T w \quad (1)$$

$$\text{subject to } y_i (w^T x^{(i)} + b) \geq 1, \quad i = 1, 2, \dots, m$$

Linear SVM: Hard Margin

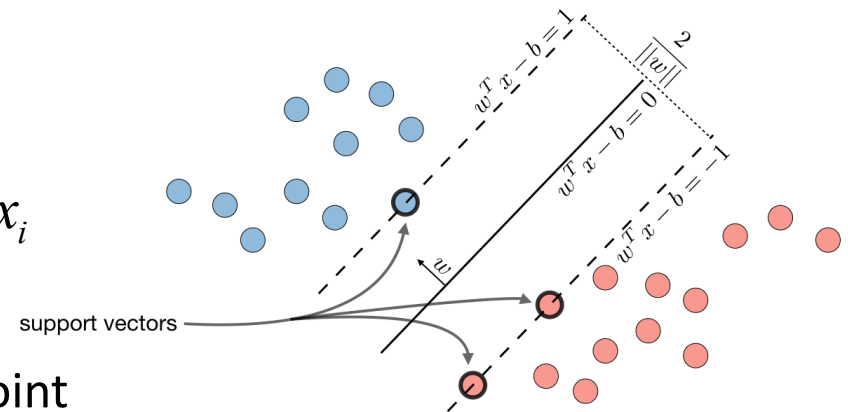
Quadratic optimization problems are a well-known class of mathematical programming problem, and many (intricate) algorithms exist for solving them (with many special ones built for SVMs)

The solution involves constructing a *dual problem* where a *Lagrange multiplier* λ_i is associated with every constraint in the primary problem:

$$a(x) = \text{sign}\left(\sum_{i=1}^k \lambda_i y_i \langle x^i, x \rangle + b\right)$$

Each non-zero λ_i indicates that corresponding x_i is a support vector.

It relies on an *inner product* between the test point $\mathbf{x}$ and the support vectors $\mathbf{x}_i$

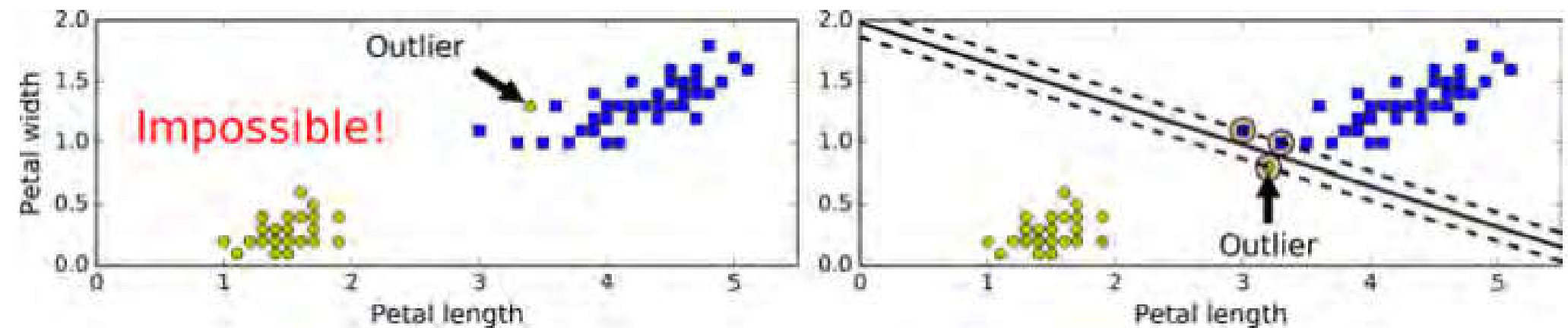


Linear SVM: Hard Margin

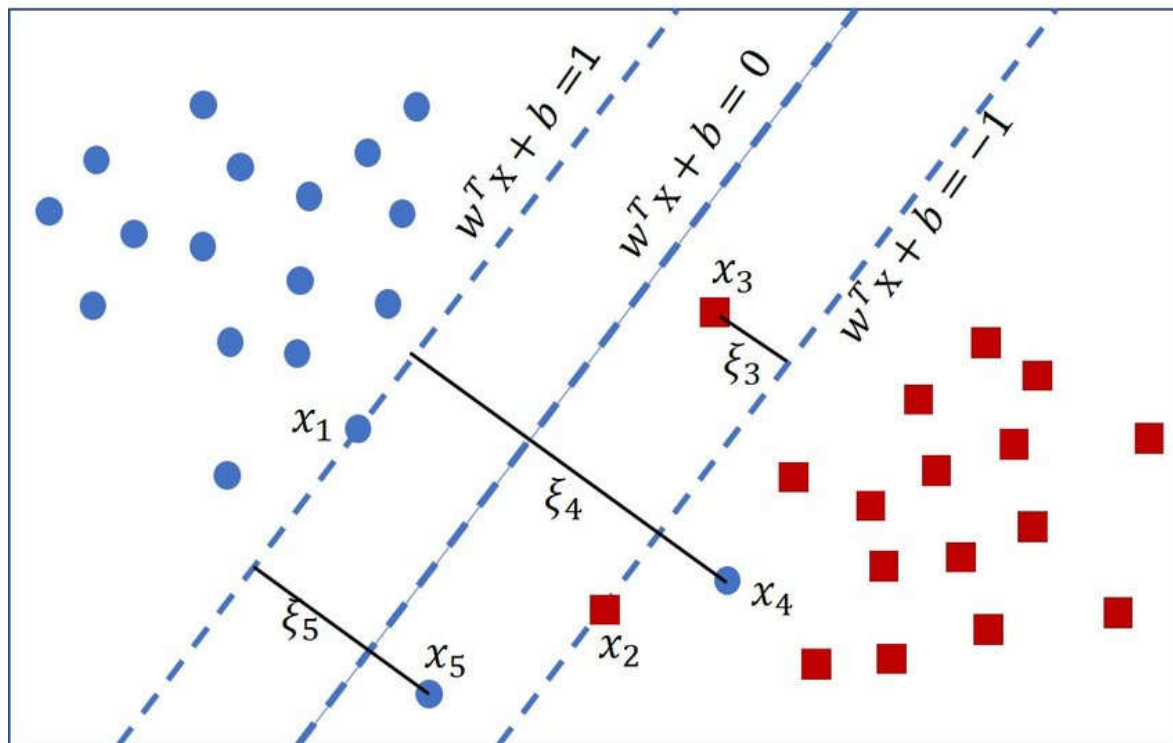
We strictly impose that all instances be off the margin and on the right side.

Issues:

- It only works if data is linearly separable
- It may plot decision boundary based on outliers



Linear SVM: Soft Margin



$$0 < \xi_3 < 1,$$
$$\xi_5 > 1$$

If the training data is not linearly separable, *slack variables* ξ_i can be added to allow misclassification of difficult or noisy examples.

Allow some errors

Let some points be moved to where they belong, at a cost

Still, try to minimize training set errors, and to place hyperplane “far” from each class (large margin)

Linear SVM: Soft Margin

The *quadratic optimization* problem incorporating slack variables

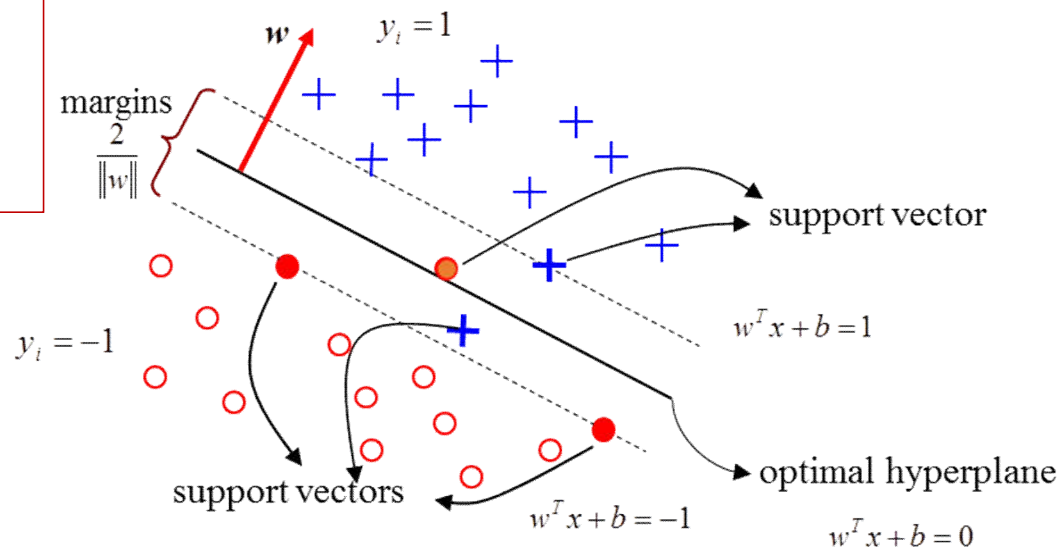
$$\begin{aligned} & \underset{w, b, \xi}{\text{minimize}} && \frac{1}{2} w^T w + C \sum_i \xi_i \\ & \text{subject to} && y_i (w^T x^{(i)} + b) \geq 1 - \xi_i, \\ & && \xi_i \geq 0, \quad i = 1, 2, \dots, m \end{aligned}$$

Parameter C can be viewed as a way to control overfitting (regularization)

The solution:

$$a(x) = \text{sign}\left(\sum_{i=1}^k \lambda_i y_i \langle x^i, x \rangle + b\right)$$

Again, x_i with non-zero λ_i will be support vectors.



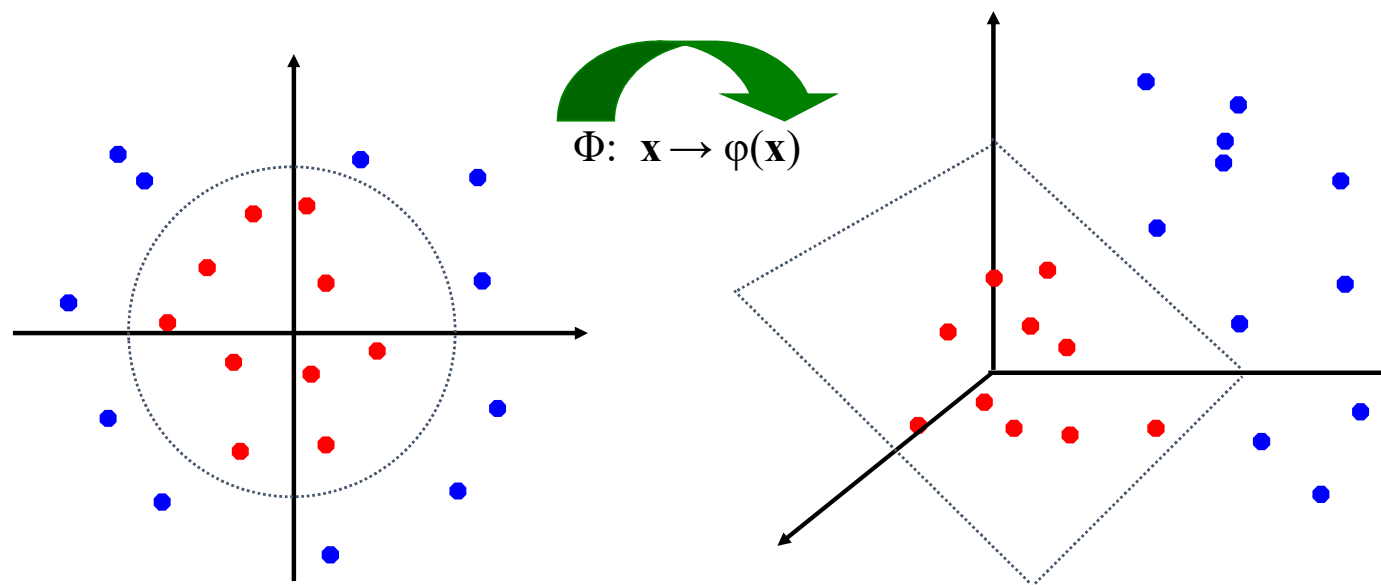
Linear SVM: Summary

- The classifier is a *separating hyperplane*.
- The most “important” training points are the support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points $\mathbf{x}_i$ are support vectors with non-zero Lagrangian multipliers λ_i .
- Both in the dual formulation of the problem and in the solution, training points appear only inside inner products:

$$a(x) = \text{sign}\left(\sum_{i=1}^k \lambda_i y_i \langle x^i, x \rangle + b\right)$$

Non-linear SVMs: Feature spaces

- General idea: the original feature space can be mapped to some higher-dimensional feature space where the training set is separable:



'Kernel Trick'

The linear classifier relies on an inner product between vectors

If every datapoint is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$, the inner product becomes:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

A *kernel function* is some function that corresponds to an inner product in some expanded feature space.

Solution:

$$a(x) = \text{sign}\left(\sum_{i=1}^k \lambda_i y_i K(x^i, x) + b\right)$$

Kernels

Why use kernels?

Make non-separable problem separable.

Map data into better representational space

Common Kernels:

- Linear:

$$K(a, b) = a^T b$$

- Polynomial:

$$K(a, b) = (\gamma a^T b + r)^d$$

- Radial basis function(RBF)
(infinite dimensional space)

$$K(a, b) = \exp(-\gamma \|a - b\|^2)$$

- Sigmoid:

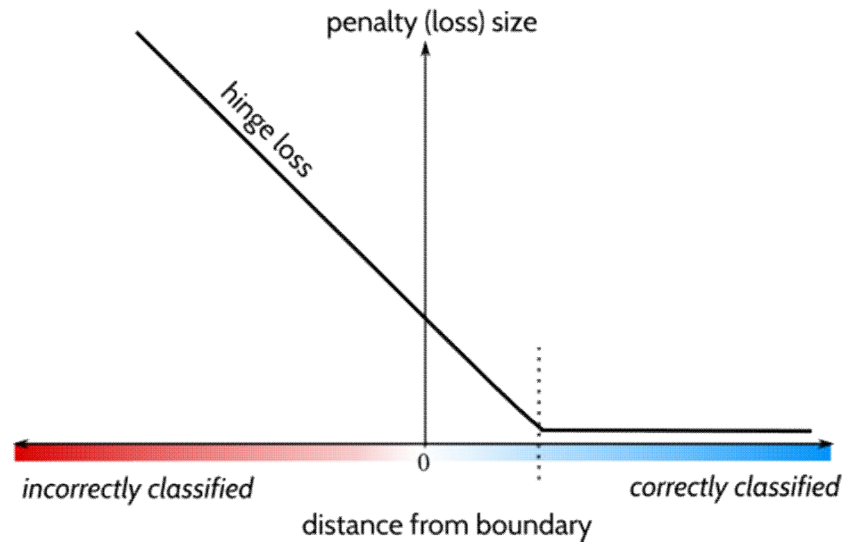
$$K(a, b) = \tanh(\gamma a^T b + r)$$

Hinge Loss

Linear SVM classifier cost function:

$$J(w, b) = C \sum_{i=1}^m \max(0, 1 - y_i (w^T x^{(i)} + b)) + \frac{1}{2} w^T w$$

hinge loss : $\max(0, 1 - t)$



SVM in sklearn lib

```
sklearn.svm.LinearSVC()  
sklearn.linear_model.SGDClassifier(loss='hinge')  
sklearn.svm.SVC(kernel='rbf')
```

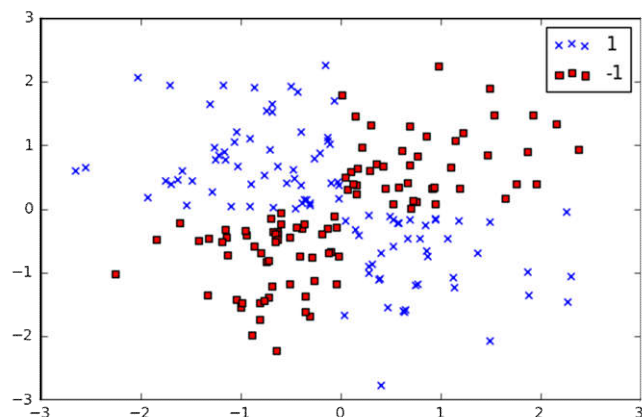
Class	Time complexity	Scaling required	Kernel trick
LinearSVC	$O(m \times n)$	Yes	No
SGDClassifier	$O(m \times n)$	Yes	No
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	Yes	Yes

SVMs are sensitive to the feature scaling!!!

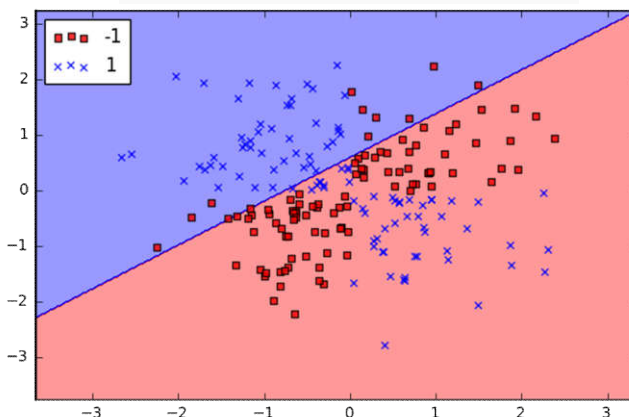
SVM software packages:

- libsvm – most commonly used implementation of kernalized svm, sklearn uses wrapper over it
- liblinear – gradient descent based implementation of linear SVM

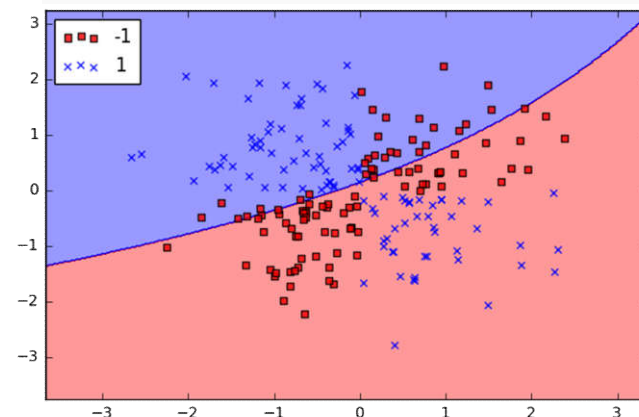
SVM Example



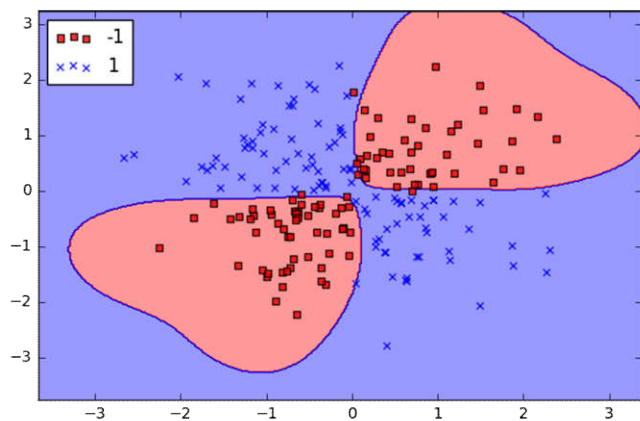
SVC(kernel='linear', C=1)



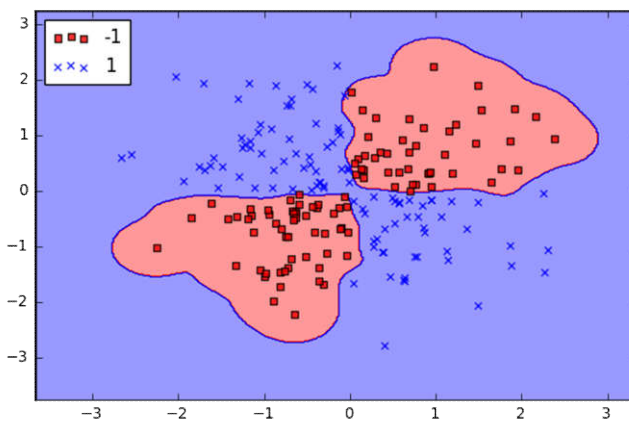
SVC(kernel='rbf', gamma=.01, C=1)



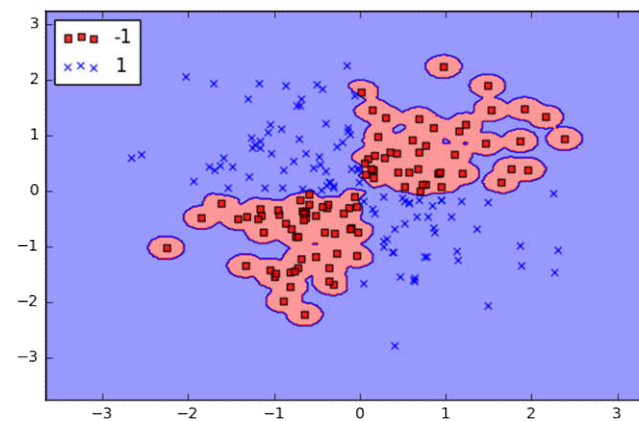
SVC(kernel='rbf', gamma=1, C=1)



SVC(kernel='rbf', gamma=10, C=1)

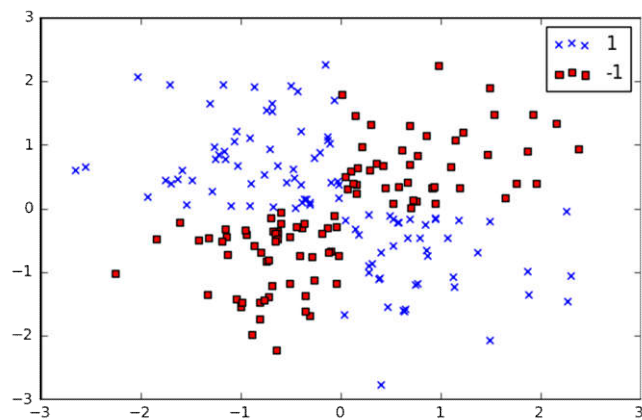


SVC(kernel='rbf', gamma=100, C=1)

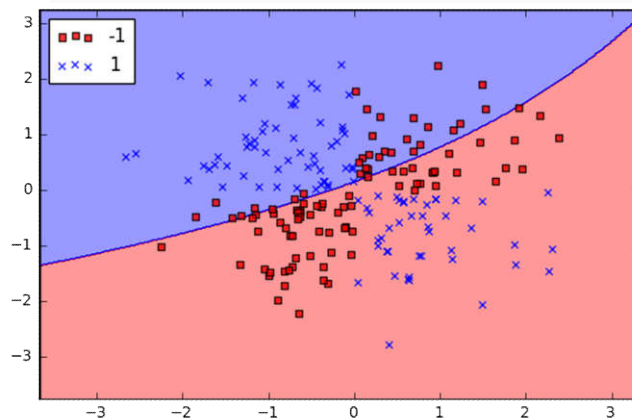


[https://chrisalbon.com/machine learning/support vector machines/svc parameters using rbf kernel/](https://chrisalbon.com/machine_learning/support_vector_machines/svc_parameters_using_rbf_kernel/)

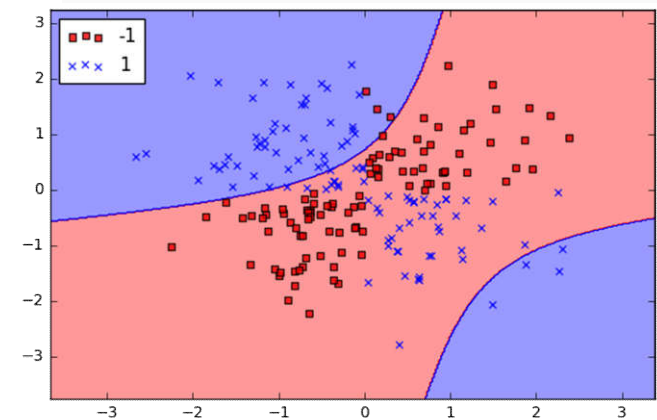
SVM Example



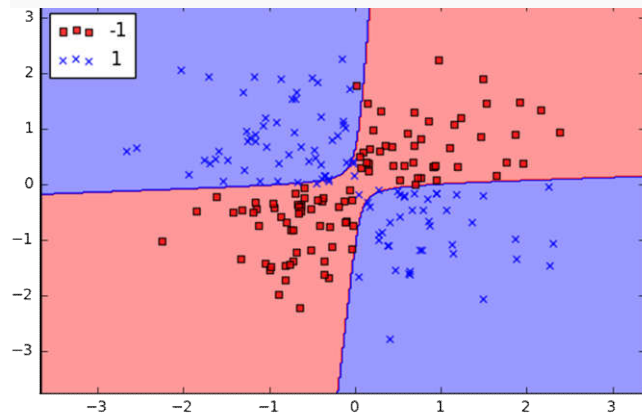
SVC(kernel='rbf', gamma=.01, C=1)



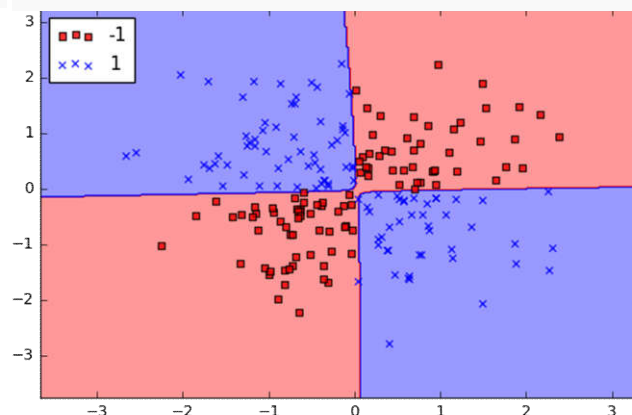
SVC(kernel='rbf', gamma=.01, C=10)



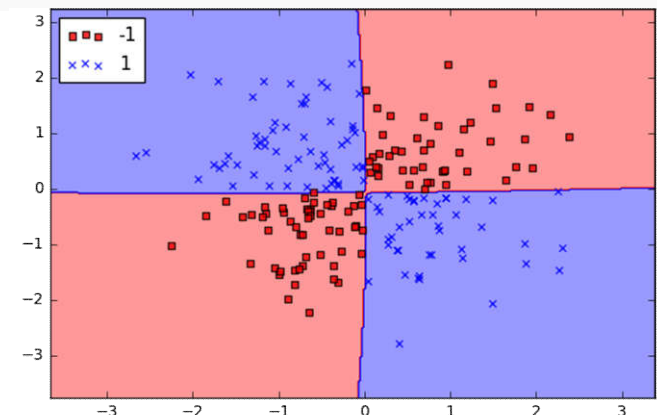
SVC(kernel='rbf', gamma=.01, C=1000)



SVC(kernel='rbf', gamma=.01, C=10000)



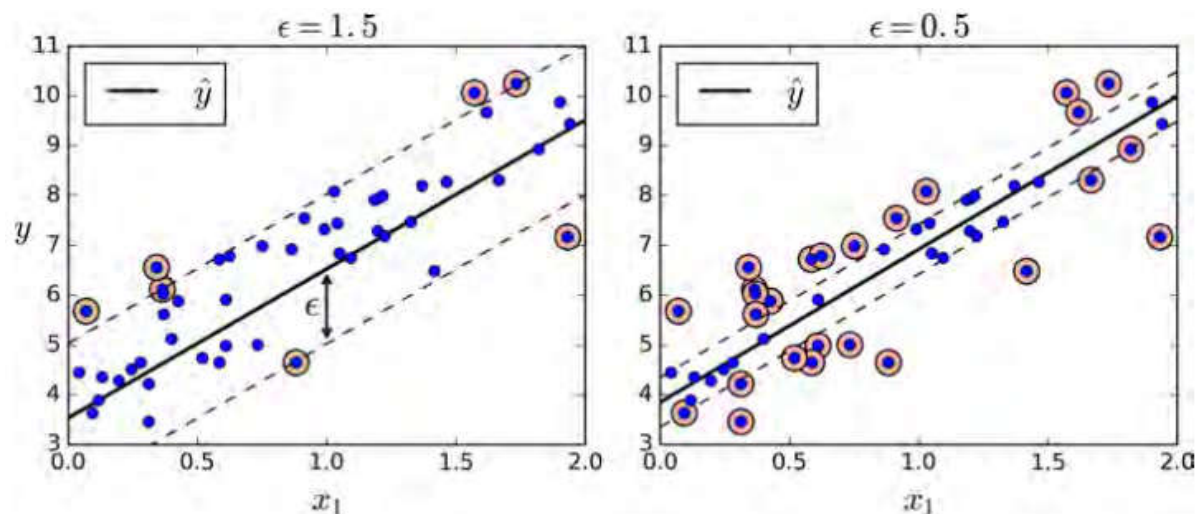
SVC(kernel='rbf', gamma=.01, C=100000)



Grid Search

```
param_grid = {'C': [0.1, 1, 10, 100, 1000], 'gamma': [1, 0.1, 0.01, 0.001, 0.0001], 'kernel': ['rbf']}  
from sklearn.model_selection import GridSearchCV  
grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=3)  
grid.fit(X, y)  
print(grid.best_params_)
```

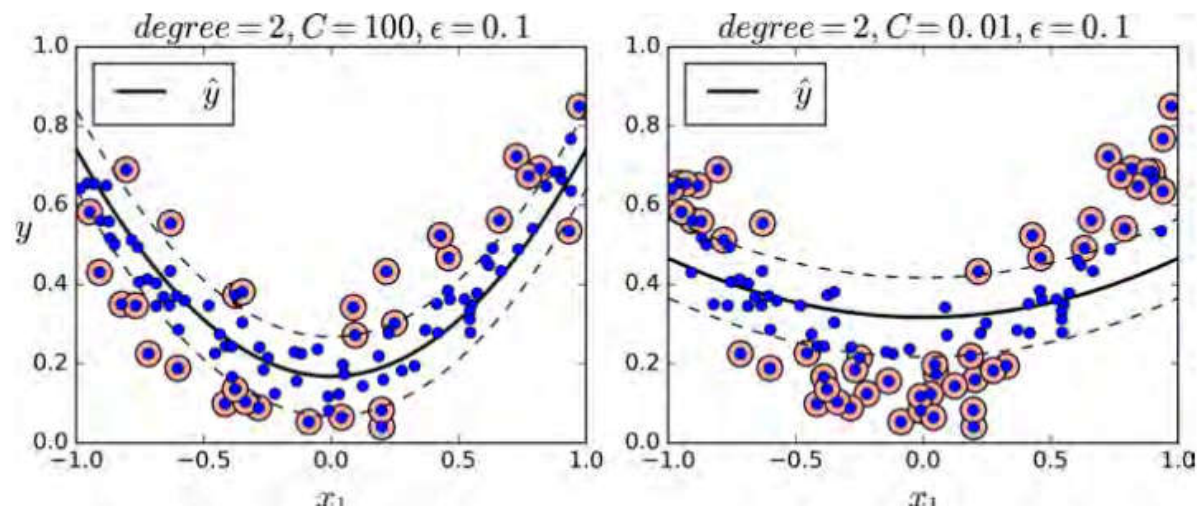
SVM Regression



SVM regression reverses the objective: instead of trying to fit largest margin between two classes, it tries to fit as many instances as possible inside the margin. The width is controlled by a hyperparameter 'epsilon'.

```
sklearn.svm.LinearSVR(epsilon = 0.5)
```

Kernelized SVM Regression



2nd – degree polynomial kernel is applied

```
sklearn.svm.LinearSVR(kernel = 'poly', degree = 2, C = 100, epsilon = 0.1)
```


Naïve Bayes Classifier

Bayesian Classification

Problem statement:

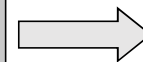
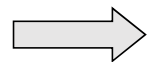
- Given features $X_1, X_2, \dots, X_n$
- Predict a label Y

Our goal is to find:

$$\arg \max_Y P(Y | X_1, \dots, X_n)$$

the probability that the image represents 5 given its pixels

Digit Recognition



5



$X_1, \dots, X_n \in \{0, 1\}$ (Black vs. White pixels)

$Y \in \{5, 6\}$ (predict whether a digit is a 5 or a 6)

Bayesian Classification

Bayes Rule

Posterior Probability

Likelihood

Prior

$$P(Y|X_1, \dots, X_n) = \frac{P(X_1, \dots, X_n|Y)P(Y)}{P(X_1, \dots, X_n)}$$

Normalization Constant

Likelihood: we think that we might be able to specify how features are “generated” by the class label

For our digit recognition task:

$$P(Y = 5|X_1, \dots, X_n) = \frac{P(X_1, \dots, X_n|Y = 5)P(Y = 5)}{P(X_1, \dots, X_n|Y = 5)P(Y = 5) + P(X_1, \dots, X_n|Y = 6)P(Y = 6)}$$
$$P(Y = 6|X_1, \dots, X_n) = \frac{P(X_1, \dots, X_n|Y = 6)P(Y = 6)}{P(X_1, \dots, X_n|Y = 5)P(Y = 5) + P(X_1, \dots, X_n|Y = 6)P(Y = 6)}$$

To classify, we'll simply compute these two probabilities and predict based on which one is greater

Naïve Bayes Model

For the Bayes classifier, we need to “learn” two functions, the likelihood and the prior.

The Naïve Bayes Assumption: Assume that all features are independent given the class label Y

$$P(X_1, \dots, X_n | Y) = \prod_{i=1}^n P(X_i | Y)$$

$$P(y | x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i | y)$$

Naïve Bayes Training

- Estimate $P(Y=v)$ as the fraction of records with $Y=v$

$$P(Y = v) = \frac{\text{Count}(Y = v)}{\# \text{ records}}$$

- Estimate $P(X_i=u | Y=v)$ as the fraction of records with $Y=v$ for which $X_i=u$

$$P(X_i = u | Y = v) = \frac{\text{Count}(X_i = u \wedge Y = v)}{\text{Count}(Y = v)}$$

This corresponds to Maximum Likelihood estimation of model parameters

Naïve Bayes Example

Data set of weather and corresponding target variable 'Play' (suggesting possibilities of playing).

We need to classify whether players will play or not based on weather condition.

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
All	5	9
	=5/14	=9/14
	0.36	0.64

Problem: Players will play if weather is sunny. Is this statement is correct?

$$P(\text{Yes} \mid \text{Sunny}) = P(\text{Sunny} \mid \text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

$$P(\text{Sunny} \mid \text{Yes}) = 3/9 = 0.33, P(\text{Sunny}) = 5/14 = 0.36, P(\text{Yes}) = 9/14 = 0.64$$

Now, $P(\text{Yes} \mid \text{Sunny}) = 0.33 * 0.64 / 0.36 = 0.60$, which has higher probability.

Naïve Bayes

- When assumption of independence holds, a Naive Bayes classifier performs better compare to other models
- It perform well in case of categorical input variables compared to numerical variable(s).
- If categorical variable has a category (in test data set), which was not observed in training data set, then model will assign a 0 (zero) probability and will be unable to make a prediction. This is often known as “Zero Frequency”. To solve this, we can use the smoothing technique. One of the simplest smoothing techniques is called Laplace estimation.

```
sklearn.naive_bayes.MultinomialNB
```

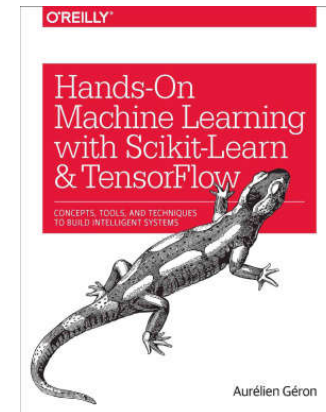
References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow',

Coursera, Machine Learning, Andrew Ng

<https://www.coursera.org/learn/machine-learning/home/welcome>



<https://github.com/yanshengjia/ml-road/tree/master/resources>

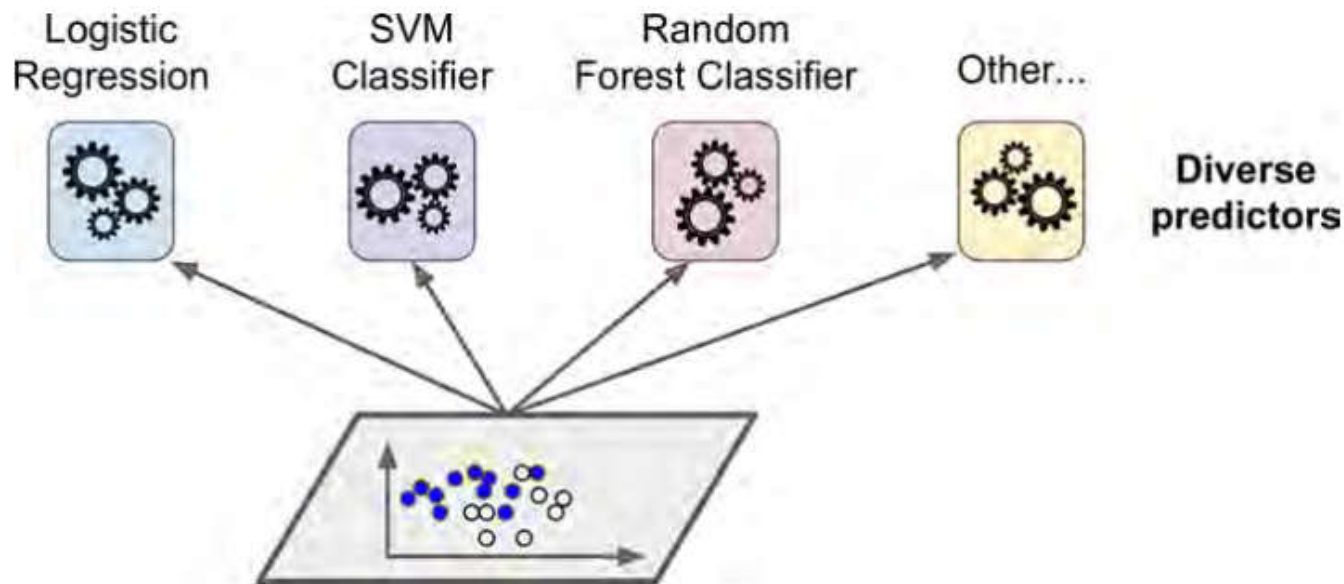
Data Analysis

Lecture 4: Ensemble classifiers

Dr. Nataliya K. Sakhnenko

Voting Classifiers

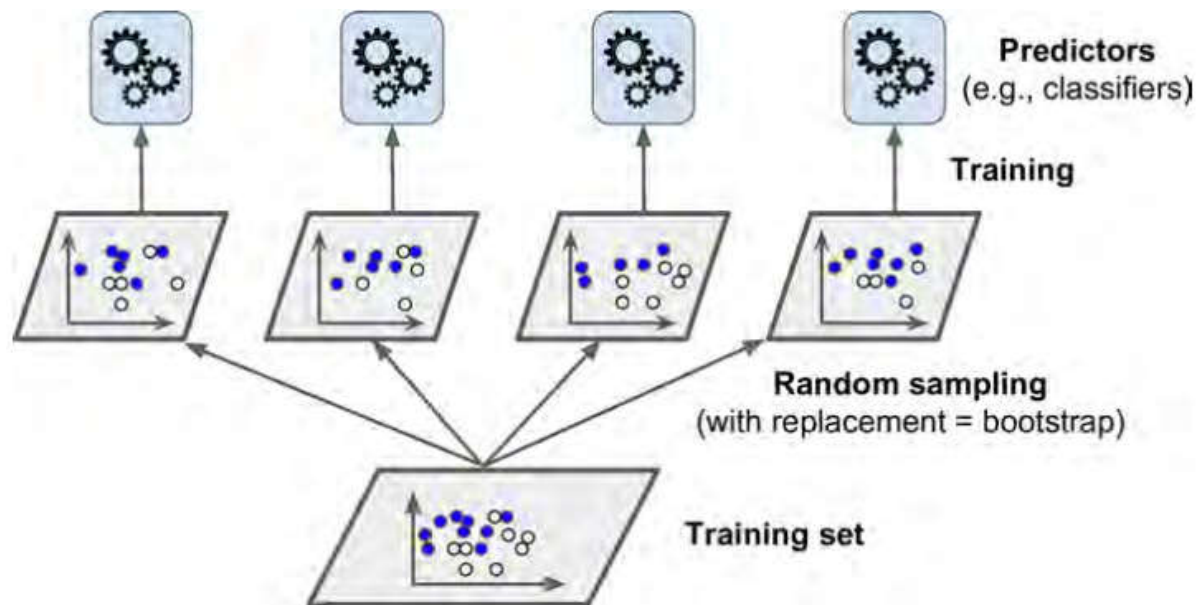
Suppose you trained a few classifiers. A simple way to create a better classifier is to aggregate the predictions of each classifier and predict the class that gets the most votes.



Bagging

Bagging

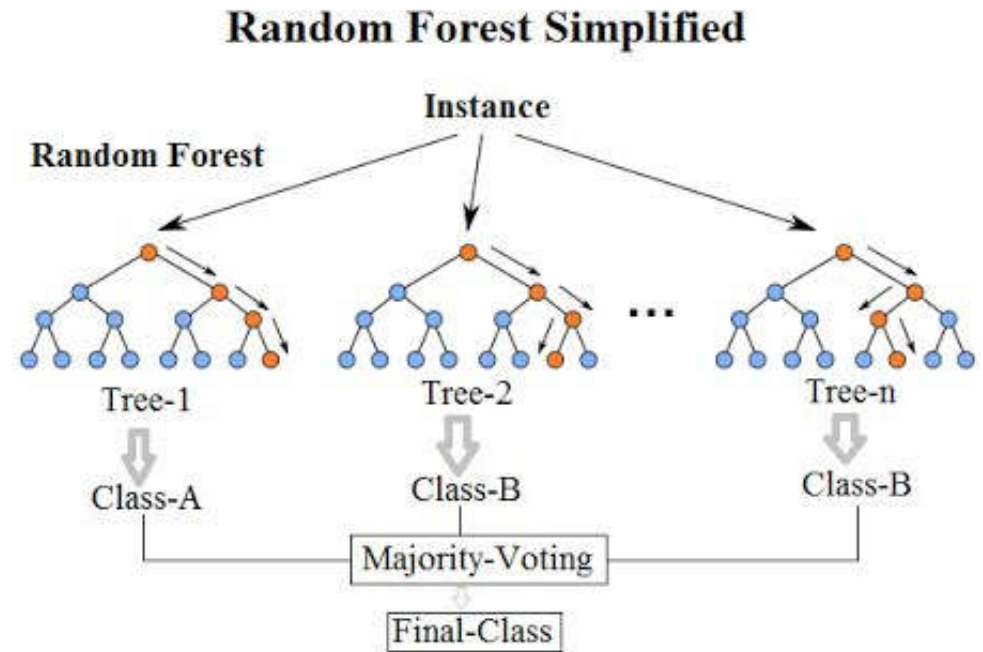
You can use the same training algorithm for every predictor. But to train them on different random subset of the training set.



Random Forest is an ensemble of Decision Trees, trained via the bagging method

Random Forest: Tin Kam Ho (1995)

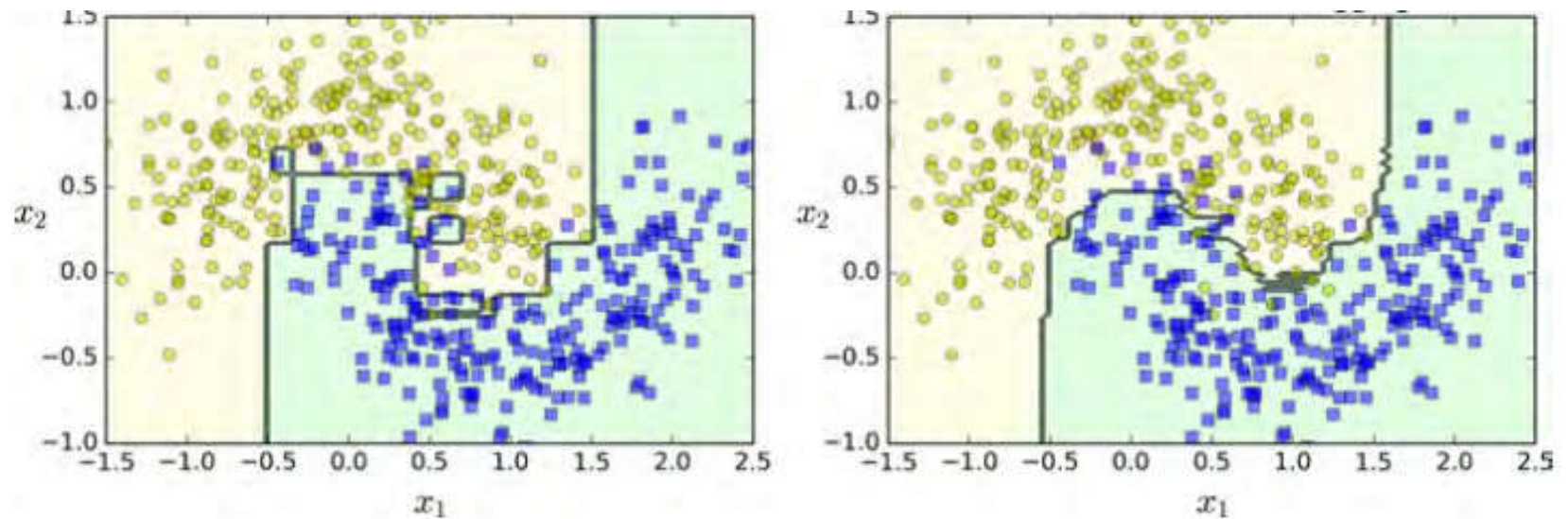
The RF algorithm introduces extra randomness when growing trees: instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features.



```
sklearn.ensembles.RandomForestClassifier
```

Contrary to the simple decision tree, RF is highly uninterpretable

Random Forest: Decision Boundary

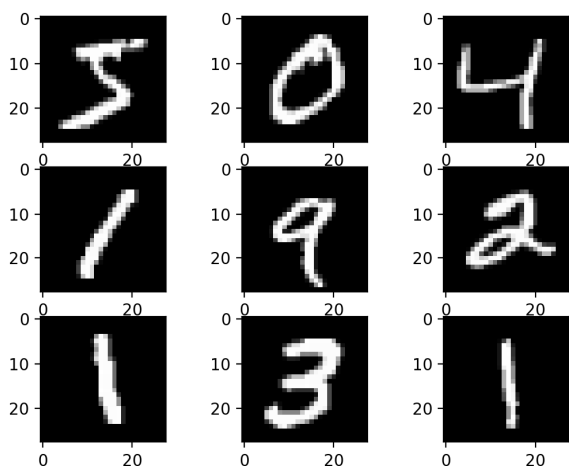


A single Decision Tree vs a Random Forest with 500 trees

Random Forest: Feature Importance

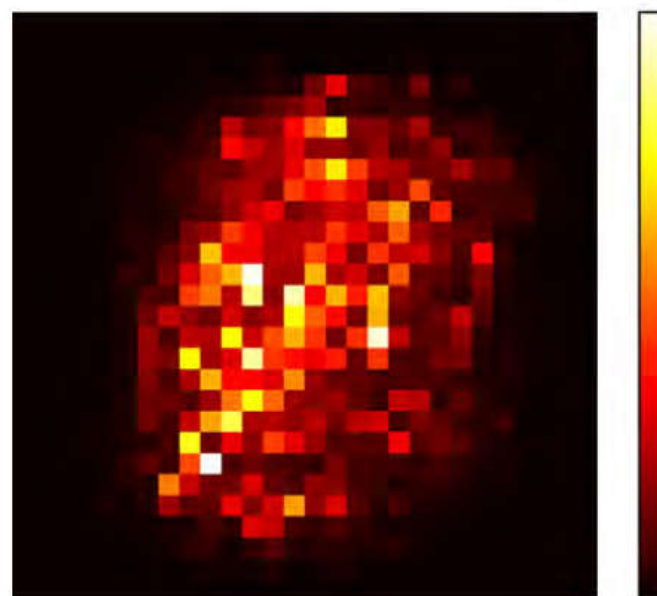
- A single DT: important features are likely to appear closer to the root of tree, while unimportant features often appear closer to the leaves
- RF: feature importance as average depth at which it appears across all trees in the forest

```
RandomForestClassifier().feature_importances_
```



The MNIST database is a large **database** of handwritten digits (28x28 pixels, 60,000 training images and 10,000 testing images)

MNIST pixel importance with accordance to RF classifier



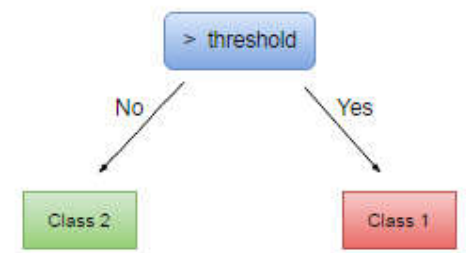
Boosting

Boosting

Boosting — The idea of boosting methods is to combine several weak learners to form a stronger one.

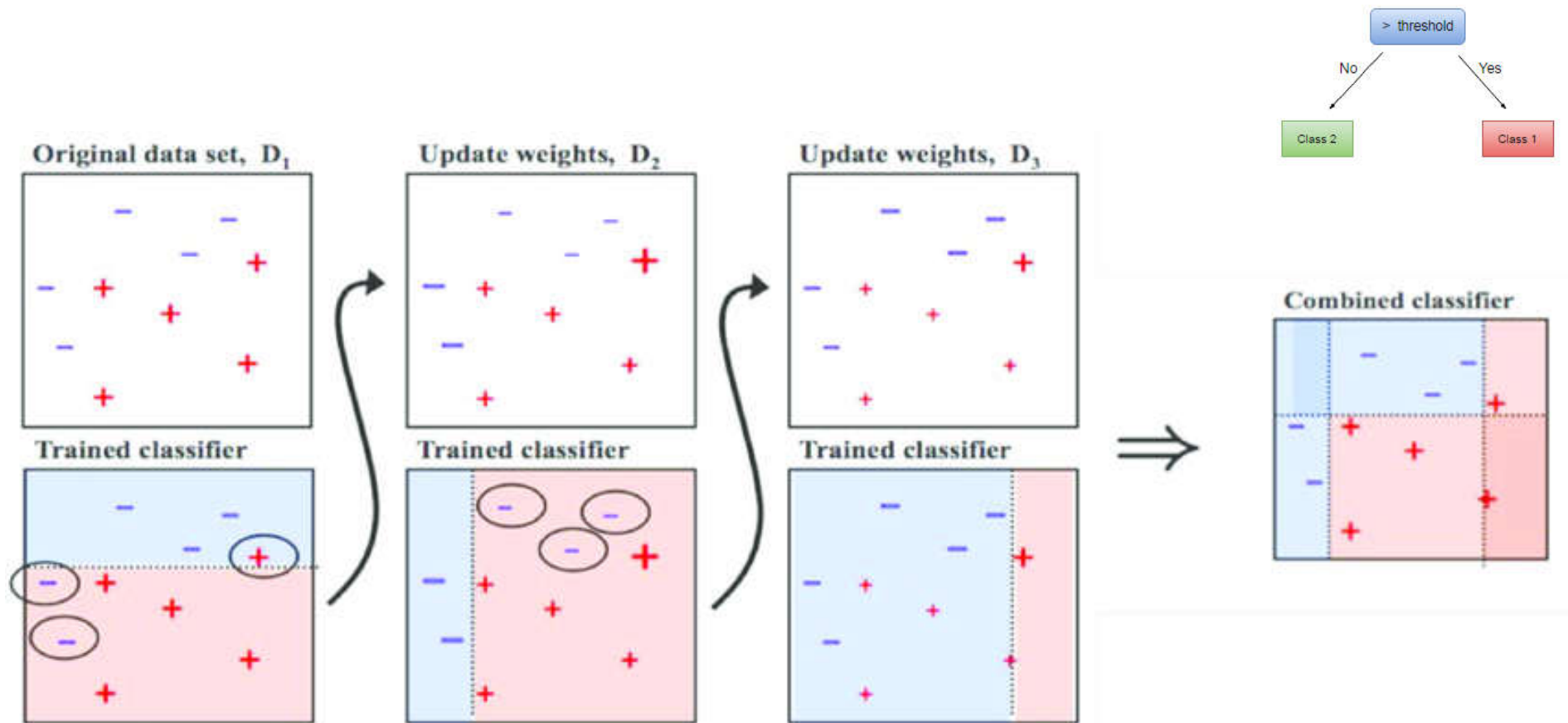
Adaptive boosting	Gradient boosting
• Known as Adaboost • High weights are put on errors to improve at the next boosting step	• Weak learners trained on remaining errors

A 'weak' learner (classifier, predictor, etc) is just one which performs relatively poorly--its accuracy is above chance, but just barely.



Decision stump

AdaBoost: Intuition



AdaBoost: (Freund&Schapire, 1996)

$X \rightarrow Y = \{-1, 1\}$ M is number of classifiers, m is number of train instances

Step 1. Set initial weights: $w^{(i)} = \frac{1}{m}, i = 1..m$

Step 2. For $j = 1 .. M$ repeat steps:

- Fit a classifier G_j to the training data
- Calculate weighted error rate of the j^{th} predictor:

$$r_j = \frac{\sum_{i=1, \hat{y}_j^{(i)} \neq y^{(i)} }^m w^{(i)}}{\sum_{i=1}^m w^{(i)}}, \hat{y}_j^{(i)} \text{ is the } j^{\text{th}} \text{ classifier prediction for the } i^{\text{th}} \text{ instance}$$

- Compute classifier weight: $a_j = \eta \log \frac{1-r_j}{r_j}$, η is learning rate hyperparameter
- Update weights: $w^{(i)} := \begin{cases} w^{(i)}, & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \cdot \exp(\alpha_j), & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$ and normalize them to sum to 1

Step 3. Output: $G(x) = \text{sign}(\sum_{j=1}^M \alpha_j G_j(x))$

`sklearn.ensembles.AdaBoostClassifier`

Gradient Boost: Intuition

Regression problem

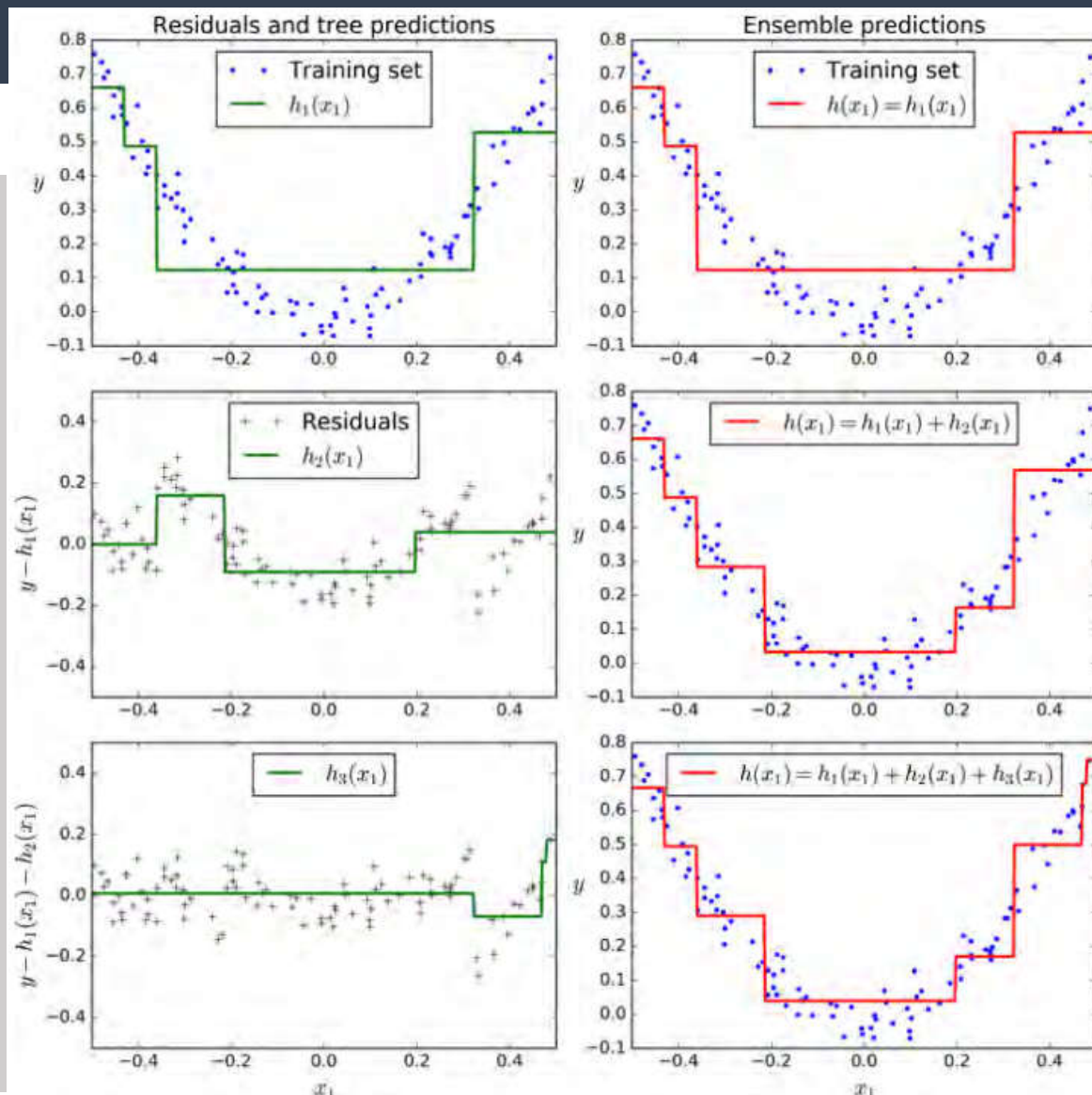
```
from sklearn.tree import DecisionTreeRegressor
```

```
tree_reg1 =  
DecisionTreeRegressor(max_depth=2)  
tree_reg1.fit(X, y)
```

```
y2 = y - tree_reg1.predict(X)  
tree_reg2 =  
DecisionTreeRegressor(max_depth=2)  
tree_reg2.fit(X, y2)
```

```
y3 = y2 - tree_reg2.predict(X)  
tree_reg3 =  
DecisionTreeRegressor(max_depth=2)  
tree_reg3.fit(X, y3)
```

```
y_pred = sum(tree.predict(X_new) for tree in  
(tree_reg1, tree_reg2, tree_reg3))
```



Gradient Boost: Friedman 1999

Regression: $\{(x_i, y_i)\}_{i=1}^m$ M is number of classifiers, m is number of train instances

$$L(y_i, \hat{y}_i) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \text{ MSE loss function}$$

$$y = f(x), f - ?$$

Step1. Set $f_0(x) = \text{average}[y_i], i = 1..m$

Step2. For $j = 1 \dots M$ repeat steps:

Compute $r_{ij} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{j-1}(x)}$ for $i = 1..m$

$-\nabla L$

Fit a Regression Tree to the r_{ij} values

Update $f_j(x) = f_{j-1}(x) + \eta [\text{Regr Tree Result}](x)$, η is learning rate

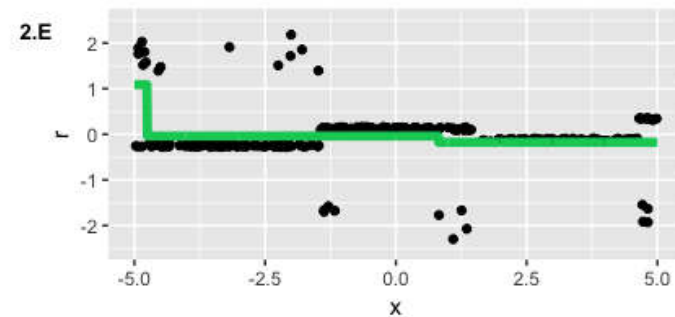
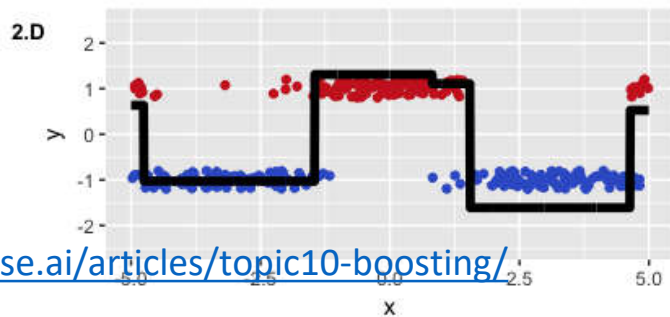
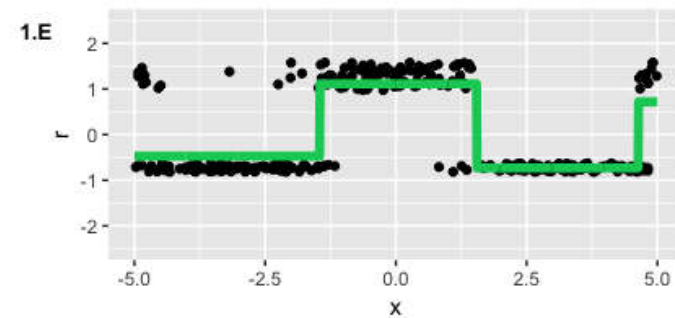
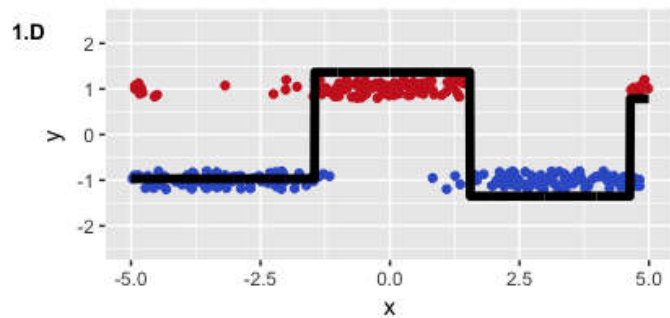
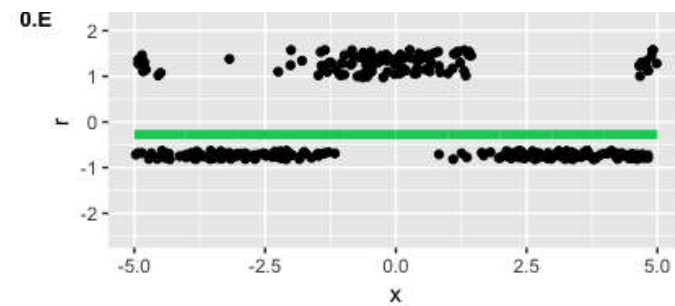
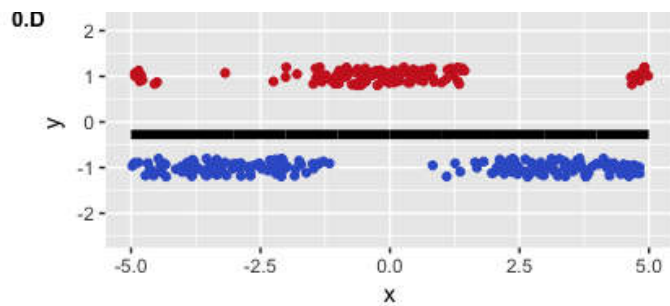
Step3. $f(x) = f_M(x)$

Use **log loss** for classification:

$$X \rightarrow Y = \{-1, 1\}$$
$$\log(1 + \exp(-y \cdot \hat{y}))$$

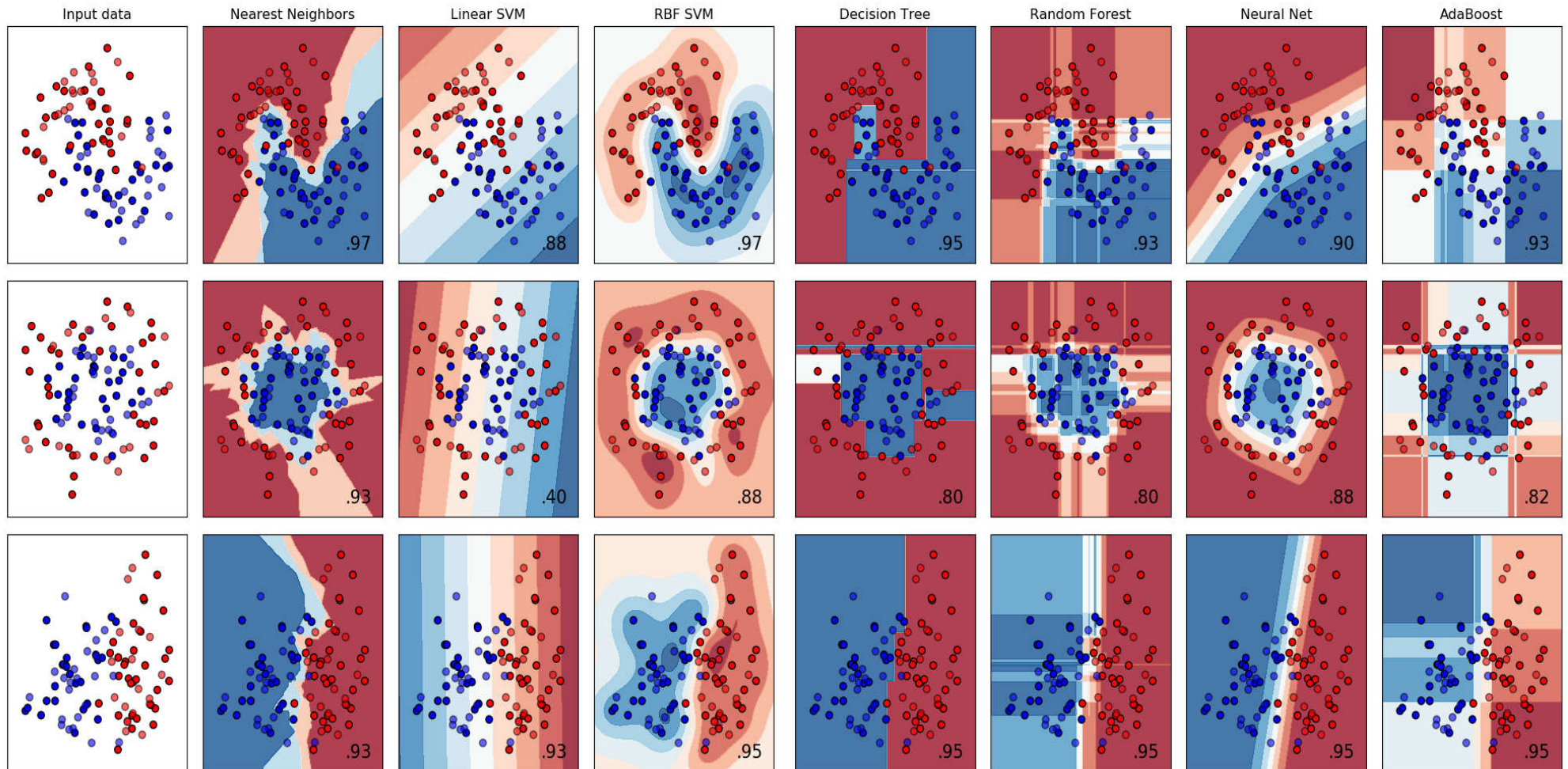
```
sklearn.ensemble.GradientBoostingClassifier  
sklearn.ensemble.GradientBoostingRegressor
```

Gradient Boost: Classification



<https://mlcourse.ai/articles/topic10-boosting/>

Classifier Comparison

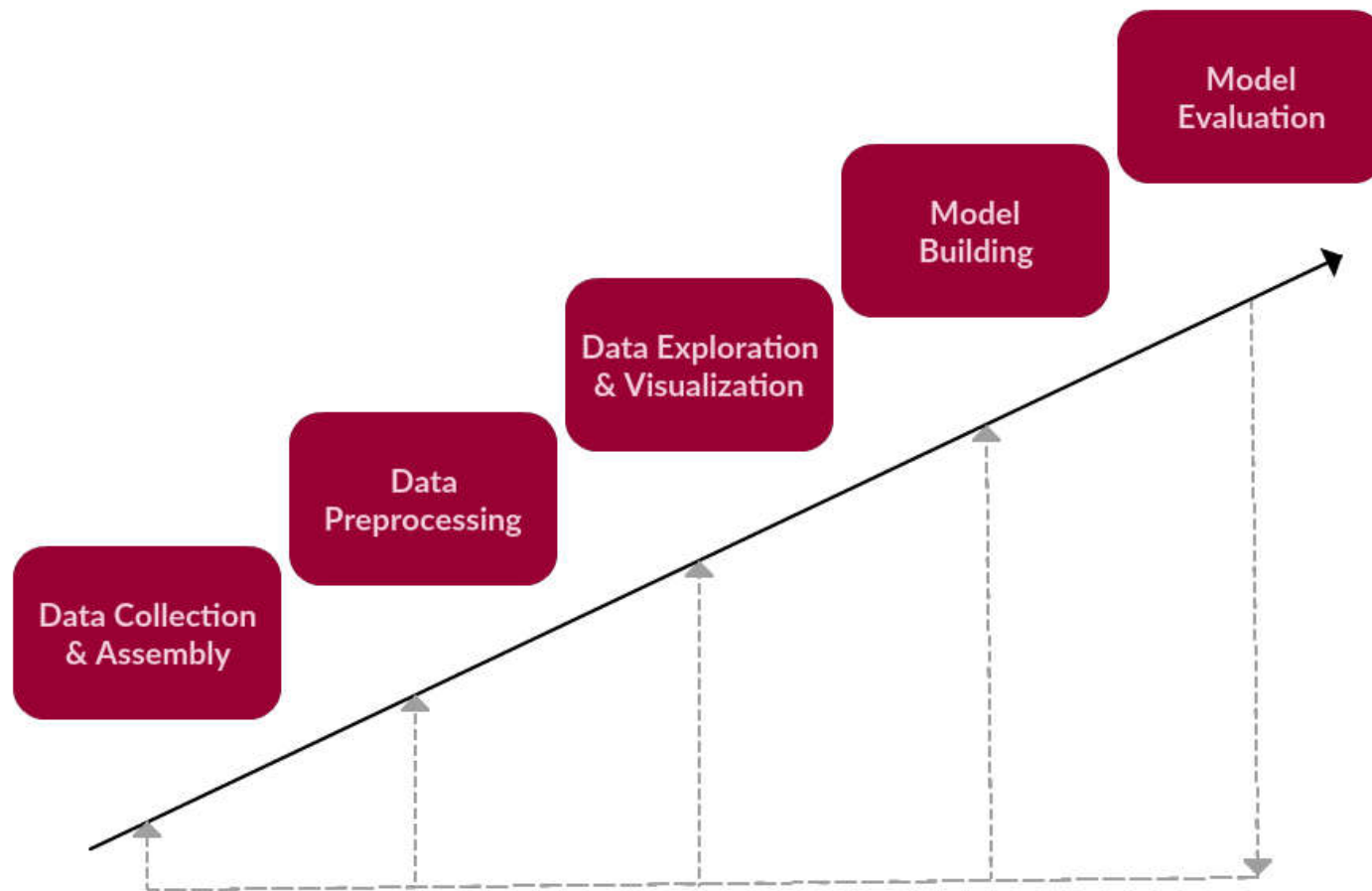


https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html

Data Visualization and Preprocessing

Pandas, matplotlib and seaborn lib

Data Analysis Steps



Pandas lib

pandas is an open source library providing high-performance, easy-to-use data structures and data analysis tools for the Python

pandas
 $y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



```
pandas.read_csv()
```

Read a comma-separated values (csv) file into DataFrame.

	id	iv2	rt
count	120.000000	120.000000	120.000000
mean	9.500000	2.000000	877.587425
std	5.790459	0.81992	309.293048
min	0.000000	1.00000	283.240752
25%	4.750000	1.00000	582.630955
50%	9.500000	2.00000	902.719888
75%	14.250000	3.00000	1114.050194
max	19.000000	3.00000	1472.688933

```
pandas.DataFrame.head()
```

Return the first n rows

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
2	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
3	R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
4	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0

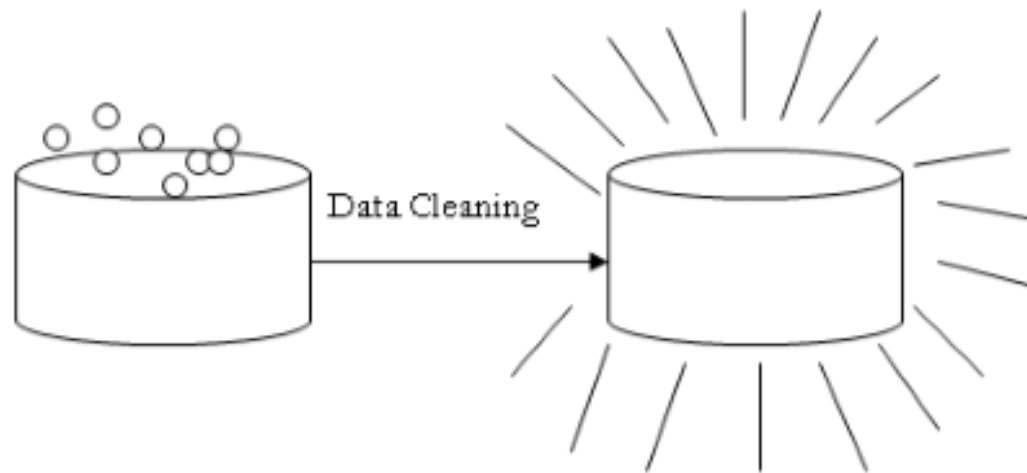
```
pandas.DataFrame.describe()
```

Generate descriptive statistics

Why data preprocessing

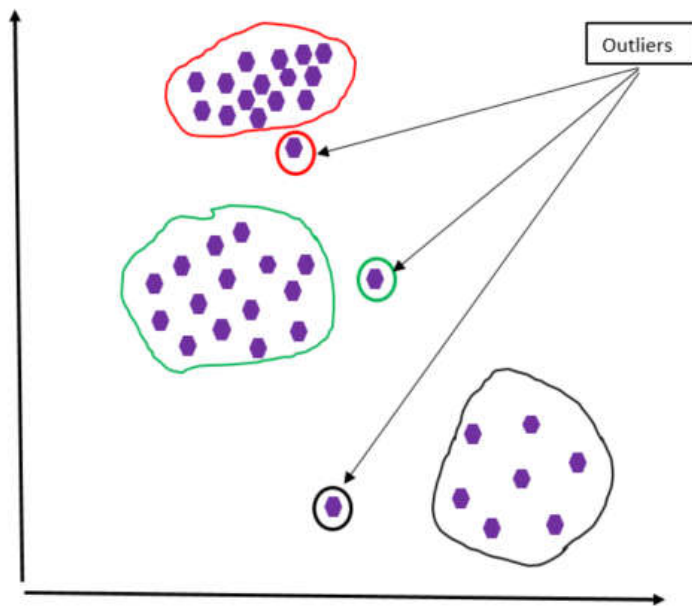
Data may be

- Incomplete (missing values)
- Noisy (containing errors or outliers)
- Inconsistent (containing discrepancies in dates, names, rates)

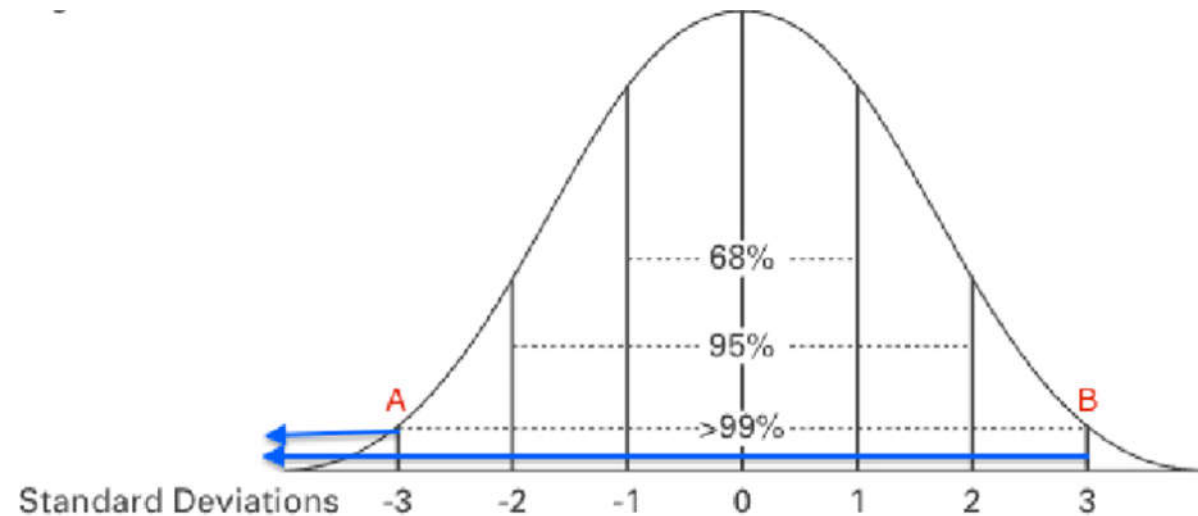


Data cleaning

Outliers detection



Three sigma rule



Normally data with $|x - \mu| > 3\sigma$
are considered as outliers

Data cleaning

Missing data

- may be deleted
- may be filled by:
 - ✓ the attribute mean
 - ✓ the attribute mean for all samples belonging to the same class
 - ✓ or other

```
pandas.DataFrame.dropna()
```

Remove missing values.

```
pandas.DataFrame.fillna()
```

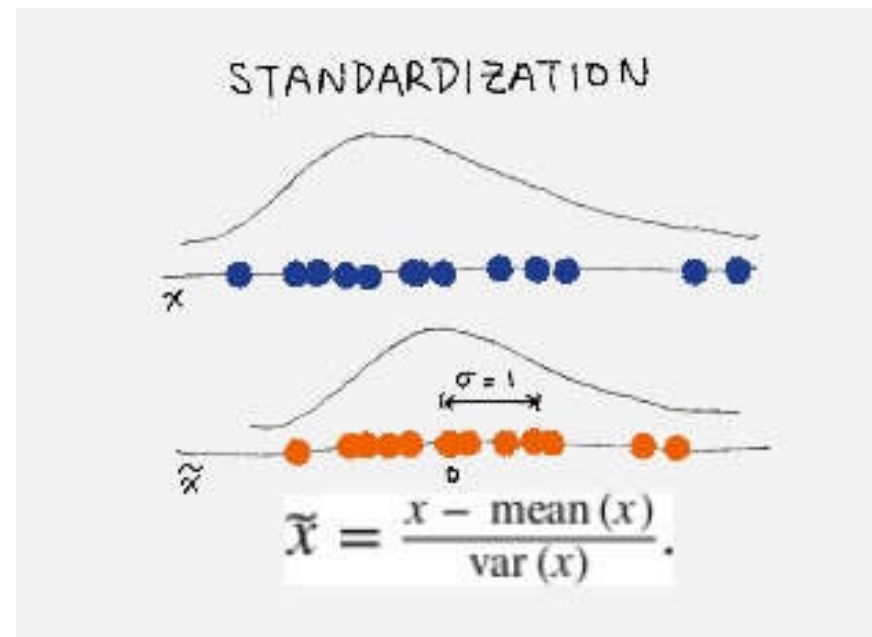
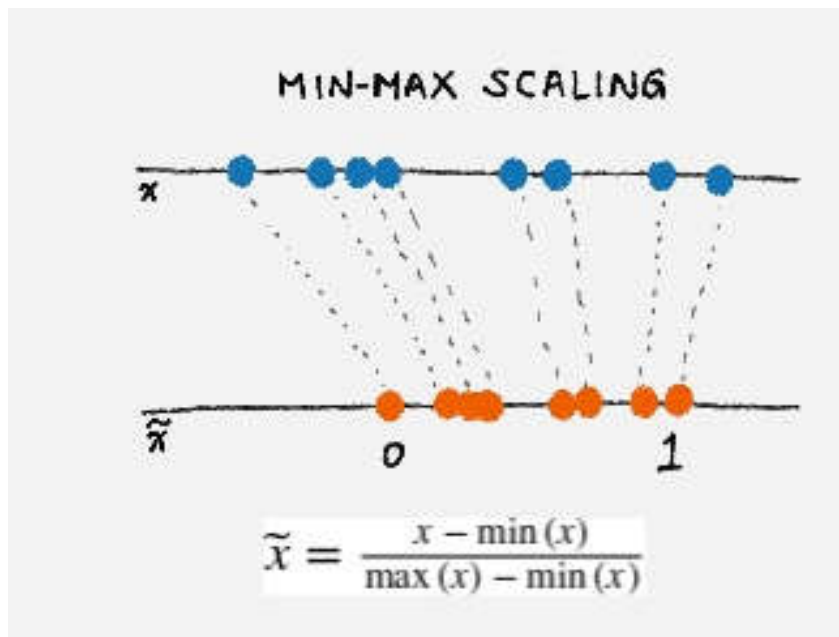
Fill missing values

	col1	col2	col3	col4	col5			col1	col2	col3	col4	col5	
0	2	5.0	3.0	6	NaN	df.fillna(0)		0	2	5.0	3.0	6	0.0
1	9	NaN	9.0	0	7.0			1	9	0.0	9.0	0	7.0
2	19	17.0	NaN	9	NaN			2	19	17.0	0.0	9	0.0

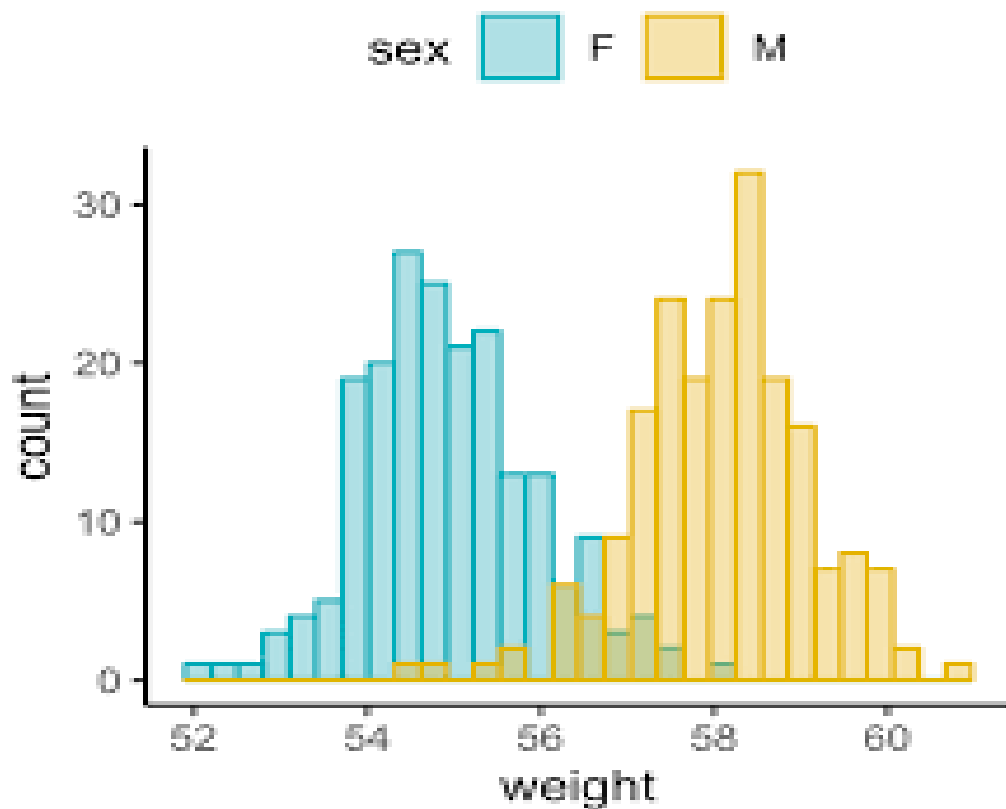
Data Normalization

After min-max scaling, all feature values are within the [0, 1] range

After standardization, a feature has mean 0 and variance 1



Data Visualization: Histogram



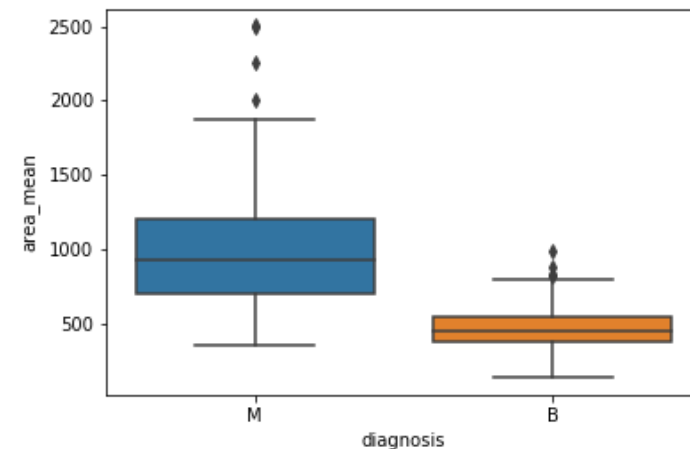
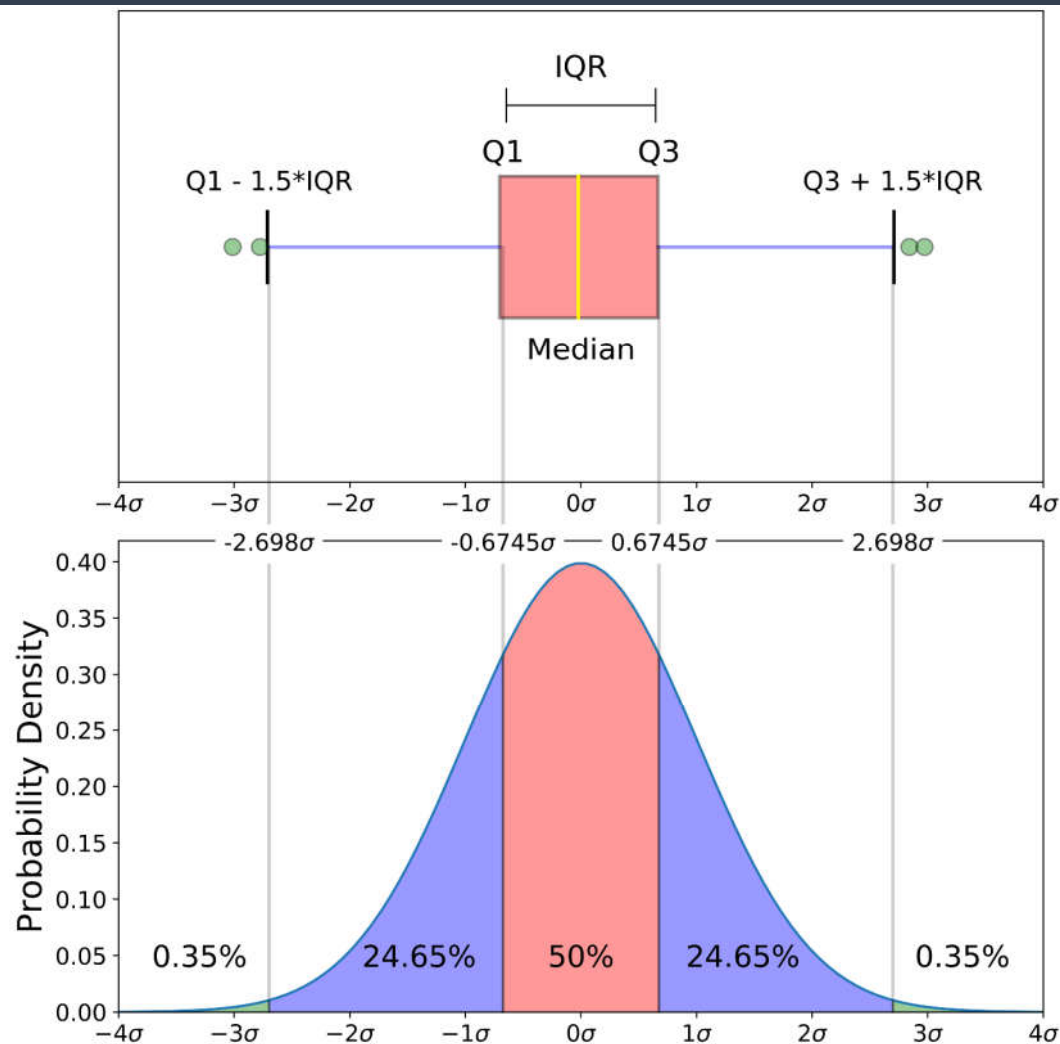
A **histogram** is an estimate of the probability distribution of a continuous variables.

To construct a histogram:

- bin the range of values;
- plot a rectangle over each bin with height proportional to the frequency

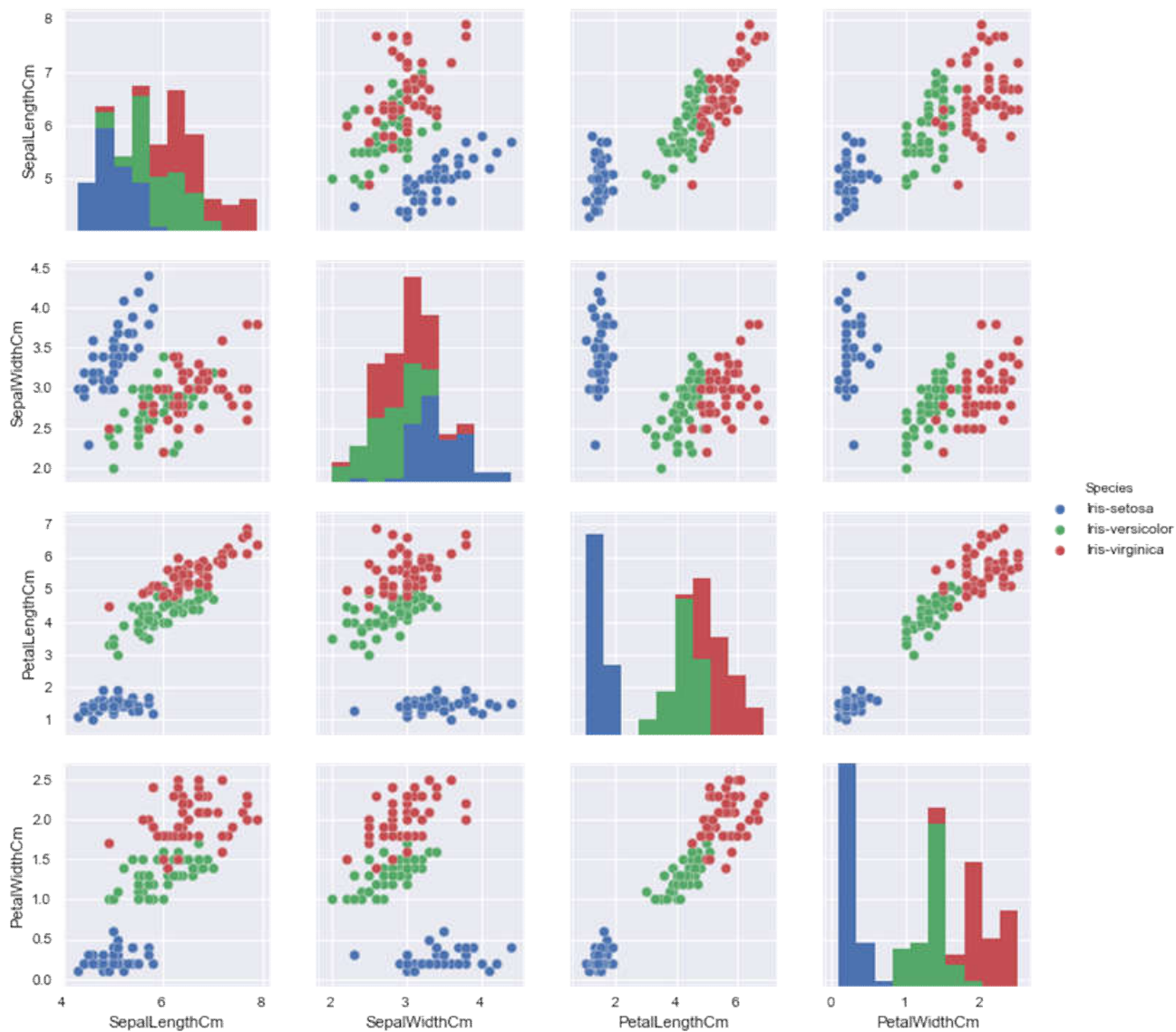
```
seaborn.distplot()
```

Data Visualization: Box Plot



A boxplot is a standardized way of displaying the distribution of data based on a five number summary (“minimum”, first quartile (Q1), median, third quartile (Q3), and “maximum”).

```
seaborn.boxplot()
```

Iris dataset

Plot pairwise
relationships in a
dataset

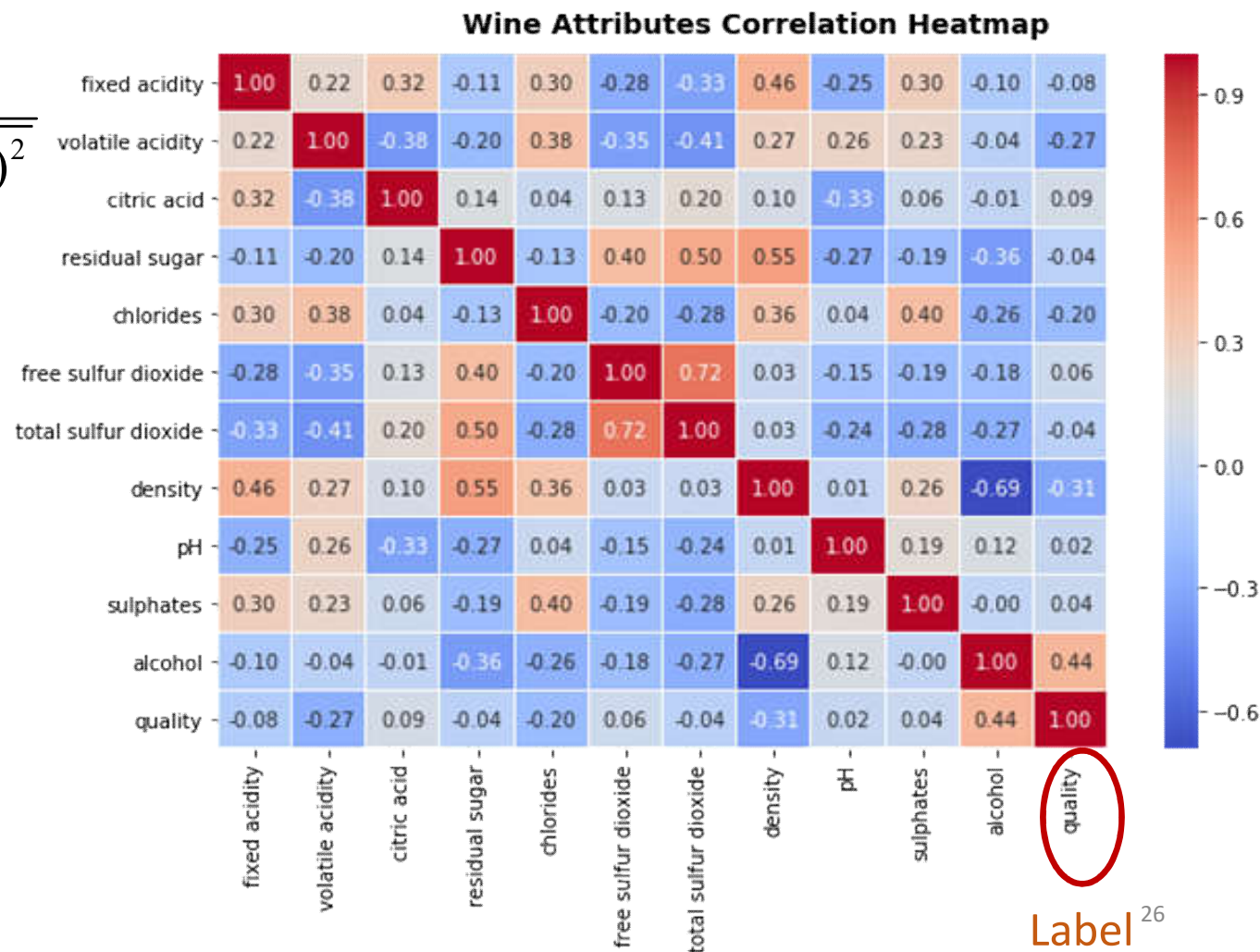
```
seaborn.pairplot()
```

Pearson correlation coefficient

$$r = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2} \sqrt{\sum (y - \bar{y})^2}}$$

Pearson's correlation coefficient is the covariance of the two variables divided by the product of their standard deviations

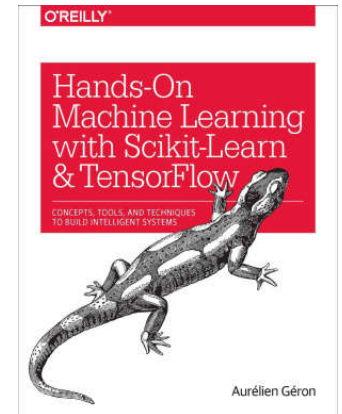
```
seaborn.heatmap()
```



References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow'



Data Analysis

Lecture 5: Unsupervised Learning

Dr. Nataliya K. Sakhnenko

Roadmap (from lecture 1)

Basic Machine Learning		Deep Learning
Supervised	Unsupervised	
☐ Regression (linear and polynomial) ☐ Classification ✓ Logistic regression, KNN, Decision Tree, SVM, NB ✓ Ensemble methods: Random Forest, AdaBoost, Gradient Boost	☐ Clustering (k-means, EM, Hierarchical clustering, DBSCAN) ☐ Dimensionality reduction (PCA, t-SNE)	☐ MLP ☐ CNN ☐ RNN ☐ Generative models: GANs, VAEs, NFs ☐ Attention and Transformers ☐ Intro to RL

Supervised Learning Summary



Unsupervised Learning

Unsupervised Learning

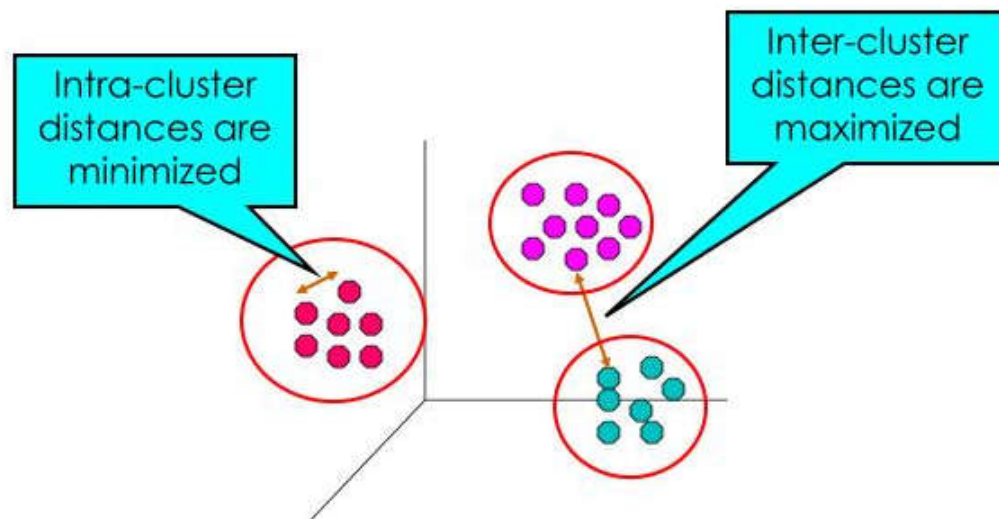
Supervised learning: discover patterns in the data that relate data attributes with a target (class) attribute. These patterns are then utilized to predict the values of the target attribute in future data instances.

Unsupervised learning: The data have no target attribute. We want to explore the data to find some intrinsic structures in them. The goal of unsupervised learning is to find hidden patterns in unlabeled data .

Supervise Learning		Unsupervised Learning	
Regression	Classification	Clustering	Dimensionality reduction

Clustering

Clustering is a technique for finding **similarity groups** in data, called **clusters**. It finds groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in the other groups



- Given a collection of text documents, we want to organize them according to their content similarities
- In marketing, segment customers according to their similarities to do targeted marketing.

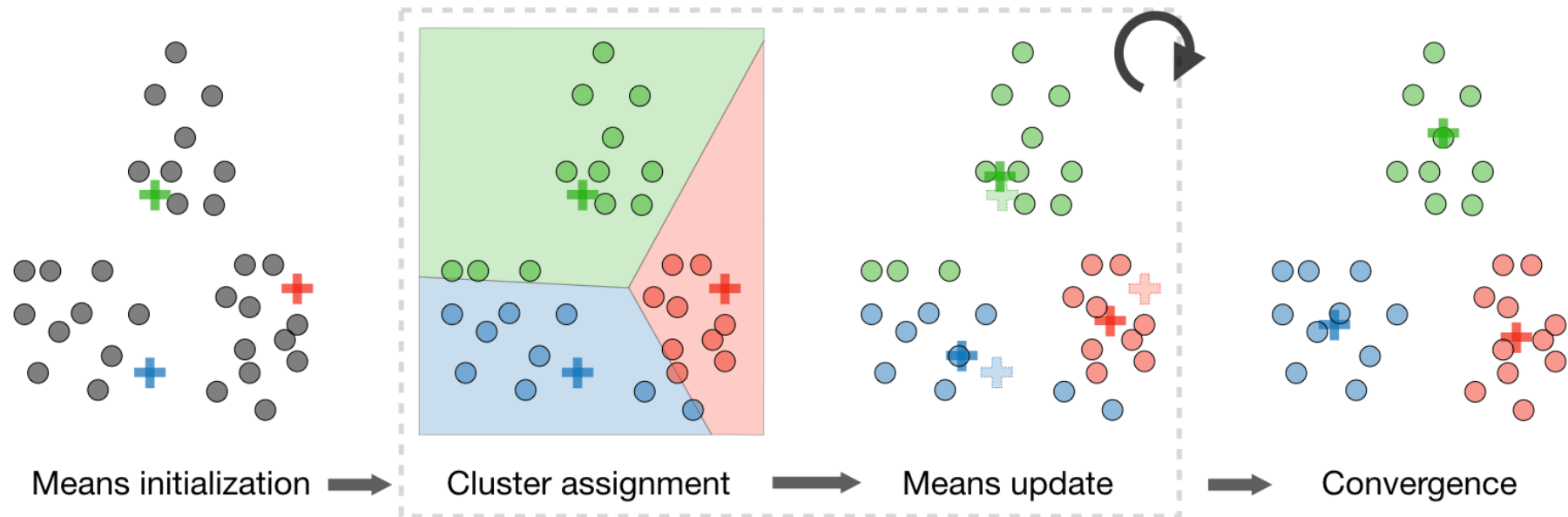
Clustering

- K-means
- EM algorithm
- Hierarchical clustering
- Density Based Clustering (DBSCAN)

K-means Clustering

K-means Clustering

K clusters



The K-means algorithm aims to choose centroids that minimize the inertia, or within-cluster sum-of-squares criterion:

$$J(c, \mu) = \frac{1}{M} \sum_{i=1}^M \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

centroids

K-means Algorithm

Data points $\{x_i\}_{i=1}^M$

The k -means algorithm partitions the given data into k clusters. Each cluster has a cluster **center**, called **centroid**, k is specified by the user

Given k , the *k-means* algorithm works as follows:

- 1) Randomly choose k data points (seeds) to be the initial centroids, cluster centers
- 2) Assign each data point to the closest centroid

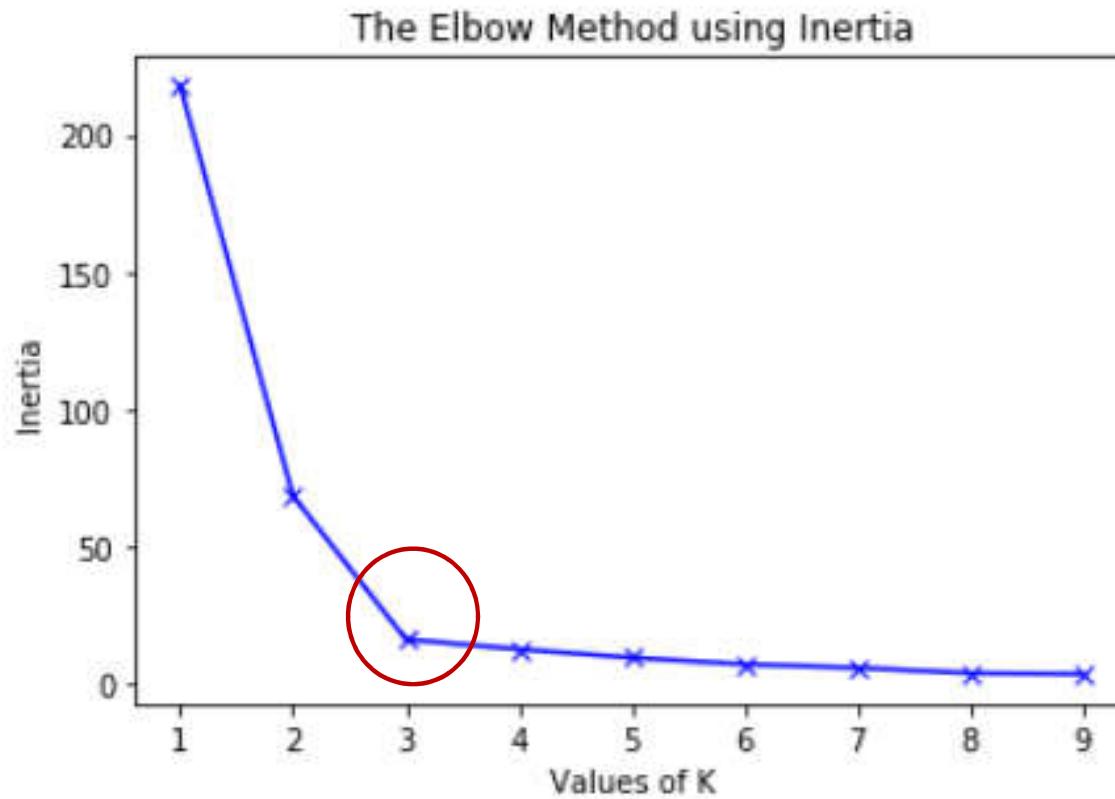
$$c^{(i)} = \arg \min_j \|x^{(i)} - \mu_j\|^2 \quad - \text{index of cluster centroid closest to } x^{(i)}$$

- 3) Re-compute the centroids using the current cluster memberships
- 4) If a convergence criterion is not met, go to 2).

Stopping/convergence criterion:

- no (or minimum) re-assignments of data points to different clusters
- no (or minimum) change of centroids

Selecting number of clusters with Elbow plot



$$J(c, \mu) = \frac{1}{M} \sum_{i=1}^M \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

Selecting number of clusters with silhouette analysis

Silhouette Coefficient combines ideas of both cohesion and separation

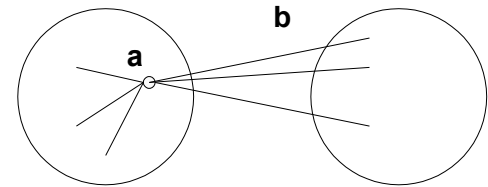
For an individual point i

a = average distance of i to the points in the same cluster

b = min (average distance of i to points in another cluster)

silhouette coefficient of i :

$$s = (b - a) / \max\{a, b\}, \text{ for each } i$$

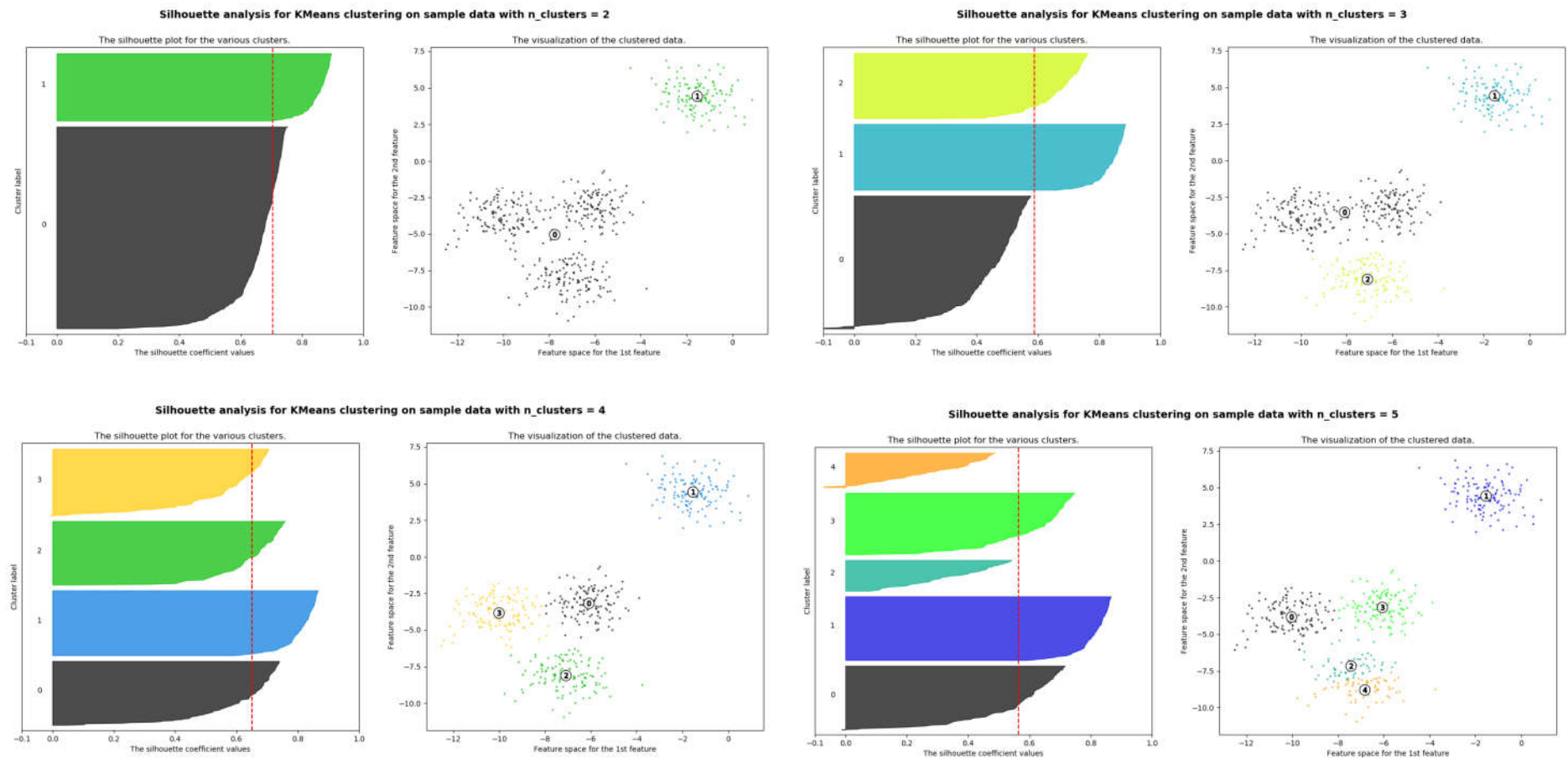


Typically between -1 and 1. The closer to 1 the better.

```
sklearn.metrics.silhouette_score  
sklearn.metrics.silhouette_samples
```

compute the mean Silhouette Coef of all samples
compute the Silhouette Coef for each sample

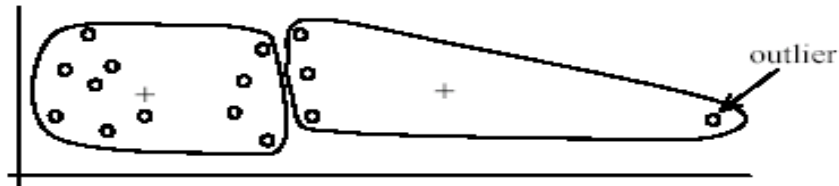
Selecting number of clusters with silhouette analysis



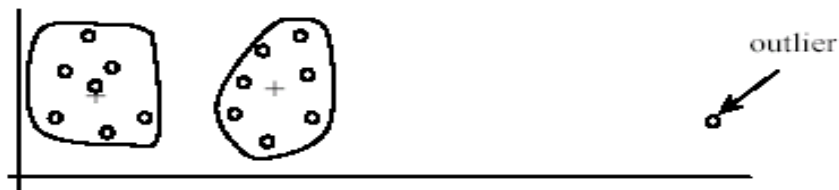
https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

K-means

The algorithm is sensitive to **outliers**

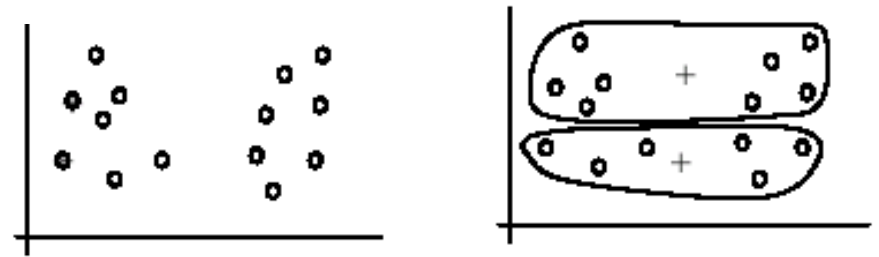


(A): Undesirable clusters



(B): Ideal clusters

The algorithm is sensitive to **initial seeds**



```
sklearn.cluster.KMeans(n_clusters=8, init='k-means++', n_init=10, max_iter=300)
```

n_init- Number of time the k-means algorithm will be run with different centroid seeds. The final results will be the best output in terms of inertia

init - 'k-means++' : selects initial cluster centers for k-mean clustering in a smart way

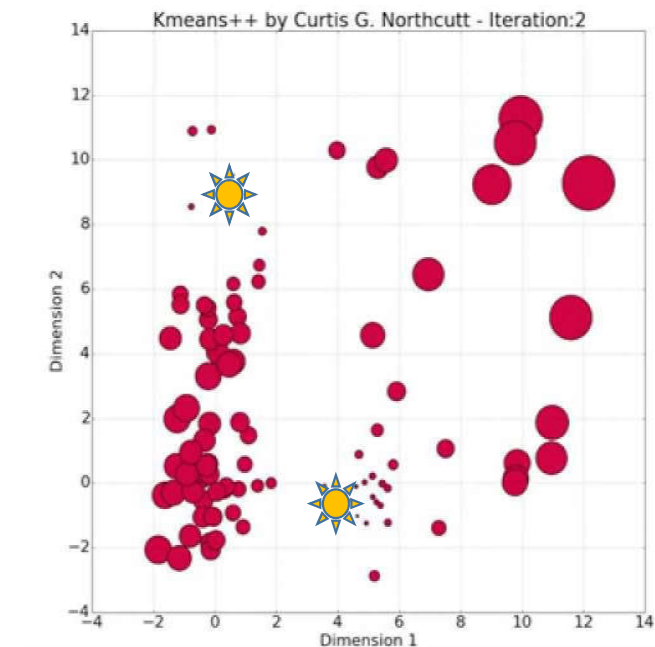
Kmeans++ initialization

The exact algorithm is as follows:

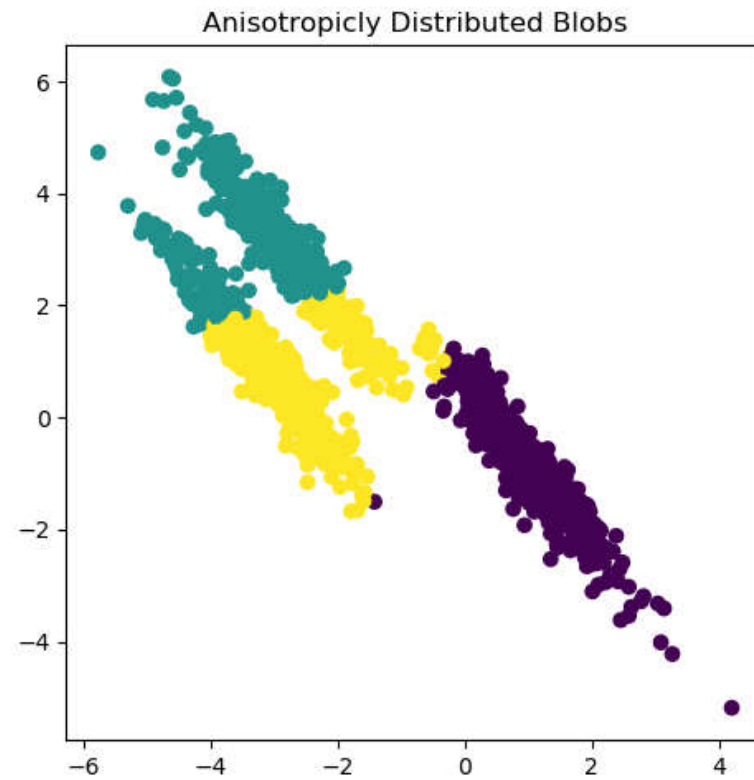
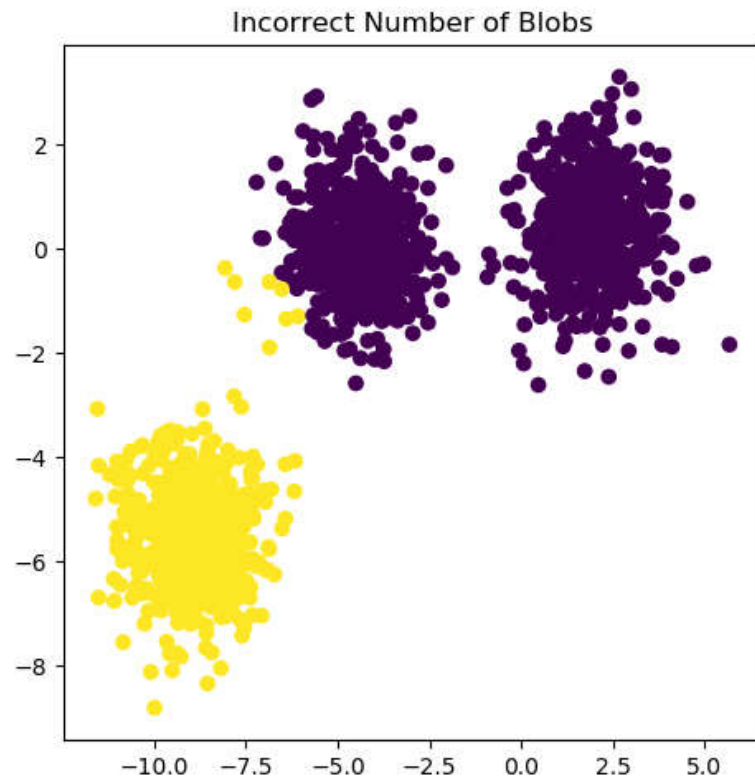
1. Choose one center uniformly at random from among the data points.
2. For each data point x , compute $D(x)$, the distance between x and the nearest center that has already been chosen.
3. Choose one new data point at random as a new center, using a weighted probability distribution where a point x is chosen with probability proportional to $D(x)^2$.
4. Repeat Steps 2 and 3 until k centers have been chosen.
5. Now that the initial centers have been chosen, proceed using standard [k-means clustering](#).

<https://en.wikipedia.org/wiki/K-means%2B%2B>

<https://www.youtube.com/watch?v=BIQDImZDuf8>



Demonstrations of K-means assumptions



https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_assumptions.html#sphx-glr-auto-examples-cluster-plot-kmeans-assumptions-py

K-means Example: Color Quantization

Original image (96,615 colors)



Quantized image (64 colors, K-Means)



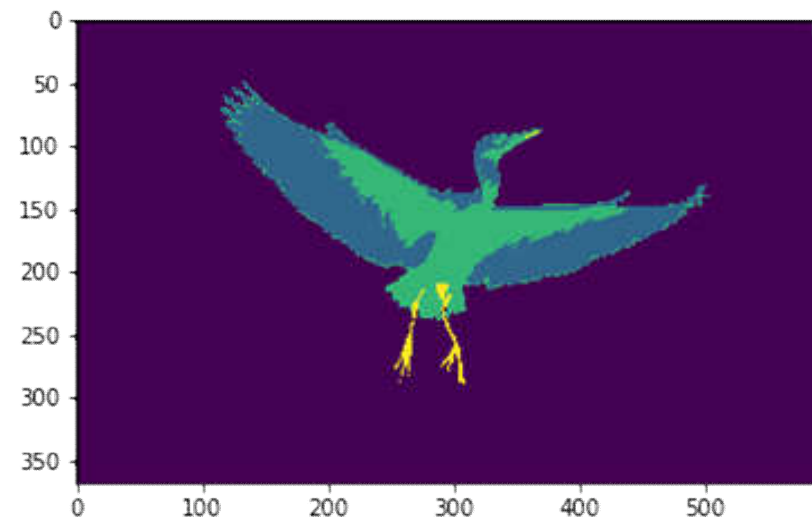
Performs a pixel-wise Vector Quantization (VQ) of an image of the summer palace (China), reducing the number of colors required to show the image from 96,615 unique colors to 64, while preserving the overall appearance quality.

https://scikit-learn.org/stable/auto_examples/cluster/plot_color_quantization.html

K-means Example: Image Segmentation



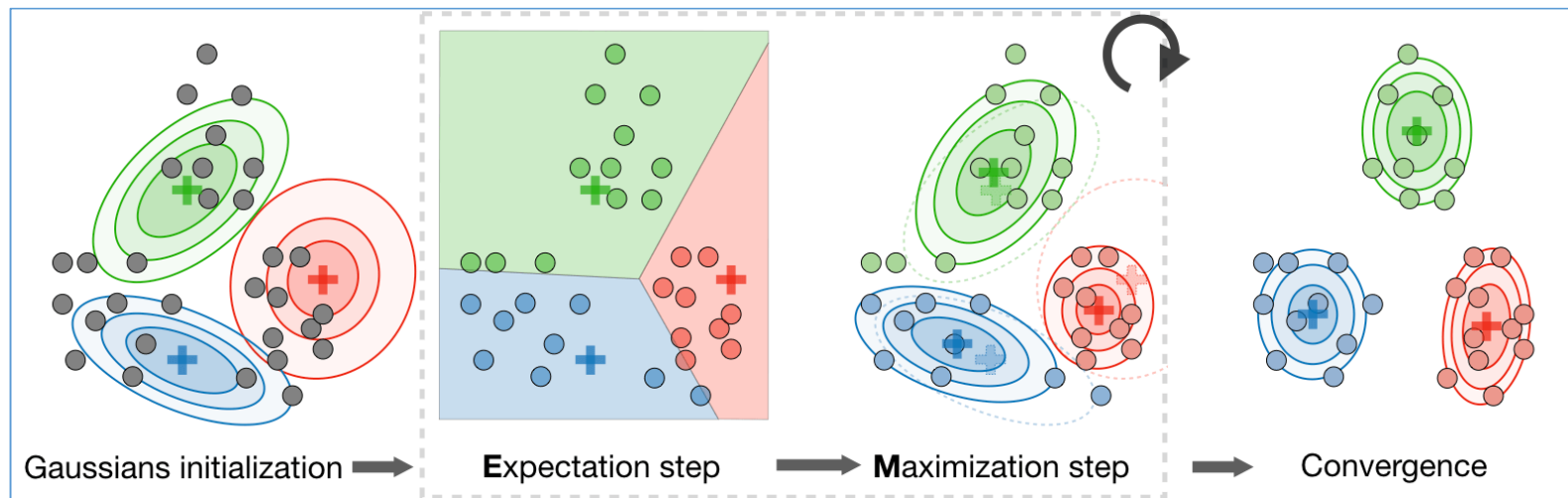
Original Image



`sklearn.cluster.KMeans(n_clusters = 4)`

Expectation Maximization (EM) algorithm

EM algorithm intuition



➤ Initialization:

Initialize K clusters: $C_1, \dots, C_K$ - (μ_j, Σ_j) and $P(C_j)$ for each cluster j .

➤ Iteration Steps:

Estimate the cluster of each data: $p(C_j | x_i)$

Expectation step: Re-estimate the cluster parameters

Maximization step: $(\mu_j, \Sigma_j), p(C_j)$ For each cluster j

Input: $\{x_i\}_{i=1}^M$

Output:

cluster parameters

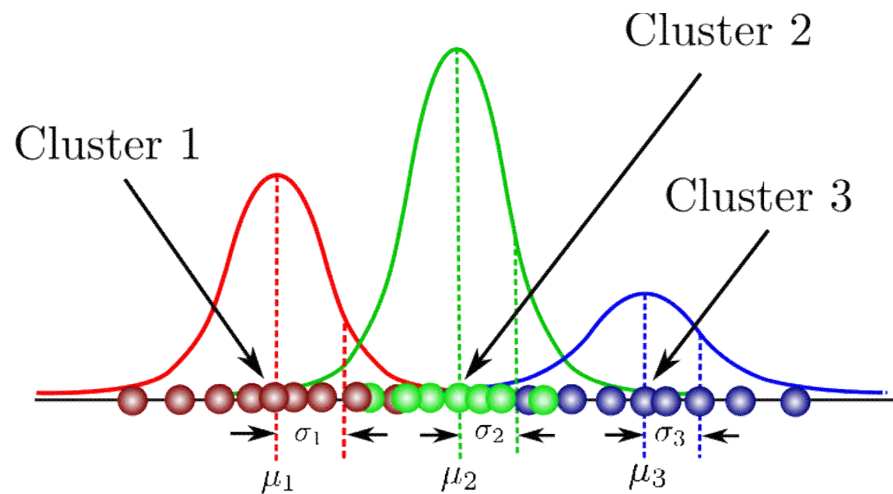
(μ_j, Σ_j) and $P(C_j)$

for each cluster j

and clustering result

$p(C_j | x_i)$

Mixture of normal distributions



$$f(\pi, \mu, \sigma) = \pi_1 N(\mu_1, \sigma_1) + \pi_2 N(\mu_2, \sigma_2) + \pi_3 N(\mu_3, \sigma_3)$$

Gaussian or normal distribution, 1D

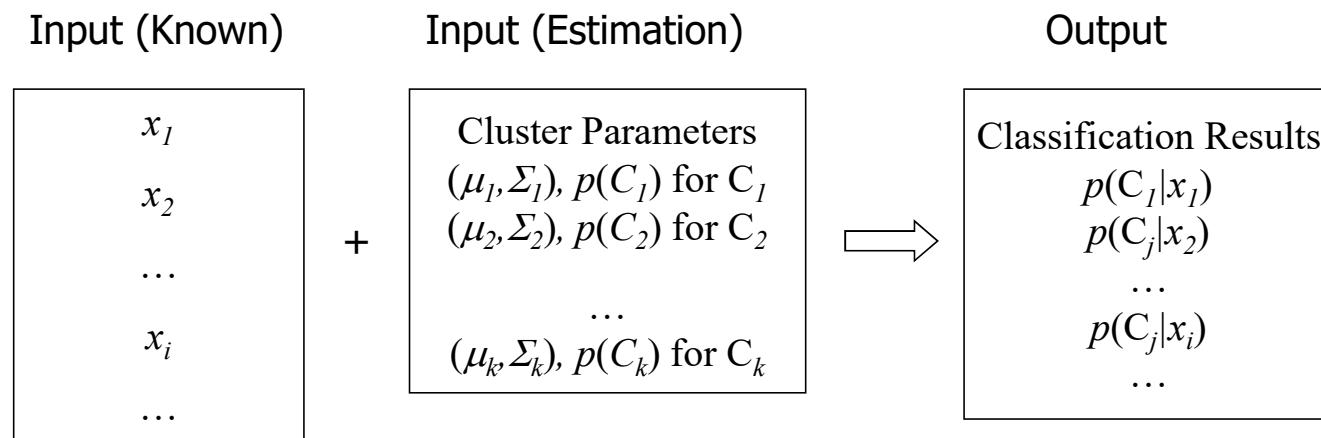
$$\mathcal{N}(x ; \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2}(x - \mu)^2 / \sigma^2 \right]$$

Parameters: mean μ , variance σ^2
(standard deviation σ)

Maximum Likelihood estimates

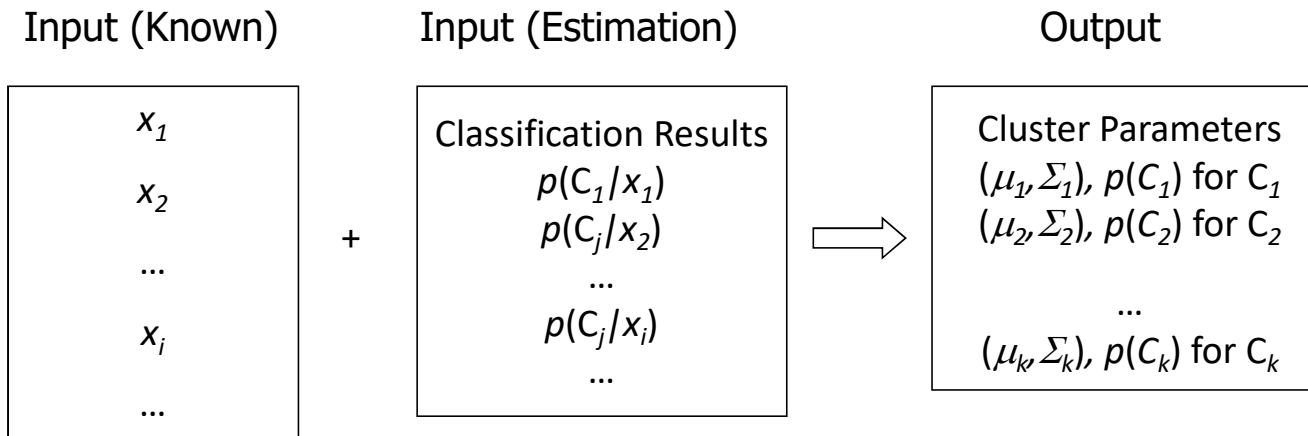
$$\hat{\mu} = \frac{1}{N} \sum_i x^{(i)}$$
$$\hat{\sigma}^2 = \frac{1}{N} \sum_i (x^{(i)} - \hat{\mu})^2$$

Expectation Step



$$p(C_j | x_i) = \frac{p(x_i | C_j) \cdot p(C_j)}{p(x_i)} = \frac{p(x_i | C_j) \cdot p(C_j)}{\sum_j p(x_i | C_j) \cdot p(C_j)}$$

Maximization Step



$$\mu_j = \frac{\sum_i p(C_j | x_i) \cdot x_i}{\sum_i p(C_j | x_i)}$$

$$\Sigma_j = \frac{\sum_i p(C_j | x_i) \cdot (x_i - \mu_j) \cdot (x_i - \mu_j)^T}{\sum_i p(C_j | x_i)}$$

$$p(C_j) = \frac{\sum_i p(C_j | x_i)}{N}$$

EM algorithm

- Initial step:

Initialize K clusters: $C_1, \dots, C_K$

(μ_j, Σ_j) and $P(C_j)$ for each cluster j .

- Iteration Steps:

Expectation Step

$$p(C_j | x_i) = \frac{p(x_i | C_j) \cdot p(C_j)}{p(x_i)} = \frac{p(x_i | C_j) \cdot p(C_j)}{\sum_j p(x_i | C_j) \cdot p(C_j)}$$

Maximization Step

$$\mu_j = \frac{\sum_i p(C_j | x_i) \cdot x_i}{\sum_i p(C_j | x_i)} \quad \Sigma_j = \frac{\sum_i p(C_j | x_i) \cdot (x_i - \mu_j) \cdot (x_i - \mu_j)^T}{\sum_i p(C_j | x_i)} \quad p(C_j) = \frac{\sum_i p(C_j | x_i)}{N}$$

EM algorithm in Sklearn

```
sklearn.mixture.GaussianMixture(n_components=1, covariance_type='full',  
init_params='kmeans')
```

covariance_type : {'full' (default), 'tied', 'diag', 'spherical'}

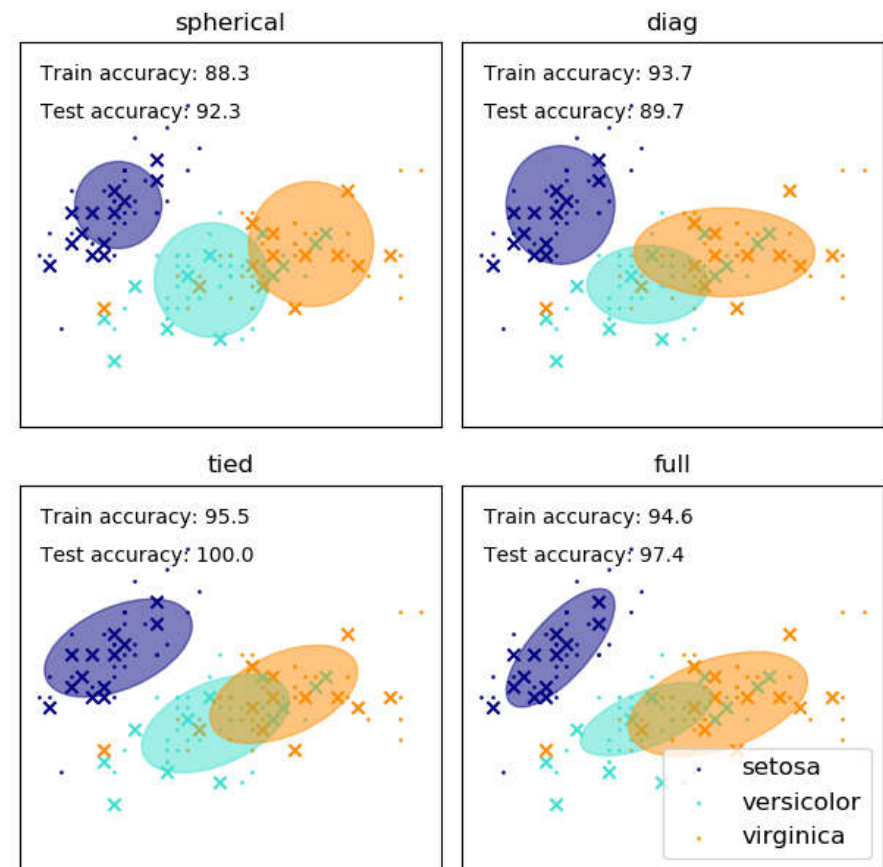
String describing the type of covariance parameters to use. Must be one of:

'full' - each component has its own general covariance matrix

'tied' - all components share the same general covariance matrix

'diag' - each component has its own diagonal covariance matrix

'spherical' - each component has its own single variance



EM algorithm in Sklearn

Attributes:

weights_ : array-like, shape (n_components,)

The weights of each mixture components.

means_ : array-like, shape (n_components, n_features)

The mean of each mixture component.

covariances_ : array-like

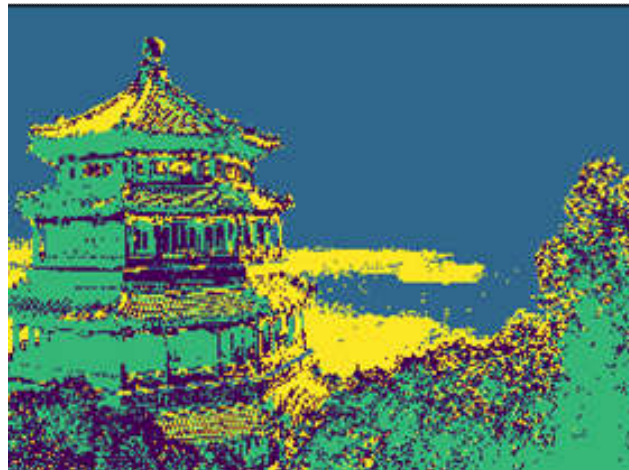
The covariance of each mixture component.

EM algorithm example

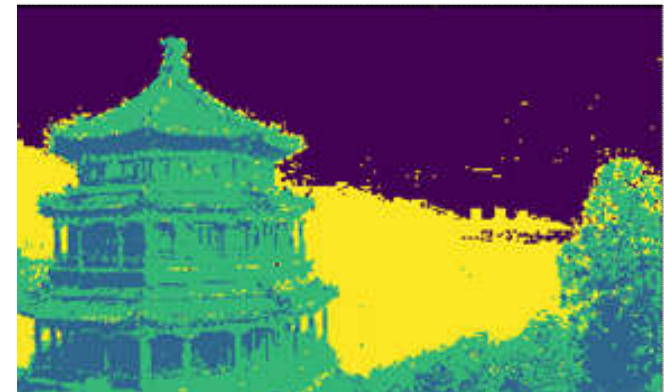


Original image

```
sklearn.cluster.  
KMeans(n_clusters = 4)
```



```
sklearn.mixture.  
GaussianMixture  
(n_components=4)
```



Hierarchical clustering

Hierarchical clustering

Hierarchical algorithms

- Bottom-up, **agglomerative**
 - Top-down, divisive
-
- Cluster based on similarities/distances
 - Distance measure between instances $\mathbf{x}^r$ and $\mathbf{x}^s$

Minkowski (L_p) (Euclidean for $p = 2$)

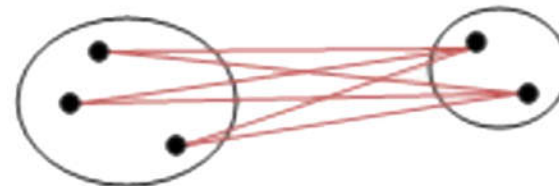
$$d_m(\mathbf{x}^r, \mathbf{x}^s) = \left[\sum_{j=1}^d (x_j^r - x_j^s)^p \right]^{1/p}$$

City-block distance $d_{cb}(\mathbf{x}^r, \mathbf{x}^s) = \sum_{j=1}^d |x_j^r - x_j^s|$

Agglomerative clustering

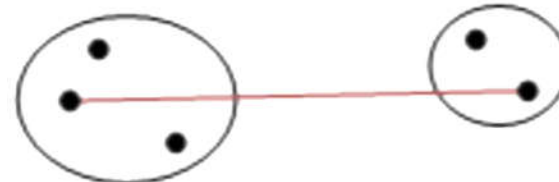
- Start with N groups each with one instance and merge two closest groups at each iteration
- Distance between two groups G_i and G_j :

- Average-link



- Complete link:

$$d(G_i, G_j) = \max_{\mathbf{x}^r \in G_i, \mathbf{x}^s \in G_j} d(\mathbf{x}^r, \mathbf{x}^s)$$



- Single link:

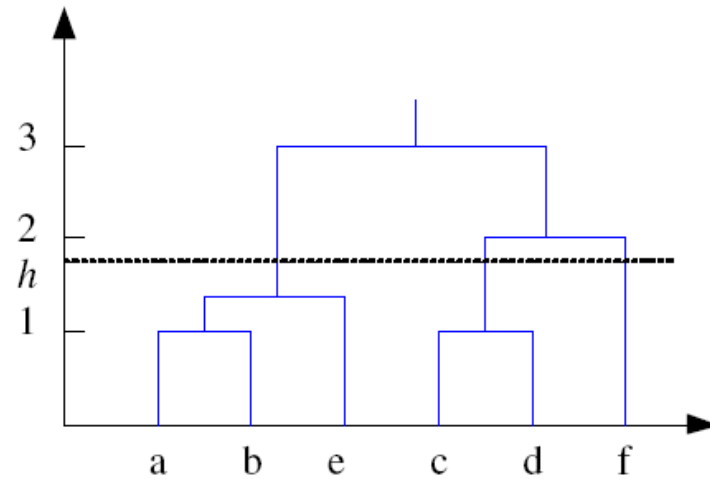
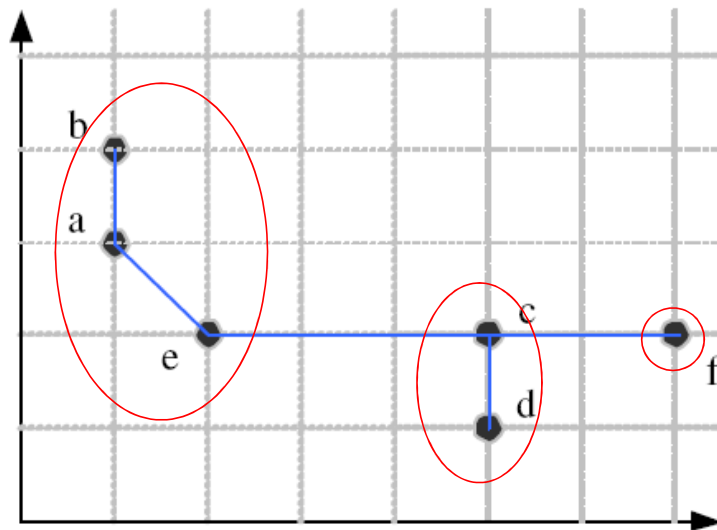
$$d(G_i, G_j) = \min_{\mathbf{x}^r \in G_i, \mathbf{x}^s \in G_j} d(\mathbf{x}^r, \mathbf{x}^s)$$



- Ward link: minimizes the variance of the clusters being merged

```
sklearn.cluster.AgglomerativeClustering(linkage='ward')
```

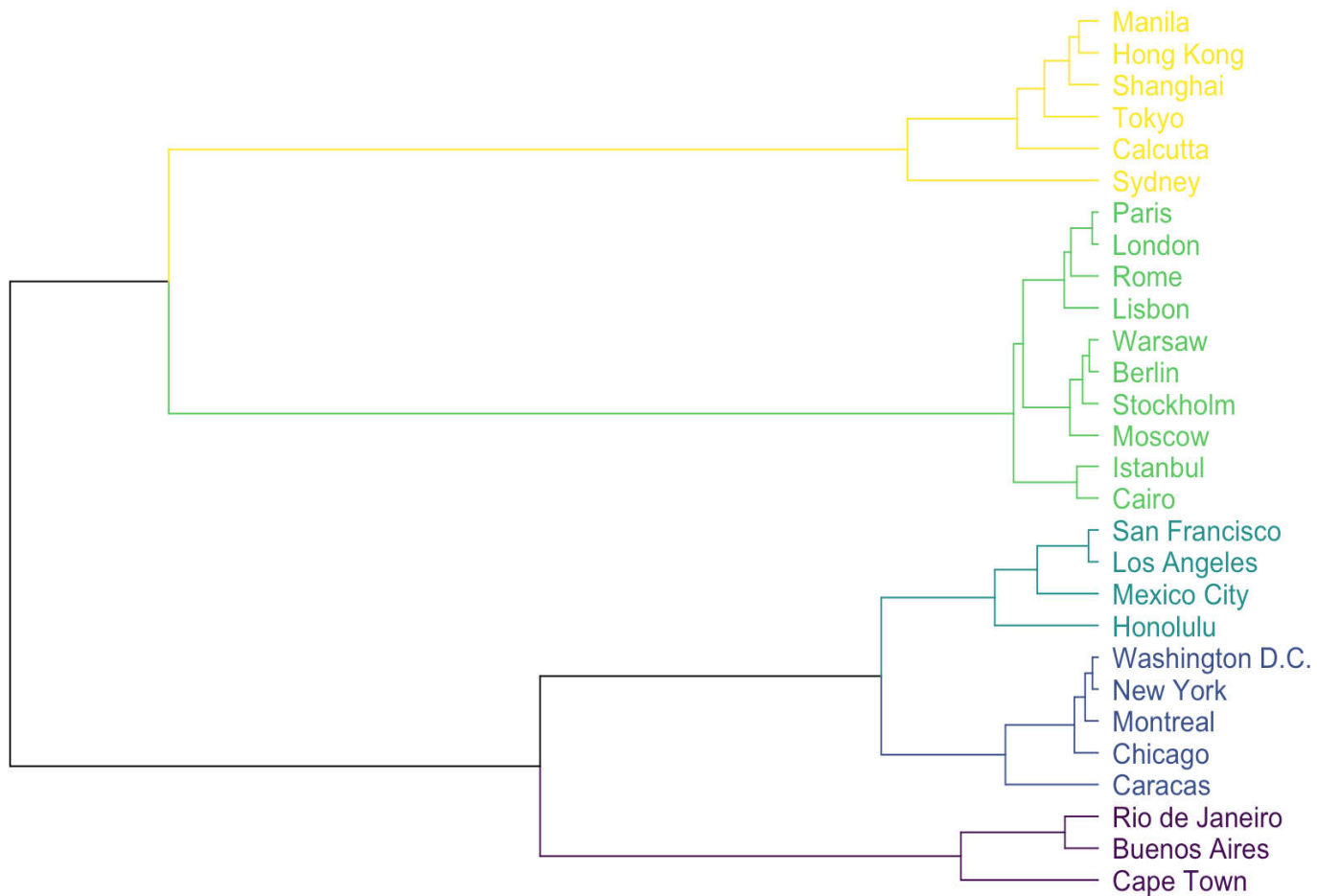
Example: Single link clustering



Dendrogram

```
scipy.cluster.hierarchy.dendrogram()
```

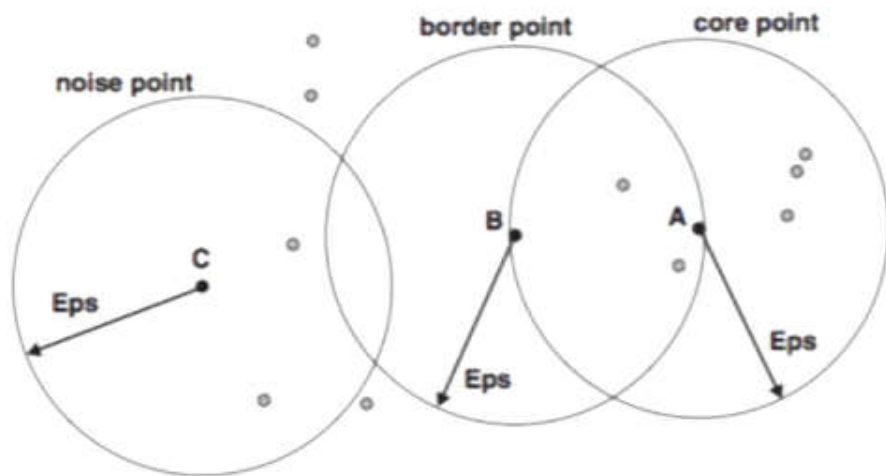
Example: Dendrogram



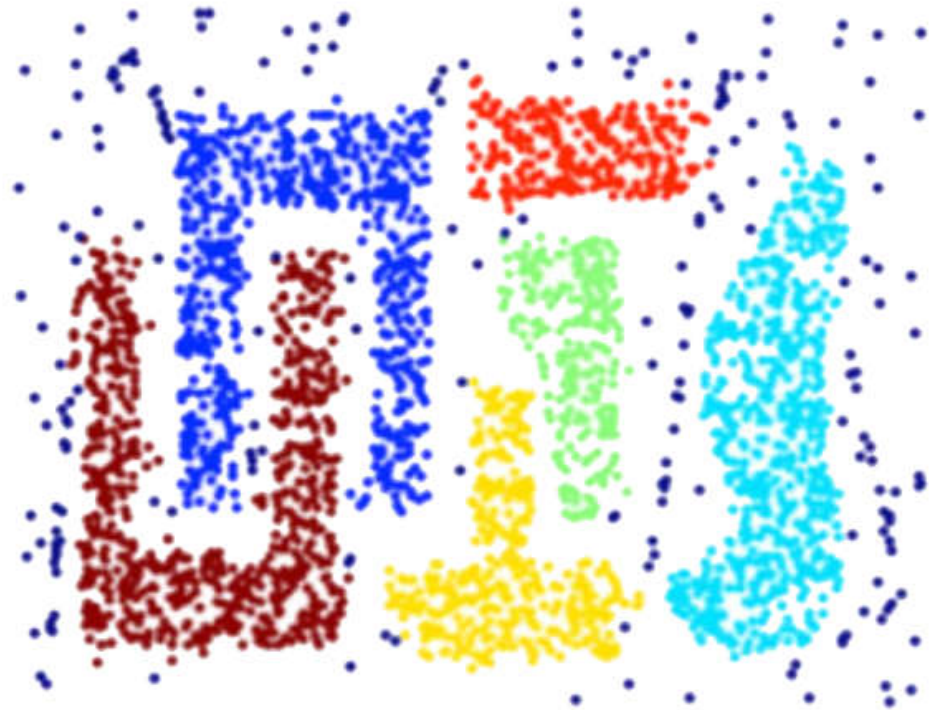
Clustering of
28 cities
based on
distance
between
them

Density Based Clustering (DBSCAN)

DBSCAN

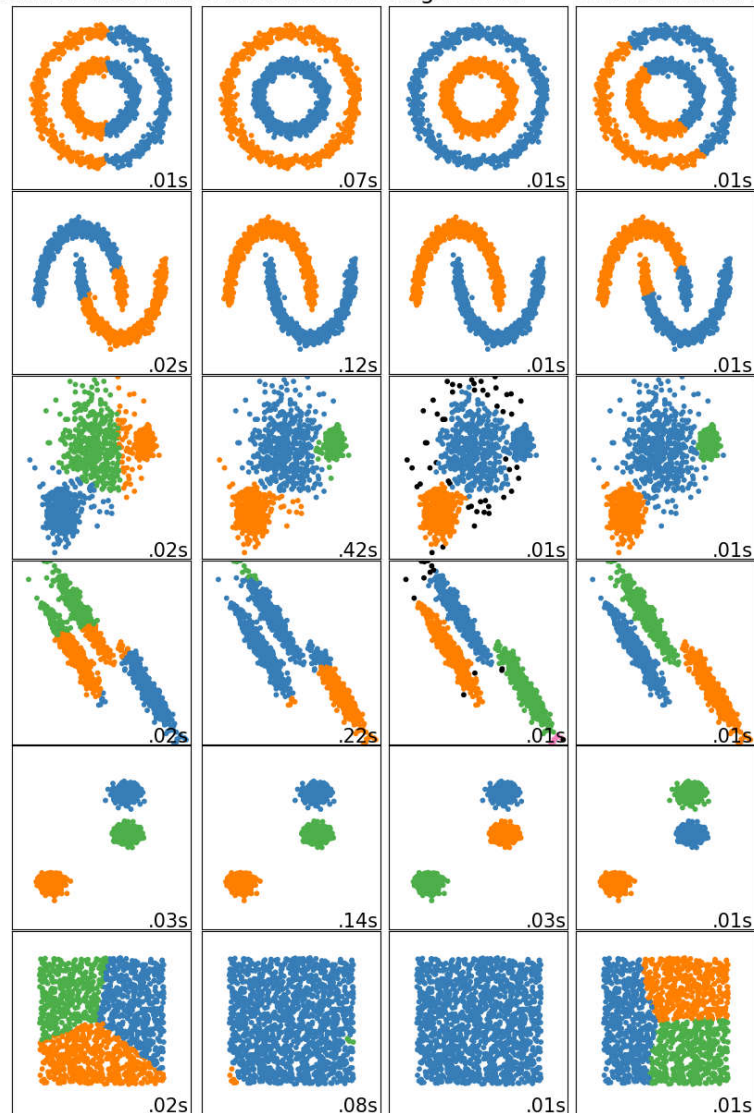


Given a set of points, it groups together points that are closely packed together (points with many [nearby neighbors](#)), marking as outliers points that lie alone in low-density regions (whose nearest neighbors are too far away).



```
class sklearn.cluster.DBSCAN(eps=0.5, min_  
samples=5, metric='euclidean')
```

MiniBatchKMeans IterativeClustering DBSCAN GaussianMixture



Clustering algorithms performance

Example shows characteristics of different clustering algorithms on 2D datasets

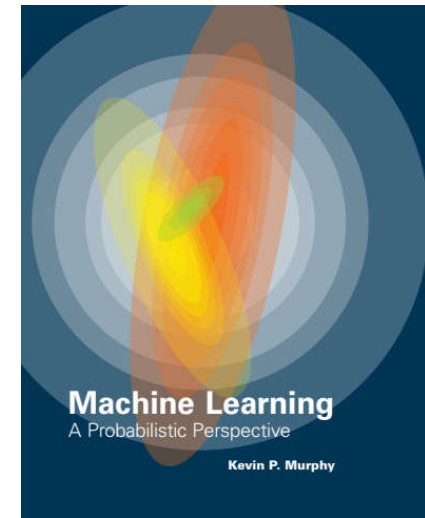
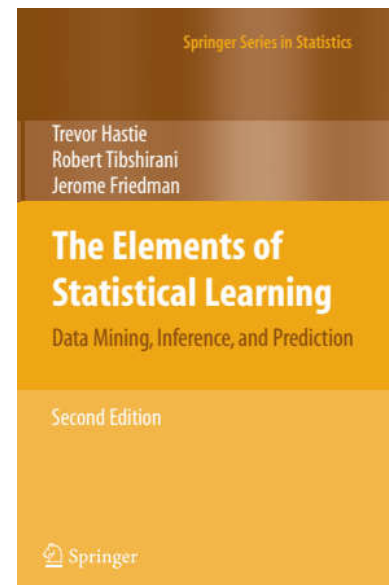
The last dataset is an example of a 'null' situation for clustering: the data is homogeneous, and there is no good clustering.

https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html

References

Books:

- K.Murphy, "Machine Learning, a probabilistic perspective"
- T.Hastie, "The Elements of Statistical Learning"



Appendix

Multivariate normal distribution

The multivariate normal distribution in n -dimensions, also called the multivariate Gaussian distribution, is parameterized by a **mean vector** $\mu \in \mathbb{R}^n$ and a **covariance matrix** $\Sigma \in \mathbb{R}^{n \times n}$, where $\Sigma \geq 0$ is symmetric and positive semi-definite. Also written “ $\mathcal{N}(\mu, \Sigma)$ ”, its density is given by:

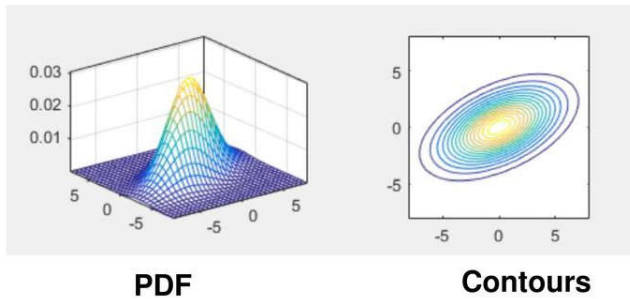
$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right).$$

In the equation above, “ $|\Sigma|$ ” denotes the determinant of the matrix Σ .

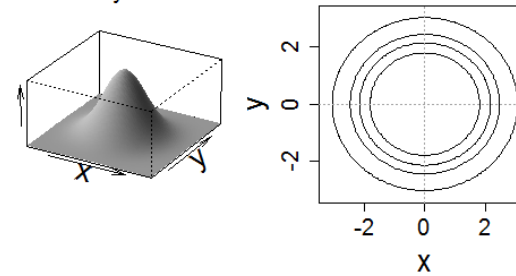
Bivariate normal distribution

$$\mu = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma = \begin{pmatrix} \sigma_x & \rho \\ \rho & \sigma_y \end{pmatrix}$$

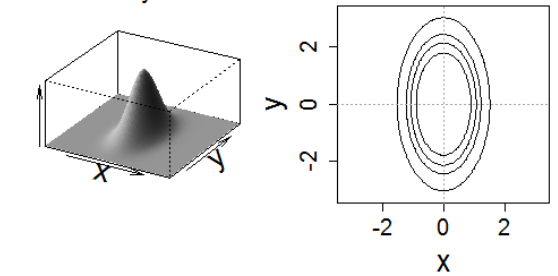
$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 9 & 3 \\ 3 & 4 \end{bmatrix}$$



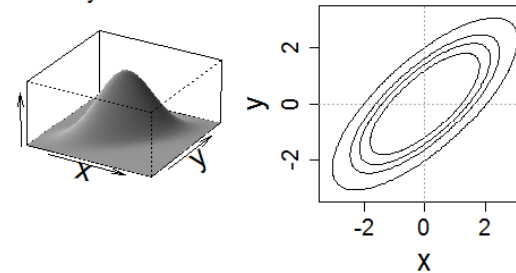
$$\sigma_x = \sigma_y, \rho = 0$$



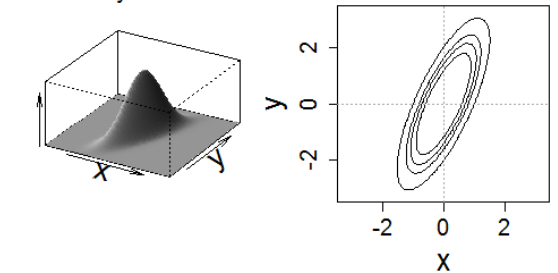
$$2\sigma_x = \sigma_y, \rho = 0$$



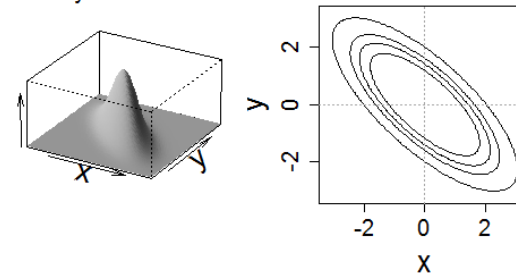
$$\sigma_x = \sigma_y, \rho = 0.75$$



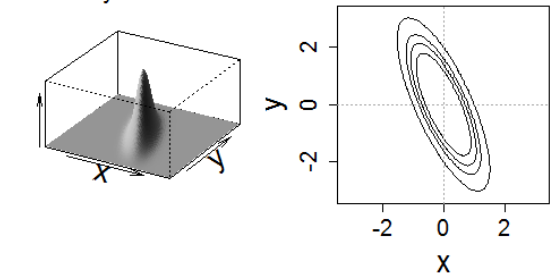
$$2\sigma_x = \sigma_y, \rho = 0.75$$



$$\sigma_x = \sigma_y, \rho = -0.75$$



$$2\sigma_x = \sigma_y, \rho = -0.75$$



Normal distribution: estimation of parameters

Having a sample $(x_1, \dots, x_m)$ from a normal distribution we would like to learn the approximate values of parameters:

- 1 dim
$$\hat{\mu} = \frac{1}{m} \sum_{i=1}^m x_i, \quad \hat{\sigma}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \hat{\mu})^2$$

- N-dim:
$$\hat{\vec{\mu}} = \frac{1}{m} \sum_{i=1}^m \vec{x}_i, \quad \hat{\Sigma} = \frac{1}{m} \sum_{i=1}^m (\vec{x}_i - \hat{\vec{\mu}})(\vec{x}_i - \hat{\vec{\mu}})^T$$

Data Analysis

Lecture 6: Unsupervised Learning

Dr. Nataliya K. Sakhnenko

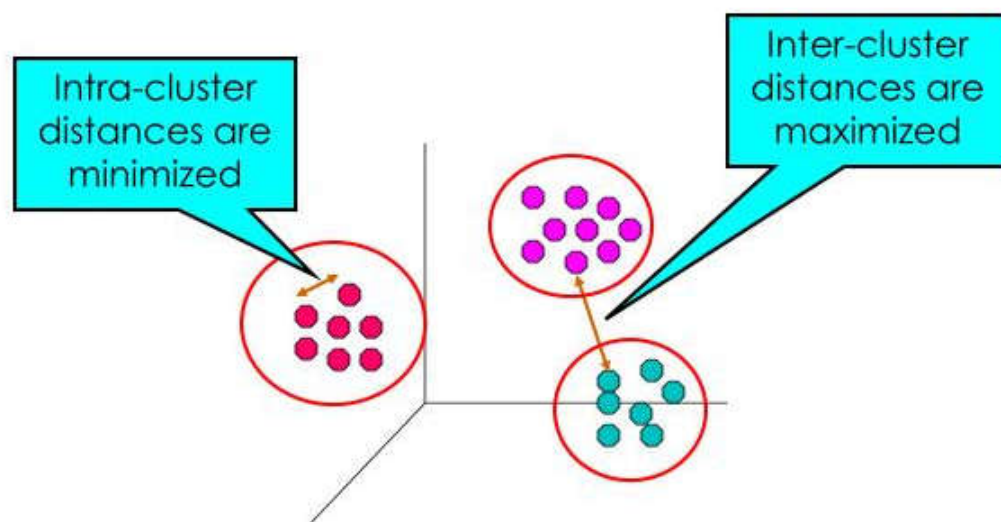
Roadmap

Basic Machine Learning		Deep Learning
Supervised	Unsupervised	
☐ Regression (linear and polynomial) ☐ Classification ✓ Logistic regression, KNN, Decision Tree, SVM, NB ✓ Ensemble methods: Random Forest, AdaBoost, Gradient Boost	☐ Clustering (k-means, EM, Hierarchical clustering, DBSCAN) ☐ Dimensionality reduction (PCA, LDA, t-SNE) 	☐ MLP ☐ CNN ☐ RNN ☐ Generative models: GANs, VAEs, NFs ☐ Attention and Transformers ☐ Intro to RL

Clustering

Recap: Clustering

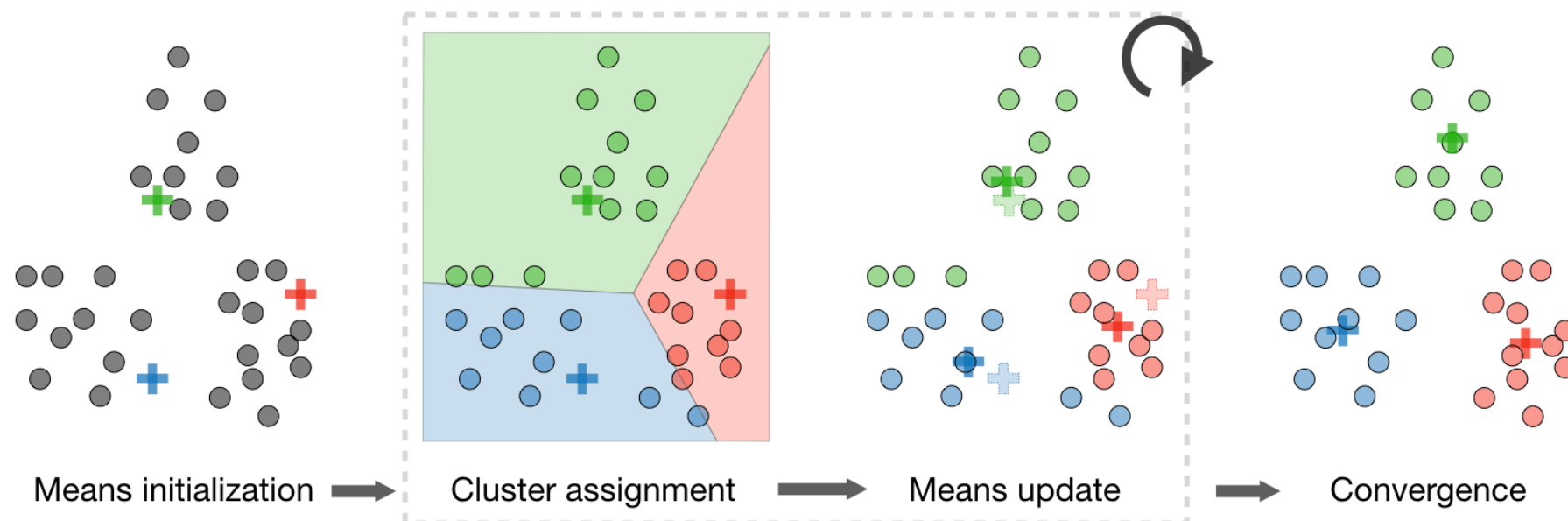
Clustering is a technique for finding **similarity groups** in data, called **clusters**. It finds groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in the other groups



- Given a collection of text documents, we want to organize them according to their content similarities
- In marketing, segment customers according to their similarities to do targeted marketing.

Recap: K-means Clustering

K clusters



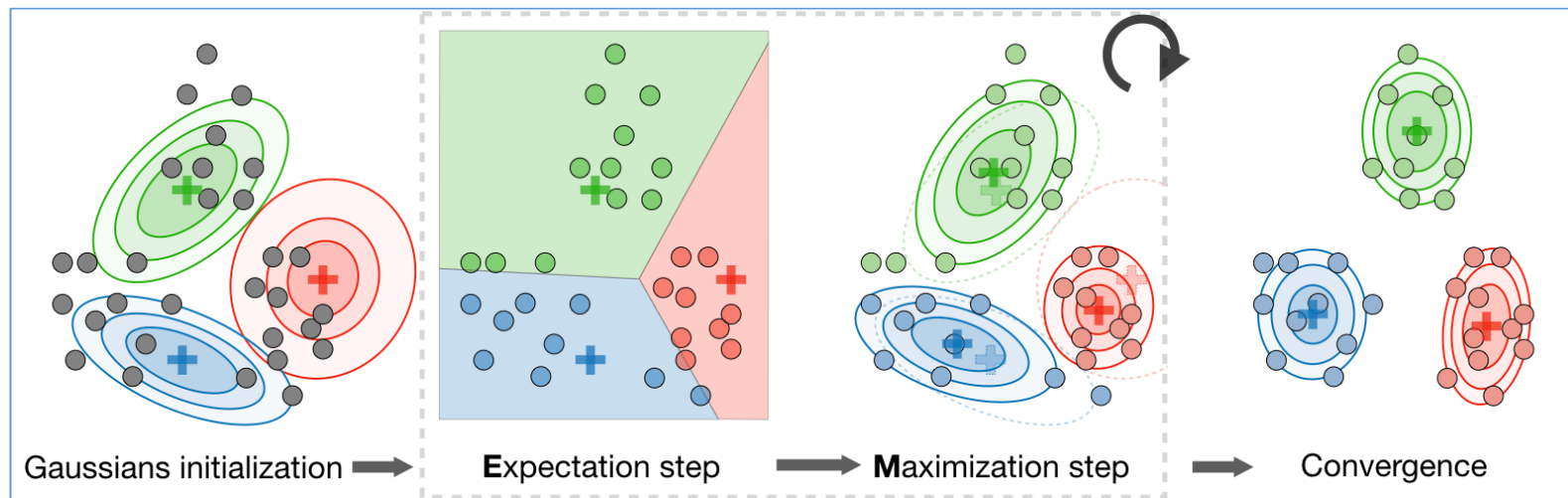
The K-means algorithm aims to choose centroids that minimize the inertia, or within-cluster sum-of-squares criterion:

$$J(c, \mu) = \frac{1}{M} \sum_{i=1}^M \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

centroids

```
sklearn.cluster.KMeans(n_clusters=8, init='k-means++', n_init=10, max_iter=300)
```


Recap: EM algorithm



➤ Initialization:

Initialize K clusters: $C_1, \dots, C_K$ - (μ_j, Σ_j) and $P(C_j)$ for each cluster j .

➤ Iteration Steps:

Expectation step: Estimate the cluster of each data $p(C_j | x_i)$

Maximization step: Re-estimate the cluster parameters $(\mu_j, \Sigma_j), p(C_j)$ For each cluster j

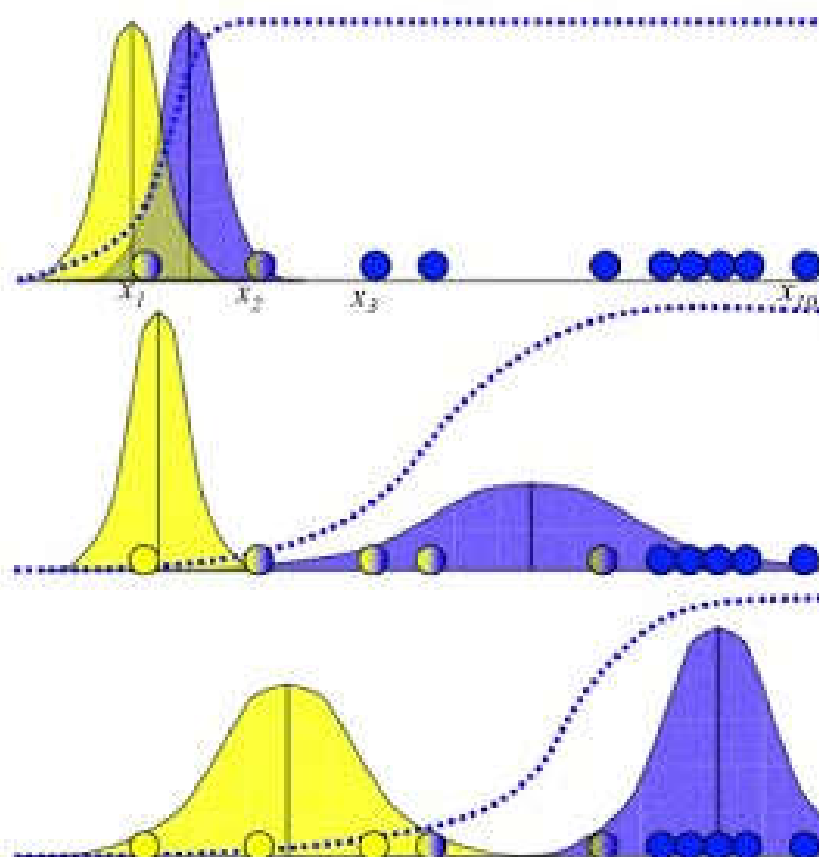
Input: $\{x_i\}_{i=1}^M$

Output:
cluster parameters (μ_j, Σ_j) and $P(C_j)$
for each cluster j
and clustering result $p(C_j | x_i)$

```
sklearn.mixture.GaussianMixture(n_components=1, covariance_type='full', init_params='kmeans')
```

Recap: EM algorithm

EM: 1-d example



$$P(x_i | b) = \frac{1}{\sqrt{2\pi\sigma_b^2}} \exp\left\{-\frac{(x_i - \mu_b)^2}{2\sigma_b^2}\right\}$$

$$b_i = P(b | x_i) = \frac{P(x_i | b)P(b)}{P(x_i | b)P(b) + P(x_i | a)P(a)}$$

$$a_i = P(a | x_i) = 1 - b_i$$

$$\mu_b = \frac{b_1 x_1 + b_2 x_2 + \dots + b_n x_n}{b_1 + b_2 + \dots + b_n}$$

$$\sigma_b^2 = \frac{b_1 (x_1 - \mu_b)^2 + \dots + b_n (x_n - \mu_b)^2}{b_1 + b_2 + \dots + b_n}$$

$$\mu_a = \frac{a_1 x_1 + a_2 x_2 + \dots + a_n x_n}{a_1 + a_2 + \dots + a_n}$$

$$\sigma_a^2 = \frac{a_1 (x_1 - \mu_a)^2 + \dots + a_n (x_n - \mu_a)^2}{a_1 + a_2 + \dots + a_n}$$

could also estimate priors:

$$P(b) = (b_1 + b_2 + \dots + b_n) / n$$

$$P(a) = 1 - P(b)$$

Copyright © 2014 Victor Laveen

Hierarchical clustering

Hierarchical clustering

Hierarchical algorithms

- Bottom-up, **agglomerative**
 - Top-down, divisive
-
- Cluster based on similarities/distances
 - Distance measure between instances $\mathbf{x}^r$ and $\mathbf{x}^s$

Minkowski (L_p) (Euclidean for $p = 2$)

$$d_m(\mathbf{x}^r, \mathbf{x}^s) = \left[\sum_{j=1}^d (x_j^r - x_j^s)^p \right]^{1/p}$$

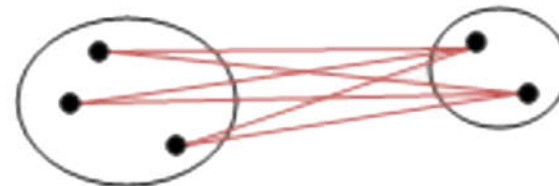
City-block distance

$$d_{cb}(\mathbf{x}^r, \mathbf{x}^s) = \sum_{j=1}^d |x_j^r - x_j^s|$$

Agglomerative clustering

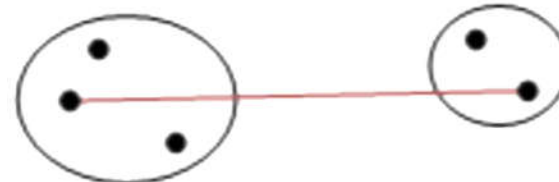
- Start with N groups each with one instance and merge two closest groups at each iteration
- Distance between two groups G_i and G_j :

- Average-link



- Complete link:

$$d(G_i, G_j) = \max_{\mathbf{x}^r \in G_i, \mathbf{x}^s \in G_j} d(\mathbf{x}^r, \mathbf{x}^s)$$



- Single link:

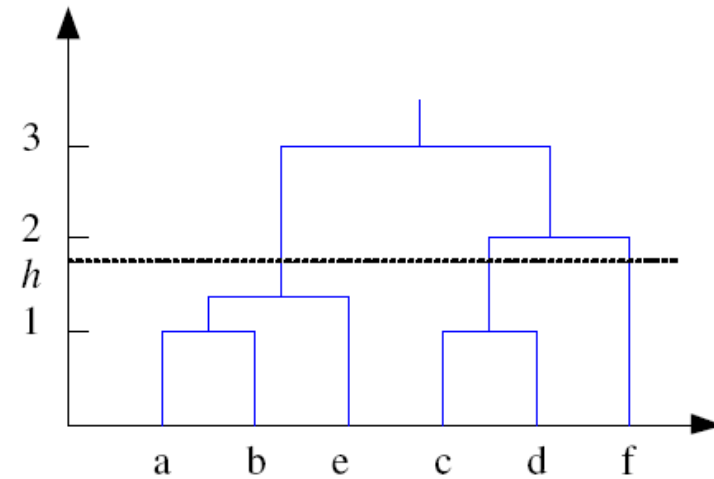
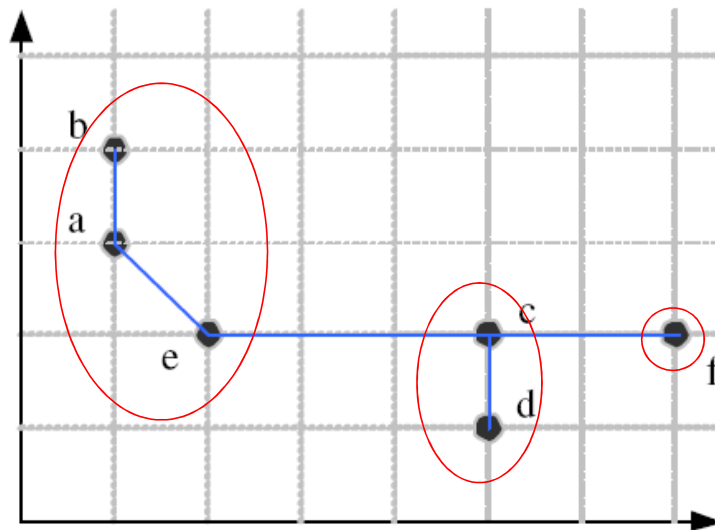
$$d(G_i, G_j) = \min_{\mathbf{x}^r \in G_i, \mathbf{x}^s \in G_j} d(\mathbf{x}^r, \mathbf{x}^s)$$



- Ward link: minimizes the variance of the clusters being merged

```
sklearn.cluster.AgglomerativeClustering(linkage='ward')
```

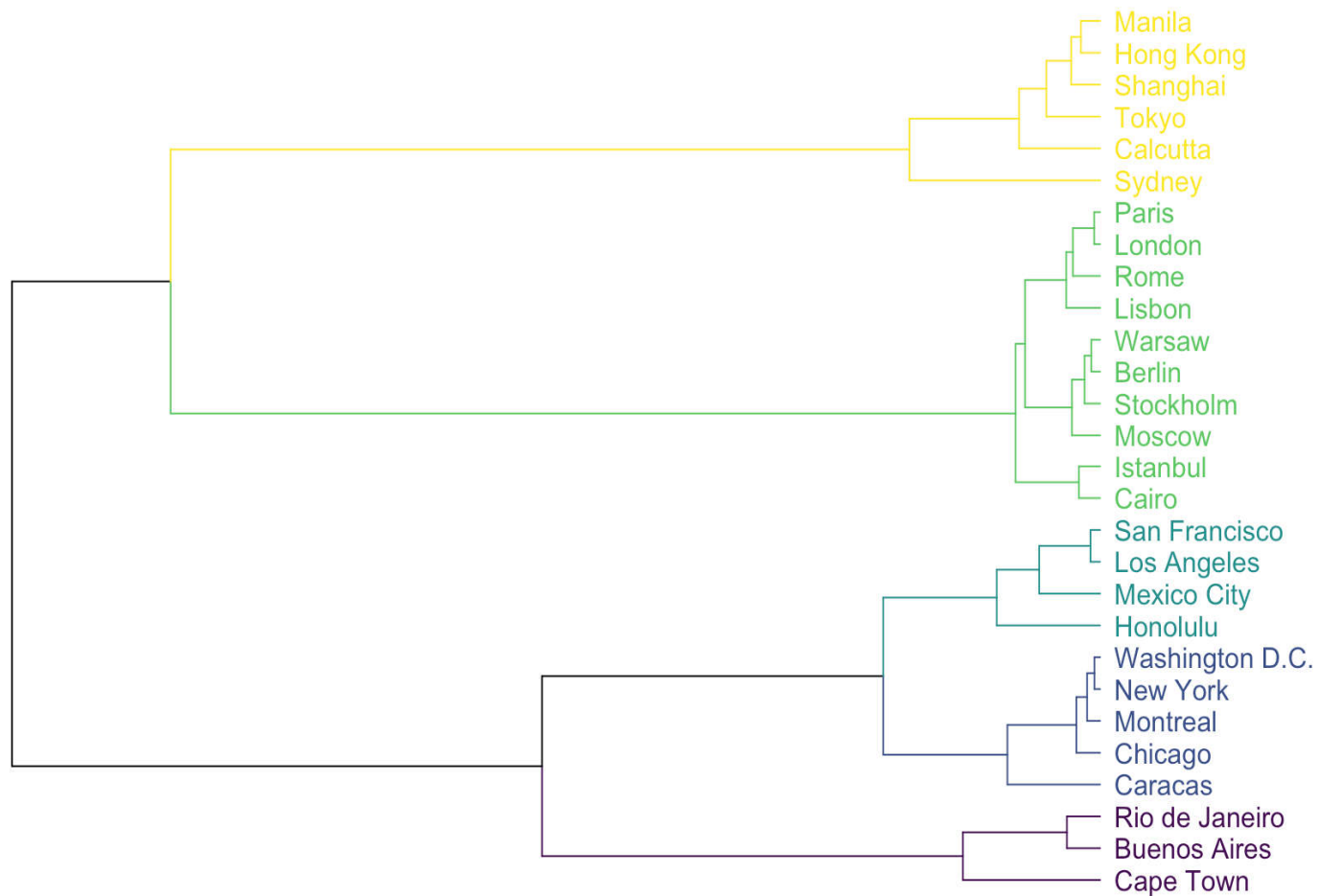
Example: Single link clustering



Dendrogram

```
scipy.cluster.hierarchy.dendrogram()
```

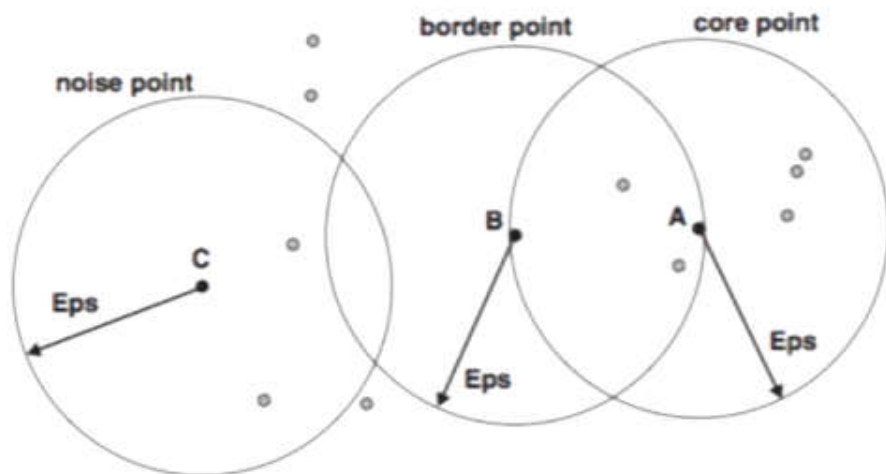
Example: Dendrogram



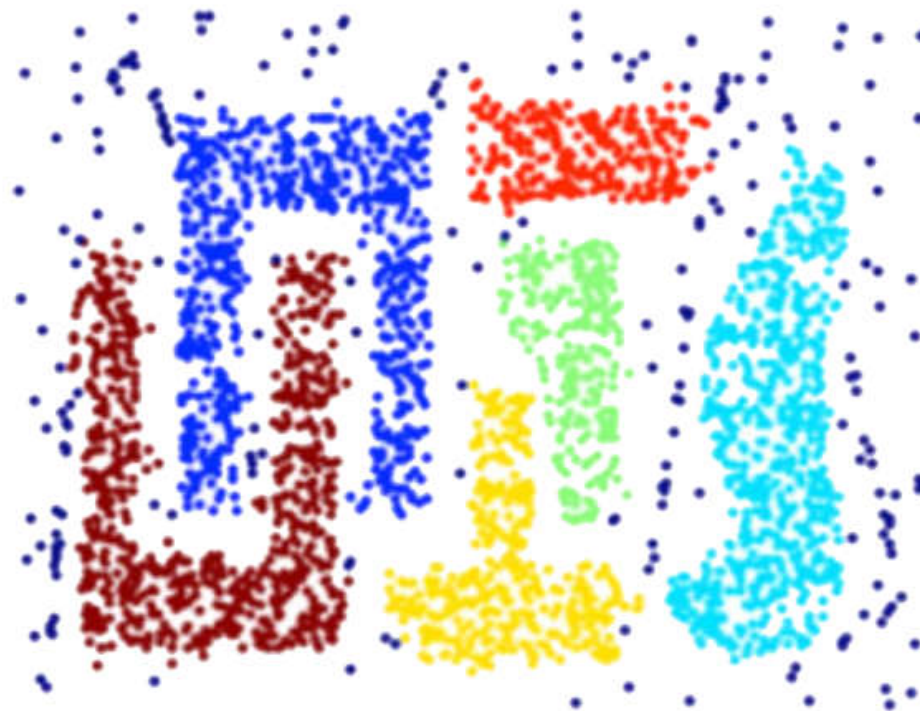
Clustering of
28 cities
based on
distance
between
them

Density Based Clustering (DBSCAN)

DBSCAN

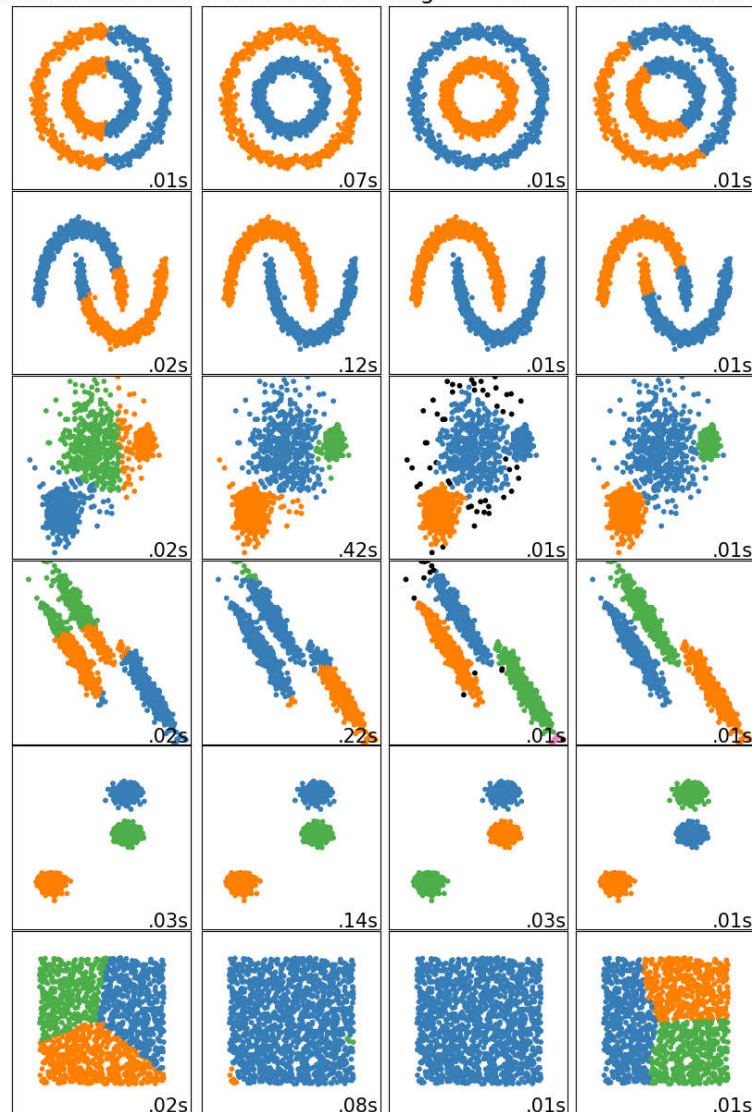


Given a set of points, it groups together points that are closely packed together (points with many [nearby neighbors](#)), marking as outliers points that lie alone in low-density regions (whose nearest neighbors are too far away).



```
class sklearn.cluster.DBSCAN(eps=0.5, min_  
samples=5, metric='euclidean')
```

MiniBatchKMeans IterativeClustering DBSCAN GaussianMixture



Clustering algorithms performance

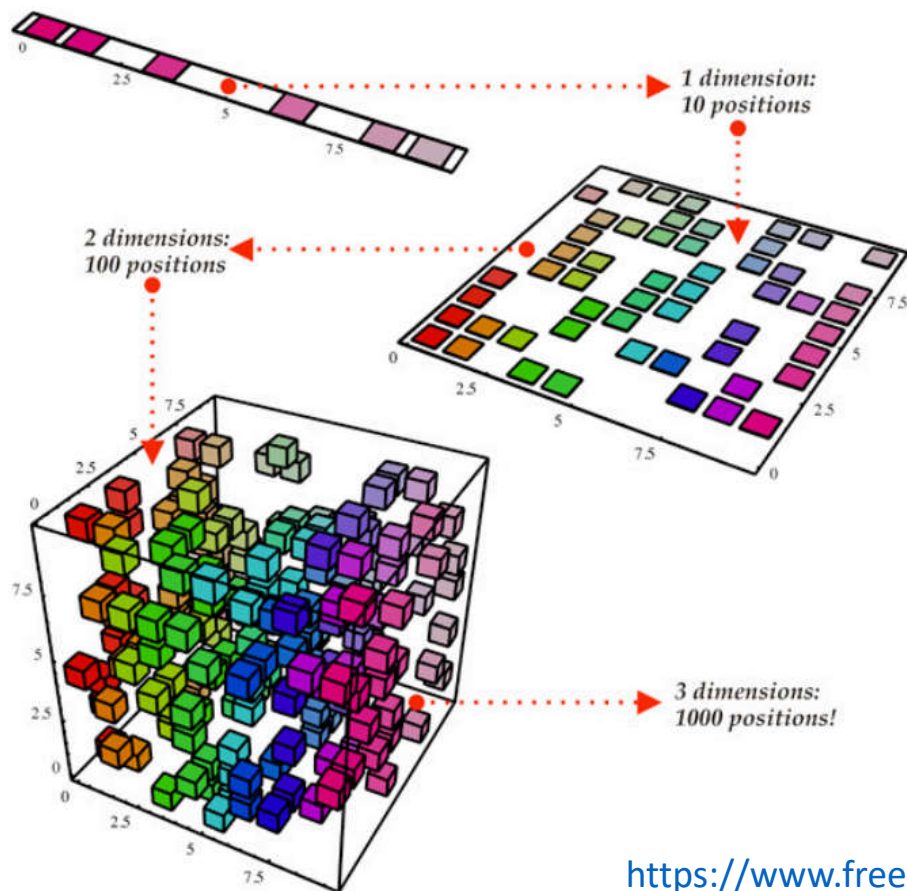
Example shows characteristics of different clustering algorithms on 2D datasets

The last dataset is an example of a 'null' situation for clustering: the data is homogeneous, and there is no good clustering.

https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html

Dimensionality Reduction

Dimensionality Reduction



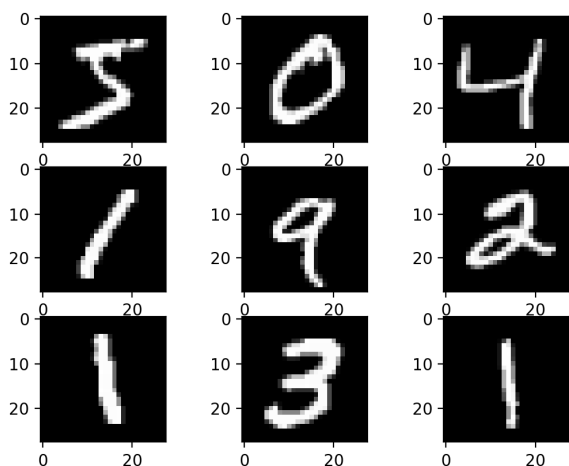
- It reduces the time and storage space required.
- Removal of multi-collinearity improves the interpretation of the parameters of the machine learning model.
- It becomes easier to visualize the data when **reduced** to very low dimensions such as 2D or 3D

<https://www.freecodecamp.org/news/an-overview-of-principal-component-analysis-6340e3bc4073/>

Recap: Random Forest - Feature Importance

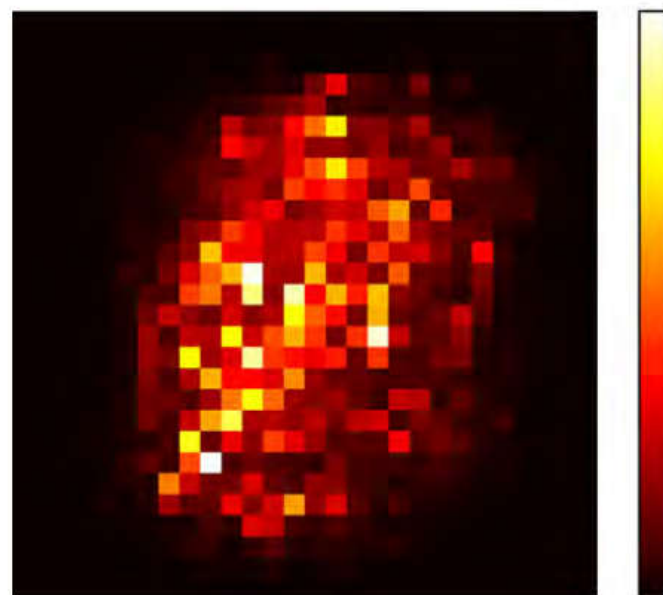
- A single DT: important features are likely to appear closer to the root of tree, while unimportant features often appear closer to the leaves
- RF: feature importance as average depth at which it appears across all trees in the forest

```
RandomForestClassifier().feature_importances_
```



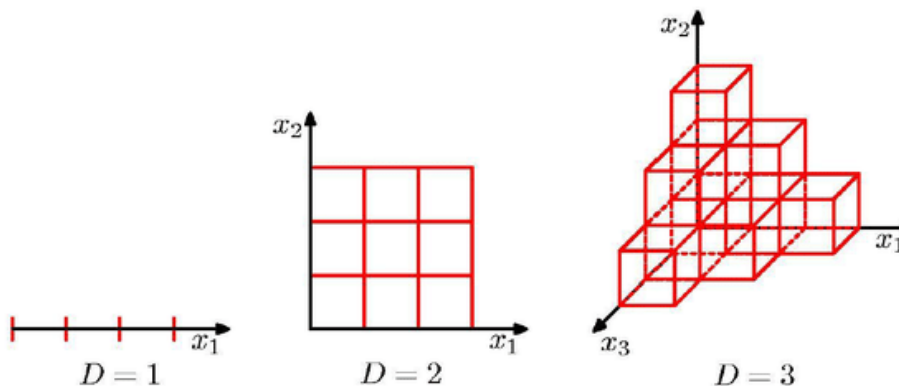
The MNIST database is a large **database** of handwritten digits (28x28 pixels, 60,000 training images and 10,000 testing images)

MNIST pixel importance with accordance to RF classifier

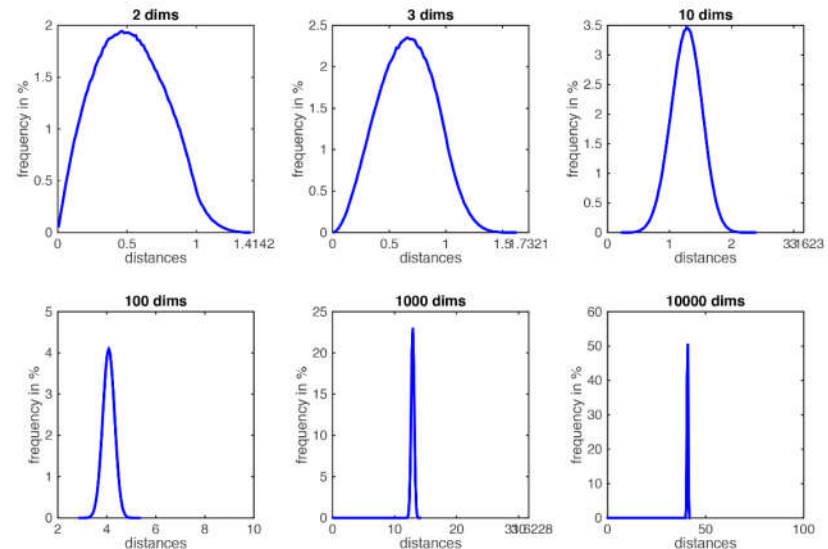


Curse of dimensionality

The curse of dimensionality refers to the phenomena that occur when classifying, organizing, and analyzing high dimensional data that does not occur in low dimensional spaces, specifically the issue of data sparsity (the volume of the space increases so fast that the available data become sparse).



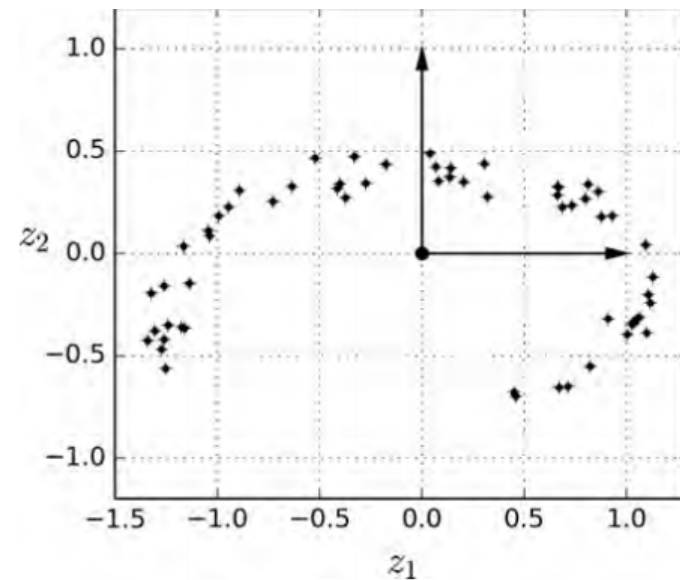
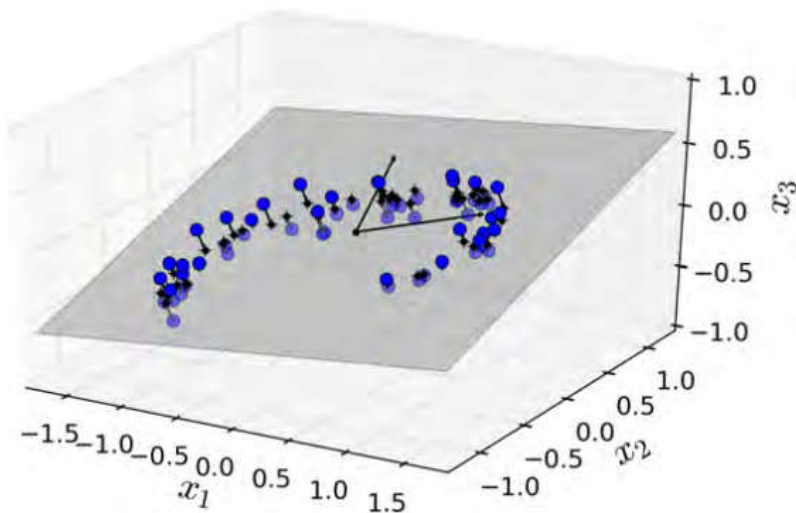
https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html



The histogram plots show the distributions of all pairwise distances between randomly distributed points within d -dimensional unit squares

Projection

In many real-world problems, training instances are not spreading across all dimensions. Many features may be nearly constant, while others are highly correlated. As a result, all training instances actually lie within (or close to) a much lower-dimensional subspace of the high-dimensional space.



Reduction of dimensionality from 3d to 2d

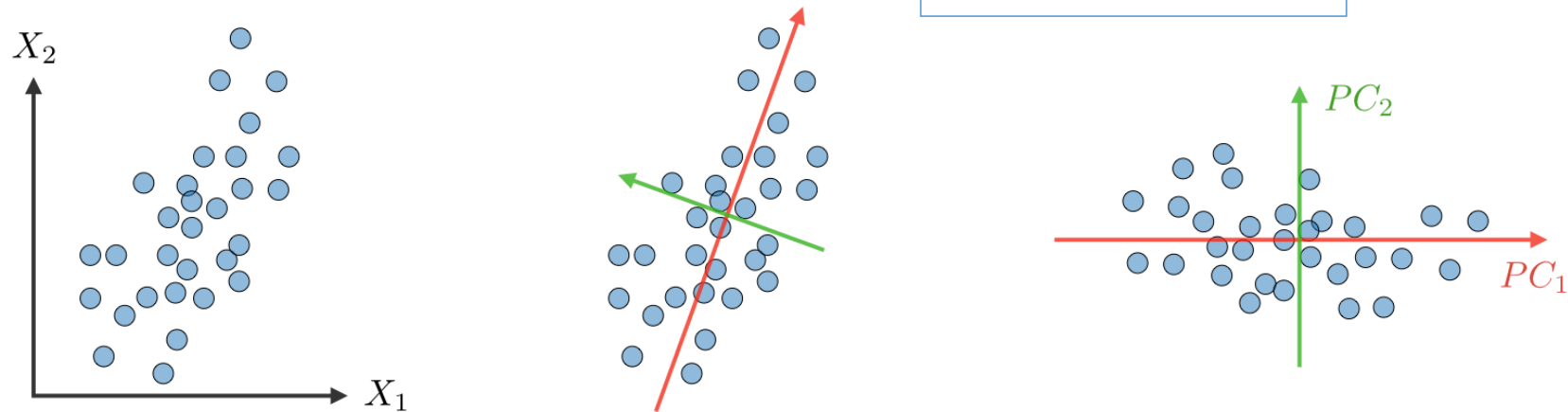
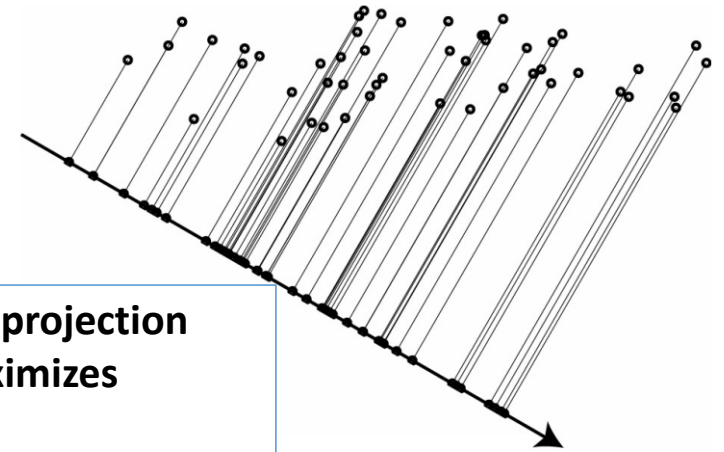
Principal Component Analysis (PCA)

Unsupervised Learning

Principal Component Analysis (PCA): 2D case, intuition

PCA is the most popular dimensionality reduction algorithm. First it identifies the hyperplane that lies closest to the data, and then it projects the data onto it.

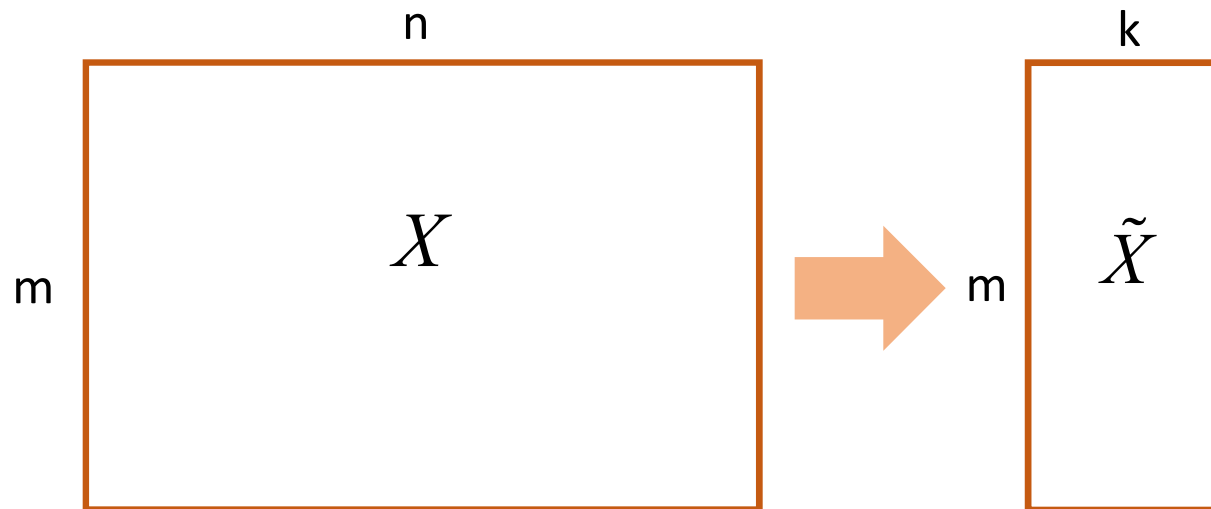
It is essential rotating the coordinate axes so higher-variance come first



Data in feature space → Find principal components → Data in principal components space

Principal Component Analysis (PCA)

- summarization of data with many (n) variables by a smaller set of (k) derived (synthetic, composite) variables.
- takes a data matrix of m objects by n variables, which may be correlated, and summarizes it by uncorrelated axes (principal components or principal axes) that are linear combinations of the original n variables
- the first k components display as much as possible of the variation among objects.



PCA

Dataset: $X_{m \times n}$, $x_j^{(i)}$

- mean of each variable:
$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$
- variance of each variable: (the average squared deviation around the mean)
$$\text{var}_j = \frac{1}{m-1} \sum_{i=1}^m \left(x_j^{(i)} - \mu_j \right)^2$$
- covariance (degree to which the variables are linearly correlated):

$$\sigma_{jk} = \frac{1}{m-1} \sum_{i=1}^m \left(x_j^{(i)} - \mu_j \right) \left(x_k^{(i)} - \mu_k \right)$$

PCA

Objective of PCA is to rigidly rotate the axes of n -dimensional space to new positions (principal axes) that have the following properties:

- ordered such that principal axis 1 has the highest variance, axis 2 has the next highest variance, , and axis p has the lowest variance
- covariance among each pair of the principal axes is zero (the principal axes are uncorrelated).

Using covariances among variables only makes sense if they are measured in the same units

Standardizing each variable to unit variance and zero mean.

$$x_j^{(i)} := \frac{x_j^{(i)} - \mu_j}{\text{std}_j}$$

```
sklearn.preprocessing.StandardScaler().fit_transform(X)
```

Covariance matrix:

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \dots & \sigma_{1n} \\ \dots & \dots & \dots \\ \sigma_{1n} & \dots & \sigma_n^2 \end{pmatrix}$$

$$\sigma_{jk} = \frac{1}{m-1} \sum_{i=1}^m (x_j^{(i)} - \mu_j)(x_k^{(i)} - \mu_k)$$

For standardized data:

$$\Sigma = \frac{1}{m-1} X^T X$$

Principal axes are eigenvectors of matrix Σ

$$\Sigma \vec{v} = \lambda \vec{v}, \quad \det(\Sigma - \lambda E) = 0$$

λ are eigenvalues

$\vec{v}$ are eigenvectors

Proof: https://en.wikipedia.org/wiki/Rayleigh_quotient

PCA

- eigenvectors are uncorrelated (orthogonal)
- the eigenvalues, $\lambda_1, \lambda_2, \dots, \lambda_n$ are the variances of the coordinates on each principal component axis
- the sum of all n eigenvalues equals to the sum of the variances of the original variables
- each eigenvector represents the “contribution” of each variable to the principal component axis

$$\frac{\lambda_j}{\sum_{j=1}^n \lambda_j}$$

- **explained variance ratio** (it indicates the proportion of the dataset's variance that lies along the axis of each principal component)

PCA: Build the new reduced dataset

Sort eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ in descending order

Choose first k eigenvalues; compose the matrix of chosen k eigenvectors coordinates.

$$W = \begin{pmatrix} | & \dots & | \\ \vec{v}_1 & \dots & \vec{v}_k \\ | & \dots & | \end{pmatrix}$$

Projection the training set down to k dimension

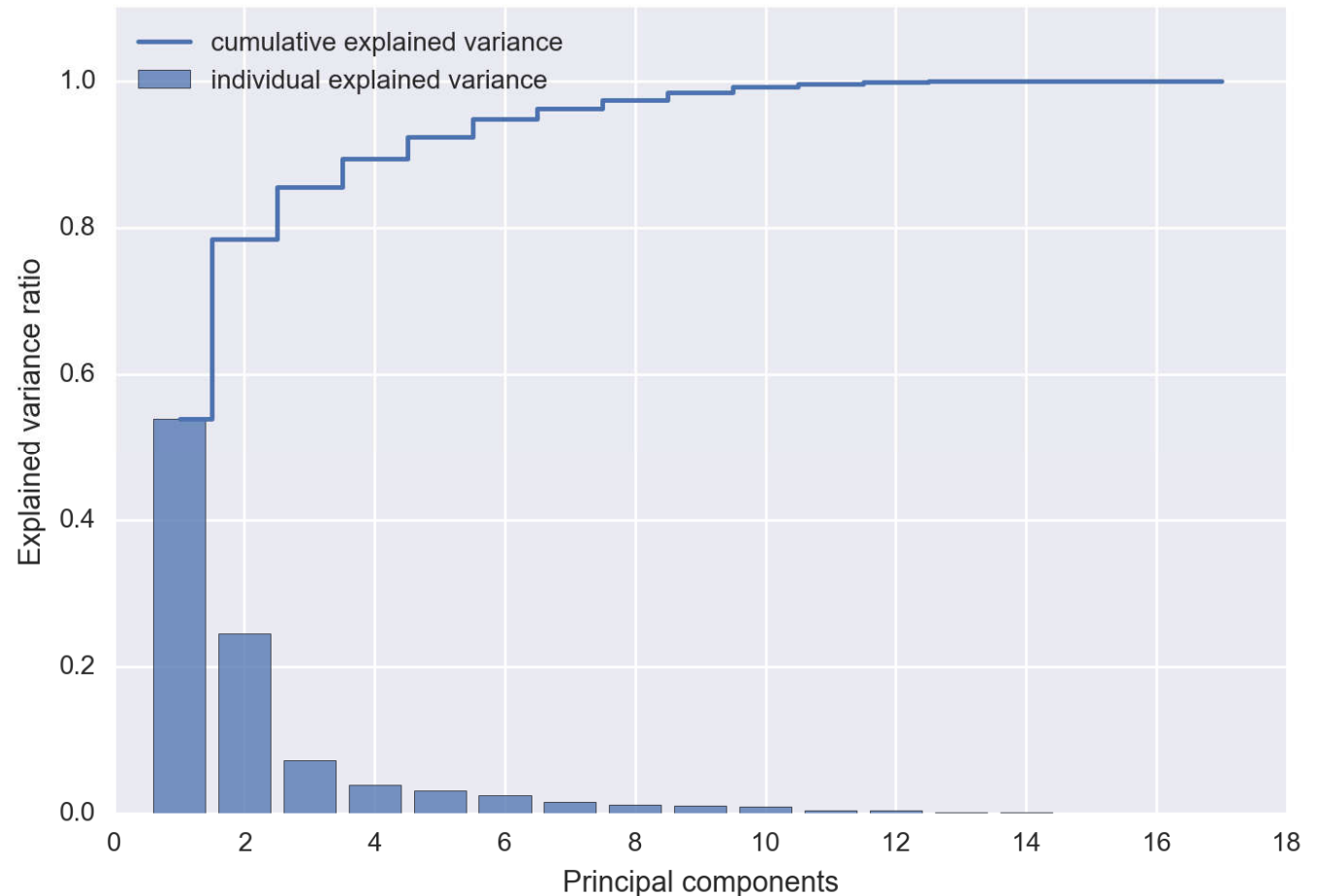
$$\tilde{X}_{mxk} = X_{mxn} \cdot W_{nxk}$$

Initial data recovering $X_{mxn} = \tilde{X}_{mxk} \cdot W^T$

PCA: Choosing the right number of dimensions

It is generally preferred to choose the number of dimensions that add up to a sufficiently large portion of variance (e.g. 95%).

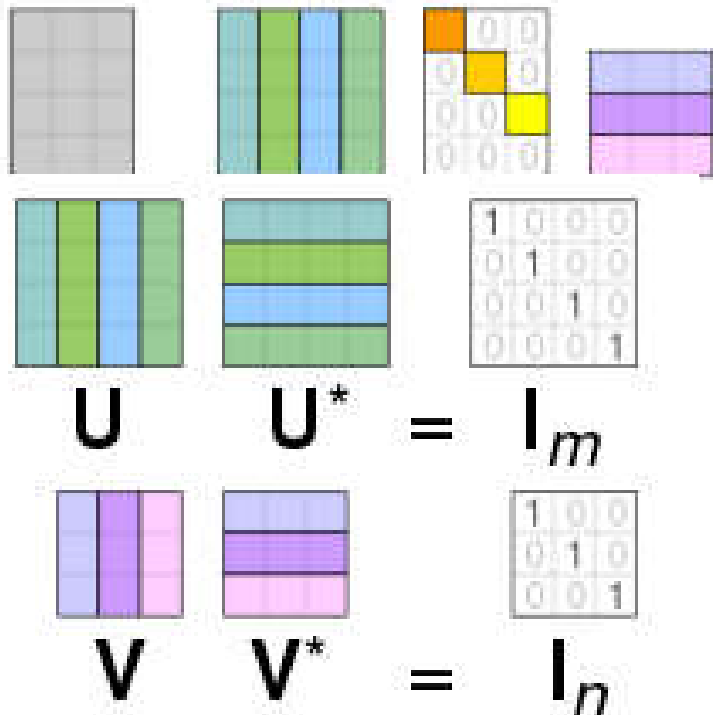
Note: for data visualization $k = 2$ or 3



PCA and SVD decomposition

Singular value decomposition (SVD) is standard matrix factorization technique.

$$X_{m \times n} = U_{m \times n} \cdot D_{n \times n} \cdot V_{n \times m}^T$$



Columns of U matrix are eigenvectors of XX^T

Columns of V matrix are eigenvectors of X^TX

Diagonal elements of D matrix are known as singular values of X

$$D = \begin{pmatrix} s_1 & 0 & & \\ 0 & s_2 & & \\ & & \dots & \end{pmatrix}, s_1 \geq s_2 \dots$$

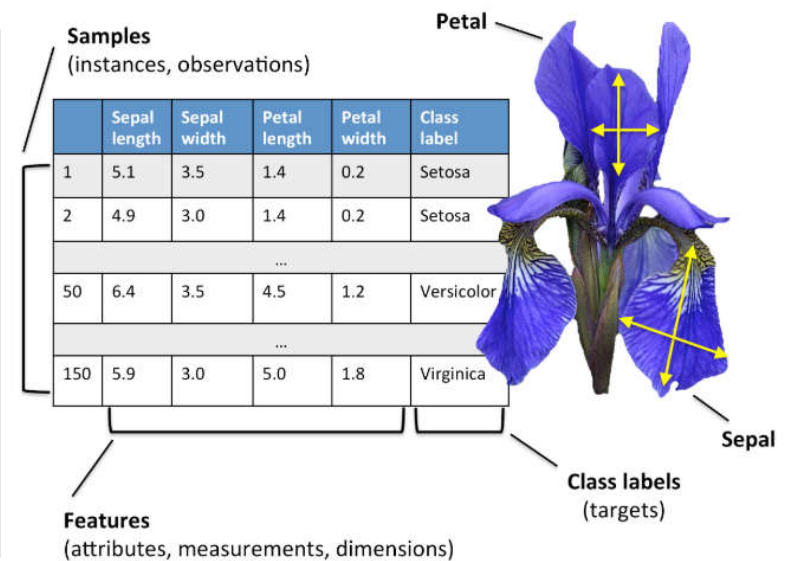
PCA in Python (Iris dataset)

```
from sklearn.preprocessing import StandardScaler
from sklearn import datasets
```

```
iris = datasets.load_iris()
X = iris.data
X_std = StandardScaler().fit_transform(X)
cov_mat = X_std.T.dot(X_std) / (X_std.shape[0]-1)
eig_vals, eig_vecs = np.linalg.eig(cov_mat)
```

```
eig_vecs
[[ 0.5223 -0.3723 -0.7210 0.2619]
 [-0.2633 -0.9255  0.2420 -0.1241]
 [ 0.5812 -0.02109 0.1408 -0.8011]
 [ 0.5656 -0.0654  0.6338  0.5235]]
eig_vals
[2.9303  0.9274  0.14834  0.0207]
```

```
u,s,v = np.linalg.svd(X_std)
print(v.T)
```



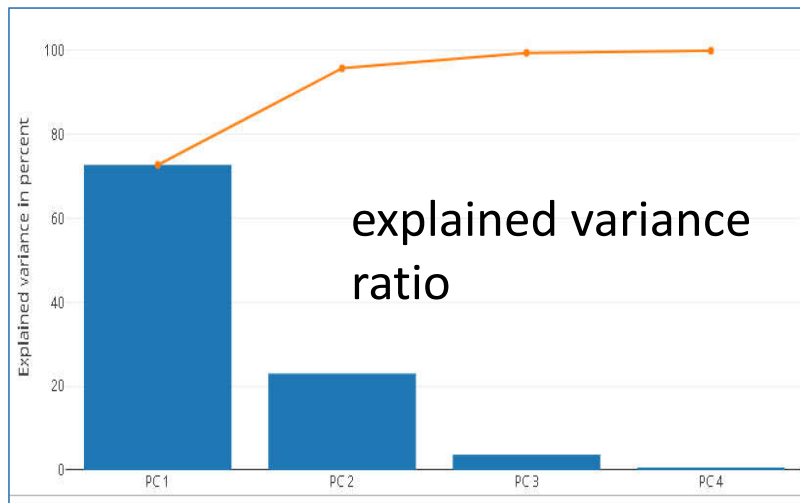
[s://plot.ly/python/v3/ipython-notebooks/principal-component-analysis/](https://plot.ly/python/v3/ipython-notebooks/principal-component-analysis/)

PCA in Python (Iris dataset)

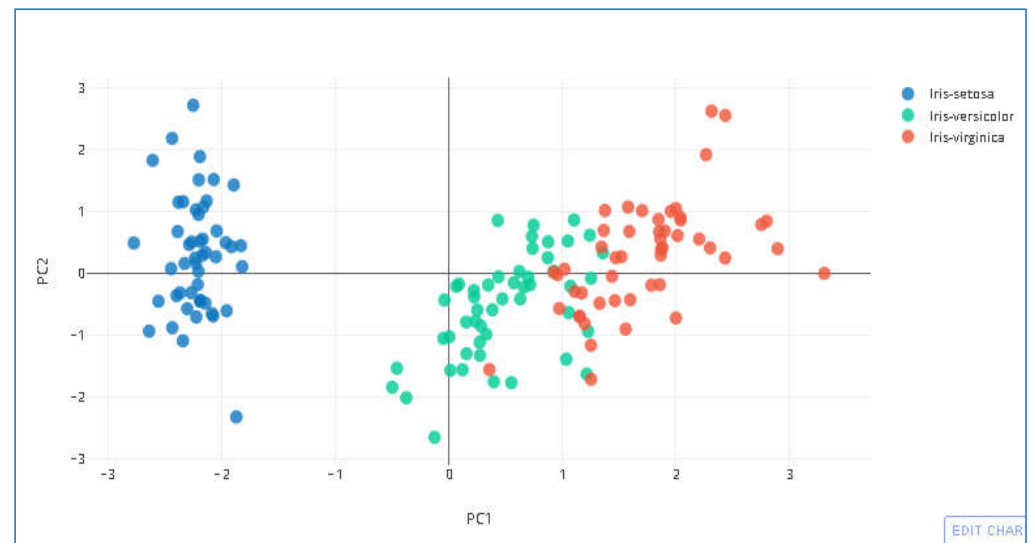
Matrix_w:

```
[[ 0.5223 -0.3723]  
 [-0.2633 -0.9255]  
 [ 0.5812 -0.0210]  
 [ 0.5656 -0.0654]]
```

```
X_new = X_std.dot(matrix_w)
```



PCA 4d -> 2d



PCA in sklearn (Iris dataset)

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_sklearn = pca.fit_transform(X_std)
```

Attributes:

explained_variance_

The amount of variance explained by each of the selected components

explained_variance_ratio_

Percentage of variance explained by each of the selected components

<https://habr.com/ru/company/ods/blog/325654/>

```
pca = PCA(n_components=0.95)
```

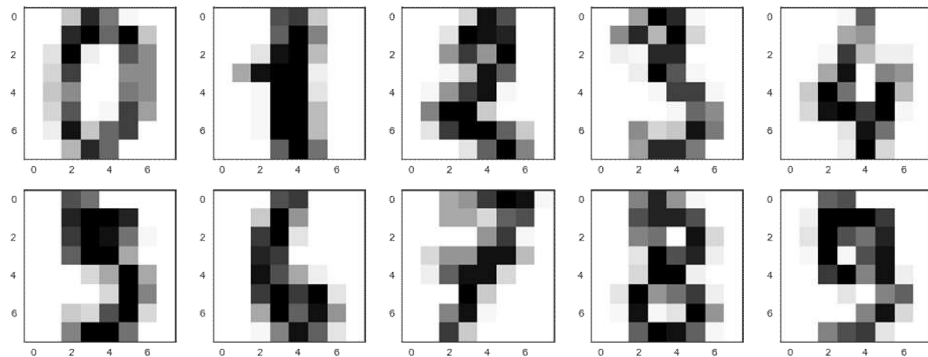
If $0 < n_components < 1$, select the number of components such that the amount of variance is greater than the percentage specified by `n_components`.

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(max_depth=2)
clf.fit(X_train, y_train)
preds = clf.predict(X_test)
accuracy_score(y_test, preds)      0.88889
...
clf.fit(pca_X_train, y_train) # 2 components
preds = clf.predict(pca_X_test)
accuracy_score(y_test, preds)      0.91111
```

PCA in sklearn (MNIST dataset)

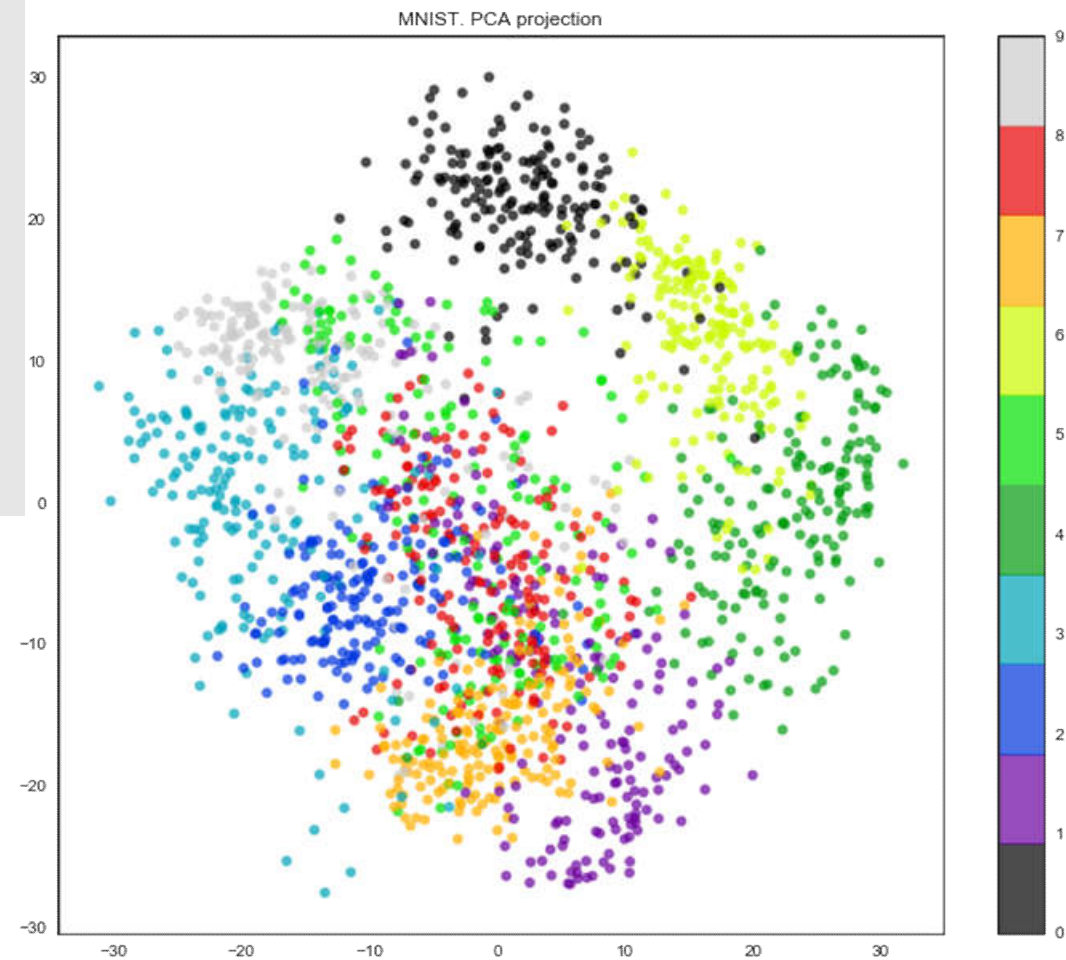
```
from sklearn import datasets  
from sklearn.decomposition import PCA
```

```
digits = datasets.load_digits()  
X = digits.data  
y = digits.target  
pca = PCA(n_components=2)  
X_reduced = pca.fit_transform(X)
```

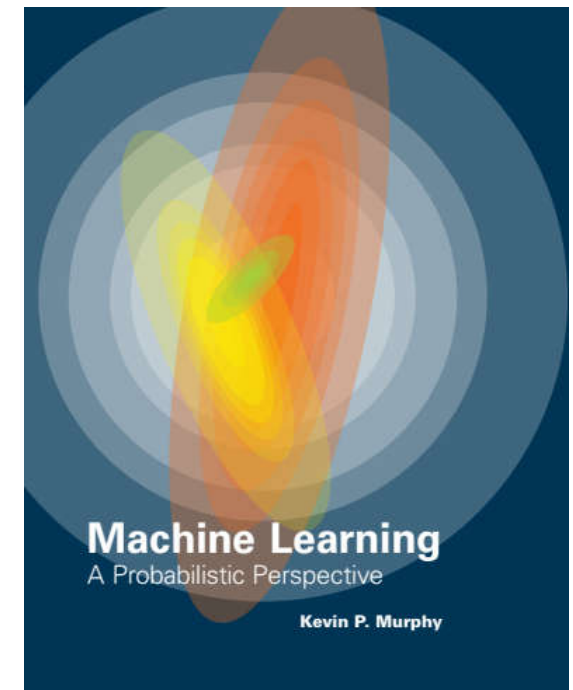
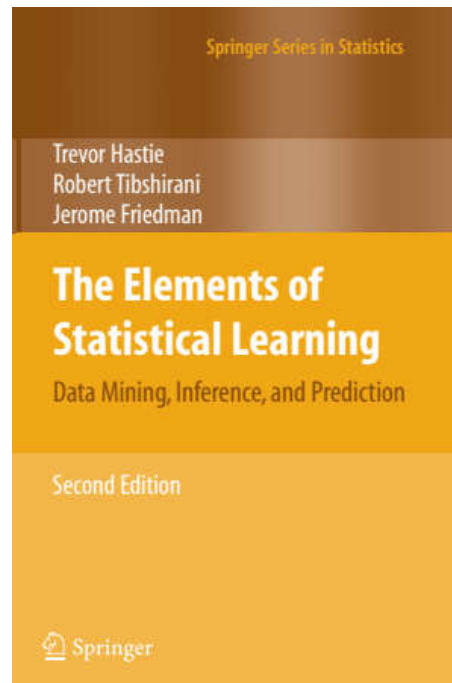
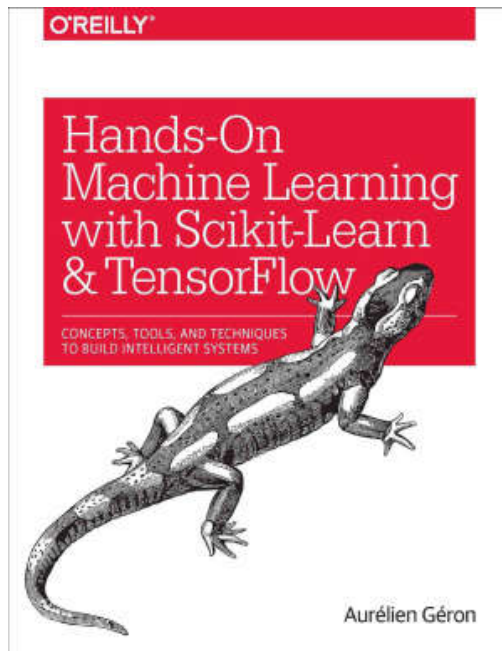


8x8

<https://habr.com/ru/company/ods/blog/325654/>



References



Appendix

Recap: Multivariate normal distribution

The multivariate normal distribution in n -dimensions, also called the multivariate Gaussian distribution, is parameterized by a **mean vector** $\mu \in \mathbb{R}^n$ and a **covariance matrix** $\Sigma \in \mathbb{R}^{n \times n}$, where $\Sigma \geq 0$ is symmetric and positive semi-definite. Also written “ $\mathcal{N}(\mu, \Sigma)$ ”, its density is given by:

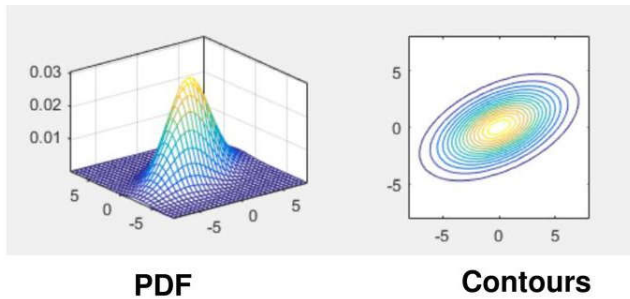
$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right).$$

In the equation above, “ $|\Sigma|$ ” denotes the determinant of the matrix Σ .

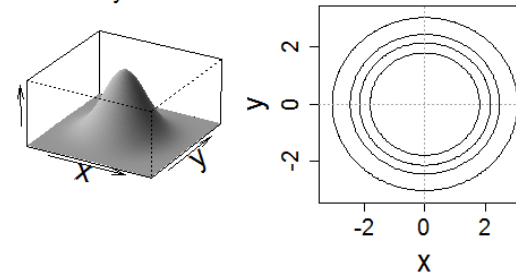
Recap: Bivariate normal distribution

$$\mu = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma = \begin{pmatrix} \sigma_x & \rho \\ \rho & \sigma_y \end{pmatrix}$$

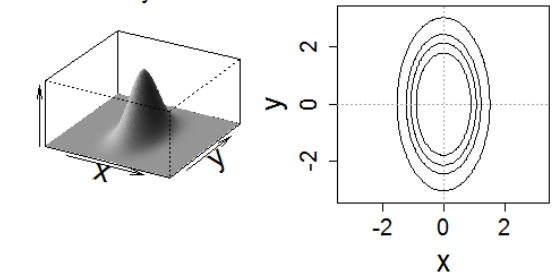
$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 9 & 3 \\ 3 & 4 \end{bmatrix}$$



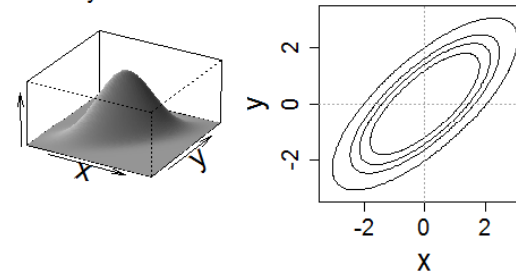
$$\sigma_x = \sigma_y, \rho = 0$$



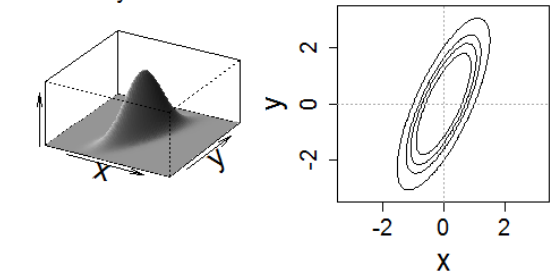
$$2\sigma_x = \sigma_y, \rho = 0$$



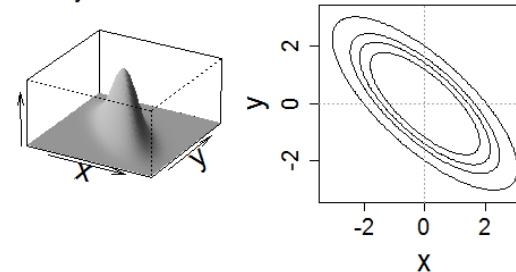
$$\sigma_x = \sigma_y, \rho = 0.75$$



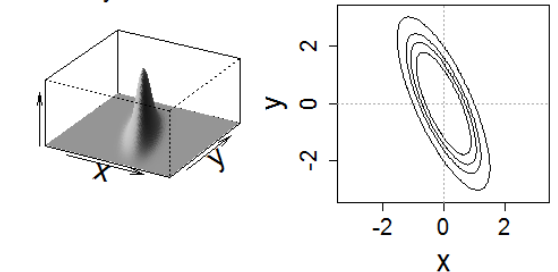
$$2\sigma_x = \sigma_y, \rho = 0.75$$



$$\sigma_x = \sigma_y, \rho = -0.75$$



$$2\sigma_x = \sigma_y, \rho = -0.75$$



Recap: Normal distribution: estimation of parameters

Having a sample $(x_1, \dots, x_m)$ from a normal distribution we would like to learn the approximate values of parameters:

- 1 dim
$$\hat{\mu} = \frac{1}{m} \sum_{i=1}^m x_i, \quad \hat{\sigma}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \hat{\mu})^2$$

- N-dim:
$$\hat{\vec{\mu}} = \frac{1}{m} \sum_{i=1}^m \vec{x}_i, \quad \hat{\Sigma} = \frac{1}{m} \sum_{i=1}^m (\vec{x}_i - \hat{\vec{\mu}})(\vec{x}_i - \hat{\vec{\mu}})^T$$

Data Analysis

Lecture 7: Dimensionality Reduction

Dr. Nataliya K. Sakhnenko

Roadmap

Basic Machine Learning		Deep Learning
Supervised	Unsupervised	
☐ Regression (linear and polynomial) ☐ Classification ✓ Logistic regression, KNN, Decision Tree, SVM, NB ✓ Ensemble methods: Random Forest, AdaBoost, Gradient Boost	☐ Clustering (k-means, EM, Hierarchical clustering, DBSCAN) ☐ Dimensionality reduction (PCA, LDA, MDS, t-SNE) 	☐ MLP ☐ CNN ☐ RNN ☐ Generative models: GANs, VAEs, NFs ☐ Attention and Transformers ☐ Intro to RL

Dimensionality Reduction

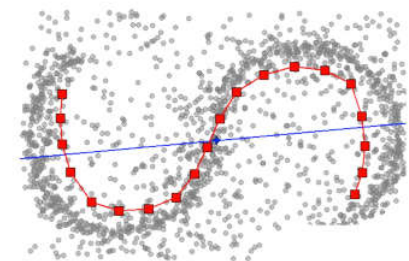
Dimensionality Reduction Methods

Linear:

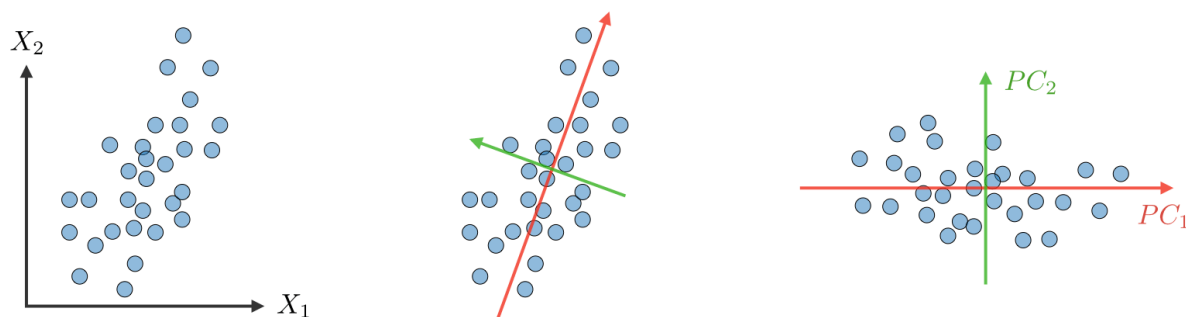
- PCA (unsupervised)
- LDA (supervised)

Nonlinear :

- MDS
- t-SNE



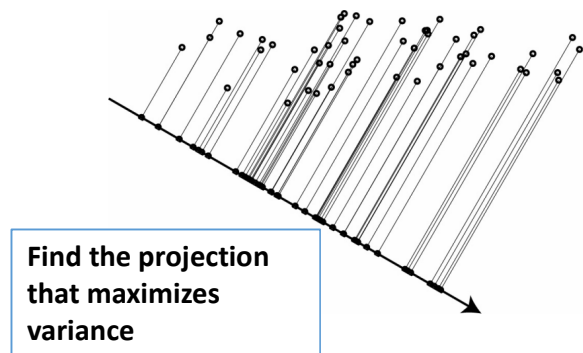
Recap: PCA



Data in feature space $\rightarrow$ Find principal components $\rightarrow$ Data in principal components space

Principal axes are eigenvectors of matrix covariance Σ

Projection the training set down to k dimension: $\tilde{X}_{mxk} = X_{mxn} \cdot W_{nxk}$



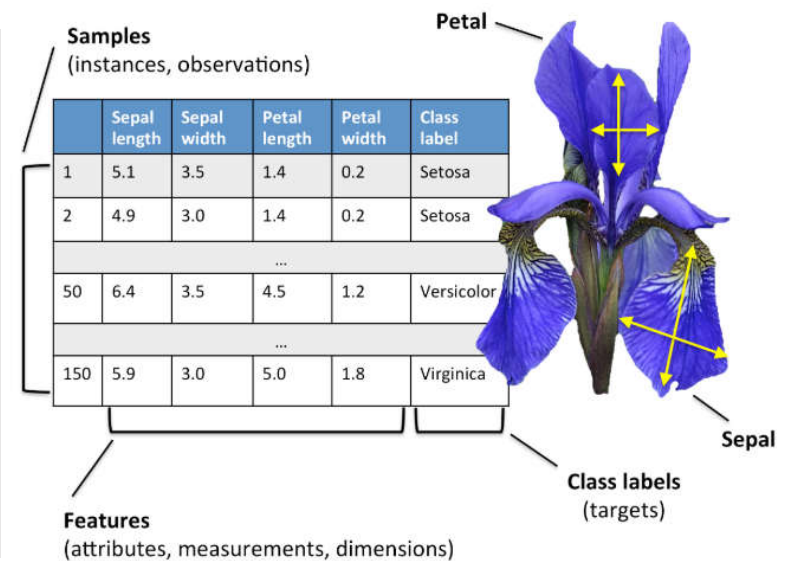
PCA in Python (Iris dataset)

```
from sklearn.preprocessing import StandardScaler
from sklearn import datasets
```

```
iris = datasets.load_iris()
X = iris.data
X_std = StandardScaler().fit_transform(X)
cov_mat = X_std.T.dot(X_std) / (X_std.shape[0]-1)
eig_vals, eig_vecs = np.linalg.eig(cov_mat)
```

```
eig_vecs
[[ 0.5223 -0.3723 -0.7210  0.2619]
 [-0.2633 -0.9255  0.2420 -0.1241]
 [ 0.5812 -0.02109 0.1408 -0.8011]
 [ 0.5656 -0.0654  0.6338  0.5235]]
eig_vals
[2.9303  0.9274  0.14834  0.0207]
```

```
u,s,v = np.linalg.svd(X_std)
print(v.T)
```



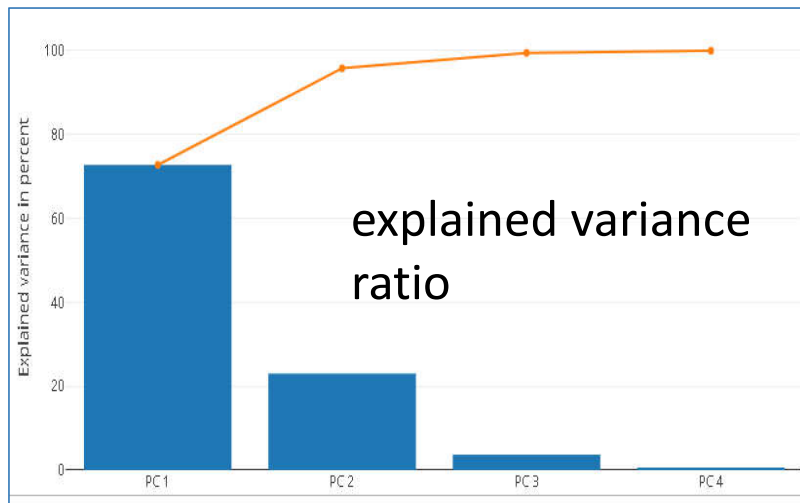
[s://plot.ly/python/v3/ipython-notebooks/principal-component-analysis/](https://plot.ly/python/v3/ipython-notebooks/principal-component-analysis/)

PCA in Python (Iris dataset)

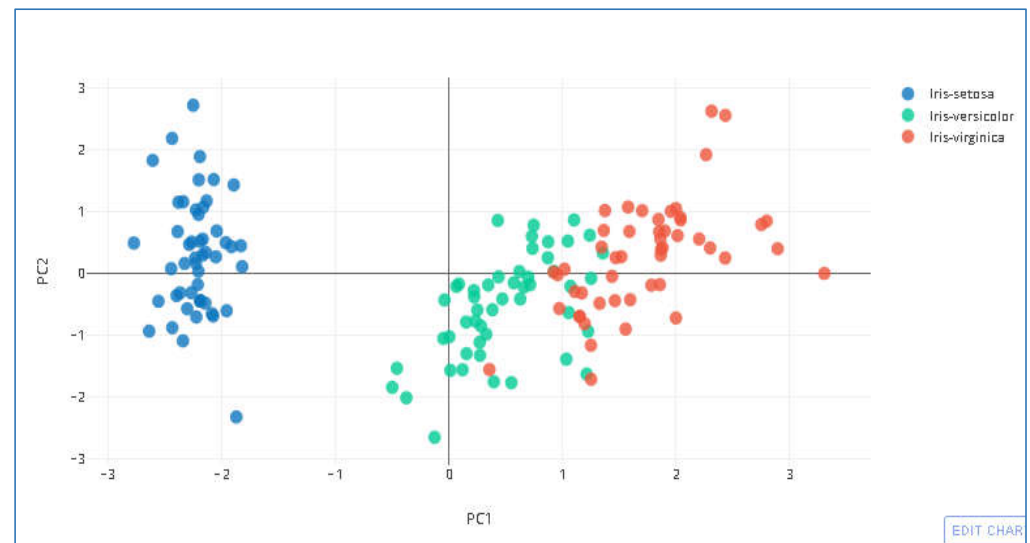
Matrix_w:

```
[[ 0.5223 -0.3723]  
 [-0.2633 -0.9255]  
 [ 0.5812 -0.0210]  
 [ 0.5656 -0.0654]]
```

```
X_new = X_std.dot(matrix_w)
```



PCA 4d -> 2d



PCA in sklearn (Iris dataset)

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_sklearn = pca.fit_transform(X_std)
```

Attributes:

explained_variance_

The amount of variance explained by each of the selected components

explained_variance_ratio_

Percentage of variance explained by each of the selected components

<https://habr.com/ru/company/ods/blog/325654/>

```
pca = PCA(n_components=0.95)
```

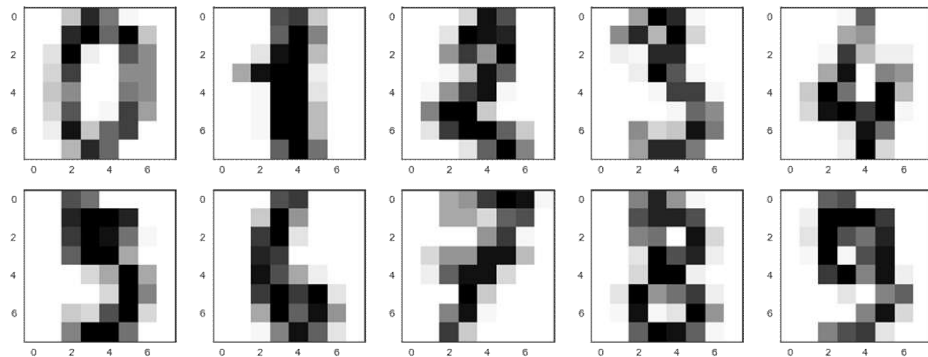
If $0 < n_components < 1$, select the number of components such that the amount of variance is greater than the percentage specified by `n_components`.

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(max_depth=2)
clf.fit(X_train, y_train)
preds = clf.predict(X_test)
accuracy_score(y_test, preds)      0.88889
...
clf.fit(pca_X_train, y_train) # 2 components
preds = clf.predict(pca_X_test)
accuracy_score(y_test, preds)      0.91111
```


PCA in sklearn (MNIST dataset)

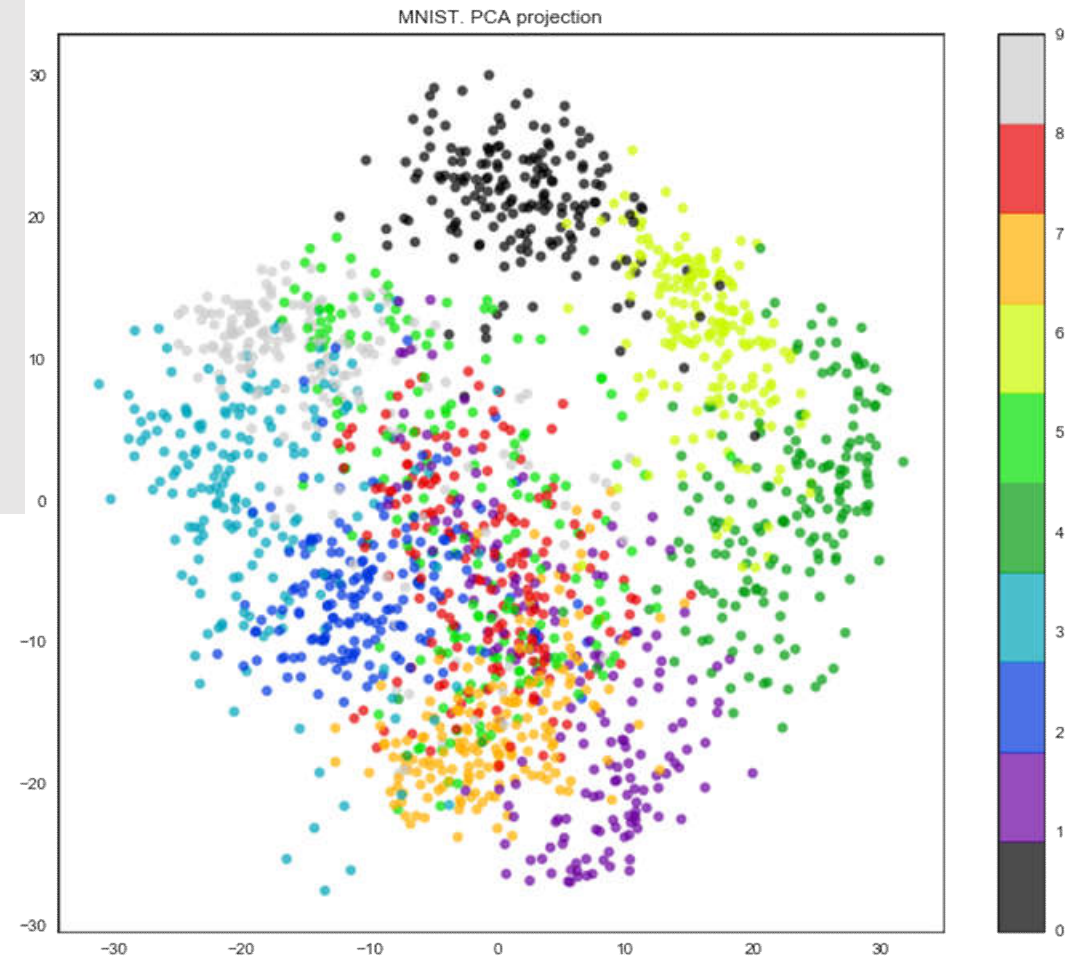
```
from sklearn import datasets  
from sklearn.decomposition import PCA
```

```
digits = datasets.load_digits()  
X = digits.data  
y = digits.target  
pca = PCA(n_components=2)  
X_reduced = pca.fit_transform(X)
```



8x8

<https://habr.com/ru/company/ods/blog/325654/>

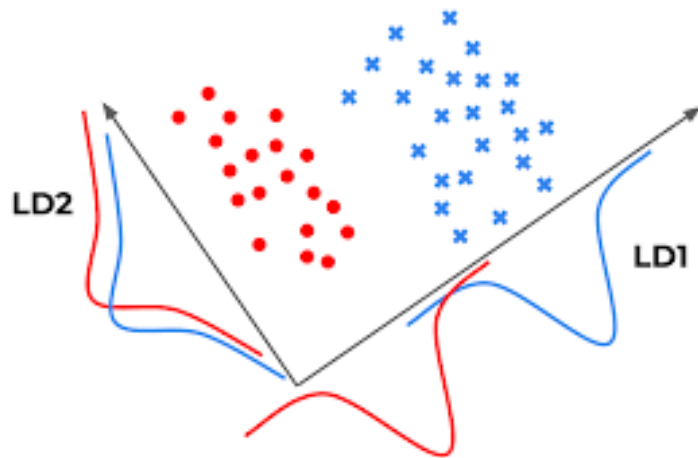


Linear Discriminant Analysis (LDA)

Supervised Learning

Linear Discriminant Analysis (LDA)

Ronald A. Fisher formulated the *Linear Discriminant* in 1936

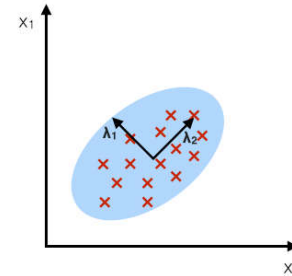


The goal is to project a dataset onto a lower-dimensional space with good class-separability

https://sebastianraschka.com/Articles/2014_python_lda.html

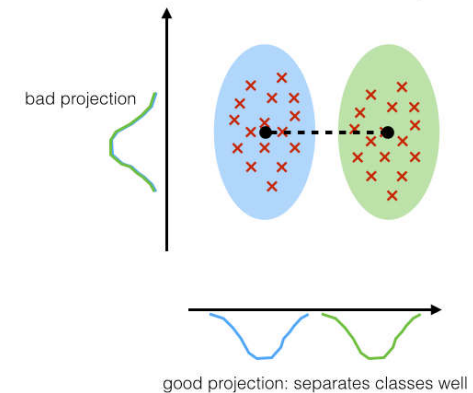
PCA:

component axes that maximize the variance



LDA:

maximizing the component axes for class-separation



LDA assumptions:

- Gaussian Mixture
- Covariance matrixes are identical

LDA algorithm

$X_{m \times n}$ Let we have C classes

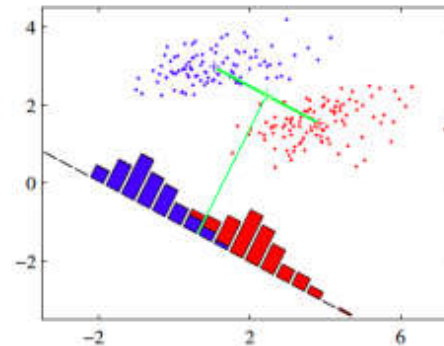
w_1 : 1st class, m_1 points, mean μ_1

w_2 : 2nd class, m_2 points, mean μ_2

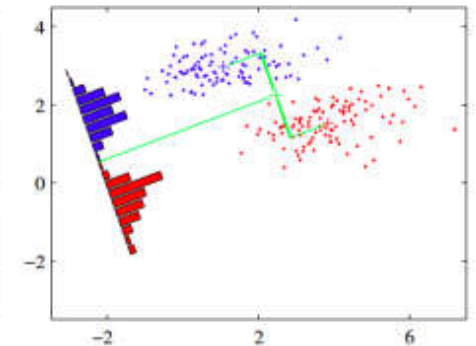
...

} μ

The separation is not great



Nice separation



Within class scatter matrix: $S_{w_j} = \sum_{i \in w_j} (\vec{x}_i - \vec{\mu}_j)(\vec{x}_i - \vec{\mu}_j)^T$

$$S_w = \sum_{i=1}^C S_{w_i}$$

In between class scatter matrix: $S_{B_j} = (\vec{\mu}_j - \vec{\mu})(\vec{\mu}_j - \vec{\mu})^T \cdot m_j$

$$S_B = \sum_{i=1}^C S_{B_i}$$

Solve the eigenvalue problem for the matrix

$$S_W^{-1} S_B$$

Further steps are similar to PCA algorithm

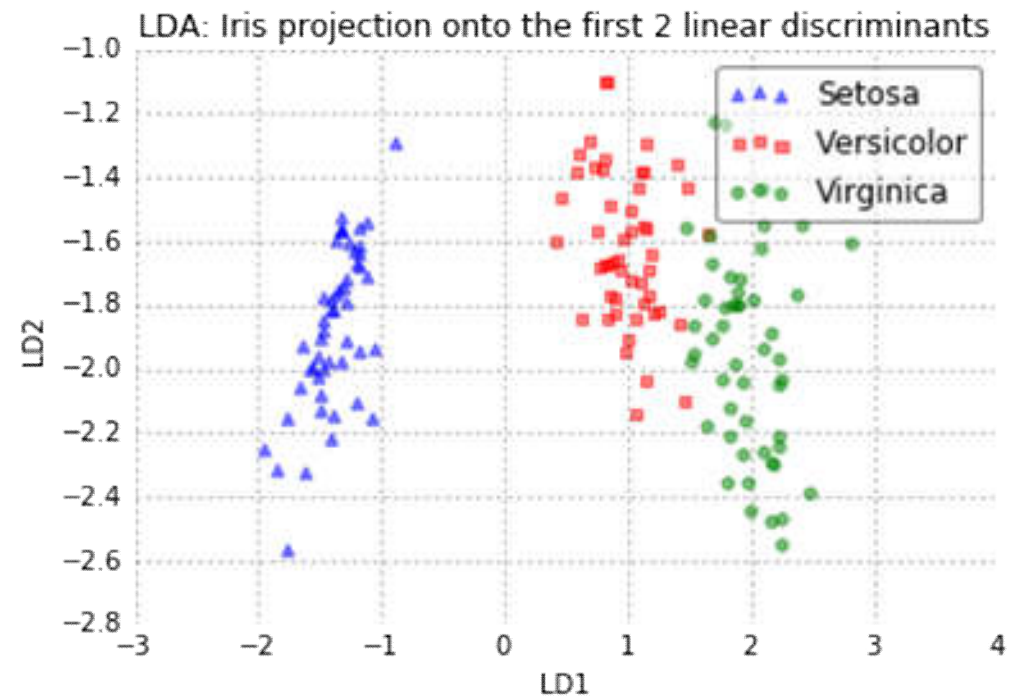
LDA algorithm (Iris dataset)

see https://sebastianraschka.com/Articles/2014_python_lda.html

Rank the eigenvectors from highest to lowest corresponding eigenvalue and choose the top k eigenvectors

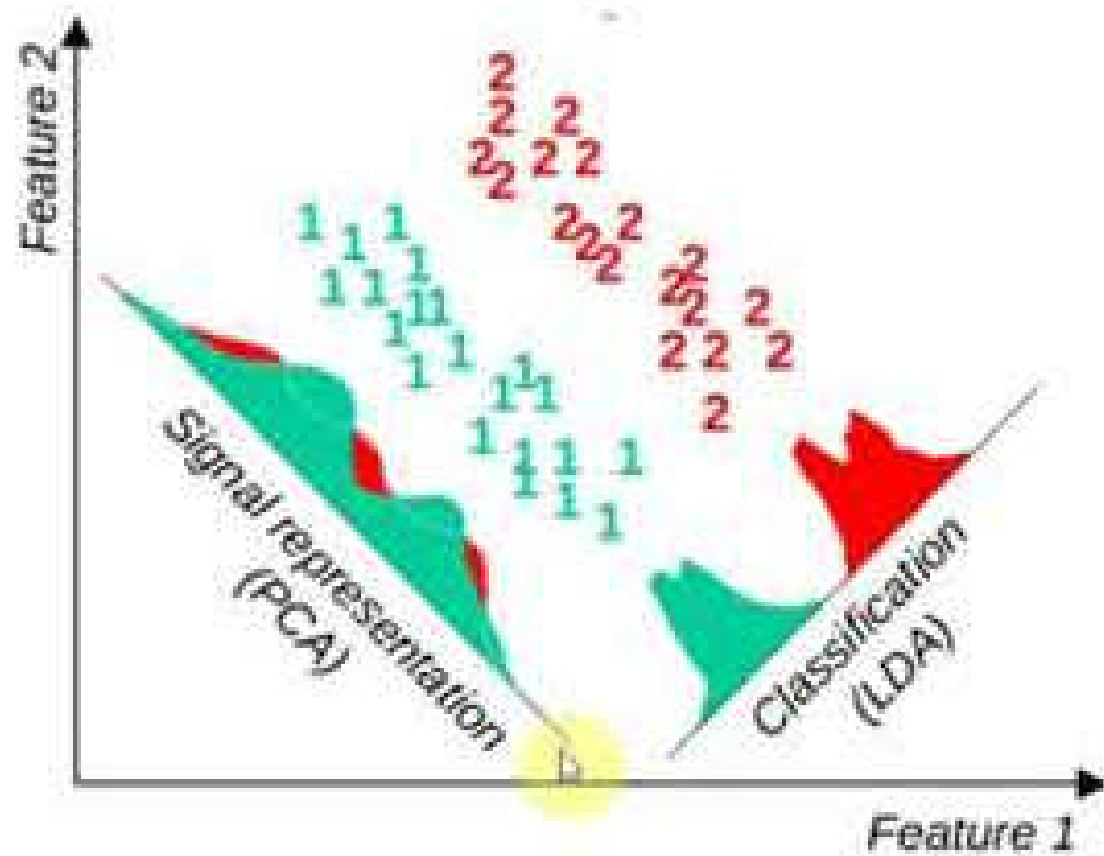
Those eigenvectors are called the linear discriminants

In LDA the number of linear discriminants is at most $C-1$, where C is number of classes



```
sklearn.discriminant_analysis.LinearDiscriminantAnalysis()
```

Difference between PCA and LDA



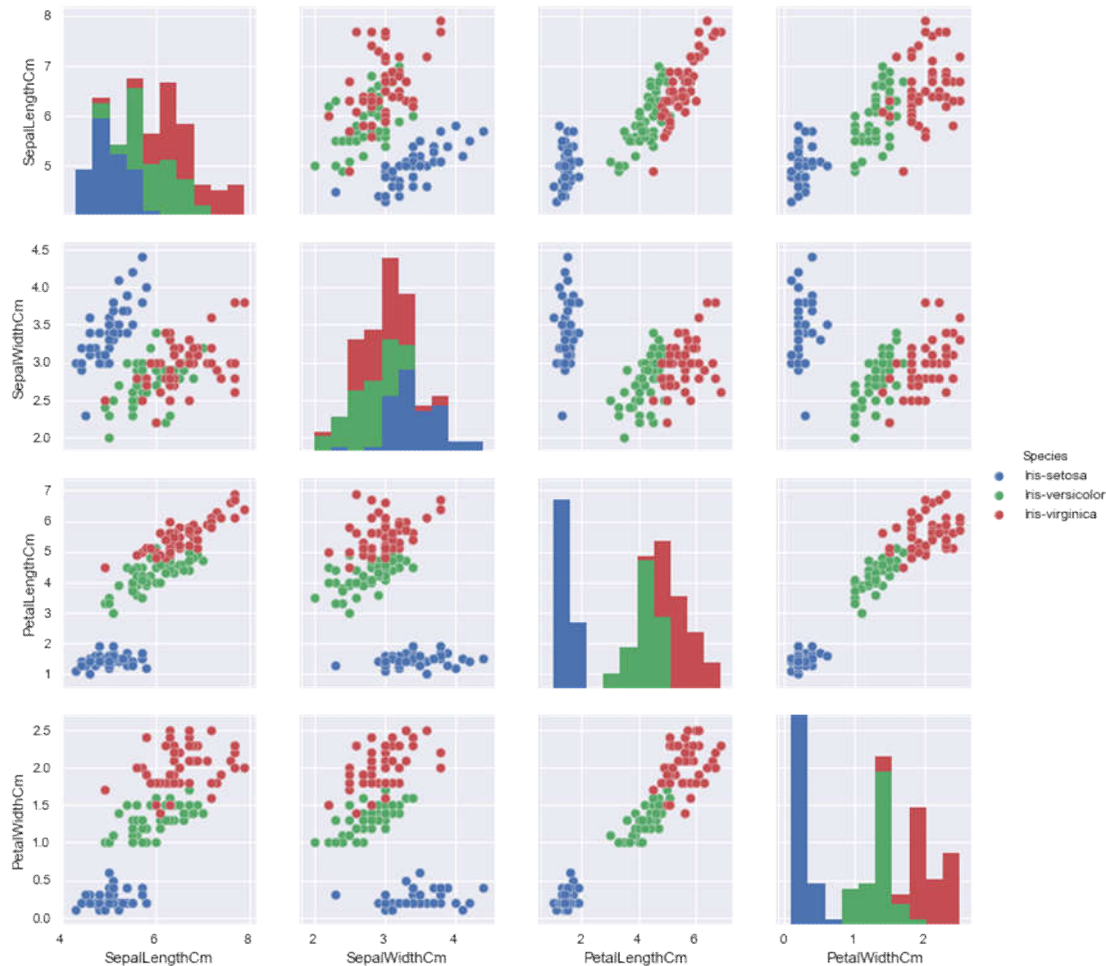
Dimensionality reduction for data visualization

Recap: Data visualization

Iris dataset

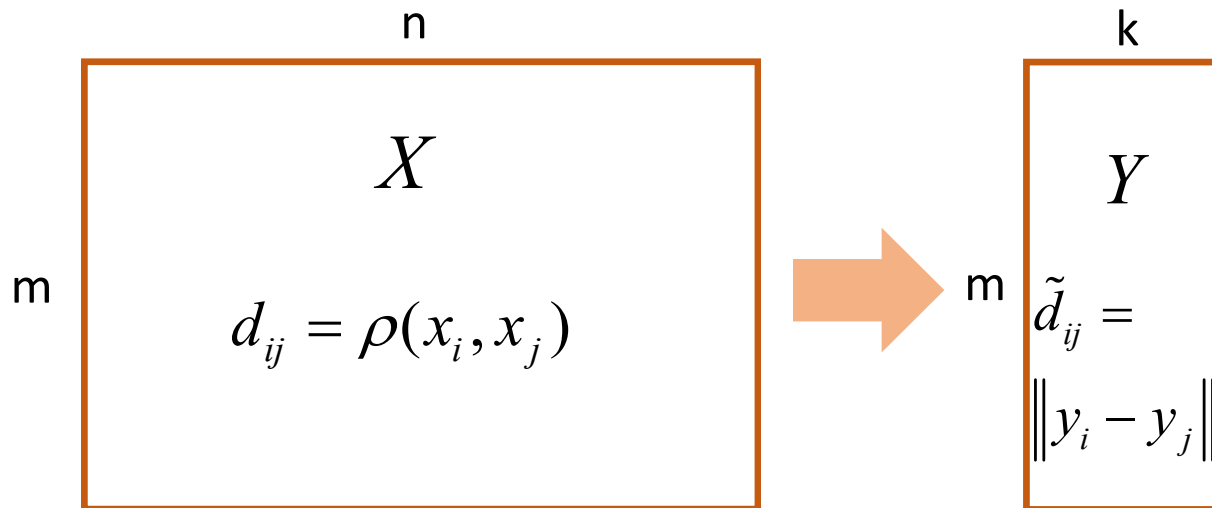
Plot pairwise relationships in a dataset

```
seaborn.pairplot()
```



Multi-Dimensional Scaling (MDS)

Multi-Dimensional Scaling arranges the low-dimensional points so as to minimize the discrepancy between the pairwise distances in the original space and the pairwise distances in the low-dimensional space.



ρ is any metrics

$\|...\|$ is euclidian distance

Given a distances between each pair of objects in a set X , and a chosen number of dimensions, k , an MDS algorithm places each object into k -dimensional space such that the between-object distances are preserved as well as possible.

Multi-Dimensional Scaling (MDS)

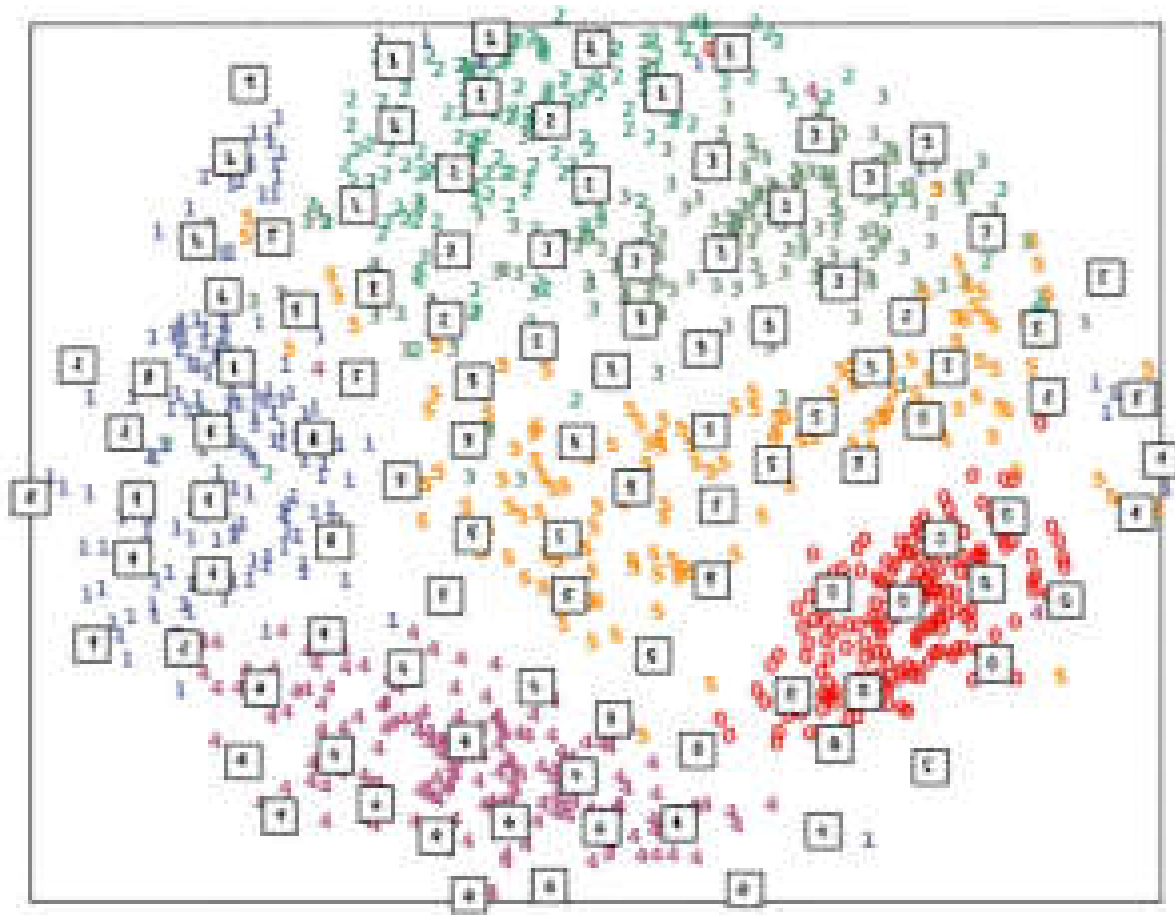
Optimization objective:

$$\sum_{i < j}^m \left(\|y_i - y_j\| - \rho(x_i, x_j) \right)^2 \rightarrow \min_{y_1, \dots, y_m} \quad (\text{SMACOF optimization algorithm})$$

Non-linear MDS is slow to optimize and also gets stuck in different local optima

```
sklearn.manifold.MDS(n_components=2, max_iter=300)
```

MDS: MNIST dataset (28x28 -> 2)

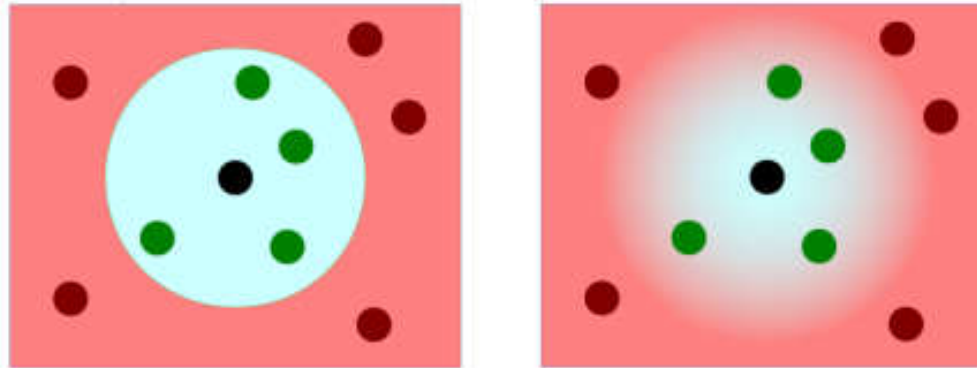


MDS method
focuses on
preserving
distances (not easy
when $n \gg k$)

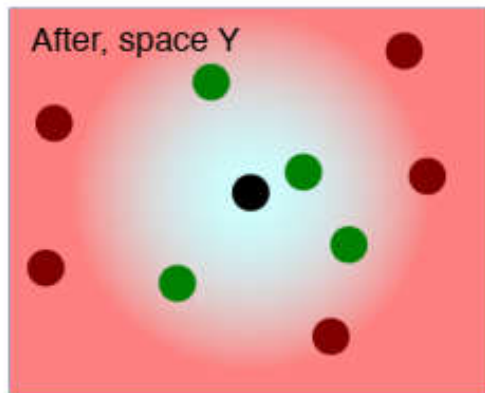
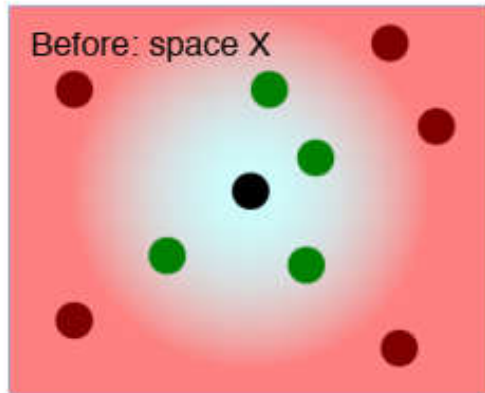
An alternative idea
is preserving
neighborhoods

Dimensionality reduction: preserving neighborhoods

- Neighbors are an important notion in data analysis, e.g. social networks, friends, twitter followers...
- Object nearby (in a metric space) are considered neighbors
- Consider **hard neighborhood** and **soft neighborhood**
- Hard: each point is a neighbor (green) or a non-neighbor (red)
- Soft: each point is a neighbor (green) or a non-neighbor (red) with some weight



Stochastic neighbor embedding (SNE)



$$p(j|i) = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma^2)}$$

Probabilistic input neighborhood

σ is hyperparameter

$$q(j|i) = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}$$

Probabilistic output neighborhood

Stochastic neighbor embedding (SNE)

Kullback-Leibler divergence

$$D(p \parallel q) = H(p, q) - H(p) = \sum_{x \in \Omega} p(x) \log \frac{p(x)}{q(x)}$$

$$H(X) = H(p) = \sum_{x \in \Omega} -p(x) \log p(x) \quad \Omega = \text{all possible values}$$

$$H(p, q) = E_p[-\log q(x)] = -\sum_{x \in \Omega} p(x) \log q(x) \quad (\text{cross-entropy})$$

Properties:

- $D(p \parallel q) \geq 0$
- $D(p \parallel q) \neq D(q \parallel p)$
- $D(p \parallel q) = 0$ if $p = q$

$$H(p, q) \geq H(p)$$

SNE: minimize the KL divergence using gradient descent

$$\text{Cost} = \sum_i KL(P_i \parallel Q_i) = \sum_i \sum_j p(j|i) \log \frac{p(j|i)}{q(j|i)}$$

t-distributed SNE (t-SNE)

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2\sigma_i^2)}$$

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

Probabilistic **input** neighborhood:

Probability to be picked as a neighbor in space X (input coordinates)

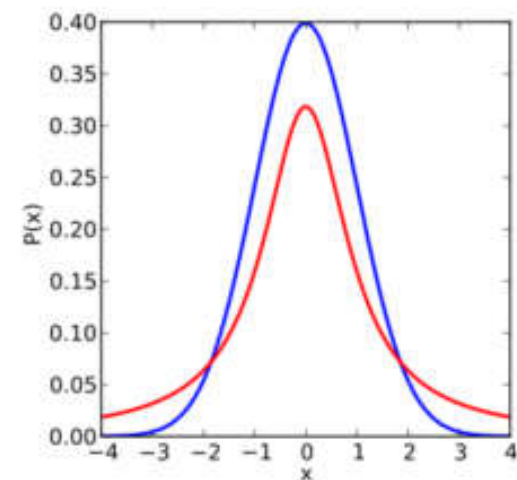
$$q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||y_k - y_l||^2)^{-1}}$$

Probabilistic **output** neighborhood:

Probability to be picked as a neighbor in space Y (display coordinates)

$$\text{Cost} = KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Visualizing Data using t-SNE,
2008, L.Maaten&G.Hinton



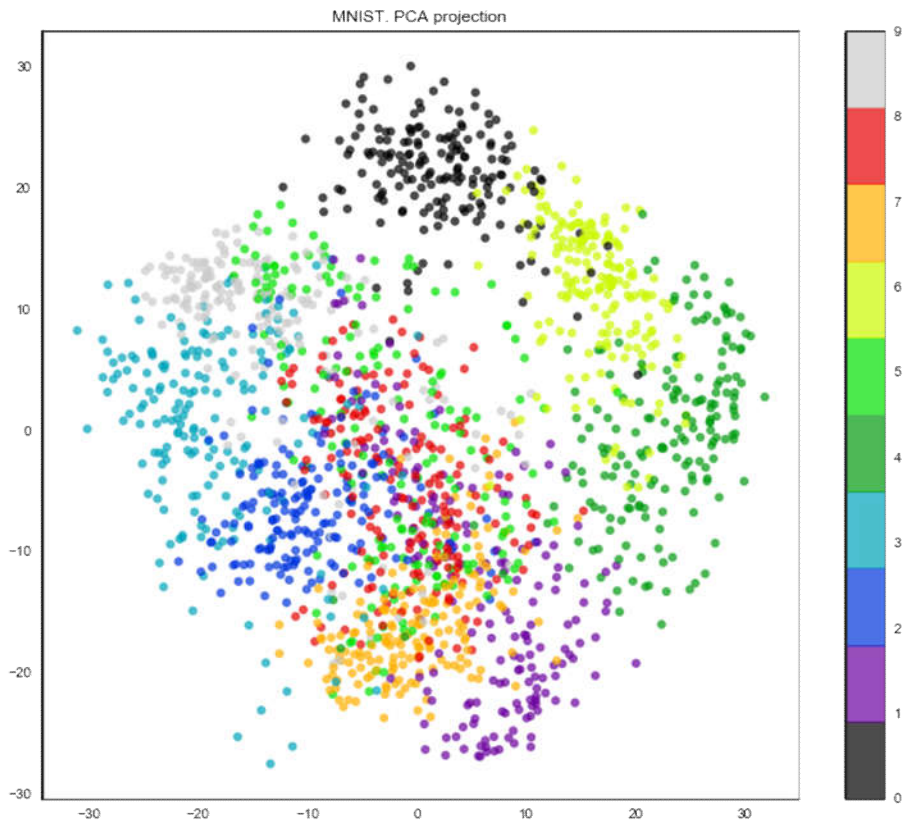
Blue: normal distribution
Red: t-distribution (heavy-tailed)

MNIST (8x8) Example

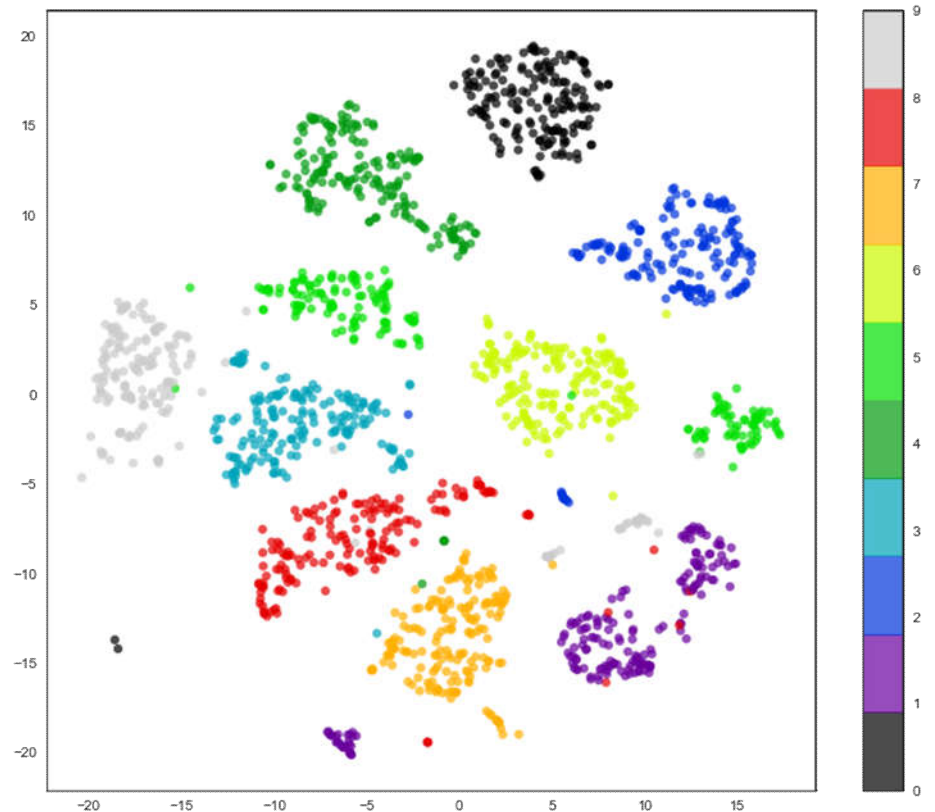
```
digits = sklearn.datasets.load_digits()
```

<https://habr.com/ru/company/ods/blog/325654/>

```
sklearn.manifold.TSNE(n_components=2)
```



PCA



t-SNE

References

https://sebastianraschka.com/Articles/2014_python_lda.html

<https://habr.com/ru/company/ods/blog/325654/>

t-SNE

Original t-SNE paper: Visualizing Data using t-SNE, 2008, L.Maaten&G.Hinton

<https://www.oreilly.com/learning/an-illustrated-introduction-to-the-t-sne-algorithm>

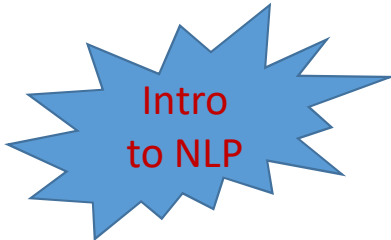
<https://habr.com/ru/post/267041/>

Data Analysis

Lecture 8: Introduction to Natural Language Processing (NLP)

Dr. Nataliya K. Sakhnenko

Roadmap

Basic Machine Learning		Deep Learning
Supervised	Unsupervised	
☐ Regression (linear and polynomial) ☐ Classification ✓ Logistic regression, KNN, Decision Tree, SVM, NB ✓ Ensemble methods: Random Forest, AdaBoost, Gradient Boost	☐ Clustering (k-means, EM, Hierarchical clustering, DBSCAN) ☐ Dimensionality reduction (PCA, LDA, MDS, t-SNE) 	☐ MLP ☐ CNN ☐ RNN ☐ Generative models: GANs, VAEs, NFs ☐ Attention and Transformers ☐ Intro to RL

Natural language processing

Natural language processing (NLP) is a subfield of linguistic, computer science, and artificial intelligence concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of natural language data.

[Wikipedia](#)



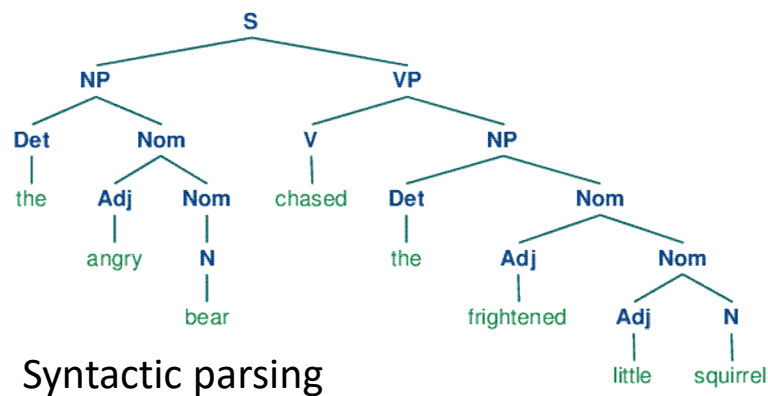
Natural language processing (NLP) tasks

- Semantic search (Google search)
- Sentiment analysis (find a rating based on review)
- Machine translation (Google translate)
- Document classification (clustering documents)
- Question answering (chat bots)



Text analysis problems

Syntax	Semantics
Part-of-speech tagging	Language models
Morphological segmentation	Information retrieval
Stemming and lemmatization	Machine translation
Sentence boundary disambiguation	Sentiment analysis
Named entity recognition	Question answering
Syntactic parsing	Text generation



Top Python NLP libs

- [NLTK](#) (Natural Language Toolkit) is used for such tasks as tokenization, lemmatization, stemming, parsing, POS tagging, etc. This library has tools for almost all NLP tasks.
- [Scikit-learn](#) provides a large library for machine learning. The tools for text preprocessing are also presented here.
- [Gensim](#) is the package for topic and vector space modeling, document similarity.

<https://medium.com/activewizards-machine-learning-company/comparison-of-top-6-python-nlp-libraries-c4ce160237eb>

NLTK Exploration

Text preprocessing

- **NLTK:** natural language toolkit
- Open source library in Python
- Has support for most NLP tasks
- Over 50 corpora

Basic Preprocessing Text

- Sentence Tokenization
- Word Tokenization
- Text Lemmatization and Stemming
- Stop Words
- Regex

✓ Tokenization

Sentence tokenization (sentence segmentation) is dividing a string into its component sentences

```
sentences = nltk.sent_tokenize(text)
```

Word tokenization (word segmentation) is dividing a string of written language into its component words

```
words= nltk.word_tokenize(sentence)
```

Tokenization

```
from nltk.tokenize import word_tokenize
```

```
text = "Data science is a multi-disciplinary field  
that uses scientific methods, processes,  
algorithms and systems to extract knowledge and  
insights from structured and unstructured data."
```

```
words = word_tokenize(text)
```

```
['Data', 'science', 'is', 'a', 'multi-disciplinary',  
'field', 'that', 'uses', 'scientific', 'methods', ',',  
'processes', ',', 'algorithms', 'and', 'systems', 'to',  
'extract', 'knowledge', 'and', 'insights', 'from',  
'structured', 'and', 'unstructured', 'data', '.']
```

Text preprocessing: lemmatization and stemming

- ✓ Stemming and lemmatization are special cases of **normalization**.

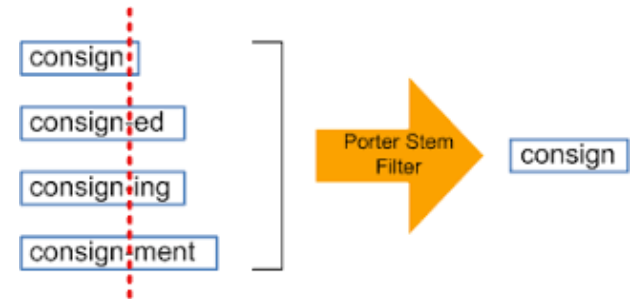
Examples:

am, are, is => be

dog, dogs, dog's, dogs' => dog

Lemmatization

was → (to) be
better → good
meeting → meeting



- **Stemming** usually refers to a heuristic process that chops off the ends of words in the hope of achieving this goal correctly most of the time
- **Lemmatization** usually use a vocabulary and morphological analysis of words, normally aiming to remove inflectional endings only and to return the base or dictionary form of a word, which is known as the lemma.

Stemming and Lemmatization

```
from nltk.stem import PorterStemmer, WordNetLemmatizer  
lemmatizer = WordNetLemmatizer()  
stemmer = PorterStemmer()
```

```
text = 'list lists listed listing listings'
```

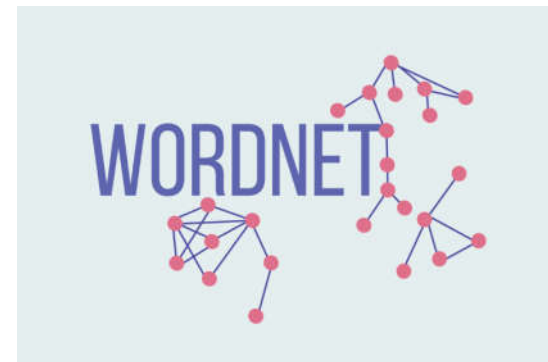
```
words = word_tokenize(text)
```

```
[stemmer.stem(t) for t in words]
```

```
['list', 'list', 'list', 'list', 'list']
```

```
[lemmatizer.lemmatize(t) for t in words]
```

```
['list', 'list', 'listed', 'listing', 'listing']
```



WordNet is a lexical database of semantic relations between words in more than 200 languages

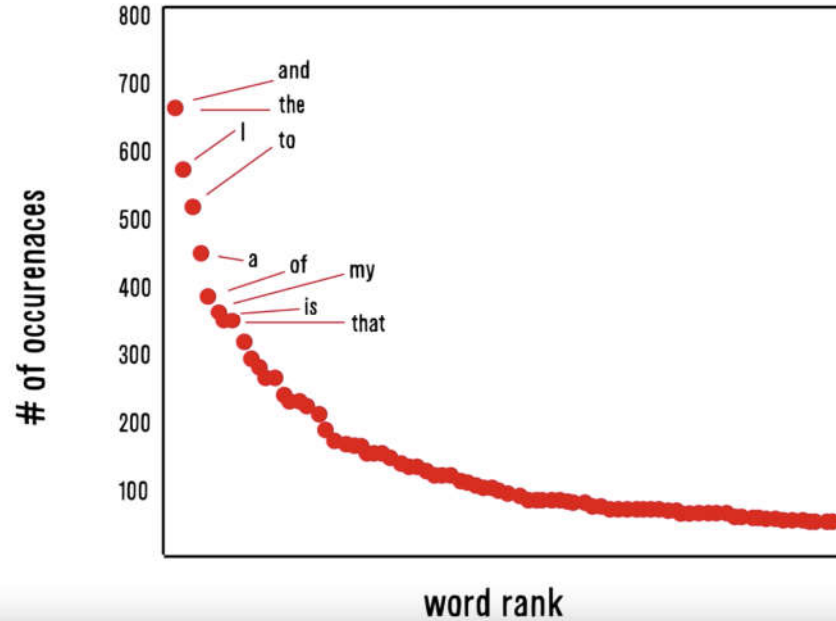
Zipf's law (1949)

Zipf's law: the frequency of any word is **inversely proportional** to its rank in the **frequency table** .

For example, in the **Brown Corpus** [60-70s] of American English text, the word **the** is the most frequently occurring word

$$\text{Freq} * \text{Rank} \sim \text{Const}$$

Romeo and Juliet



Text preprocessing: stop words

Stop words **usually** refer to the **most common words** such as “**and**”, “**the**”, “**a**” etc. These words can add a lot of **noise to data**.

There is **no single universal list** of stopwords, it can change depending on the application.

```
from nltk.corpus import stopwords  
stop_words = set(stopwords.words("english"))
```



Stop words

```
from nltk.corpus import stopwords  
stop_words = stopwords.words('english')  
stop_words[:8]
```

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves']
```

```
text = "This is a sample sentence, showing off the stop words  
filtration."
```

```
words = word_tokenize(text)  
filtered_sentence = [w for w in words if not w in stop_words]
```

```
['This', 'sample', 'sentence', ',', 'showing', 'stop', 'words',  
'filtration', '.']
```

Text preprocessing: regex

A **regular expression**, **regex**, or **regexp** is a sequence of characters that define a **search pattern**.

```
import re
```

- . - match **any character except newline**
- \w - match **word**
- \d - match **digit**
- \s - match **whitespace**
- \W - match **not word**
- \D - match **not digit**
- \S - match **not whitespace**
- [abc] - match **any** of a, b, or c
- [a-g] - match a character **between** a & g
- [^abc] - **not** match a, b, or c

• [RegEx]*

We can use regex to apply **additional filtering** to our text. For example, we can remove all the non-words characters. In many cases, we don't need the punctuation marks and it's easy to remove them with regex.

POS tagging (part of speech)

- Recall high school grammar: nouns, verbs, adjectives, ...
- Many more tags than just these

```
text = 'Bob is great'  
words = word_tokenize(text)  
nltk.pos_tag(words)
```

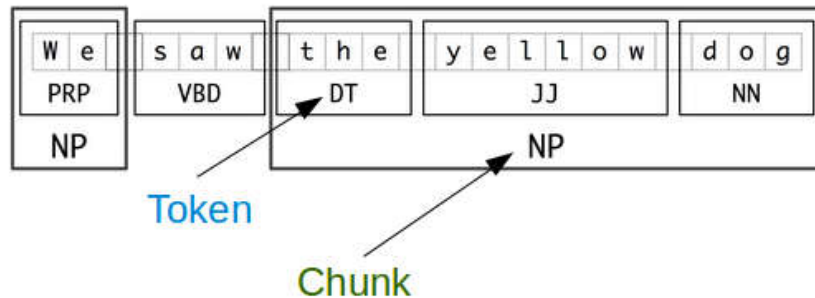
```
[('Bob', 'NNP'), ('is', 'VBZ'), ('great', 'JJ')]
```

```
nltk.help.upenn_tagset('NNP')
```

NNP: noun, proper, singular

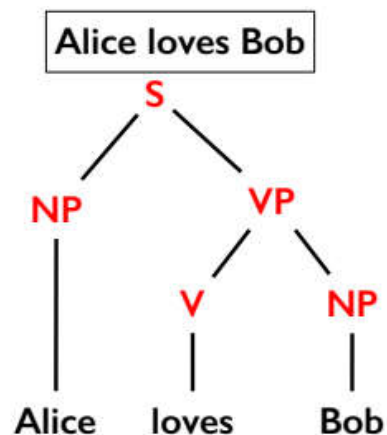
Tag	Word class	Tag	Word class	Tag	Word class
CC	Conjunction	JJ	Adjective	PRP	Pronoun
CD	Cardinal	MD	Modal	RB	Adverb
DT	Determiner	NN	Noun	SYM	Symbol
IN	Preposition	POS	Possessive	VB	Verb

What else?



- ✓ Chunking is to analyzing a sentence to identify the constituents (noun groups, verbs, verb groups, etc.). However, it does not specify their internal structure, nor their role in the main sentence.

- ✓ Parsing sentence structure



- ✓ Named Entity Recognition

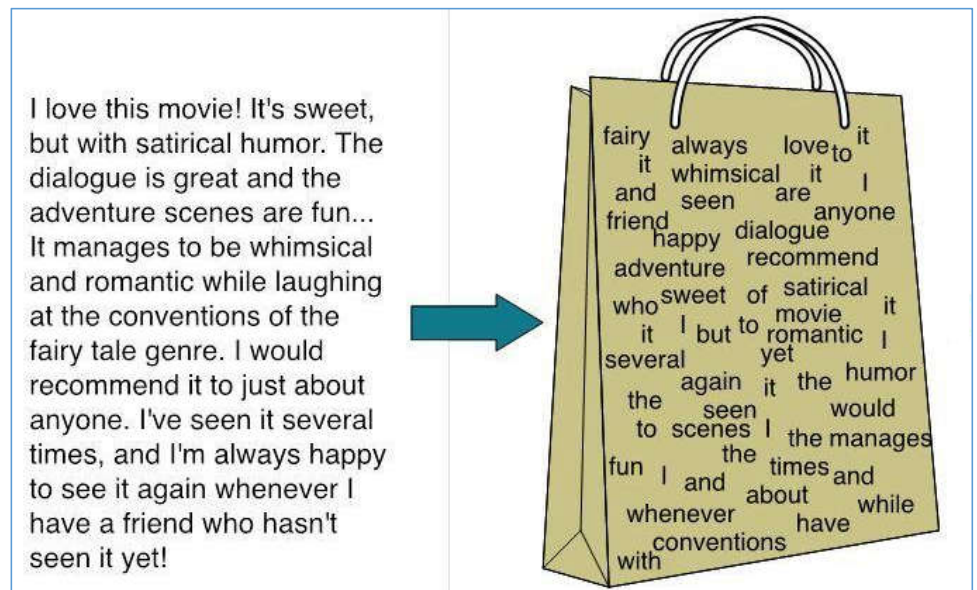


<http://www.nltk.org/book>

Text to Vector Representations

Bag-of-words

A **bag-of-words** model describes the occurrence of each word within a document.



To use this model, we need to:

- Design a **vocabulary** of known words (also called **tokens**)
- Choose a **measure of the presence** of known words
- Any information about **the order** or **structure** of words is **discarded**.

N-grams

- unigram (1-gram):

a	swimmer	likes	swimming	thus	he	swims
---	---------	-------	----------	------	----	-------

- bigram (2-gram):

a swimmer	swimmer likes	likes swimming	swimming thus	...
-----------	---------------	----------------	---------------	-----

- trigram (3-gram):

a swimmer likes	swimmer likes swimming	likes swimming thus	...
-----------------	------------------------	---------------------	-----

A **bag-of-n-grams** model represents a text document as an unordered collection of its n-grams.

Word frequency



```
from sklearn.feature_extraction.text import CountVectorizer
```

```
corpus = [ 'This is the first document.', 'This document is the second  
document.', 'And this is the third one.', 'Is this the first document?']
```

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(corpus)  
vectorizer.get_feature_names()
```

CountVectorizer: convert a collection of text documents to a matrix of token counts

```
['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third', 'this']
```

```
X.toarray()
```

```
[[0 1 1 1 0 0 1 0 1]  
 [0 2 0 1 0 1 1 0 1]  
 [1 0 0 1 1 0 1 1 1]  
 [0 1 1 1 0 0 1 0 1]]
```

```
sklearn.feature_extraction.text.CountVectorizer  
(lowercase=True, stop_words=None, token_pattern=  
'(?u)\b\w\w+\b', ngram_range=(1, 1), analyzer='w  
ord', max_df, min_df, vocabulary=None)
```

TF-IDF

One problem with **scoring word frequency** is that the most frequent words in the document start to have the highest scores. These frequent words may not contain as much “**informational gain**” to the model compared with some rarer and domain-specific words.

TF-IDF, short for **term frequency-inverse document frequency**

$$TF(\mathbf{term}) = \frac{\text{Number of times } \mathbf{term} \text{ appears in a document}}{\text{Total number of items in the document}}$$

$$IDF(\mathbf{term}) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents with } \mathbf{term} \text{ in it}}\right)$$

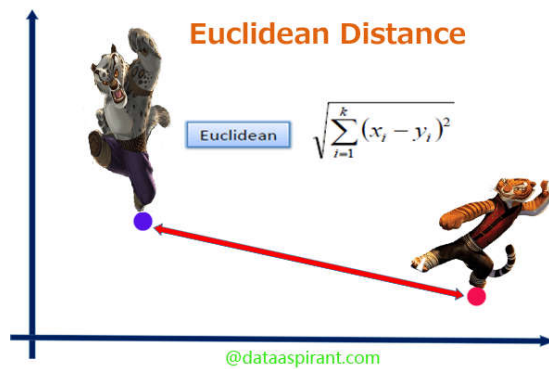
$$TFIDF(\mathbf{term}) = TF(\mathbf{term}) * IDF(\mathbf{term})$$

Parameters are similar to CounVectorizer

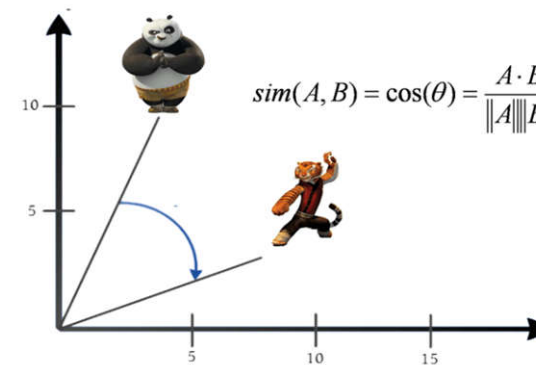
`sklearn.feature_extraction.text.TfidfVectorizer()`

Distance Measurement in Text Mining

1) Euclidean Distance

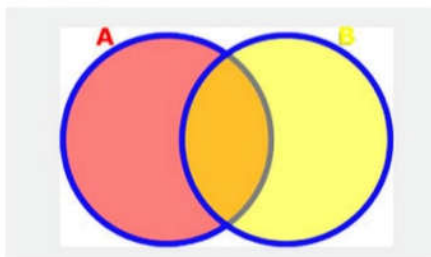


2) Cosine Distance



3) Jaccard Similarity

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$



Jaccard Similarity



Examples of Text Classification

Examples of text classification

- ✓ Topic identification (text classification): Is this news article about Politics, Sports, or Technology?
- ✓ Spam detection: Is this email a spam or not?
- ✓ Sentiment analysis: Is this movie review positive or negative?



Text classification example

```
from sklearn.datasets import fetch_20newsgroups
train_all = fetch_20newsgroups(subset='train')
train_all.target_names
```

The 20 Newsgroups data set is a collection of approximately 20,000 newsgroup documents, partitioned evenly across 20 different newsgroups.

```
['alt.atheism', 'comp.graphics', 'comp.os.ms-windows.misc',
'comp.windows.x', 'misc.forsale', 'rec.autos', 'rec.motorcycles',
'sci.electronics', 'sci.med', 'sci.space',...]
```

```
categories = ['alt.atheism', 'soc.religion.christian', 'comp.graphics', 'sci.med']
data_train = fetch_20newsgroups(subset='train', categories=categories)
```

https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html

Example of text classification (cont)

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(max_df=500, min_df=10)
tfidf_train = vectorizer.fit_transform(data_train.data)
vectorizer.get_feature_names()[10]
```

```
`1993apr5'
```

```
print (tfidf_train.shape) (2257, 4726)
```

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(tfidf_train, simple_dataset.target)
```

```
data_test = fetch_20newsgroups(subset='test', categories=categories, shuffle=True)
tfidf_test = vectorizer.transform(data_test.data)
predicted = clf.predict(tfidf_test)
from sklearn.metrics import accuracy_score
accuracy_score(predicted, data_test.target)
```

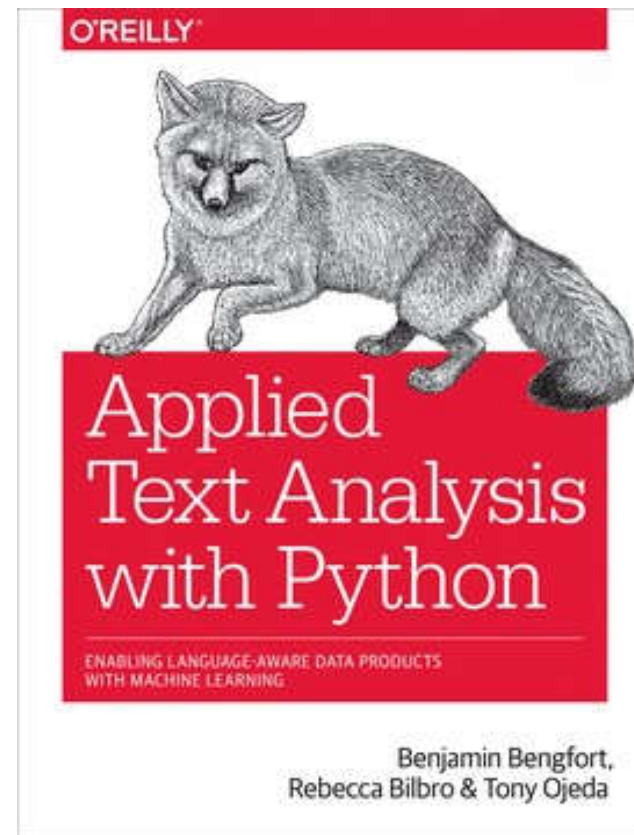
```
0.8874833555259654
```

How we can improve the result?

References

B.Bengfor, R. Bilbro & T.Ojeda, “Applied Text Analysis with Python”

<https://www.coursera.org/lecture/python-text-mining/introduction-to-text-mining-y5C24>

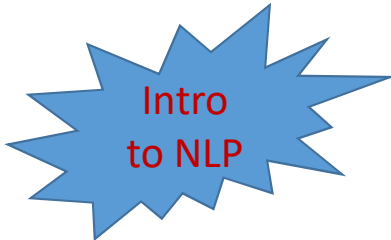


Data Analysis

Lecture 9: Introduction to Natural Language Processing (NLP)

Dr. Nataliya K. Sakhnenko

Roadmap

Basic Machine Learning		Deep Learning
Supervised	Unsupervised	
☐ Regression (linear and polynomial) ☐ Classification ✓ Logistic regression, KNN, Decision Tree, SVM, NB ✓ Ensemble methods: Random Forest, AdaBoost, Gradient Boost	☐ Clustering (k-means, EM, Hierarchical clustering, DBSCAN) ☐ Dimensionality reduction (PCA, LDA, MDS, t-SNE) 	☐ MLP ☐ CNN ☐ RNN ☐ Generative models: GANs, VAEs, NFs ☐ Attention and Transformers ☐ Intro to RL

Recap: NLP

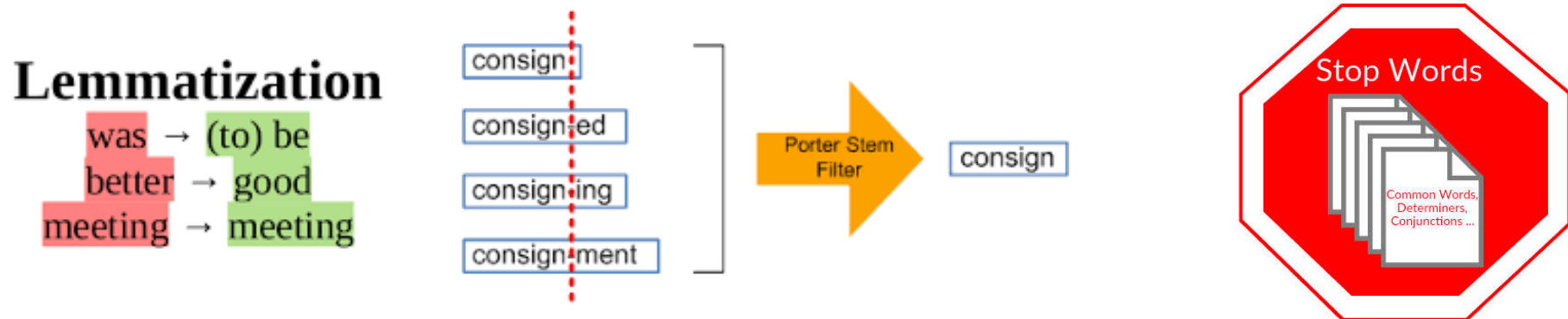
Natural language processing (NLP) is a subfield of linguistic, computer science, and artificial intelligence concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of natural language data.



Recap: Text Basic Preprocessing

- Sentence Tokenization
- Word Tokenization
- Text Lemmatization and Stemming
- Stop Words
- Regex

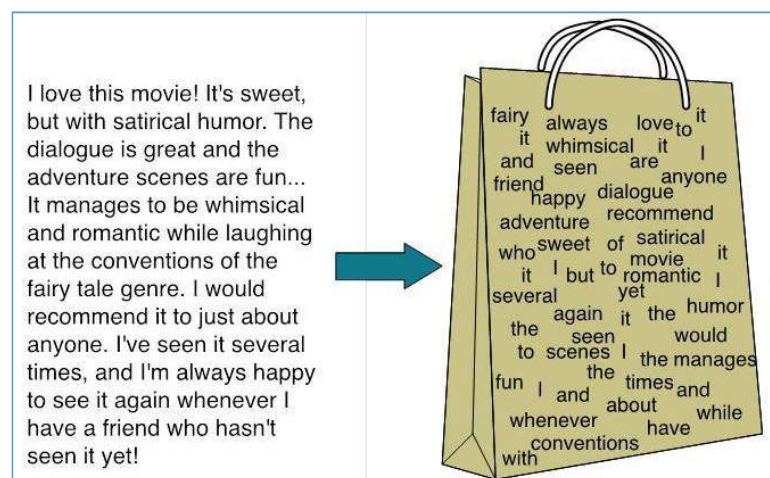
- **NLTK**: natural language toolkit



Stop words **usually** refer to the **most common words** such as “and”, “the”, “a” etc. These words can add a lot of **noise to data**.

Recap: Text to Vector Representations

Bag-of-words model



Bag-of-n-grams model

- unigram (1-gram):

a	swimmer	likes	swimming	thus	he	swims
---	---------	-------	----------	------	----	-------

- bigram (2-gram):

a swimmer	swimmer likes	likes swimming	swimming thus	...
-----------	---------------	----------------	---------------	-----

- trigram (3-gram):

a swimmer likes	swimmer likes swimming	likes swimming thus	...
-----------------	------------------------	---------------------	-----

Recap: Word frequency



```
from sklearn.feature_extraction.text import CountVectorizer
```

```
corpus = [ 'This is the first document.', 'This document is the second  
document.', 'And this is the third one.', 'Is this the first document?']
```

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(corpus)  
vectorizer.get_feature_names()
```

CountVectorizer: convert a collection of text documents to a matrix of token counts

```
['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third', 'this']
```

```
X.toarray()
```

```
[[0 1 1 1 0 0 1 0 1]  
 [0 2 0 1 0 1 1 0 1]  
 [1 0 0 1 1 0 1 1 1]  
 [0 1 1 1 0 0 1 0 1]]
```

```
sklearn.feature_extraction.text.CountVectorizer  
(lowercase=True, stop_words=None, token_pattern=  
'(?u)\b\w\w+\b', ngram_range=(1, 1), analyzer='w  
ord', max_df, min_df, vocabulary=None)
```

Recap: TF-IDF

One problem with **scoring word frequency** is that the most frequent words in the document start to have the highest scores. These frequent words may not contain as much “**informational gain**” to the model compared with some rarer and domain-specific words.

TF-IDF, short for **term frequency-inverse document frequency**

$$TF(\mathbf{term}) = \frac{\text{Number of times } \mathbf{term} \text{ appears in a document}}{\text{Total number of items in the document}}$$

$$IDF(\mathbf{term}) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents with } \mathbf{term} \text{ in it}}\right)$$

$$TFIDF(\mathbf{term}) = TF(\mathbf{term}) * IDF(\mathbf{term})$$

Parameters are similar to CounVectorizer

`sklearn.feature_extraction.text.TfidfVectorizer()`

Examples of Text Classification

Spam Classifier

<https://www.kaggle.com/uciml/sms-spam-collection-dataset>

```
import pandas as pd
spam_data = pd.read_csv('spam.csv')
spam_data.head()
```

	text	target
0	Go until jurong point, crazy.. Available only ...	ham
1	Ok lar... Joking wif u oni...	ham
2	Free entry in 2 a wkly comp to win FA Cup fina...	spam
3	U dun say so early hor... U c already then say...	ham
4	Nah I don't think he goes to usf, he lives aro...	ham

Only 13% of data spam_data are spam

WordCloud visualization of spam and ham data



Spam Classifier (cont)

Recap: Bayesian Classification

Problem statement:

- Given features $X_1, X_2, \dots, X_n$
- Predict a label Y

Bayes Rule

$$\begin{array}{ccc} \text{Posterior Probability} & \text{Likelihood} & \text{Prior} \\ \downarrow & \downarrow & \downarrow \\ P(Y|X_1, \dots, X_n) & = & \frac{P(X_1, \dots, X_n|Y)P(Y)}{P(X_1, \dots, X_n)} \end{array}$$

The Naïve Bayes Assumption: Assume that all features are independent given the class label Y

$$P(X_1, \dots, X_n|Y) = \prod_{i=1}^n P(X_i|Y)$$

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

Spam Classifier (cont)

Recap: Naïve Bayes Training

Likelihood: $\Pr(x_i | y)$ for all features x_i and labels y in Y

- Count how many times feature x_i appears in instances labeled as class y
- If there are p instances of class y , and x_i appears in k of those, $\Pr(x_i | y) = k/p$

What happens if $\Pr(x_i | y) = 0$?

- Feature never occurs in documents labeled by y
- But then, the posterior probability $\Pr(y | x_i)$ will be 0!!!

Smooth the parameters!!!

Laplace smoothing:

$$\Pr(x_i | y) = (k + \alpha) / (p + \alpha n)$$

α is the hyperparameter
usually $\alpha = 1$

Spam Classifier (cont)

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix
```

```
vectorizer = CountVectorizer()
X_train_transformed = vectorizer.fit_transform(X_train)
X_test_transformed = vectorizer.transform(X_test)
clf = MultinomialNB(alpha=0.1)
clf.fit(X_train_transformed, y_train)
y_predicted = clf.predict(X_test_transformed)
```

```
from sklearn.naive_bayes import MultinomialNB, GaussianNB, BernoulliNB
```

We can choose distribution that is appropriate for our feature vectors:

If real-valued -> Gaussian

If counts -> Multinomial

If binary -> Bernoulli

Sentiment Analysis

SENTIMENT ANALYSIS



Discovering people opinions, emotions and feelings about
a product or service

Sentiment analysis (also known as **opinion mining** or **emotion AI**) refers to the use of NLP to systematically identify, extract, quantify, and study affective states and subjective information.

Sentiment Analysis, IMDB movies



<http://ai.stanford.edu/~amaas/data/sentiment/>



- ✓ We'll be using a dataset of 50,000 movie reviews taken from IMDb.
- ✓ The data is split evenly with 25k reviews intended for training and 25k for testing your classifier.
- ✓ Each set has 12.5k positive and 12.5k negative reviews.
- ✓ IMDb lets users rate movies on a scale from 1 to 10. To label these reviews the curator of the data labeled anything with ≤ 4 stars as negative and anything with ≥ 7 stars as positive. Reviews with 5 or 6 stars were left out.

Sentiment Analysis, IMDB movies

Details of data downloading and cleaning (NLTK normalization, Regex) is here:

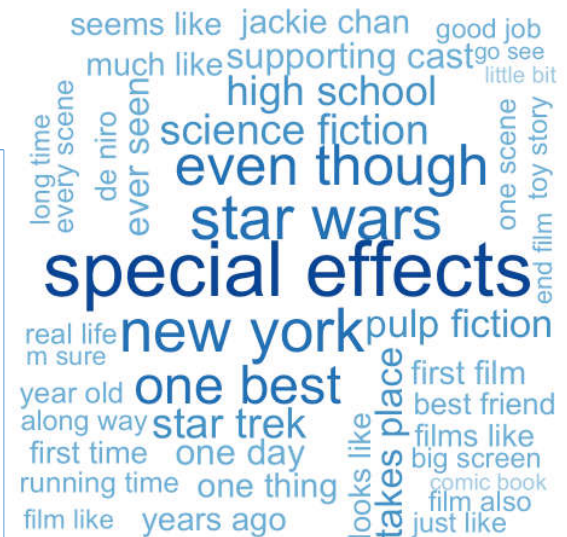
<https://towardsdatascience.com/sentiment-analysis-with-python-part-1-5ce197074184>

<https://towardsdatascience.com/sentiment-analysis-with-python-part-2-4f71e7bde59a>

```
from sklearn.feature_extraction.text import CountVectorizer
ngram_vectorizer = CountVectorizer(ngram_range=(1, 2))
X_train = ngram_vectorizer.fit_transform(reviews_train_clean)
X_test = ngram_vectorizer.transform(reviews_test_clean)
```

After extracting features, we can apply any classification algorithm

Uni-gram and Bi-gram



WordCloud of bi-grams

N-gram language model

N-gram Language model

A language model learns to predict the probability of a sequence of words.

Word ordering:
 $p(\text{the cat is small}) > p(\text{small the is cat})$

There are primarily two types of Language Models:

1. Statistical Language Models: These models use traditional statistical techniques like N-grams

2. Neural Language Models

How it works? Use the previous N-1 words in a sequence to predict the next word



<https://www.analyticsvidhya.com/blog/2019/08/comprehensive-guide-language-model-nlp-python-code/>

N-gram Language model

Compute the product of component conditional probabilities?

$$P(\text{the mythical unicorn}) = P(\text{the}) * P(\text{mythical} | \text{the}) * P(\text{unicorn} | \text{the mythical})$$

But...the *longer* the sequence, the *less likely* we are to find it in a training corpus

$P(\text{Most biologists and folklore specialists believe that in fact the mythical unicorn horns derived from the narwhal})$

What can we do?

Markov assumption: the probability of a word depends only on the probability of a limited history

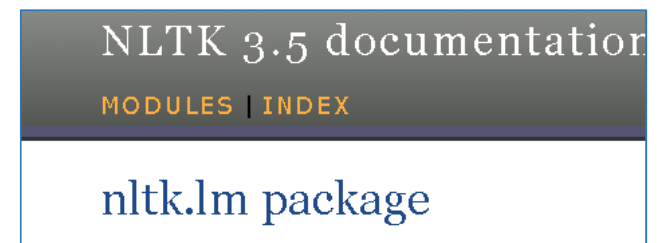
For bigram model, the probability of a sequence is just the product of the conditional probabilities of its bigrams, e.g.

$$P(\text{the, mythical, unicorn}) = P(\text{unicorn} | \text{mythical}) P(\text{mythical} | \text{the}) P(\text{the} | \text{<start>})$$

A simple bi-gram example

- Estimate the likelihood of the sentence **I want to eat Chinese food**.
 - $P(\text{I want to eat Chinese food}) = P(\text{I} \mid \langle \text{start} \rangle) P(\text{want} \mid \text{I}) P(\text{to} \mid \text{want}) P(\text{eat} \mid \text{to}) P(\text{Chinese} \mid \text{eat}) P(\text{food} \mid \text{Chinese}) P(\langle \text{end} \rangle \mid \text{food})$
- What do we need to calculate these likelihoods?
 - Calculate probabilities from a large corpus

$$\begin{array}{ll} \text{Bi-gram} & \Pr(x_i \mid x_{i-1}) = \frac{\text{count}(x_i x_{i-1})}{\text{count}(x_{i-1})} \\ \text{Tri-gram} & \Pr(x_i \mid x_{i-1}, x_{i-2}) = \frac{\text{count}(x_i x_{i-1} x_{i-2})}{\text{count}(x_{i-1} x_{i-2})} \\ \dots & \end{array}$$



N-Gram probabilities come from a training corpus

overly narrow corpus: probabilities don't **generalize**

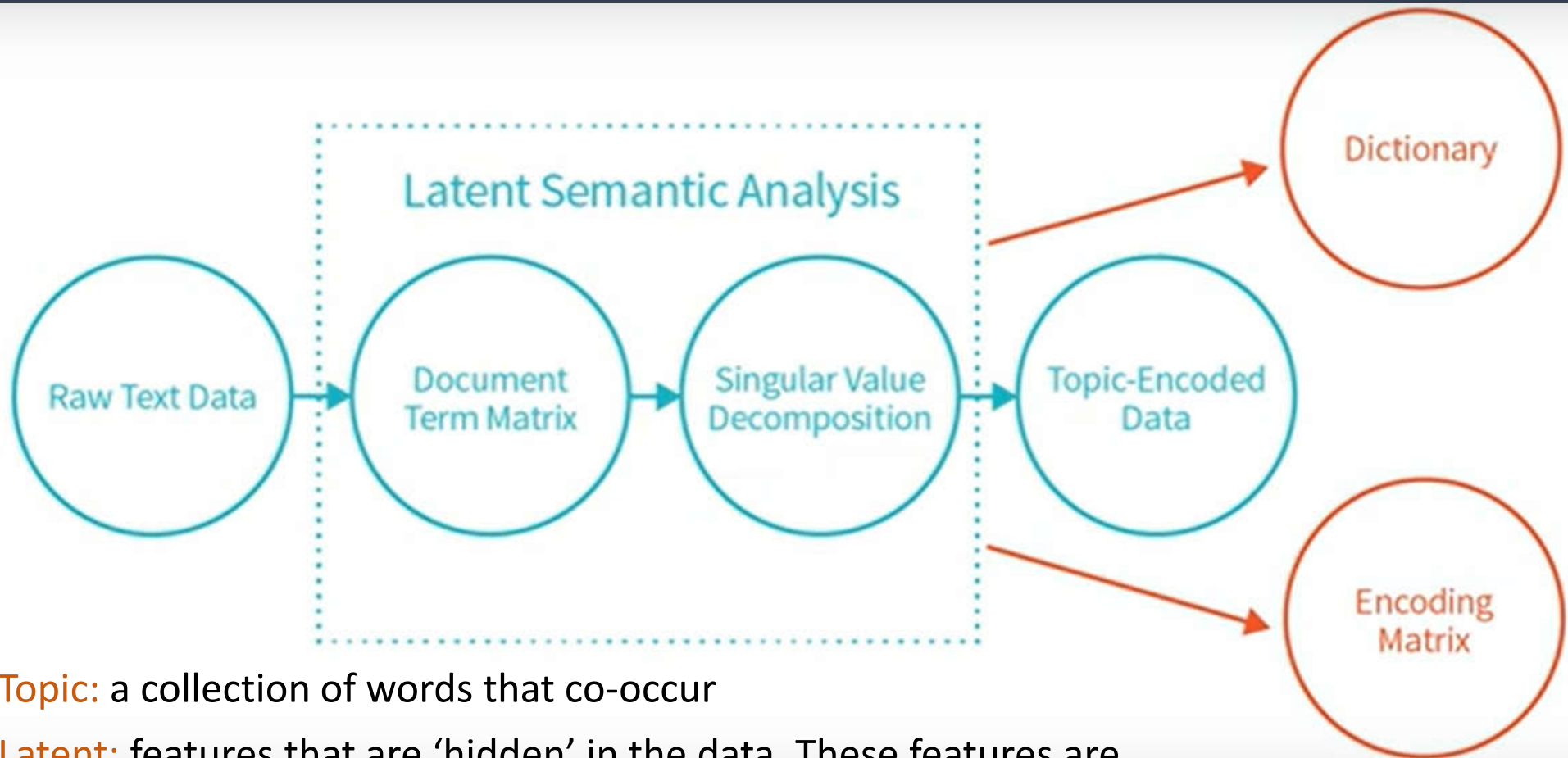
overly general corpus: probabilities don't **reflect task or domain**

Application of language modeling

- ✓ Speech recognition
- ✓ Machine translation
- ✓ Handwriting recognition
- ✓ Information retrieval
- ✓ Text generation in a given style
- ✓ Article spinning

Latent Semantic Analysis (LSA)

Latent Semantic Indexing (LSI) in information Retrieval



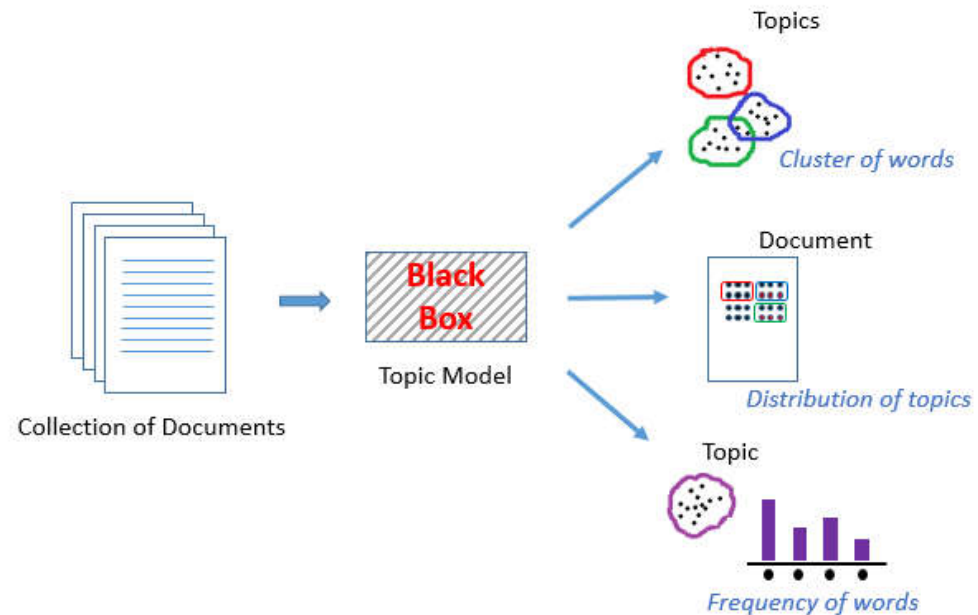
Topic: a collection of words that co-occur

Latent: features that are 'hidden' in the data. These features are essential to the data, but are not original features of the dataset

Topic Modeling

- an unsupervised technique to discover topics across various text documents;
- these topics are abstract in nature, i.e., words which are related to each other form a topic;
- there can be multiple topics in an individual document

Topic model forms clusters of similar and related words which are called topics. These topics have a certain distribution in a document, and every topic is defined by the proportion of different words it contains.



<https://www.analyticsvidhya.com/blog/2018/10/stepwise-guide-topic-modeling-latent-semantic-analysis/>

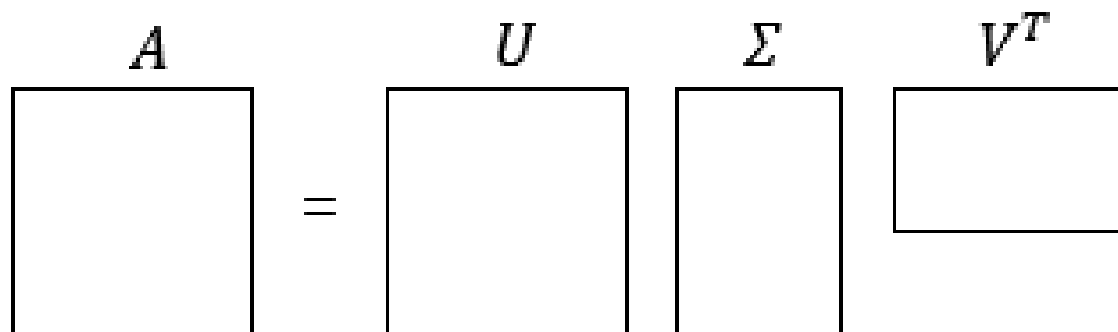
Topic Modeling

- Topic modeling helps in exploring large amounts of text data, finding clusters of words, similarity between documents, and discovering abstract topics.
- Topic modeling is also used in search engines wherein the search string is matched with the results.

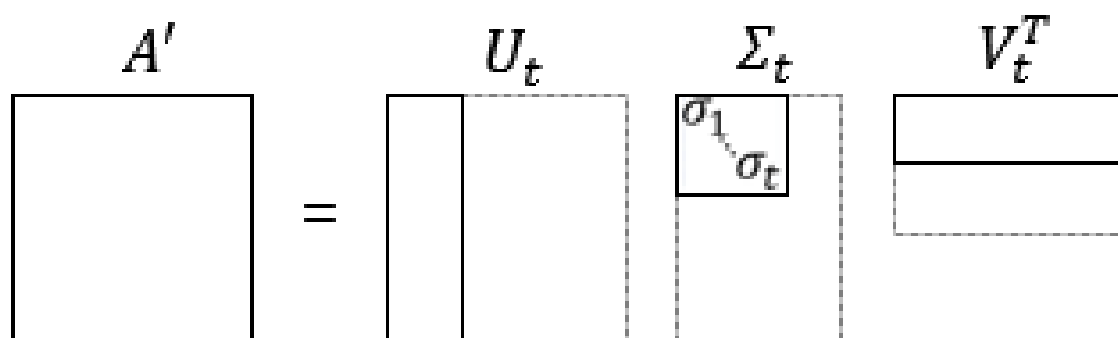


Truncated SVD

Full SVD

$$A = U \Sigma V^T$$


Truncated SVD

$$A' = U_t \Sigma_t V_t^T$$


The truncated SVD is no longer an exact decomposition of the original matrix A , but the approximate matrix A' is the closest approximation to A that can be achieved by a matrix of rank t .

LSA (Example)

```
corp= ['Machine learning is super fun', 'Python is cool', 'Python is great for machine learning', 'I like football', 'Football is great to watch']
```

```
from sklearn.feature_extraction.text import CountVectorizer  
vectorizer = CountVectorizer(stop_words = 'english')  
dtm = vectorizer.fit_transform(corp)
```

Document Term Matrix (dtm):

	cool	football	fun	great	learning	like	machine	python	super	watch
Machine learning is super fun	0	0	1	0	1	0	1	0	1	0
Python is cool	1	0	0	0	0	0	0	1	0	0
Python is great for machine learning	0	0	0	1	1	0	1	1	0	0
I like football	0	1	0	0	0	1	0	0	0	0
Football is great to watch	0	1	0	1	0	0	0	0	0	1

LSA (Example)

```
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import Normalizer
lsa = TruncatedSVD(2)
dtm_lsa = lsa.fit_transform(dtm)
dtm_lsa = Normalizer(copy = False).fit_transform(dtm_lsa)
pd.DataFrame(lsa.components_, index = ['topic_1', 'topic_2'], columns = vectorizer.get_feature_names())
```

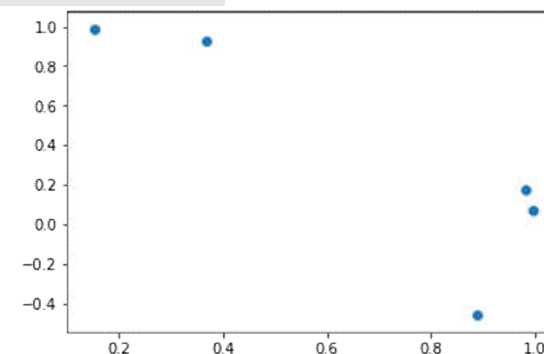
	cool	football	fun	great	learning	like	machine	python	super	watch
topic_1	0.066837	0.115504	0.249972	0.381023	0.53729	0.021798	0.53729	0.354154	0.249972	0.093706
topic_2	0.020739	0.646224	-0.218521	0.440041	-0.18366	0.241044	-0.18366	0.055599	-0.218521	0.405180

Encoding Matrix
(lsa.components_)

```
pd.DataFrame(dtm_lsa, index = example, columns = ['topic_1', 'topic_2'])
```

Topic Encoded
Data
(dtm_lsa)

	topic_1	topic_2
Machine learning is super fun	0.890525	-0.454934
Python is cool	0.983955	0.178419
Python is great for machine learning	0.997496	0.070727
I like football	0.152926	0.988238
Football is great to watch	0.367978	0.929835



LSA (Example)

```
terms = vectorizer.get_feature_names()

for i, comp in enumerate(lsa.components_):
    terms_comp = zip(terms, comp)
    sorted_terms = sorted(terms_comp, key= lambda x:x[1], reverse=True)[:3]
    print("Topic "+str(i)+" : ")
    for t in sorted_terms:
        print(t[0])
```

Topic 1:
learning
machine
great

Topic 2:
football
great
watch

	cool	football	fun	great	learning	like	machine	python	super	watch
topic_1	0.066837	0.115504	0.249972	0.381023	0.53729	0.021798	0.53729	0.354154	0.249972	0.093706
topic_2	0.020739	0.646224	-0.218521	0.440041	-0.18366	0.241044	-0.18366	0.055599	-0.218521	0.405180

LSA (Example)

Document similarity using LSA

Cosine similarity

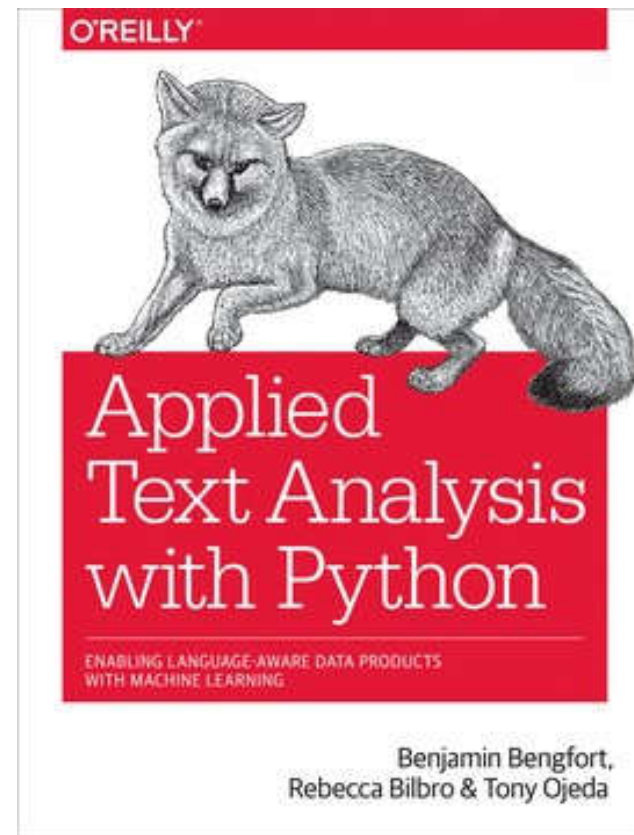
```
similarity = np.asarray(np.asmatrix(dtm_lsa) * np.asmatrix(dtm_lsa).T)  
pd.DataFrame(similarity, index = example, columns = example)
```

	Machine learning is super fun	Python is cool	Python is great for machine learning	I like football	Football is great to watch
Machine learning is super fun	1.000000	0.795067	0.856119	-0.313398	-0.095320
Python is cool	0.795067	1.000000	0.994109	0.326793	0.527974
Python is great for machine learning	0.856119	0.994109	1.000000	0.222438	0.432820
I like football	-0.313398	0.326793	0.222438	1.000000	0.975171
Football is great to watch	-0.095320	0.527974	0.432820	0.975171	1.000000

References

B.Bengfor, R. Bilbro & T.Ojeda, “Applied Text Analysis with Python”

<https://www.coursera.org/lecture/python-text-mining/introduction-to-text-mining-y5C24>

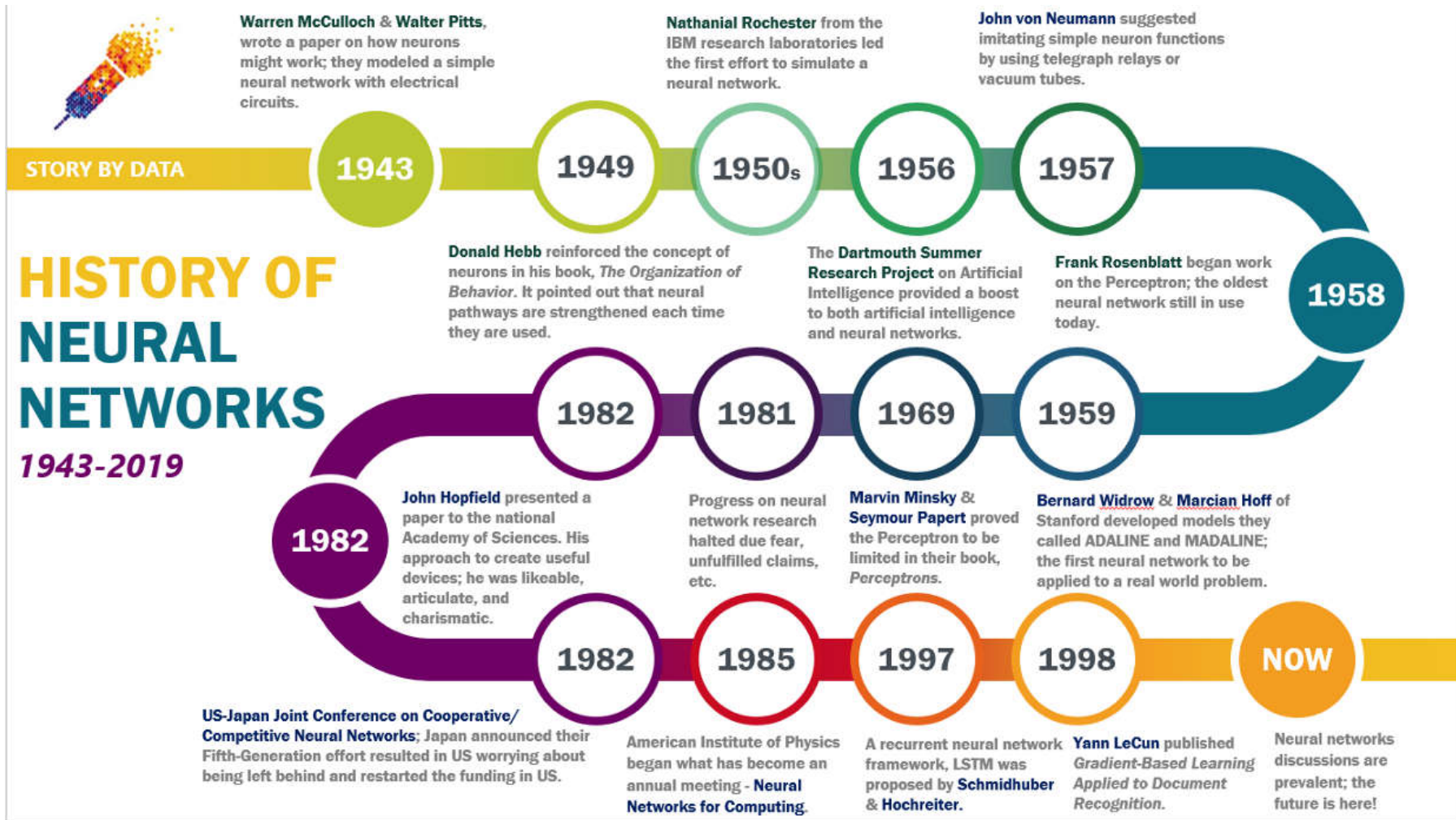


Data Analysis

Lecture 10: Introduction to Artificial Neural Networks (ANNs)

Dr. Nataliya K. Sakhnenko

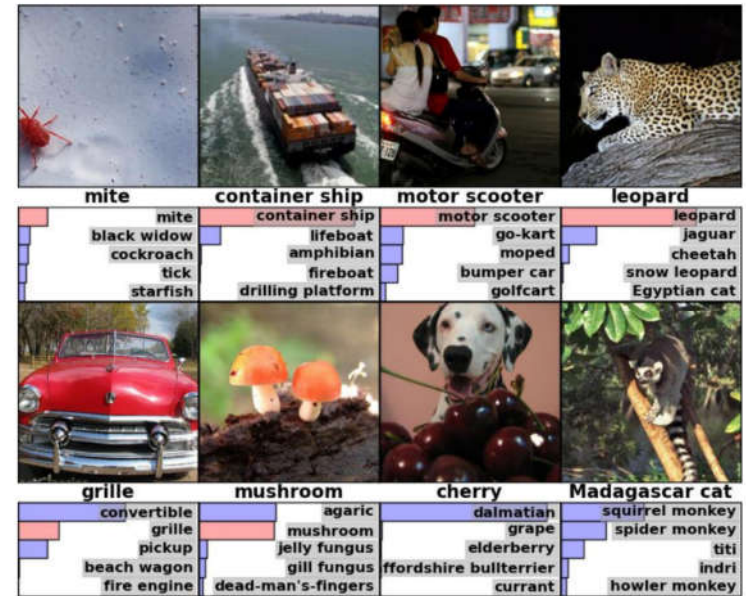
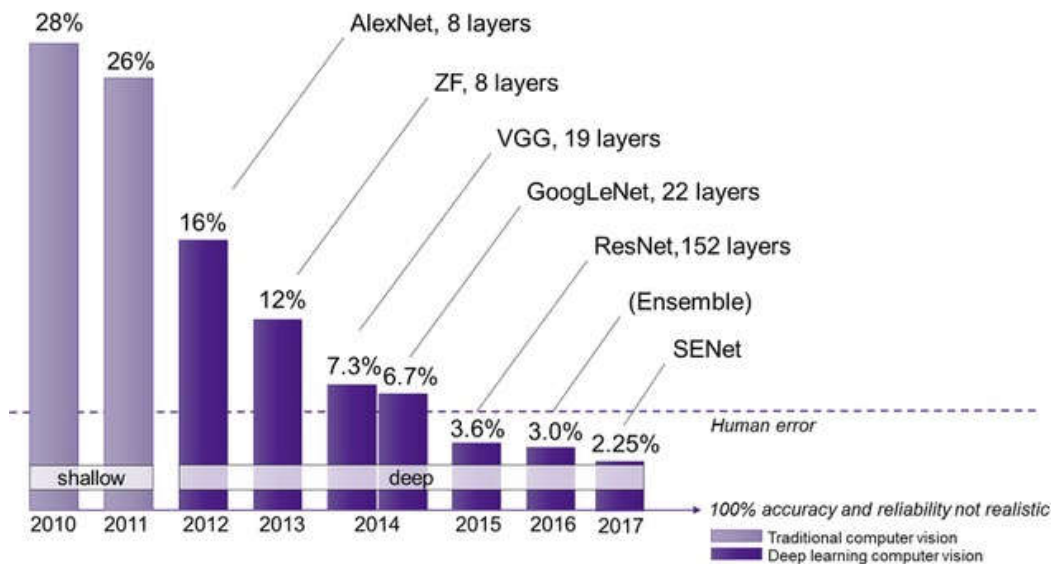
History of Neural Networks



Rise of Deep Learning

Image Recognition

ImageNet Classification Error



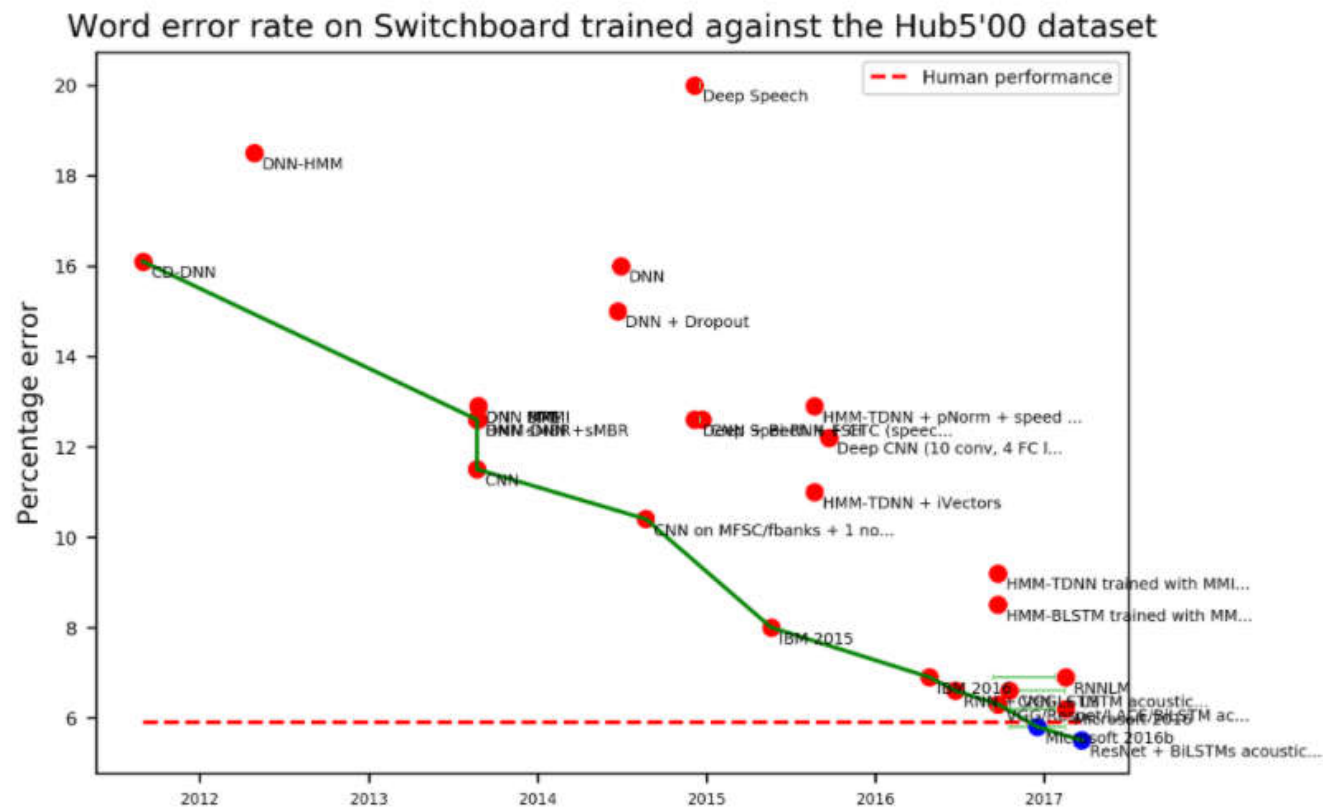
Krizhevsky A., Sutskever I., Hinton G.E. <http://www.cs.toronto.edu/~fritz/absps/imagenet.pdf>



14 million images
20,000 categories

Rise of Deep Learning

Speech Recognition



<https://www.eff.org/ai/metrics>

AI Progress Measurement

Rise of Deep Learning

Gaming: Go / Reinforcement Learning

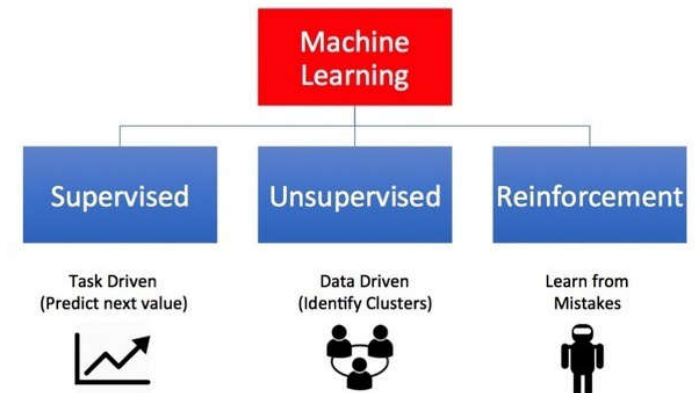
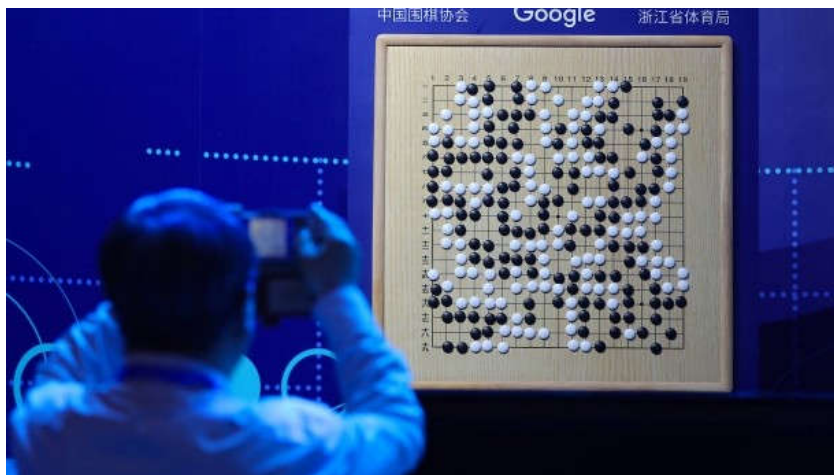
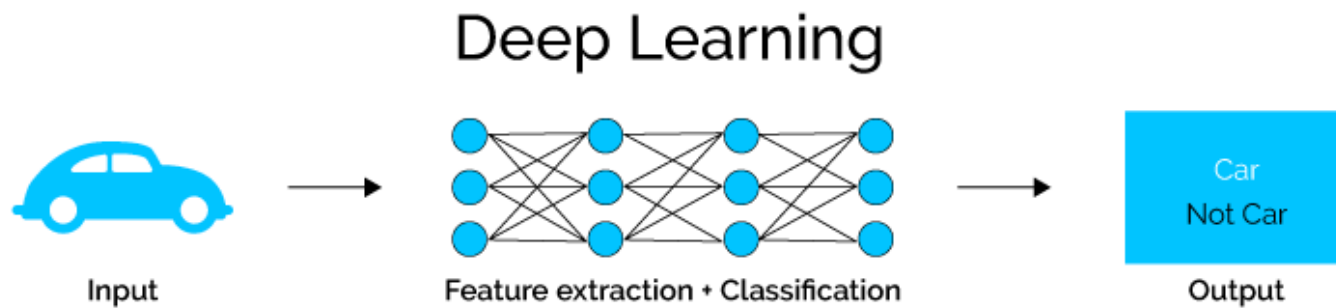
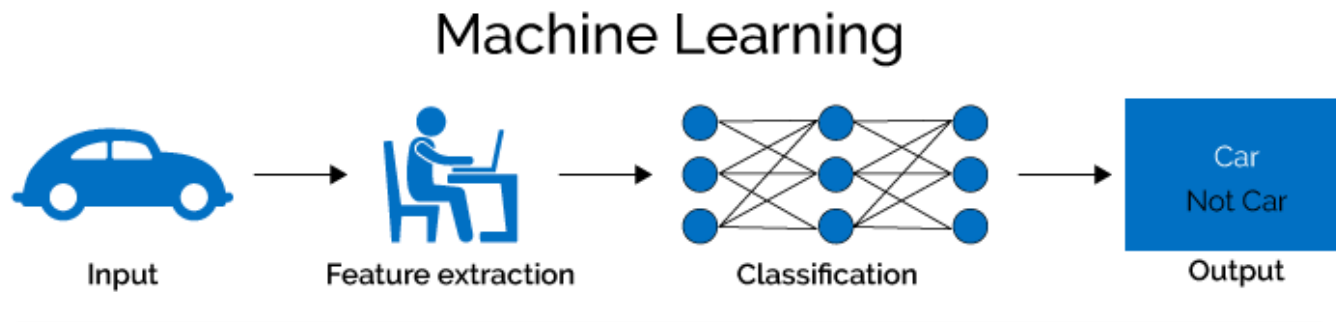


Figure 1 | AlphaGo versus Lee Sedol. In March 2018, the artificial-intelligence program AlphaGo defeated a world Go champion, Lee Sedol.

Basic ML algorithms vs Deep Learning



Turing Award (2018)



Yann LeCun , Geoffrey Hinton, Yoshua Bengio

‘Fathers of AI’ honored with Turing Award, the Nobel Prize of computing

[Yoshua Bengio](#), [Geoffrey Hinton](#), and [Yann LeCun](#) were honored of the 2018 Turing Award for conceptual and engineering breakthroughs that have made deep neural networks a critical component of computing.

- Bengio is Professor at the University of Montreal and Scientific Director at Mila, Quebec’s Artificial Intelligence Institute;
- Hinton is VP and Engineering Fellow of Google, Chief Scientific Adviser of The Vector Institute, and University Professor Emeritus at the University of Toronto;
- LeCun is Professor at New York University and VP and Chief AI Scientist at Facebook.

Biological inspiration

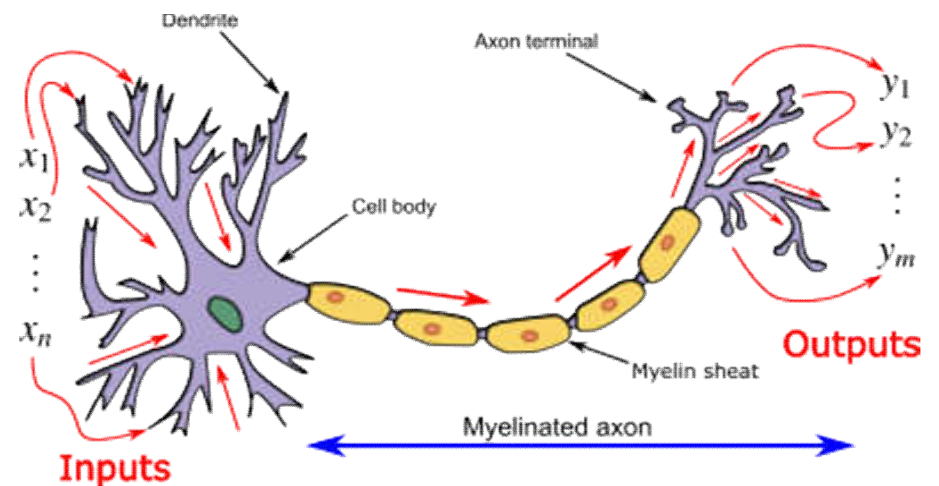
Neural network: information processing paradigm inspired by biological nervous systems, such as our brain

Structure: large number of highly interconnected processing elements (**neurons**) working together

Like people, they learn **from experience** (by example)

- From experience: examples / training data
- Strength of connection between the neurons is stored as a weight-value for the specific connection.
- Learning the solution to a problem = changing the connection weights

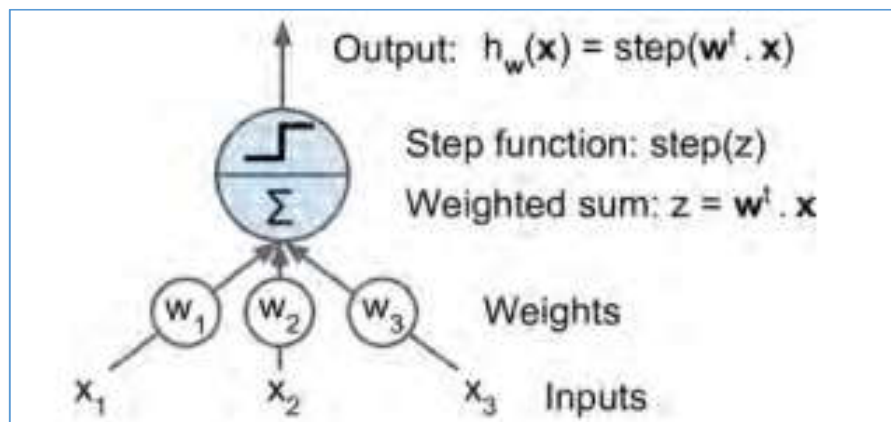
A physical neuron



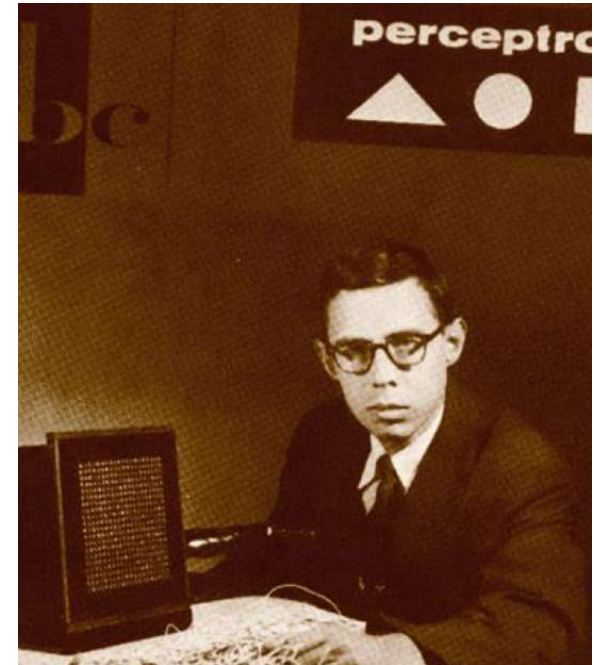
An artificial neuron

Perceptron

The Perceptron (1957, Frank Rosenblatt) is based on linear threshold unit.



$$\sum_i w_i x_i = \mathbf{w}^T \mathbf{x}$$

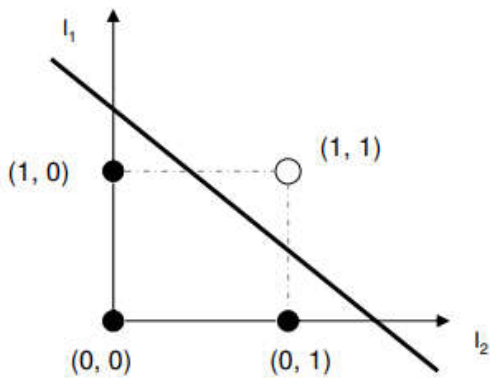


Limitation:

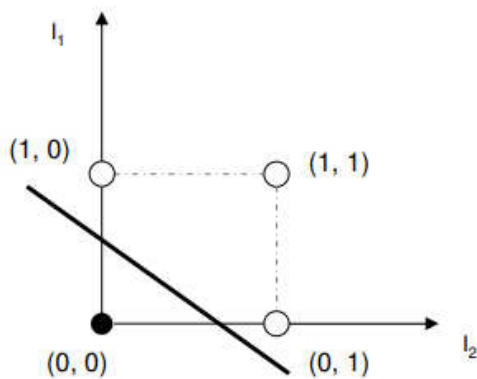
The decision boundary of each output neuron is **linear**, so Perceptron (Single Layer Perceptron) is incapable of learning complex patterns.

Logical operations with perceptron

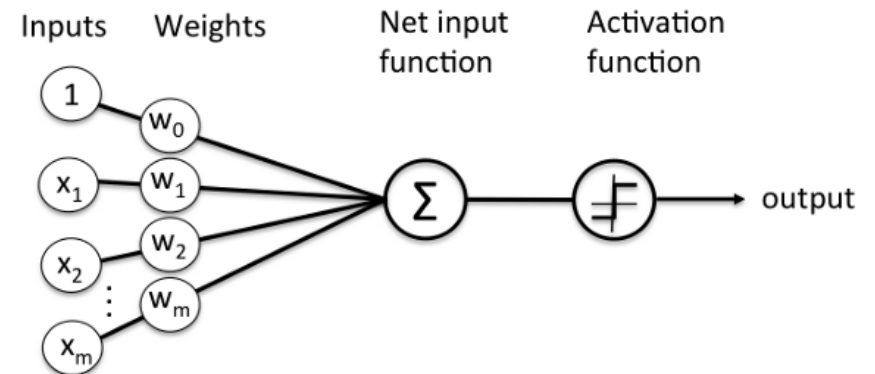
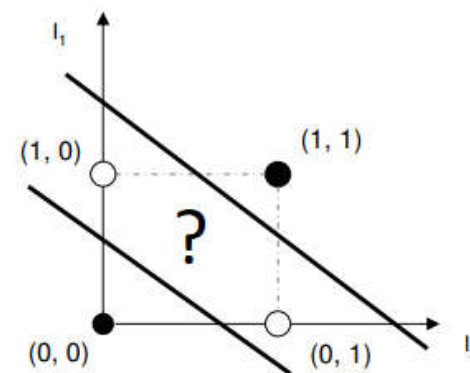
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

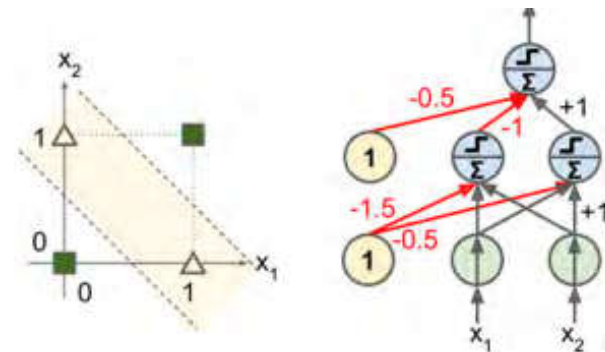
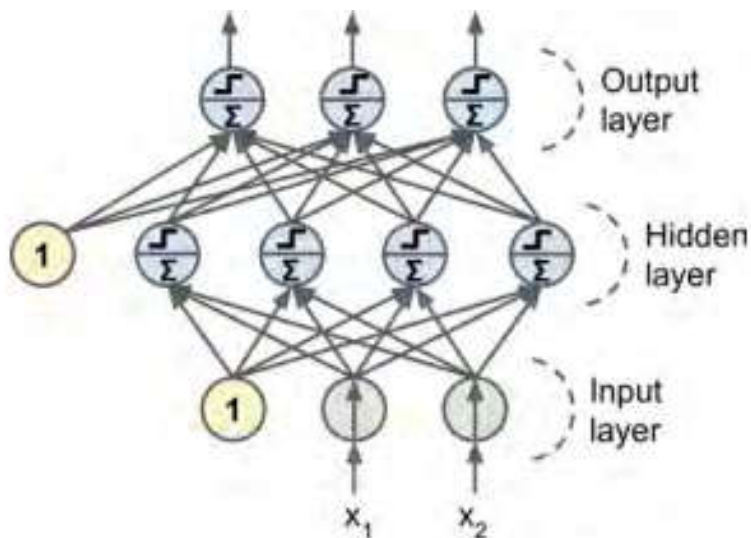


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



Multi Layer Perceptron

Linear limitation of Perceptron can be eliminated by stacking multiple Perceptrons. The resulting neural network is called Multi-Layer Perceptron



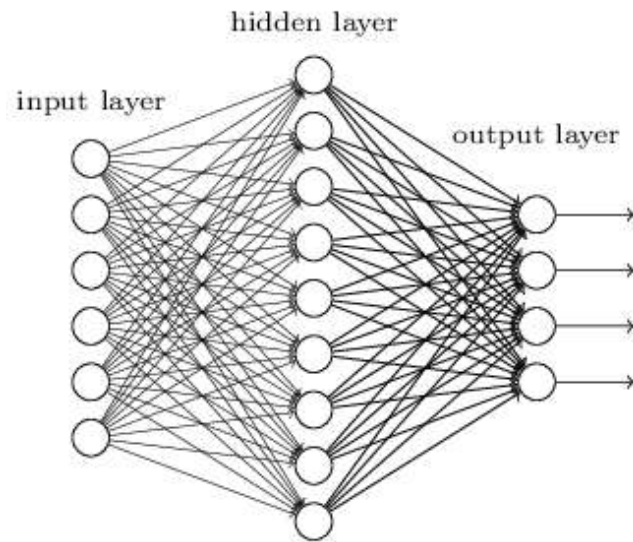
MLP can solve XOR problem

How to train MLP?

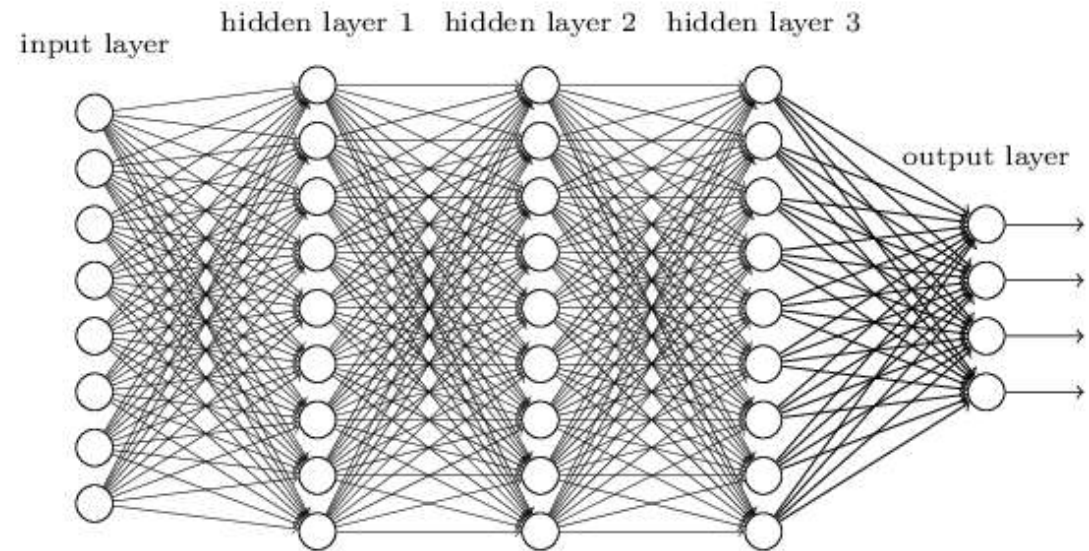
Backpropagation algorithm (1986-87)

Training of Neural Networks

**"Non-deep" feedforward
neural network**



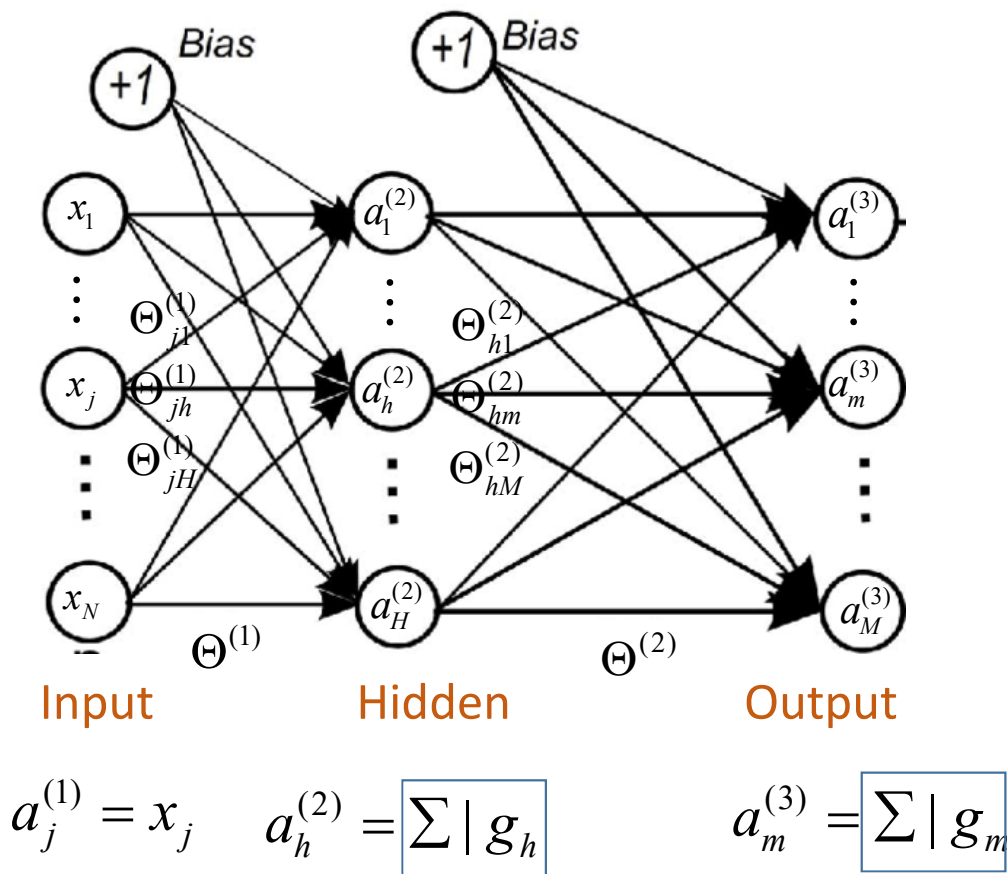
Deep neural network



<https://stats.stackexchange.com/questions/182734/what-is-the-difference-between-a-neural-network-and-a-deep-neural-network-and-w>

Training of MLP

Fully connected network



Data:

$$X_{L,N} \quad Y = R^M$$

How many parameters to train?

$$\Theta^{(1)}, \Theta^{(2)}$$

$$(N+1) \cdot H + (H+1) \cdot M$$

What if activation functions (g) are **linear**?

Forward Prop

$$a_h^{(2)} = g_h \left(\sum_{j=0}^N \Theta_{jh}^{(1)} \cdot x_j \right)$$

$$a_m^{(3)} = g_m \left(\sum_{h=0}^H \Theta_{hm}^{(2)} \cdot a_h^{(2)} \right)$$

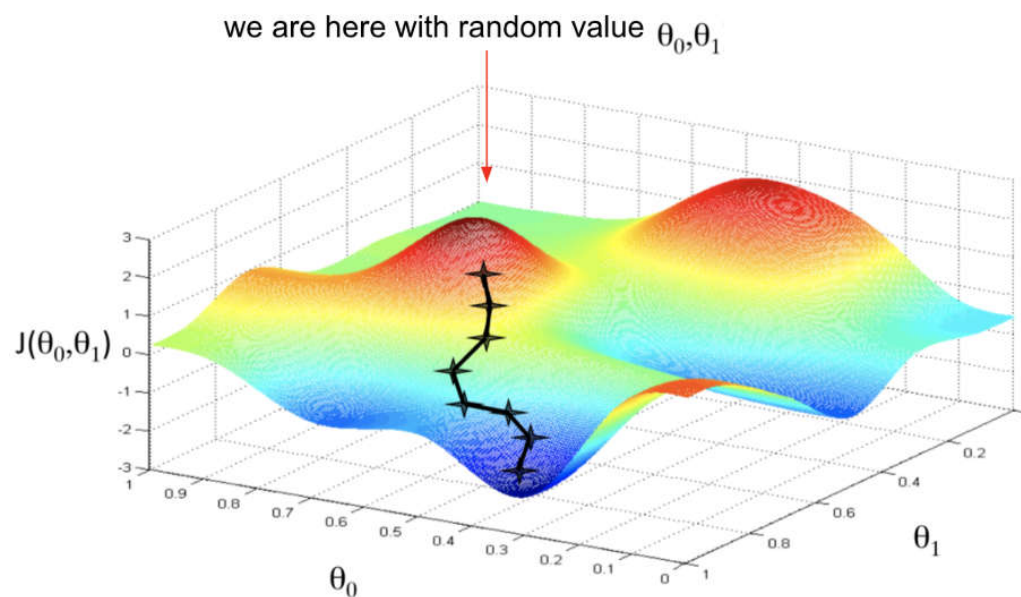
Loss function $\frac{1}{L} \sum_{i=1}^L J(\Theta, x_i, y_i),$

$$\Theta \in \mathbb{R}^{H(N+M+1)+M}$$

Stochastic gradient descent:

$$\Theta := \Theta - \eta \cdot \nabla J(\Theta, x_i, y_i)$$

How to calculate derivatives?



Backward Prop

$$\text{Let } J(\Theta) = \frac{1}{2} \sum_{m=1}^M (a_m^{(3)} - y_m)^2$$

$$\nabla J = ? \left(\frac{\partial J}{\partial \Theta^{(1)}} - ?, \frac{\partial J}{\partial \Theta^{(2)}} - ? \right)$$

$$\frac{\partial J}{\partial a_m^{(3)}} = a_m^{(3)} - y_m = \delta_m^{(3)} \quad \text{- Output Layer Error}$$

$$\frac{\partial J}{\partial a_h^{(2)}} = \sum_{m=1}^M \frac{\partial J}{\partial a_m^{(3)}} \frac{\partial a_m^{(3)}}{\partial a_h^{(2)}} = \sum_{m=1}^M \delta_m^{(3)} g'_m \Theta_{hm}^{(2)} = \delta_h^{(2)} \quad \text{Hidden Layer 'Error'}$$

$$\frac{\partial a_m^{(3)}}{\partial a_h^{(2)}} = \frac{\partial}{\partial a_h^{(2)}} \left[g_m \left(\sum_{h=0}^H \Theta_{hm}^{(2)} a_h^{(2)} \right) \right] = g'_m \Theta_{hm}^{(2)}$$

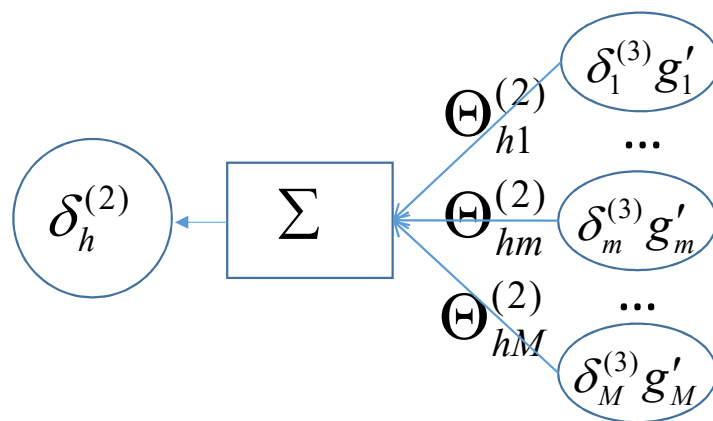
$$F = F(x(t)), \quad F'_t = F'_x x'$$

$$F = F(u(t), v(t)), \quad F'_t = F'_u u'_t + F'_v v'_t$$

Gradient Calculation

$$\frac{\partial J}{\partial \Theta_{hm}^{(2)}} = \frac{\partial J}{\partial a_m^{(3)}} \frac{\partial a_m^{(3)}}{\partial \Theta_{hm}^{(2)}} = \delta_m^{(3)} g'_m a_h^{(2)}, \quad m = 1 \dots M, \quad h = 0 \dots H$$

$$\frac{\partial J}{\partial \Theta_{jh}^{(1)}} = \frac{\partial J}{\partial a_h^{(2)}} \frac{\partial a_h^{(2)}}{\partial \Theta_{jh}^{(1)}} = \delta_h^{(2)} g'_h x_j, \quad h = 1 \dots H, \quad j = 0 \dots N$$



Back propagation

Training MLP

init weights($\Theta_{jh}^{(1)}, \Theta_{hm}^{(2)}$)

repeat :

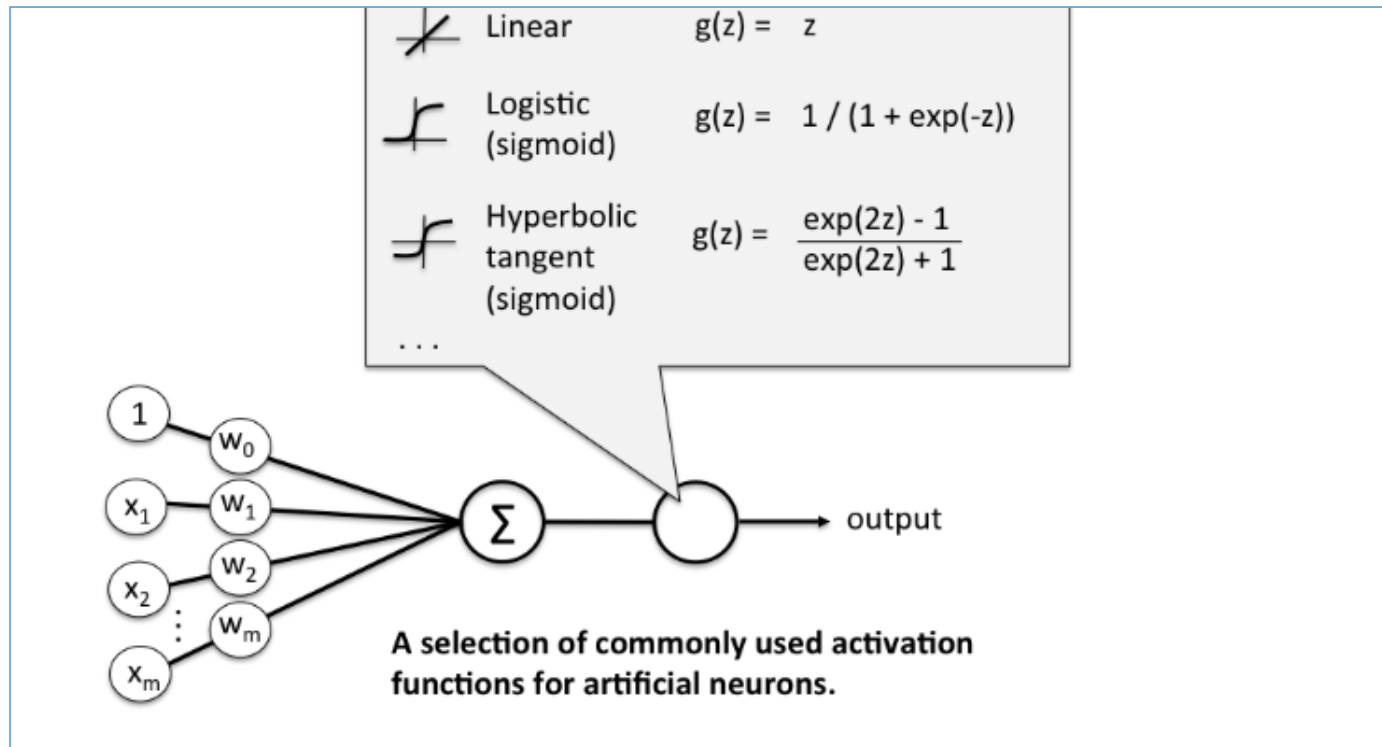
Forward pass: calculate $a_h^{(2)}, a_m^{(3)}$

Backward pass: calculate $\delta_m^{(3)}, \delta_h^{(2)}$

Update weights:

$$\Theta_{hm}^{(2)} = \Theta_{hm}^{(2)} - \eta \cdot \delta_m^{(3)} \cdot g'_m \cdot a_h^{(2)}$$
$$\Theta_{jh}^{(1)} = \Theta_{jh}^{(1)} - \eta \cdot \delta_h^{(2)} \cdot g'_h \cdot x_j$$

Activation Functions



Universal approximation theorem

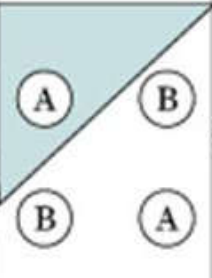
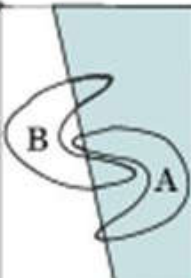
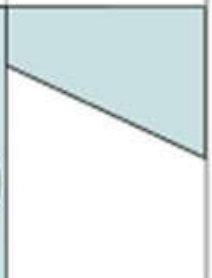
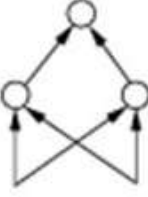
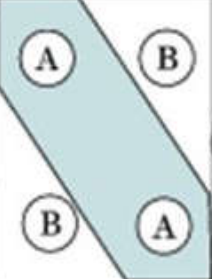
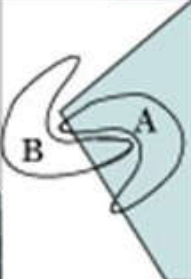
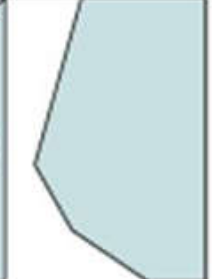
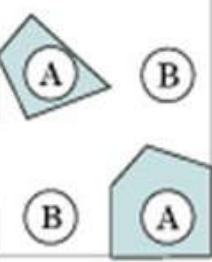
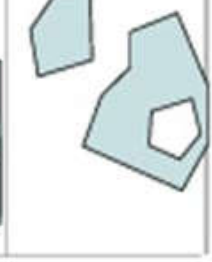
In the mathematical theory of ANNs, the universal approximation theorem states that a feed-forward network with a single hidden layer containing a finite number of neurons can approximate continuous functions on compact subset of $\mathbb{R}^n$

The theorem thus states that simple neural networks can represent a wide variety of interesting functions when given appropriate parameters; however, it does not touch upon the algorithmic learnability of those parameters.

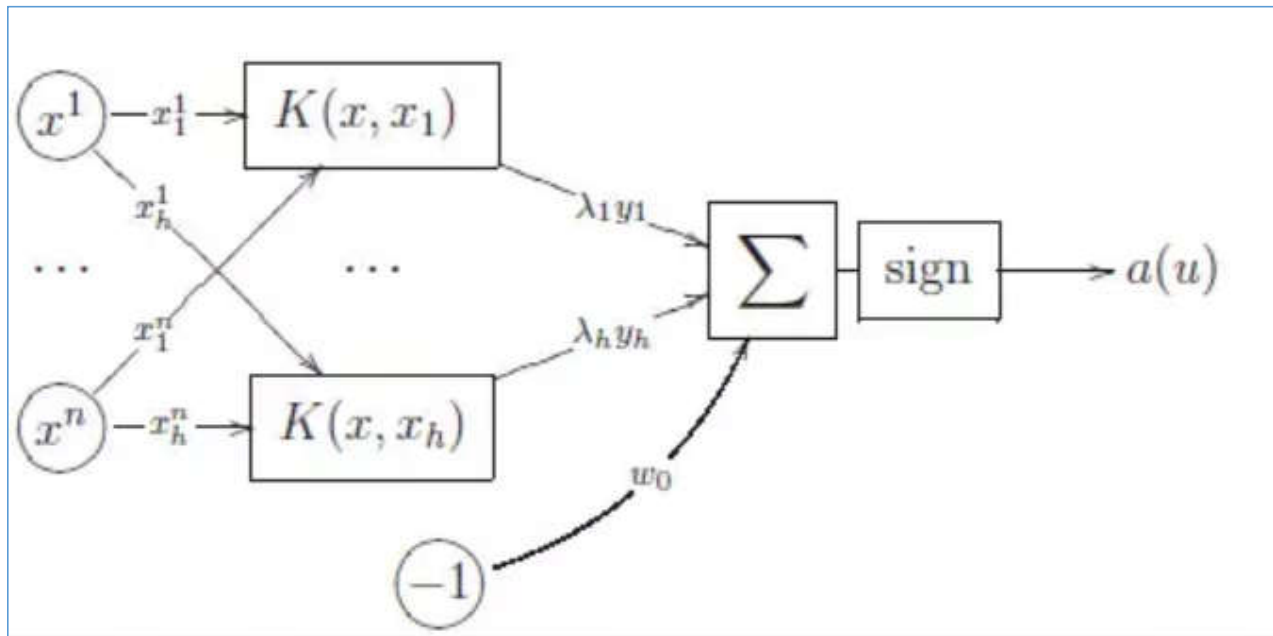
Original paper

<http://deeplearning.cs.cmu.edu/document/reading/article1.pdf>

Geometrical Interpretation of Perceptron Learning

Structure	Types of decision regions	Exclusive OR problem	Classes with meshed regions	Most general region shapes
Single-layer 	Half Plane (Bounded by hyperplane)			
Two-layer 	Convex (Open or closed regions)			
Three-layer 	Arbitrary (Complexity limited by number of neurons)			

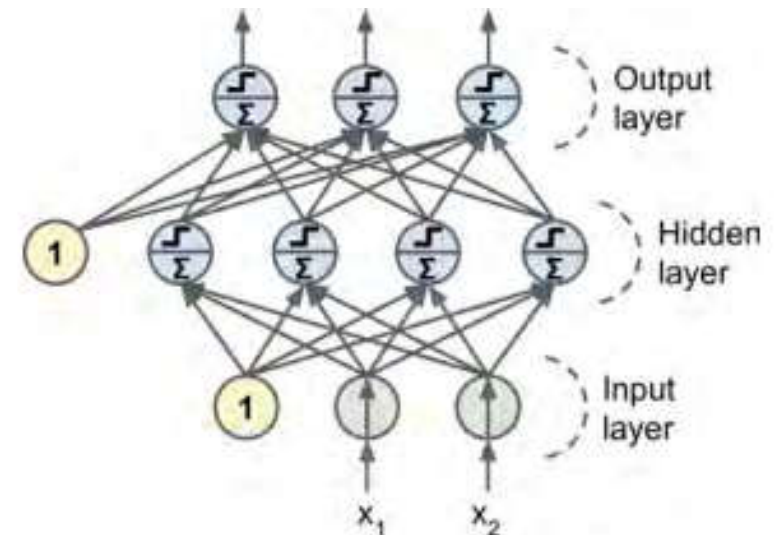
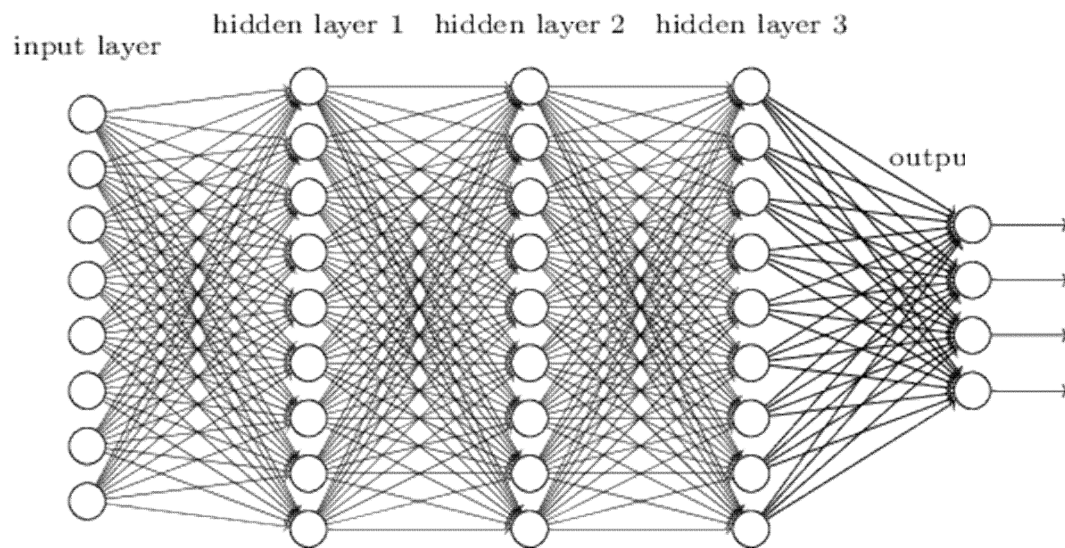
SVM as Two-Layer Perceptron



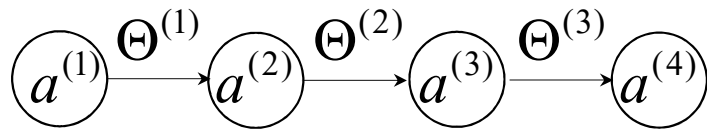
<https://www.quora.com/What-are-the-general-easy-to-understand-differences-between-support-vector-machines-and-artificial-neural-networks-Why-do-you-need-them-both>

Training ANN problems

- Vanishing gradients problem
- Training is extremely slow
- Severely risk of overfitting



What is the vanishing gradient problem?



$$a^{(i)} = \sigma(\Theta^{(i-1)} a^{(i-1)}) \quad J = \frac{1}{2} (a^{(4)}(x) - y)^2$$

$$\frac{\partial J}{\partial a^{(4)}} = a^{(4)} - y \leq \delta$$

$$\frac{\partial J}{\partial a^{(3)}} = \frac{\partial J}{\partial a^{(4)}} \frac{\partial a^{(4)}}{\partial a^{(3)}} = \delta \cdot \sigma' \cdot \Theta^{(3)} \leq \frac{\delta}{4} \cdot \Theta^{(3)}$$

$$\frac{\partial J}{\partial a^{(2)}} = \frac{\partial J}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial a^{(2)}} \leq \frac{\delta}{4^2} \cdot \Theta^{(3)} \cdot \Theta^{(2)}$$

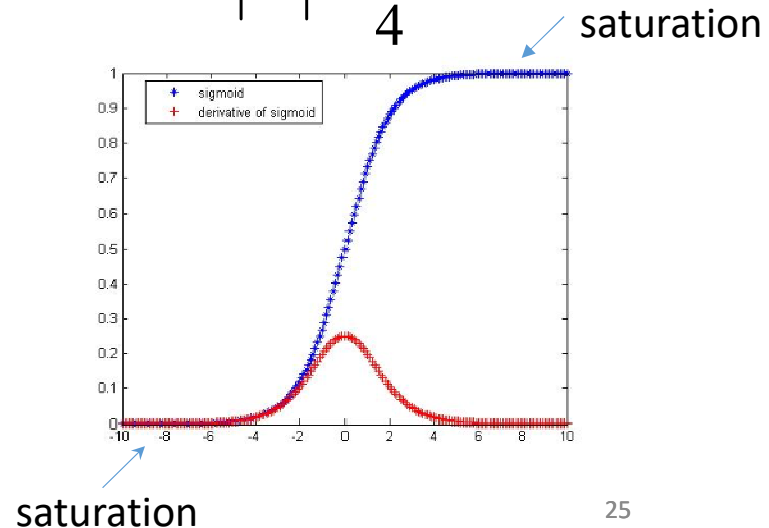
$$\frac{\partial J}{\partial a^{(1)}} = \frac{\partial J}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial a^{(1)}} \leq \frac{\delta}{4^3} \cdot \Theta^{(3)} \cdot \Theta^{(2)} \cdot \Theta^{(1)}$$

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

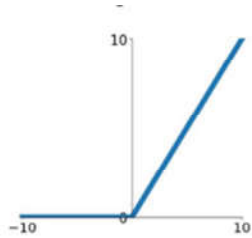
$$\sigma' = \sigma(1 - \sigma)$$

$$|\sigma'| < \frac{1}{4}$$

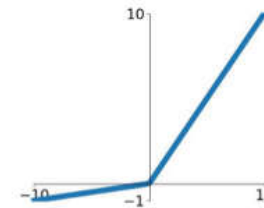


Nonsaturating activation functions

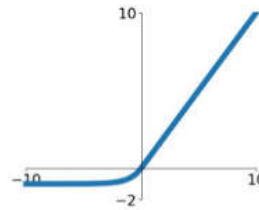
ReLU
 $\max(0, x)$



Leaky ReLU
 $\max(0.1x, x)$



ELU
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



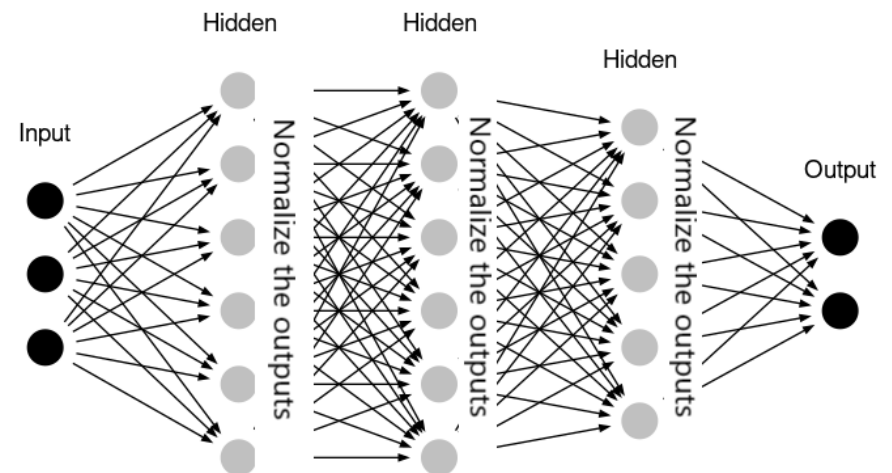
Batch normalization (S. Ioffe, 2015)

- Each unit's pre-activation is normalized (mean subtraction, stddev division)
- During training, mean and stddev is computed for each minibatch
- Backpropagation takes into account the normalization
- At test time, the global mean / stddev is used

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

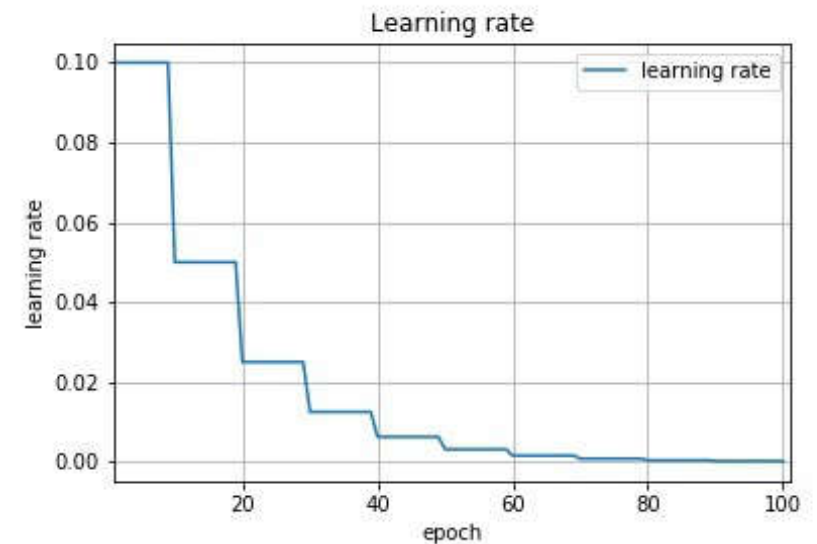
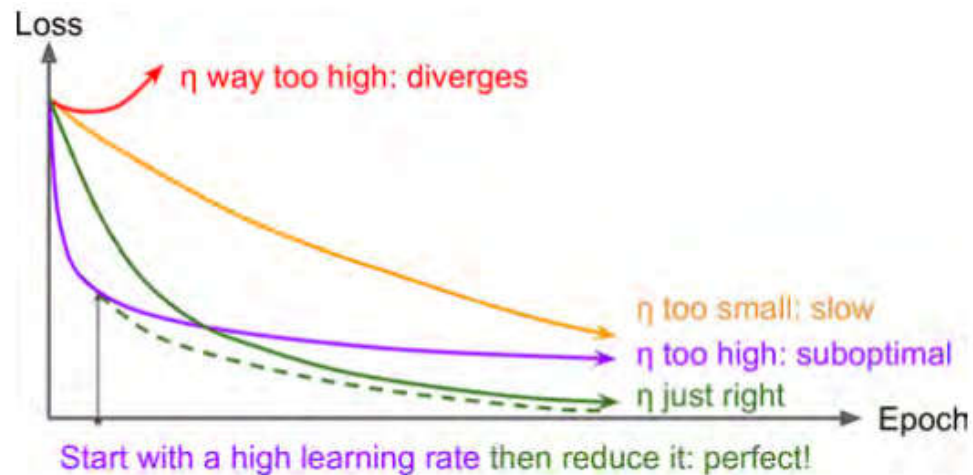


Learned linear transformation to adapt to non-linear activation function (γ and β are trained)

Learning rate scheduling

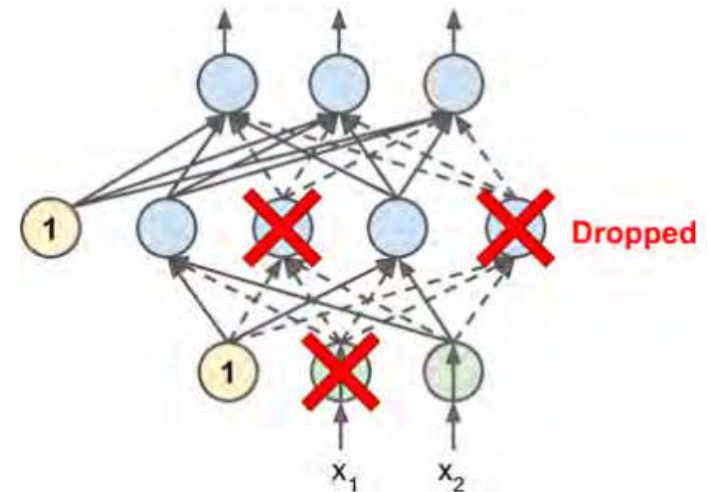
Start with high learning rate and then reduce it

- Predetermined piecewise constant learning rate
- Exponential scheduling



Avoiding Overfitting Through Regularization

- ✓ Early stopping (just interrupt training)
- ✓ L1 and L2 regularization
- ✓ Dropout (every neuron has a probability p of being temporally 'dropped out' (ignored during this training step), but it may be active during the next step)

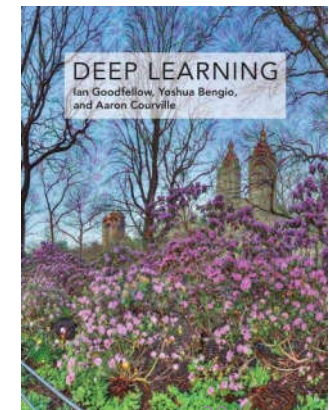
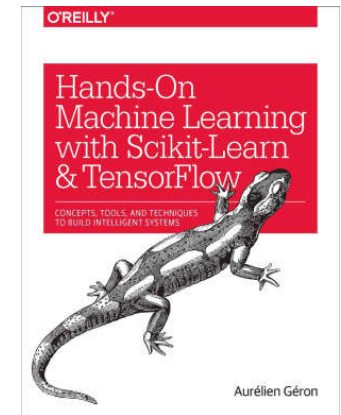


References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow'
- I.Goodfellow 'Deep Learning'

Coursera, Deep Learning Specialization, Andrew Ng



<https://github.com/yanshengjia/ml-road/tree/master/resources>

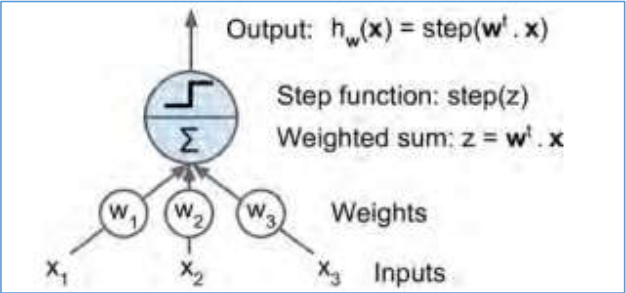
Data Analysis

Lecture 11: Artificial Neural Networks

Dr. Nataliya K. Sakhnenko

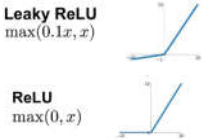
Recap: Training ANN problems

The Perceptron (1957, Frank Rosenblatt)

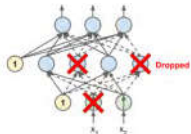


Problem	Solution
Vanishing gradients problem	Nonsaturating activation functions Batch Normalization
Severely risk of overfitting	✓ Early stopping ✓ l1 and l2 regularization ✓ Dropout
Training is extremely slow	Optimization of gradient descent

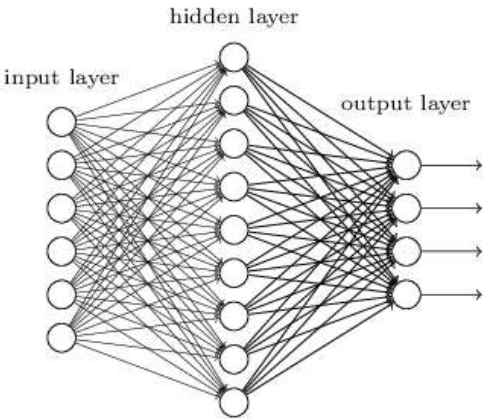
Nonsaturating activation functions



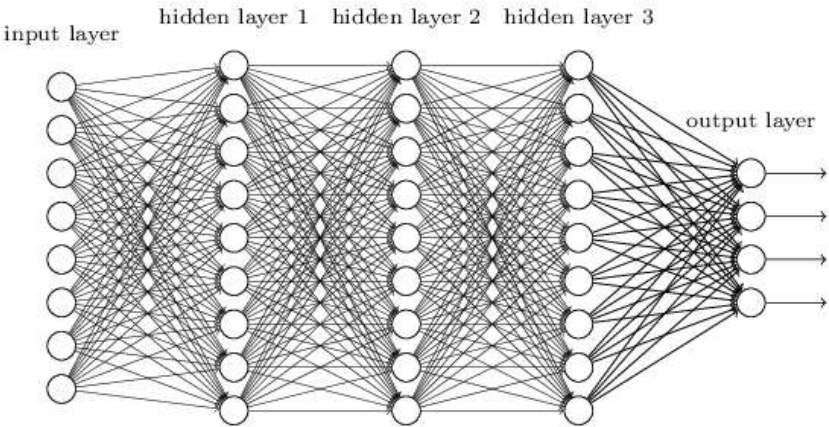
Dropout:



"Non-deep" feedforward neural network

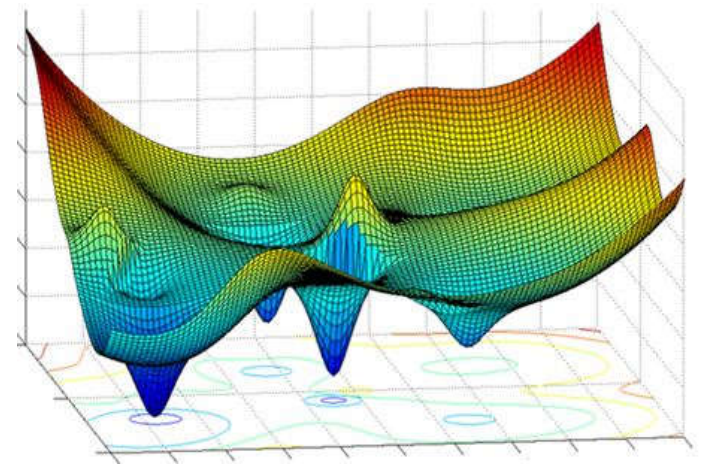


Deep neural network



Multilayered Perceptron (MLP)

Optimization of gradient descent



Optimization

SGD: $\Theta_t = \Theta_{t-1} - \eta \nabla_{\Theta} J(\Theta_{t-1})$

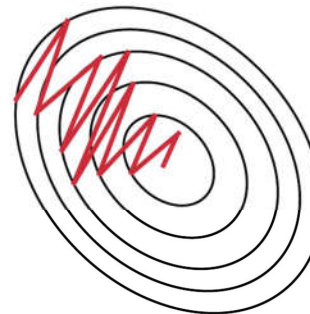
✓ Momentum optimization (Polyak 1964):

$$v_t = \gamma v_{t-1} + \eta \nabla_{\Theta} J(\Theta_{t-1})$$

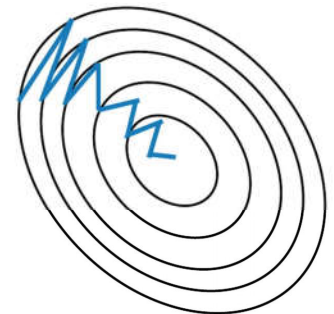
$$\Theta_t = \Theta_{t-1} - v_t$$

✓ Nesterov Accelerated gradient (Nesterov 1983):

$$v_t = \gamma v_{t-1} + \eta \nabla_{\Theta} J(\Theta_{t-1} - \gamma v_{t-1})$$



Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum

Adaptive Optimization

✓ Adagrad

$$g_{t,i} = \nabla_{\Theta_i} J(\Theta)$$

$$\Theta_{t,i} = \Theta_{t-1,i} - \frac{\eta}{\sqrt{G_{t-1,i} + \varepsilon}} g_{t-1,i}$$

$$\Theta_t = \Theta_{t-1} - \frac{\eta}{\sqrt{G_{t-1} + \varepsilon}} g_{t-1}$$

$$G_{t,i} = G_{t-1,i} + g_{t,i}^2$$

✓ RMS prop

$$\Theta_t = \Theta_{t-1} - \frac{\eta}{\sqrt{E(g^2)_{t-1} + \varepsilon}} g_{t-1}$$

$$E(g^2)_t = \gamma E(g^2)_{t-1} + (1 - \gamma) g_t^2$$

Adam Optimization (D. Kingma 2014)

- ✓ Adam (adaptive moment estimation) combines ideas of Momentum optimization and RMSProp: like Momentum optimization it keeps track of an exponentially decaying average of past gradients, and like RMSProp it keeps track of an exponentially decaying average of past squared gradients

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

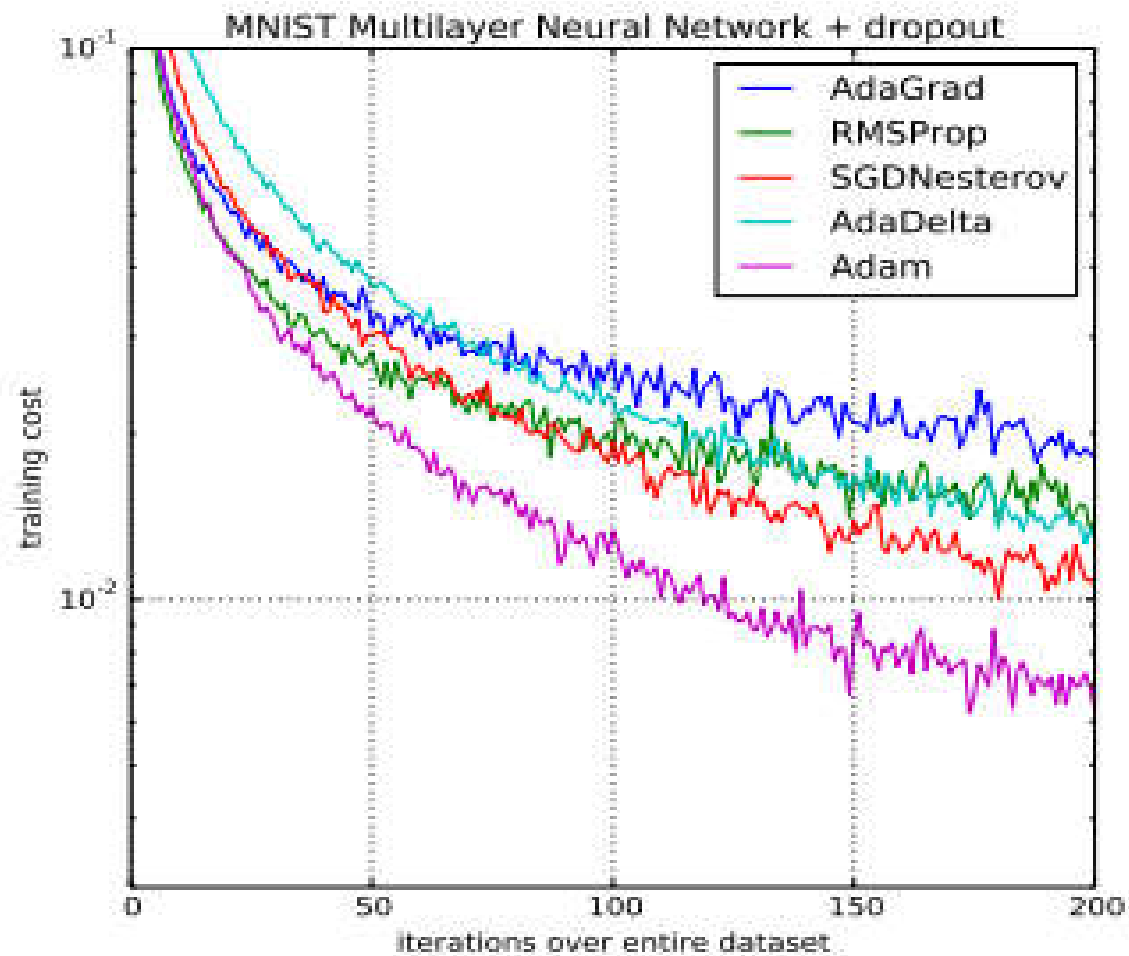
$$\Theta_{t+1} = \Theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$$

Optimization



<https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/>

Deep Learning Frameworks



Keras

Who makes Keras? Contributors and backers

 633 contributors



Keras API

TensorFlow

GPU

CPU

TPU

Keras API

The Sequential API

```
import keras
from keras import layers

model = keras.Sequential()
model.add(layers.Dense(20, activation='relu', input_shape=(10,)))
model.add(layers.Dense(20, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.fit(x, y, epochs=10, batch_size=32)
```

The functional API

```
import keras
from keras import layers

inputs = keras.Input(shape=(10,))
x = layers.Dense(20, activation='relu')(x)
x = layers.Dense(20, activation='relu')(x)
outputs = layers.Dense(10, activation='softmax')(x)

model = keras.Model(inputs, outputs)
model.fit(x, y, epochs=10, batch_size=32)
```

Recap: softmax

$$T: X \rightarrow Y, Y = \{1, 2, \dots, K\}$$

$$h_{\theta}(x) = \begin{Bmatrix} P(y=1|x; \theta) \\ P(y=2|x; \theta) \\ \dots \\ P(y=K|x; \theta) \end{Bmatrix} = \frac{1}{\sum_{i=1}^K \exp(\theta_i^T x)} \begin{pmatrix} \exp(\theta_1^T x) \\ \exp(\theta_2^T x) \\ \dots \\ \exp(\theta_K^T x) \end{pmatrix}$$

One-hot encoding

$$\begin{bmatrix} 1 \\ 0 \\ \dots \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ \dots \\ 0 \end{bmatrix}, \dots, \begin{bmatrix} 0 \\ 0 \\ \dots \\ 1 \end{bmatrix}, Y \in R^K$$

1, 2, ... K

<https://keras.io/>

MNIST - Logistic Regression

```
import keras  
from keras.datasets import mnist
```

```
def get_data():  
    (x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
    x_train = x_train.reshape(60000, 784)
```

```
    x_test = x_test.reshape(10000, 784)
```

```
    x_train = x_train.astype('float32')
```

```
    x_test = x_test.astype('float32')
```

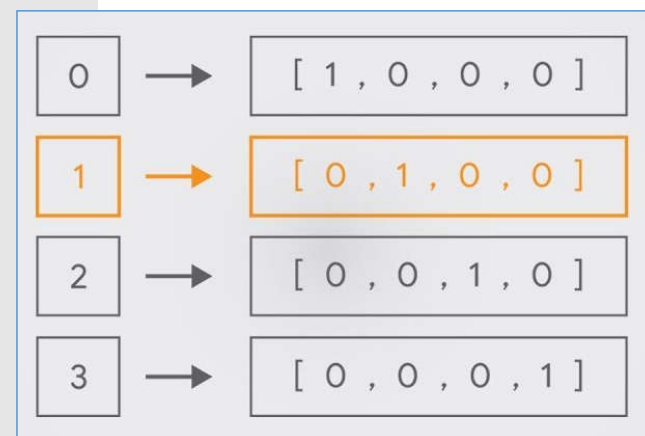
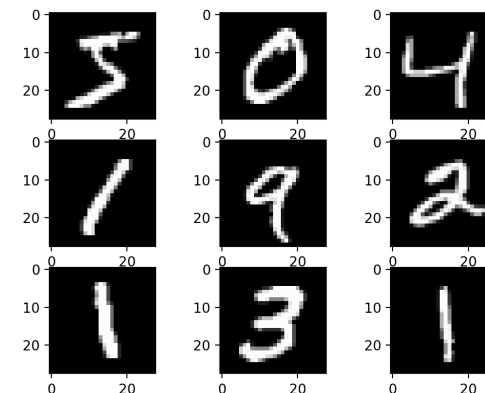
```
    x_train /= 255
```

```
    x_test /= 255
```

```
    y_train = keras.utils.to_categorical(y_train, num_classes)
```

```
    y_test = keras.utils.to_categorical(y_test, num_classes)
```

```
    return (x_train, y_train), (x_test, y_test)
```



One-hot encoding

MNIST - Logistic Regression

```
from keras.layers import Dense
from keras.models import Sequential
```

```
batch_size = 128
num_classes = 10
epochs = 20
input_dim = 784 #28*28
```

```
model = Sequential()
model.add(Dense(input_dim=input_dim, output_dim = num_classes, activation='softmax'))
model.summary()
```

```
model.compile(optimizer='sgd', loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs, verbose=1,
                    validation_data=(x_test, y_test))
```

```
score = model.evaluate(x_test, y_test, verbose=0)
print('Test accuracy:', score[1])
```

Model summary

Layer (type)	Output Shape	Param #
dense_10 (Dense)	(None, 10)	7850
Total params: 7,850		
Trainable params: 7,850		
Non-trainable params: 0		

Test accuracy: 0.9095

MNIST - MLP

```
model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(784,)))
model.add(Dense(512, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 512)	401920
dense_13 (Dense)	(None, 512)	262656
dense_14 (Dense)	(None, 10)	5130

=====
Total params: 669,706
Trainable params: 669,706
Non-trainable params: 0

MNIST - MLP

```
(x_train, y_train), (x_test, y_test) = get_data()
```

```
history = model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs, verbose=1,  
                    validation_data=(x_test, y_test))
```

```
60000 train samples  
10000 test samples  
Train on 60000 samples, validate on 10000 samples  
Epoch 1/20  
60000/60000 [=====] - 44s 727us/step - loss: 0.2147 - acc: 0.9370 - val_loss: 0.0970 - val_acc: 0.9698  
Epoch 2/20  
60000/60000 [=====] - 43s 712us/step - loss: 0.0789 - acc: 0.9755 - val_loss: 0.0882 - val_acc: 0.9723  
Epoch 3/20  
60000/60000 [=====] - 43s 712us/step - loss: 0.0511 - acc: 0.9838 - val_loss: 0.0757 - val_acc: 0.9754  
Epoch 4/20  
60000/60000 [=====] - 45s 747us/step - loss: 0.0358 - acc: 0.9886 - val_loss: 0.0649 - val_acc: 0.9798.
```

```
score = model.evaluate(x_test, y_test, verbose=0)  
print('Test accuracy:', score[1])
```

Test accuracy: 0.9818

Using Dropout

```
from keras.layers import Dropout

model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(784,)))
model.add(Dropout(0.5))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))

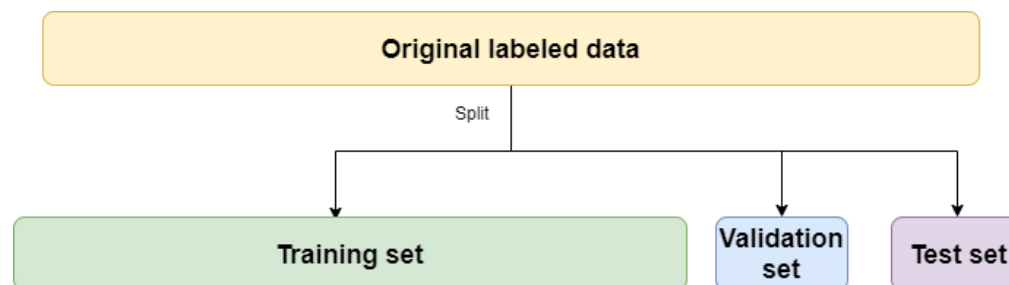
model.summary()
```


Using Batch Normalization

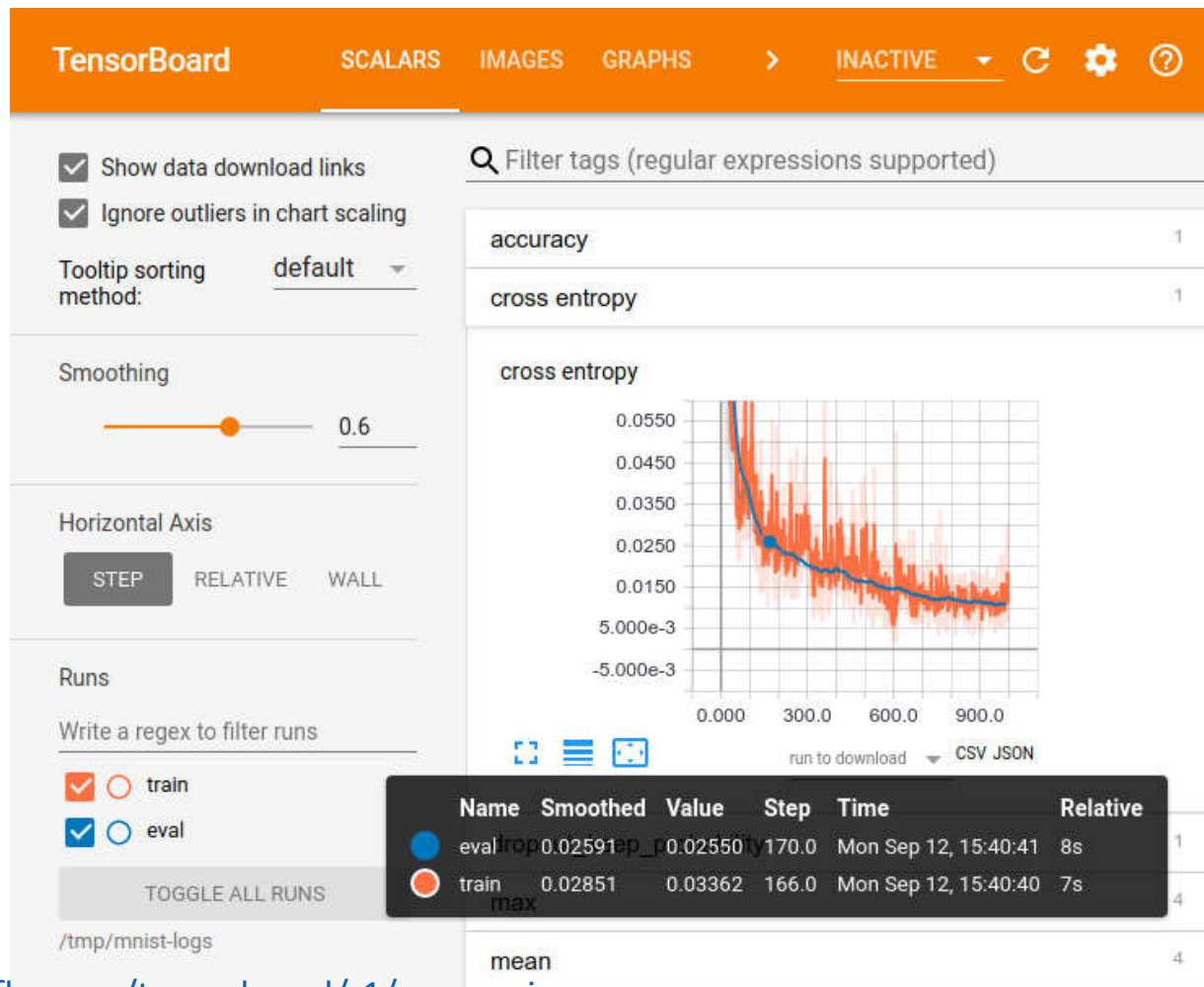
```
from keras.layers.normalization import BatchNormalization
from keras.layers import Activation
```

```
model = Sequential()
model.add(Dense(512, input_shape=(784,)))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.5))
```

...



TensorBoard: Visualizing Learning



<https://www.tensorflow.org/tensorboard/r1/summaries>

TensorBoard: Playground

<https://playground.tensorflow.org/>

Tinker With a **Neural Network** Right Here in Your Browser.
Don't Worry, You Can't Break It. We Promise.

Epoch 000,496 Learning rate 0.03 Activation Tanh Regularization None Regularization rate 0 Problem type Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 10

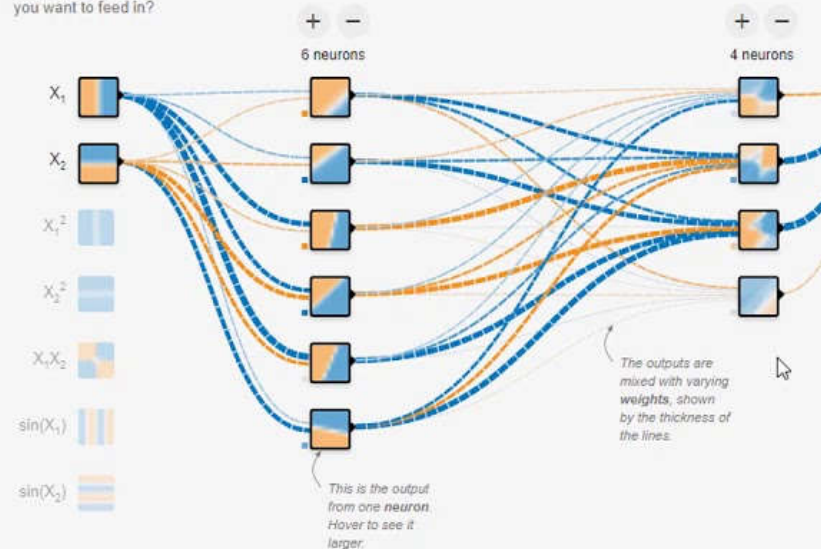
REGENERATE

FEATURES

Which properties do you want to feed in?

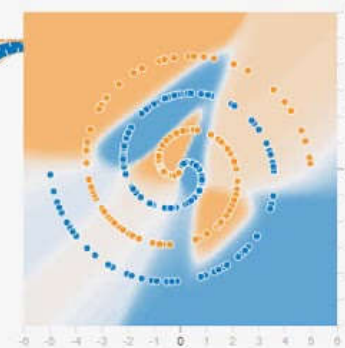
X_1
 X_2
 X_1^2
 X_2^2
 $X_1 X_2$
 $\sin(X_1)$
 $\sin(X_2)$

+ - 2 HIDDEN LAYERS



OUTPUT

Test loss 0.307
Training loss 0.208



Colors shows data, neuron and weight values.

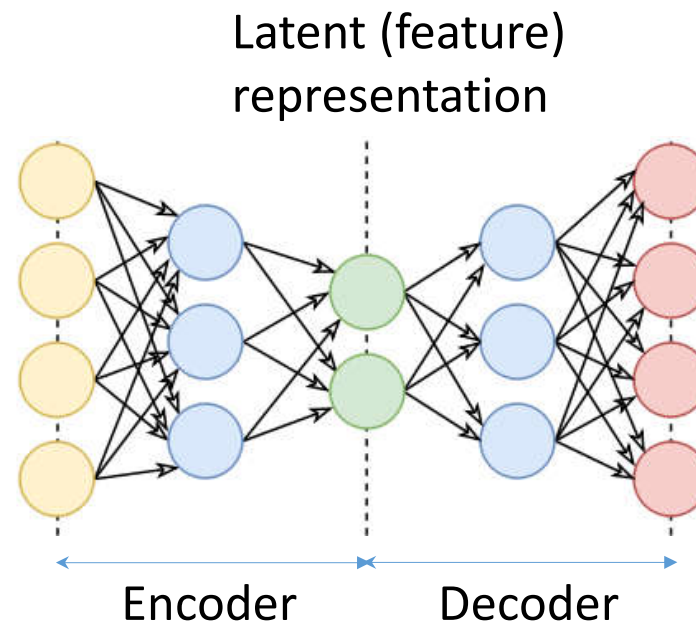
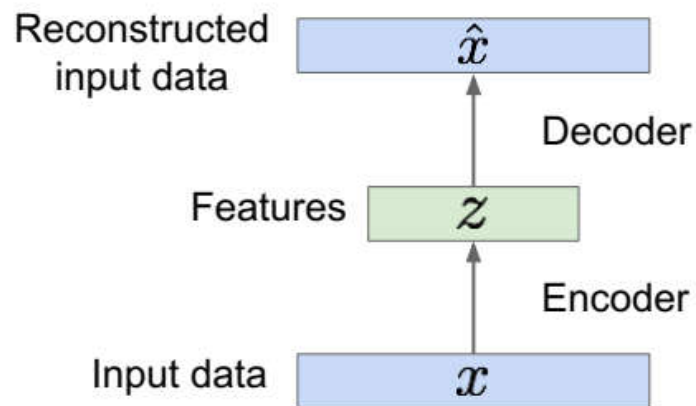
☐ Show test data ☐ Discretize output

Auto encoders

(Unsupervised Learning)

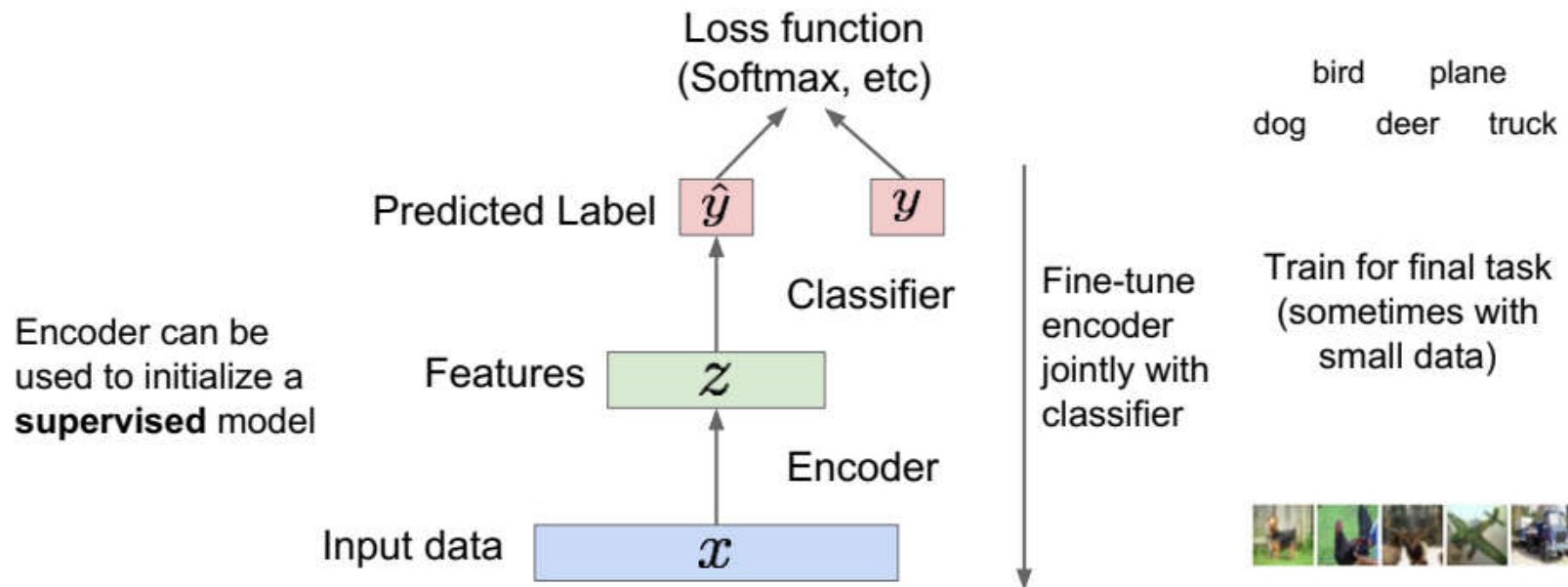
AE for dimensionality reduction

Train to reconstruct original data



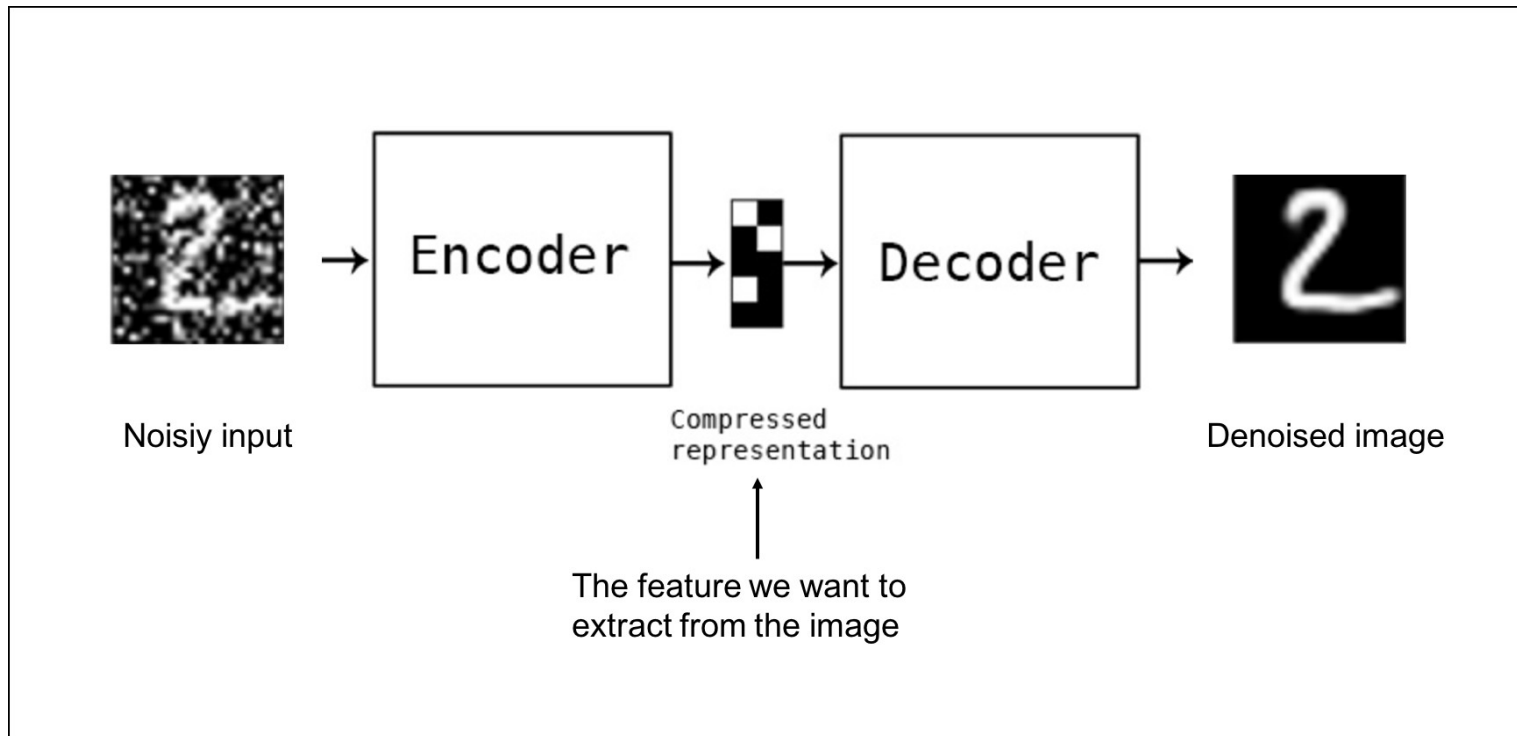
An **autoencoder (AE)** is a type of ANN used to learn efficient data coding in an unsupervised manner. The goal of any AE is to reconstruct its own input. Usually, the AE first compresses the input into a smaller form, then transforms it back into an approximation of the input.

AE for feature learning



AEs can reconstruct data, and can learn features to initialize a supervised model

Denoising AE



- Reconstruction X' computed from the corrupted input X_{noise}
- Loss function compares X' reconstruction with the noiseless input X

AE for image denoising (MNIST example)

```
from keras.layers import Input, Dense
from keras.models import Model
from keras.datasets import mnist
```

Build base AE model

```
input_layer = Input(shape=(784,))
encoding_layer = Dense(32, activation='relu')(input_layer)
decoding_layer = Dense(784, activation='sigmoid')(encoding_layer)
autoencoder = Model(input_layer, decoding_layer)
autoencoder.compile(optimizer='adam', loss='mse')
```

Load MNIST data

```
(x_train, _), (x_test, _) = mnist.load_data()

x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
```


AE for image denoising (MNIST example)

Build advanced AE model

```
input_layer = Input(shape=(784,))
encoding_layer1 = Dense(128, activation='relu')(input_layer)
encoding_layer2 = Dense(64, activation='relu')(encoding_layer1)
encoding_layer3 = Dense(32, activation='relu')(encoding_layer2)

decoding_layer1 = Dense(64, activation='relu')(encoding_layer2)
decoding_layer2 = Dense(128, activation='relu')(decoding_layer1)
output_decoding_layer = Dense(784, activation='sigmoid')(decoding_layer2)

autoencoder = Model(input_layer, output_decoding_layer)
autoencoder.compile(optimizer='adam', loss='mse')
```

AE for image denoising (MNIST example)

Add noise to images

```
noise_factor = 0.4
x_train_noise = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_train.shape)
x_test_noise = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_test.shape)

x_train_noise = np.clip(x_train_noise, 0., 1.)
x_test_noise = np.clip(x_test_noise, 0., 1.)
```

Train the model

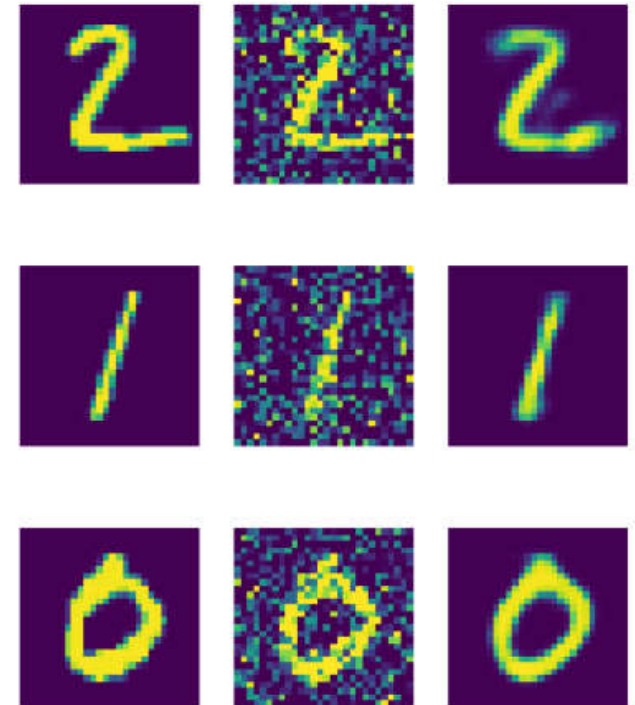
```
autoencoder.fit(x_train_noise, x_train, epochs=10, batch_size=128, shuffle=True,
validation_split = 0.15)
```

AE for image denoising (MNIST example)

Test the model

```
output_images = autoencoder.predict(x_test_noise)
```

```
for i in range(1, 4):  
    f, axes = plt.subplots(1, 3)  
    axes[0].imshow(x_test[i].reshape(28,28))  
    axes[1].imshow(x_test_noise[i].reshape(28,28))  
    axes[2].imshow(output_images[i].reshape(28,28))
```

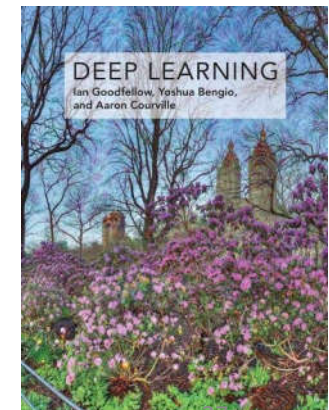
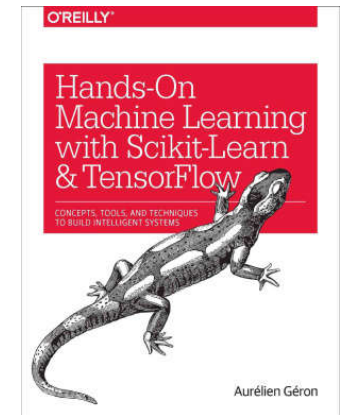


References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow'
- I.Goodfellow 'Deep Learning'

Coursera, Deep Learning Specialization, Andrew Ng



<https://github.com/yanshengjia/ml-road/tree/master/resources>

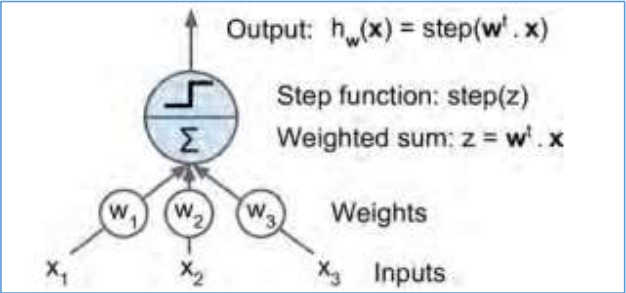
Data Analysis

Lecture 12: Intro to Convolutional Neural Networks

Dr. Nataliya K. Sakhnenko

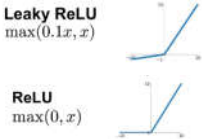
Recap: ANN

The Perceptron (1957, Frank Rosenblatt)

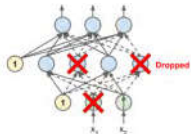


Problem	Solution
Vanishing gradients problem	Nonsaturating activation functions Batch Normalization
Severely risk of overfitting	✓ Early stopping ✓ l1 and l2 regularization ✓ Dropout
Training is extremely slow	Optimization of gradient descent

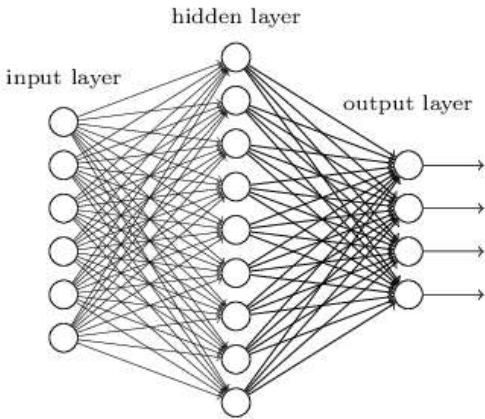
Nonsaturating activation functions



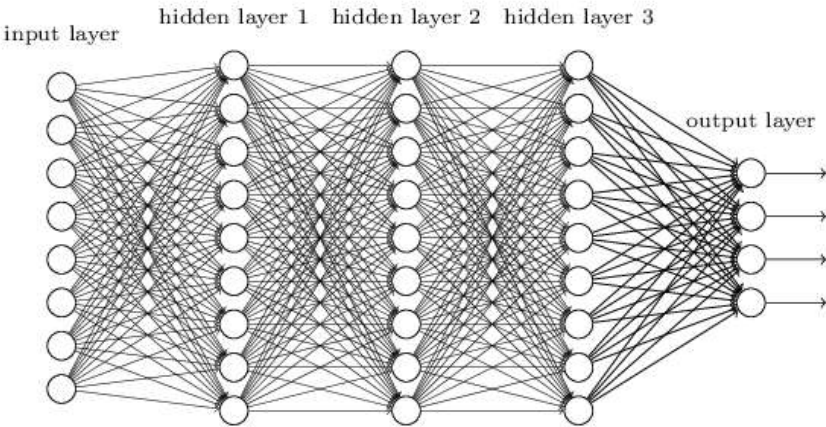
Dropout:



"Non-deep" feedforward neural network



Deep neural network



Multilayered Perceptron (MLP)



What We See

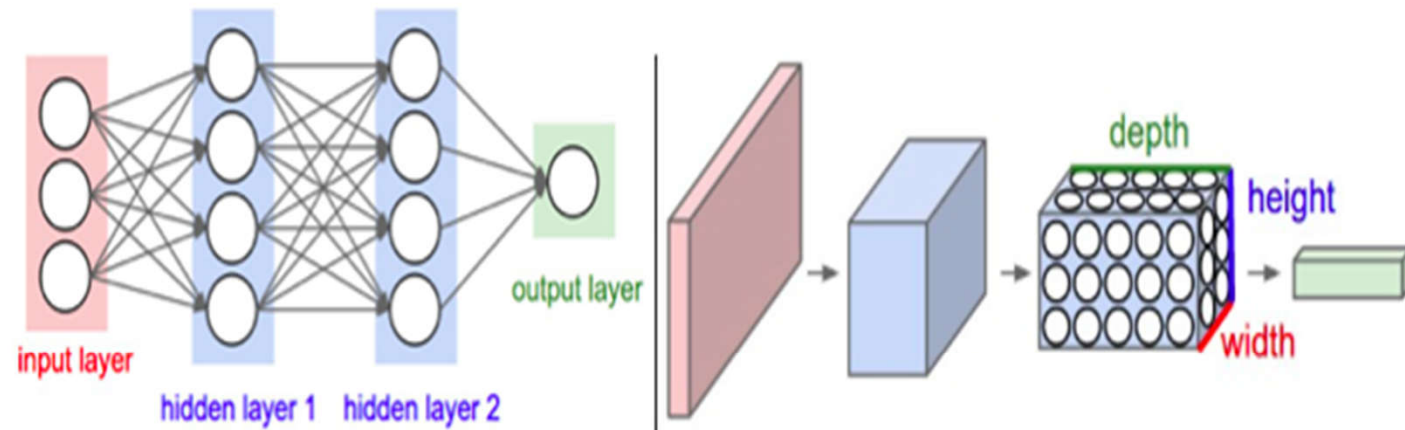
```

08 02 22 97 38 18 00 40 00 75 04 08 07 78 52 12 80 77 91 08
49 49 99 40 17 81 18 57 40 87 17 40 98 43 89 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 96 01 32 56 71 37 02 38 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 25 66 33 13 60
24 47 32 60 99 03 85 02 44 75 32 33 78 36 84 20 35 17 12 50
32 98 81 28 64 23 47 10 26 38 40 67 59 34 70 66 18 38 64 70
67 26 20 68 02 62 12 20 93 63 94 39 63 08 40 91 66 49 34 21
24 53 58 05 66 73 99 26 97 17 78 78 94 83 14 88 34 89 63 72
21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 03 44 44 37 44 60 21 55 51 54 17 58
19 80 81 68 05 94 47 65 28 73 82 13 84 32 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 97 32 16 26 28 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 03 54
01 70 34 71 83 51 54 69 16 93 33 48 41 49 32 01 89 19 67 48
  
```

What Computers See

<https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/>

Dense NN vs Convolutional NN

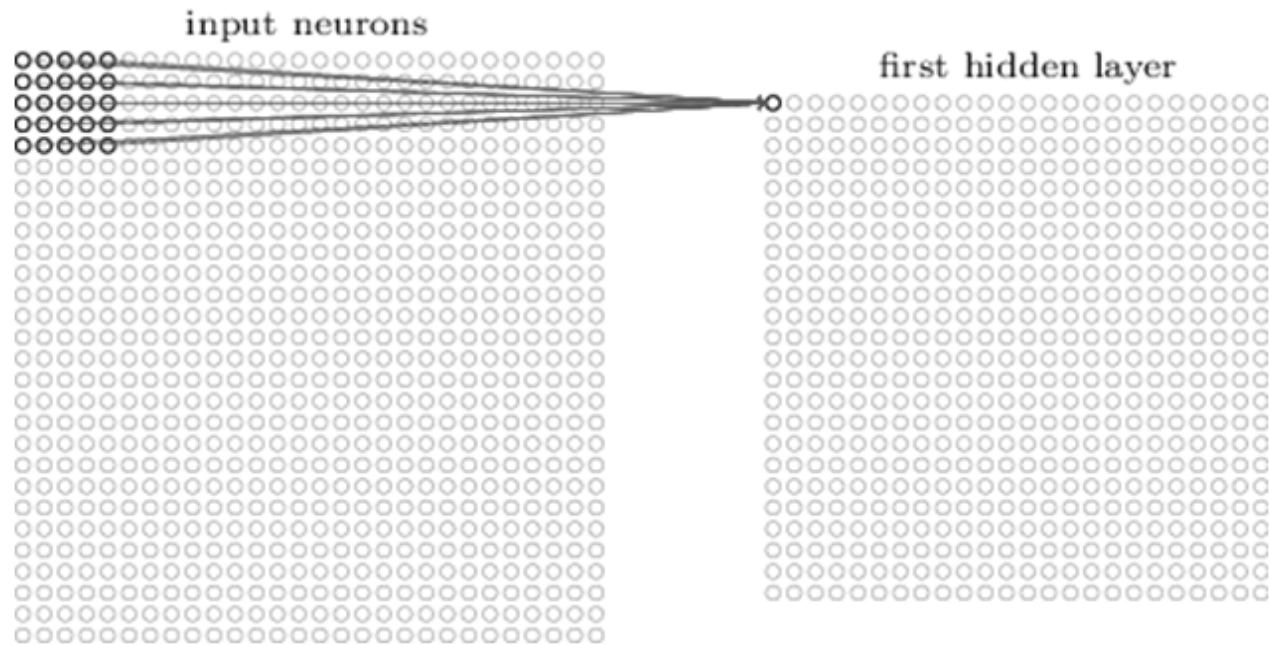


Left: A regular 3-layer Neural Network. Right: A ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers. Every layer of a ConvNet transforms the 3D input volume to a 3D output volume of neuron activations. In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels).

Building Blocks of the CNN

- Convolutional Layers
- Pooling Layer
- Fully-Connected Layer (exactly as seen in regular Neural Networks)

Convolutional Layer

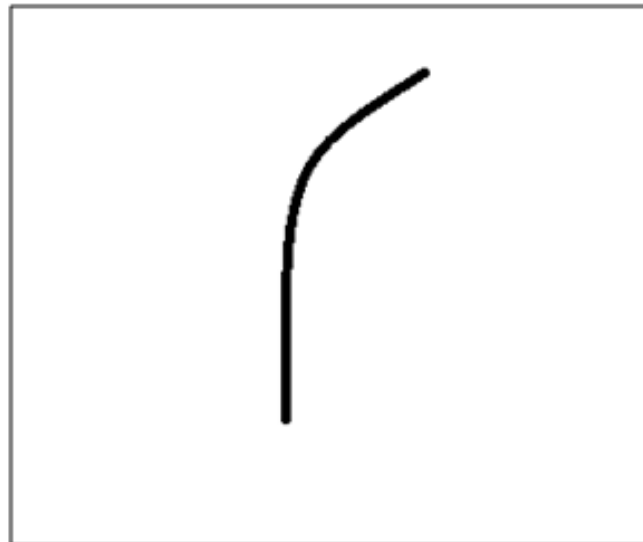


5x5 filter convolving around an input and producing a feature map

Filter (kernel)

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

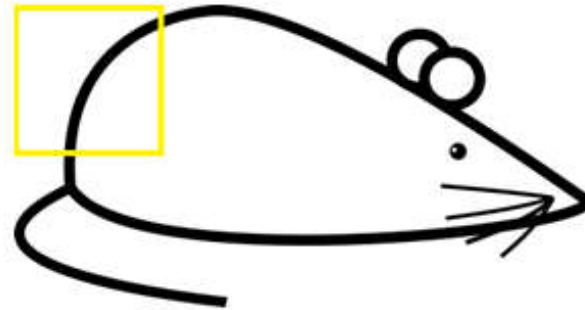


Visualization of a curve detector filter

Convolution



Original image



Visualization of the filter on the image



Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

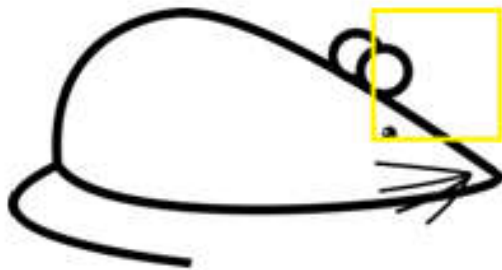
*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = $(50*30)+(50*30)+(50*30)+(20*30)+(50*30) = 6600$ (A large number!)

Convolution



Visualization of the filter on the image

0	0	0	0	0	0	0
0	40	0	0	0	0	0
40	0	40	0	0	0	0
40	20	0	0	0	0	0
0	50	0	0	0	0	0
0	0	50	0	0	0	0
25	25	0	50	0	0	0

Pixel representation of receptive field

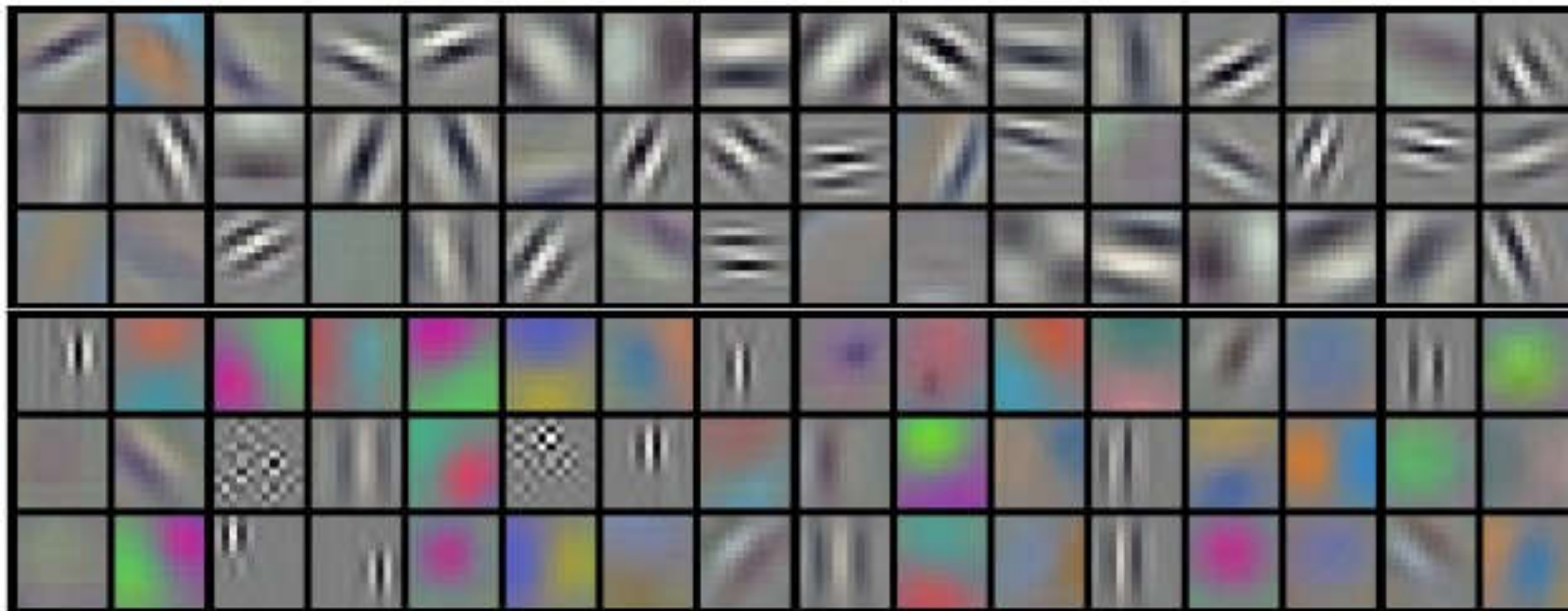
*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

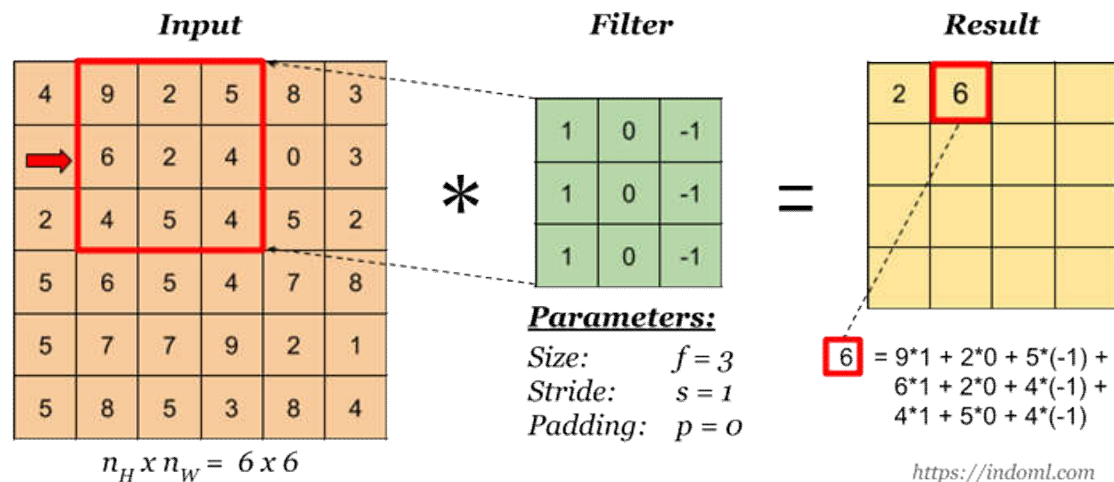
Multiplication and Summation = 0

Visualization of filters



Stride

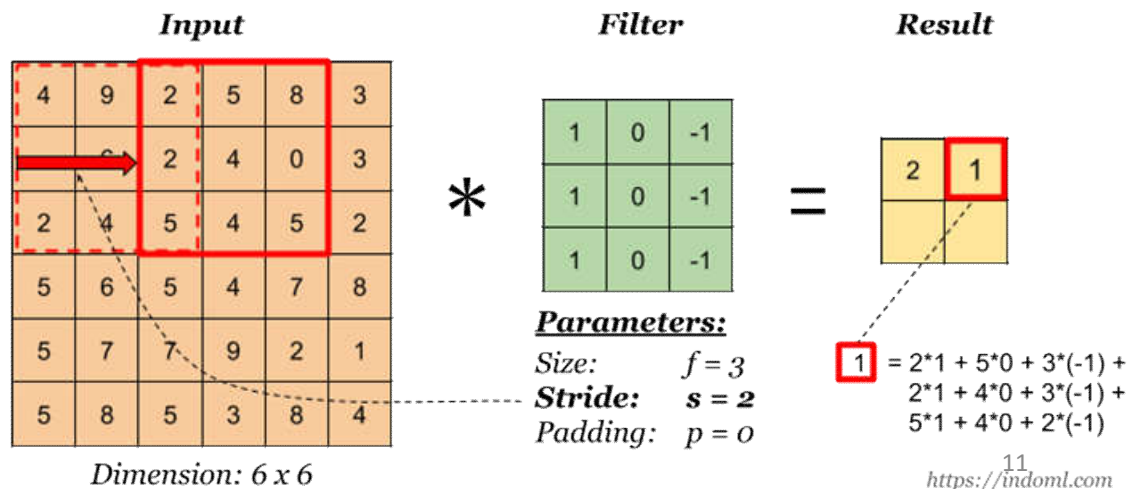
Stride governs how many cells the filter is moved in the input to calculate the next cell in the result.



The total number of multiplications to calculate the result is $(4 \times 4) \times (3 \times 3) = 144$

The total number of multiplications to calculate the result is $(2 \times 2) \times (3 \times 3) = 36$.

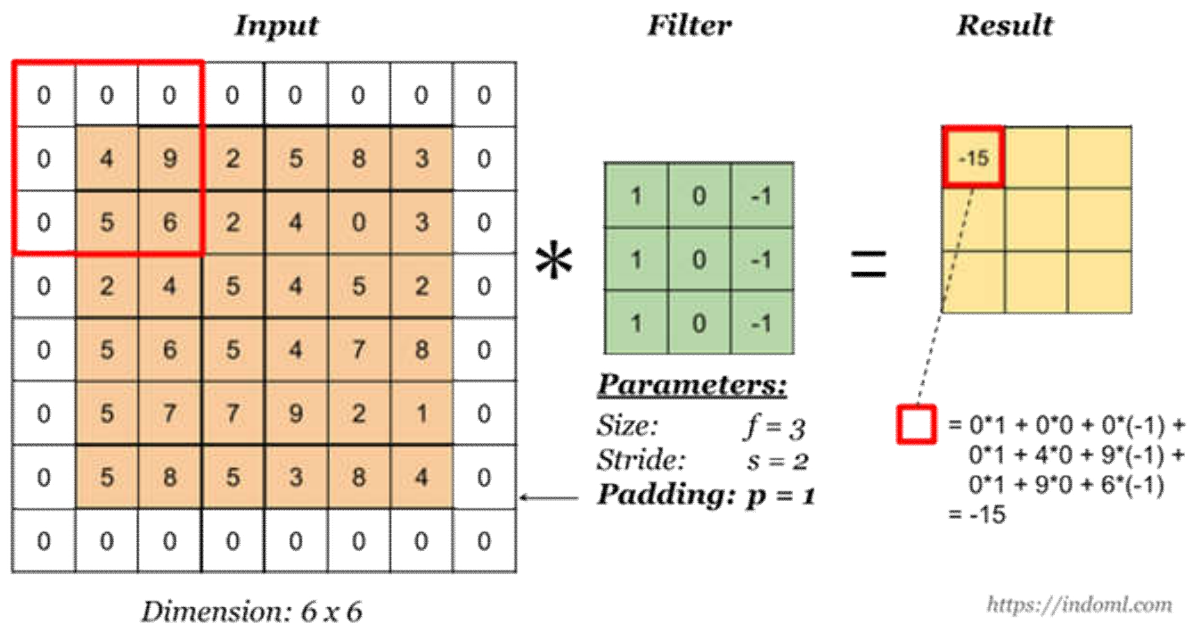
<https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/>



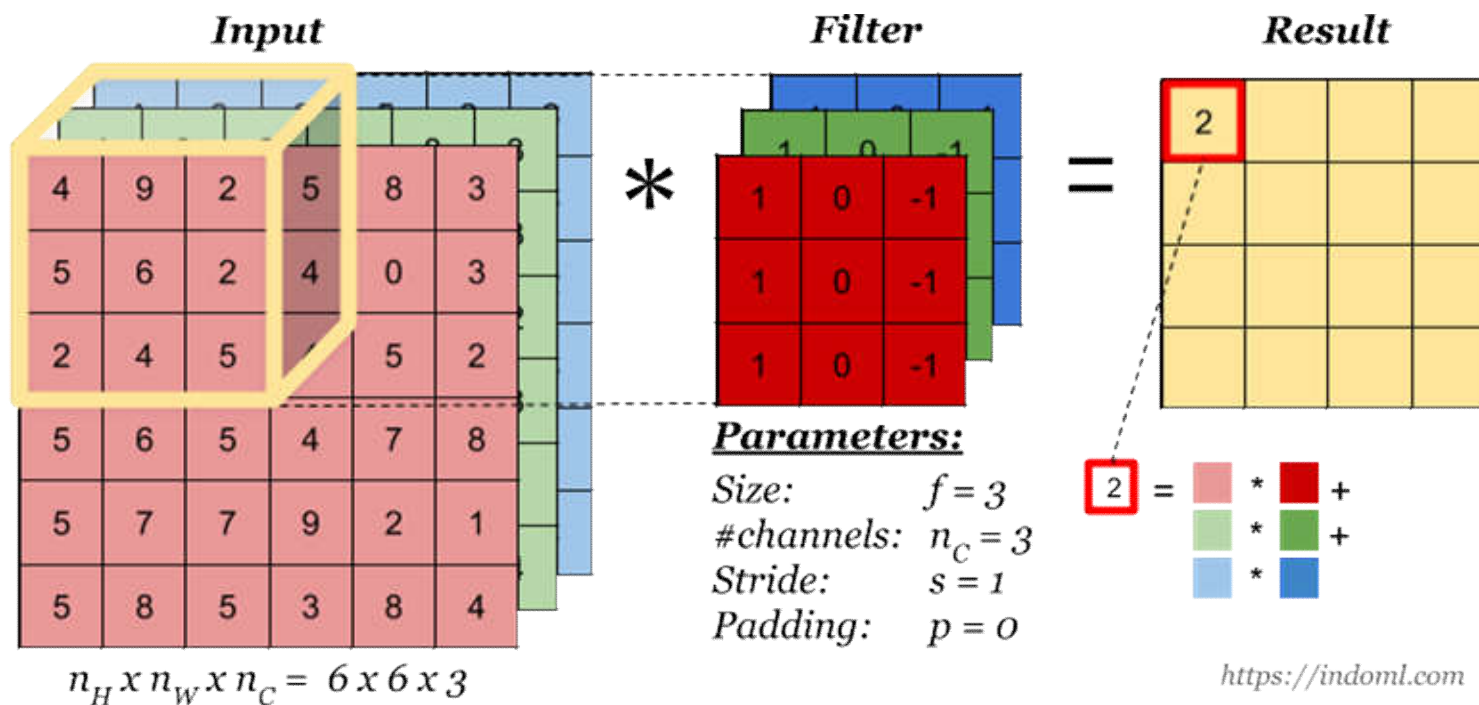
Padding

Padding :

- allows us to use a CONV layer without necessarily shrinking the height and width the height/width would shrink as we go to deeper layers.
- helps us keep more of the information at the border of an image. Without padding, very few values at the next layer would be affected by pixels as the edges of an image.



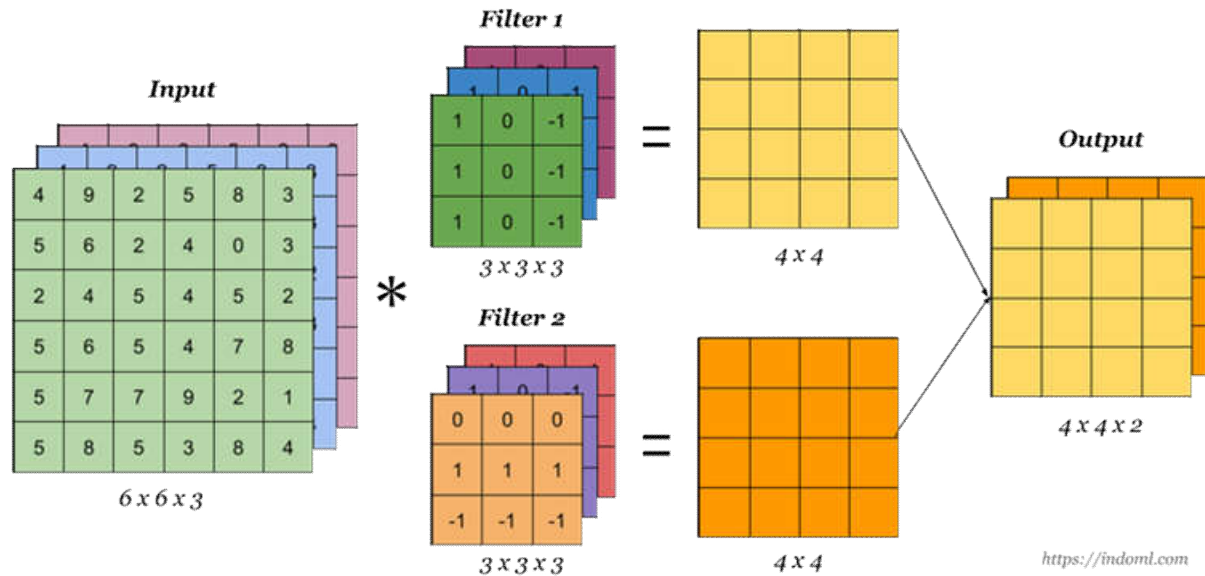
Convolution for RGB



```
from keras.layers import Input, Conv2D
in_ = Input(shape=(6,6,3))
layer = Conv2D(filters=12,kernel_size=(3,3))
out_ = layer(in_)
print("Weights shape: {}".format(layer.get_weights()[0].shape))
```

⇒ Weights shape: (3, 3, 3, 12)

Convolution with multiple filters



```
from keras.layers import Input, Conv2D
in_ = Input(shape=(6,6,3))
layer = Conv2D(filters=12,kernel_size=(3,3))
out_ = layer(in_)
print("Weights shape: {}".format(layer.get_weights()[0].shape))
```

➡ Weights shape: (3, 3, 3, 12)

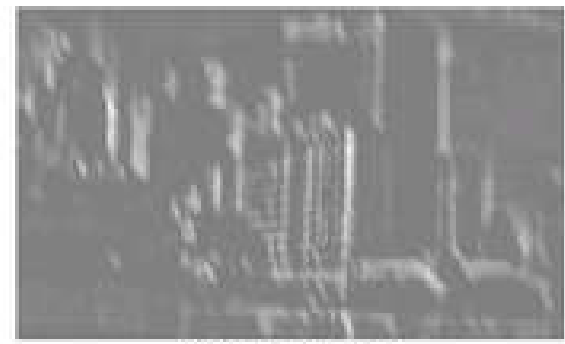
Nonlinearity (activation)



Original Image

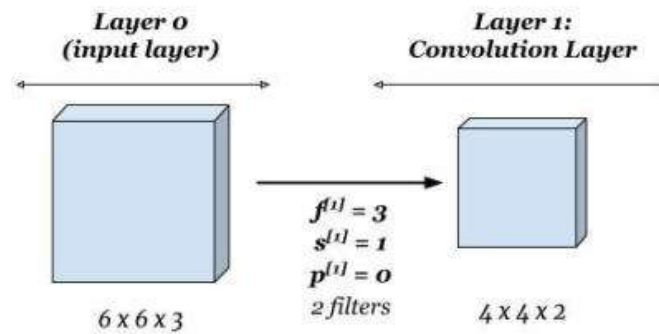
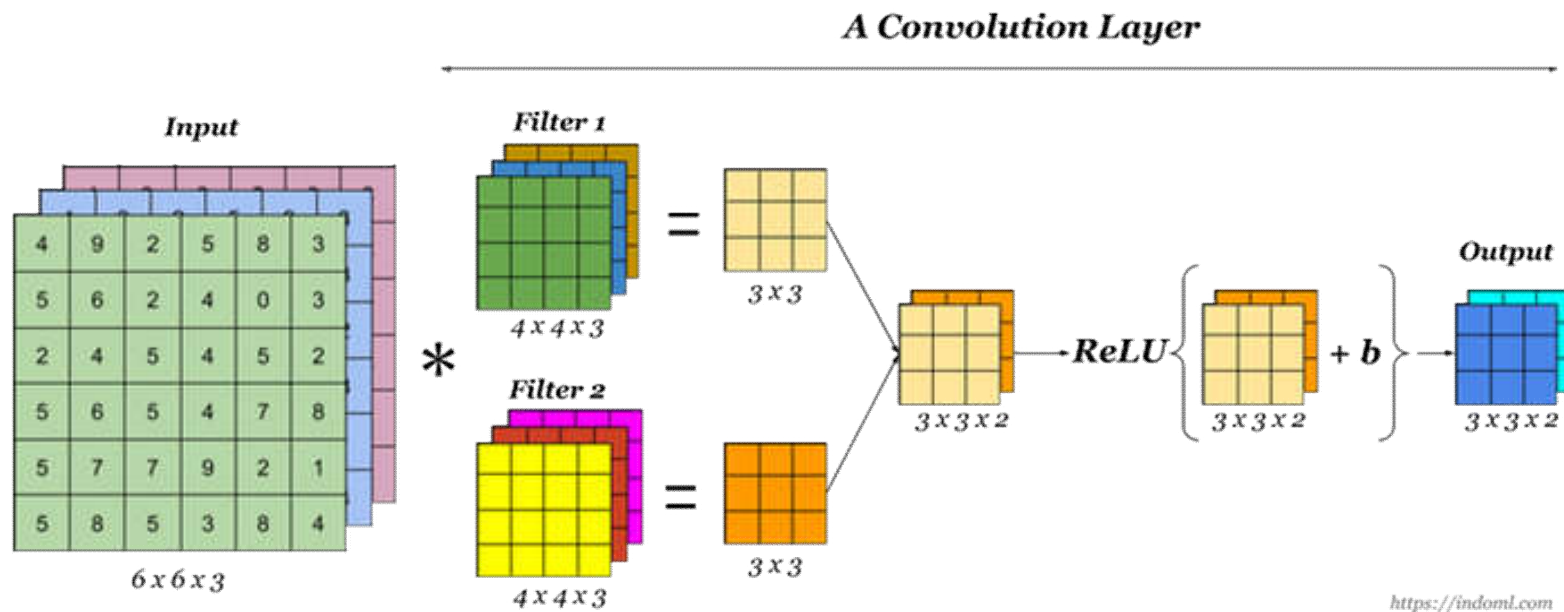


Feature Map



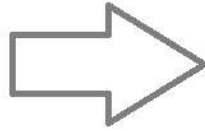
Non-Linear

Convolutional Layer



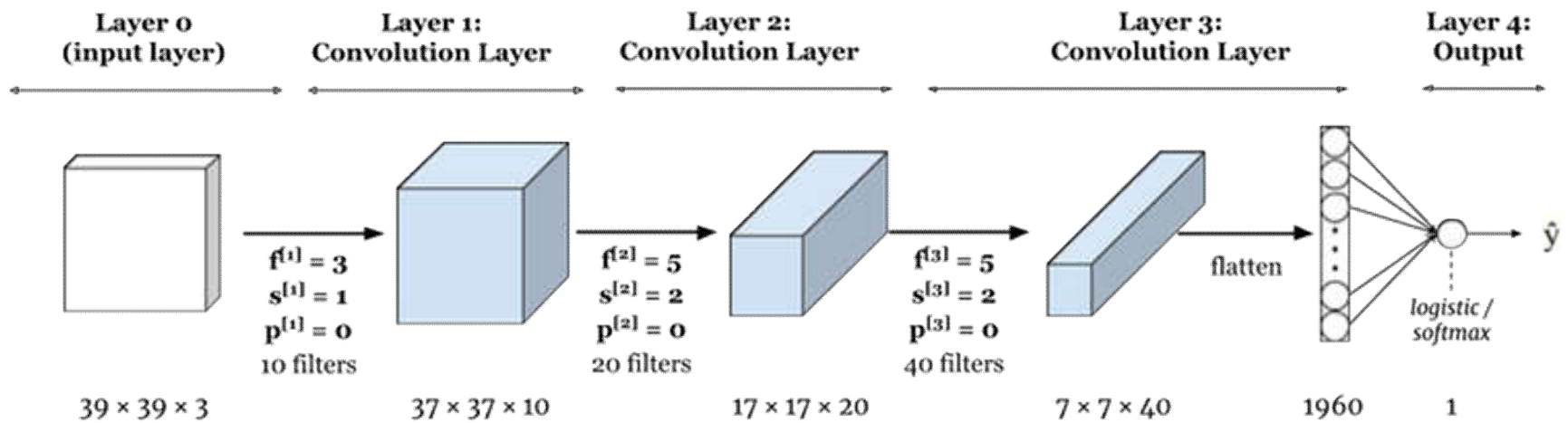
Flattening Layer

1	1	0
4	2	1
0	2	1



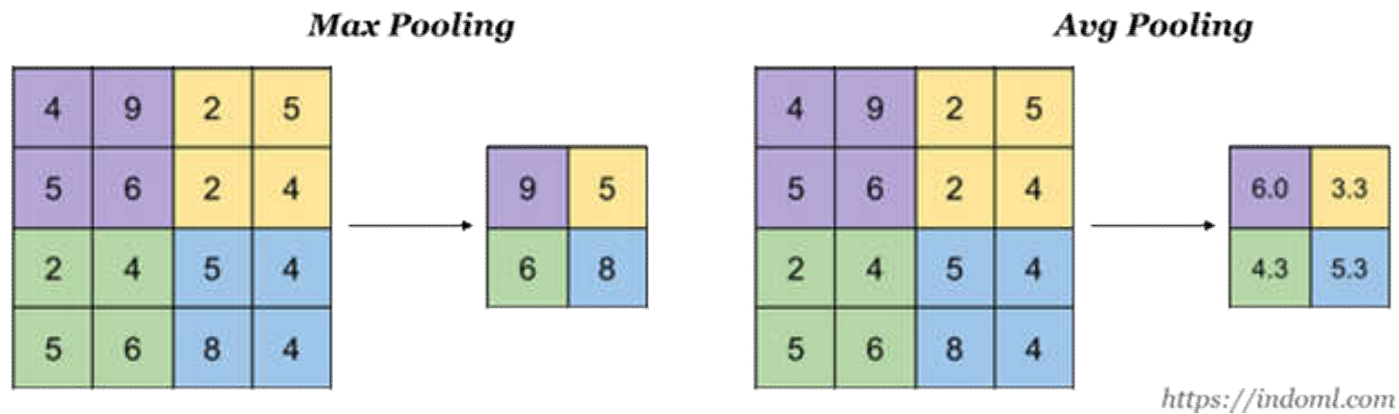
1
1
0
4
2
1
0
2
1

Sample complete network



<https://indoml.com>

Pooling Layer

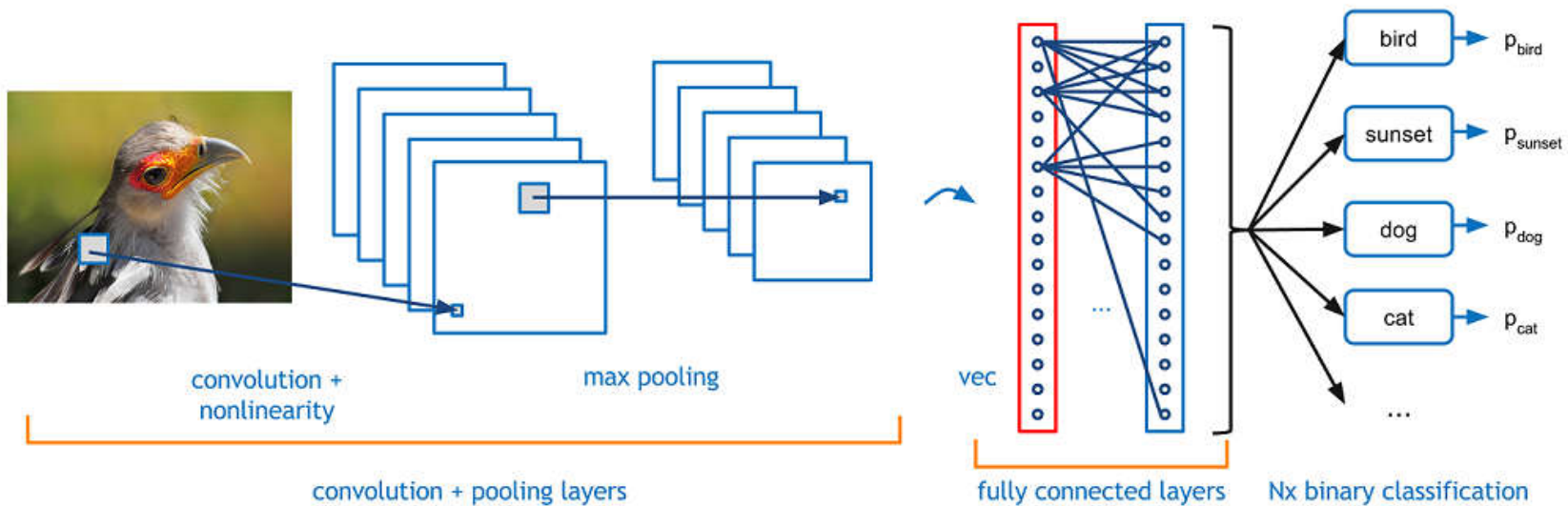


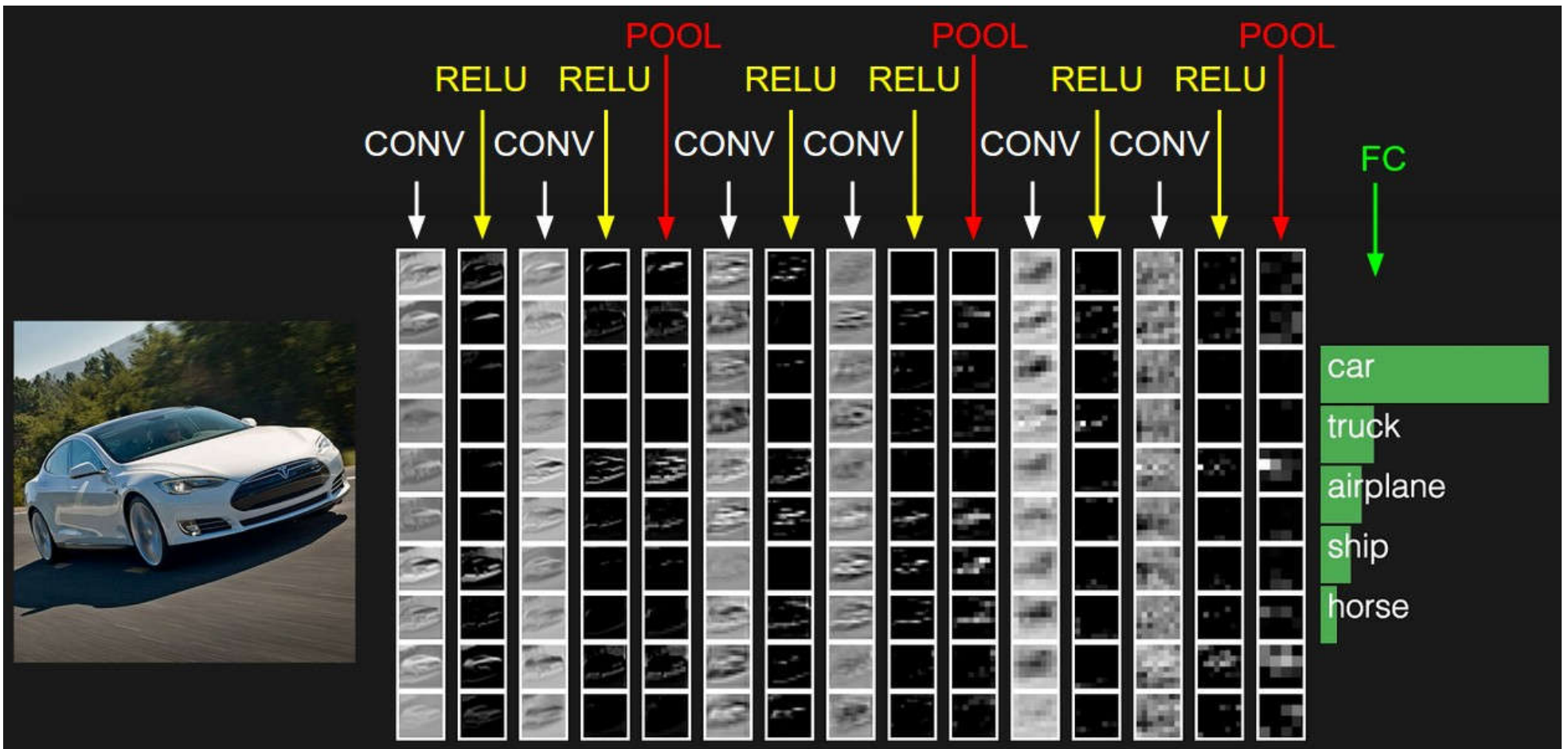
Interesting properties of pooling layer:

- it has hyper-parameters:
 - **size** (f)
 - **stride** (s)
 - **type** (max or avg)
- but it doesn't have parameter; there's nothing for gradient descent to learn

Fully-Connected Layer

This layer takes an input volume (whatever the output is of the conv or ReLU or pool layer preceding it) and outputs an N dimensional vector where N is the number of classes





CIFAR-10

airplane



automobile



bird



cat



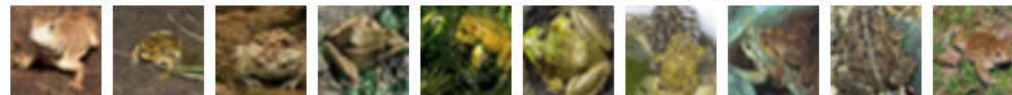
deer



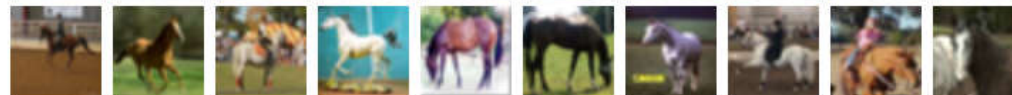
dog



frog



horse



ship



truck



CIFAR-10, CNN example

```
from keras.layers import Conv2D, MaxPooling2D, AveragePooling2D
from keras.layers import Dropout, Flatten, Dense
from keras.models import Sequential
from keras.layers.advanced_activations import LeakyReLU

from keras.datasets import cifar10
(train_features, train_labels), (test_features, test_labels) = cifar10.load_data()

print(train_features.shape)      (50000, 32, 32, 3)
print(test_features.shape)      (10000, 32, 32, 3)
```

```
train_features = train_features.astype('float32')/255
test_features = test_features.astype('float32')/255

from keras.utils import np_utils

num_classes = 10
train_labels = np_utils.to_categorical(train_labels, num_classes)
test_labels = np_utils.to_categorical(test_labels, num_classes)
```

CIFAR-10, Basic Model

```
model = Sequential()

model.add(Conv2D(filters=16, kernel_size=(2, 2), padding="same", activation="relu",
input_shape=(train_features.shape[1:]))
model.add(MaxPooling2D(pool_size=(2, 2), padding='same'))

model.add(Conv2D(filters=32, kernel_size=(3, 3), padding="same", activation="relu"))
model.add(MaxPooling2D(pool_size=(2, 2), padding='same'))

model.add(Conv2D(filters=64, kernel_size=(4, 4), padding="same", activation="relu"))
model.add(MaxPooling2D(pool_size=(2, 2), padding='same'))

model.add(Flatten())

model.add(Dense(256, activation="relu"))
model.add(Dense(10, activation="softmax"))

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(train_features, train_labels, validation_split=0.2, epochs=10, batch_size=128, verbose=1)
```

`model.summary()`

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 32, 32, 16)	208

max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 16)	0

conv2d_2 (Conv2D)	(None, 16, 16, 32)	4640

max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 32)	0

conv2d_3 (Conv2D)	(None, 8, 8, 64)	32832

max_pooling2d_3 (MaxPooling2D)	(None, 4, 4, 64)	0

flatten_1 (Flatten)	(None, 1024)	0

dense_1 (Dense)	(None, 256)	262400

dense_2 (Dense)	(None, 10)	2570
=====		
Total params: 302,650		
Trainable params: 302,650		
Non-trainable params: 0		

CIFAR-10, Advanced Model

Add Dropout and LeakyRelu to each layer

```
from keras.layers.advanced_activations import LeakyReLU
model = Sequential()
model.add(Conv2D(filters=16, kernel_size=(2, 2), padding="valid", input_shape=(train_features.shape[1:])))

model.add(Dropout(0.4))

model.add(LeakyReLU(alpha=0.01))
model.add(MaxPooling2D(pool_size=(2, 2), padding='valid'))
```

Testing models:

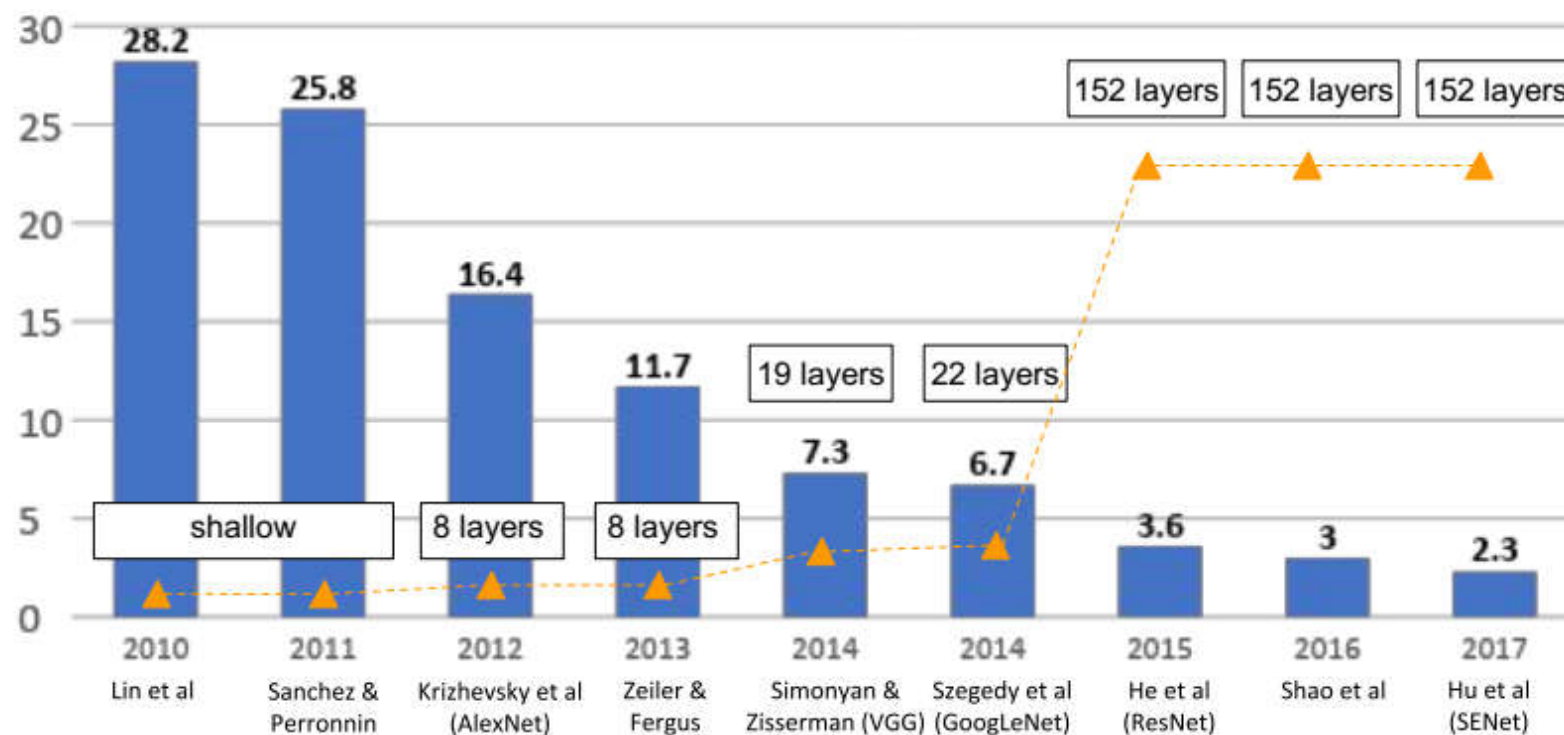
Basic model accuracy: 0.7

Advanced model accuracy: 0.76

Model with nearly 90% accuracy:

<https://appliedmachinelearning.blog/2018/03/24/achieving-90-accuracy-in-object-recognition-task-on-cifar-10-dataset-with-keras-convolutional-neural-networks/>

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

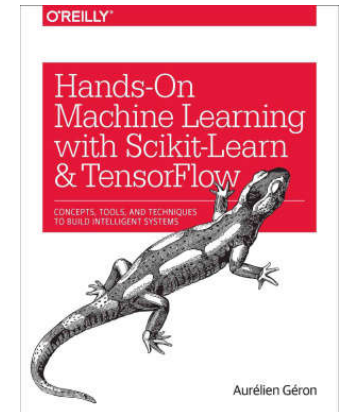


References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow'
- I.Goodfellow 'Deep Learning'

Stanford's [CS 231N course](#)



<https://github.com/yanshengjia/ml-road/tree/master/resources>

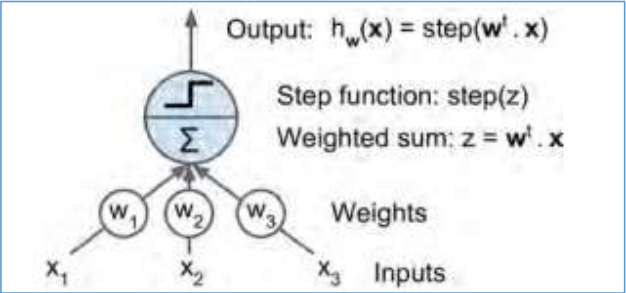
Data Analysis

Lecture 13: Recurrent Neural Networks

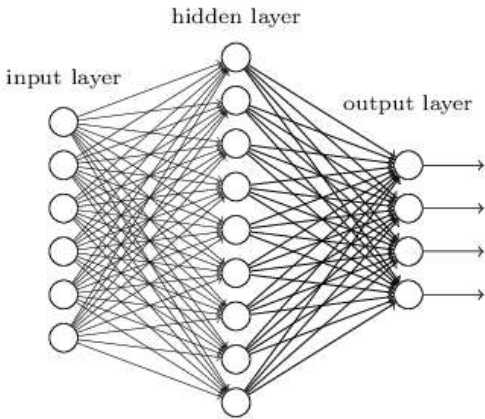
Dr. Nataliya K. Sakhnenko

Recap: ANN

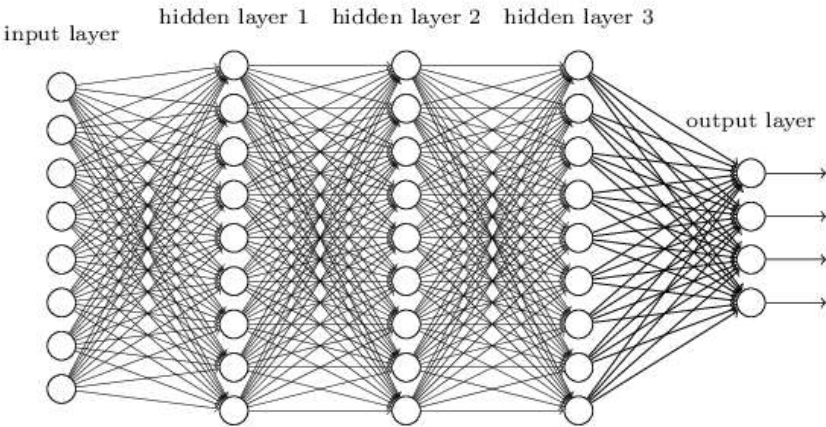
The Perceptron (1957, Frank Rosenblatt)



"Non-deep" feedforward neural network



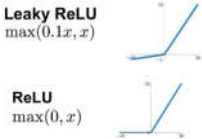
Deep neural network



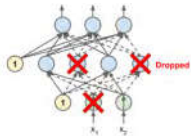
Multilayered Perceptron (MLP)

Problem	Solution
Vanishing gradients problem	Nonsaturating activation functions Batch Normalization
Severely risk of overfitting	✓ Early stopping ✓ l1 and l2 regularization ✓ Dropout
Training is extremely slow	Optimization of gradient descent

Nonsaturating activation functions

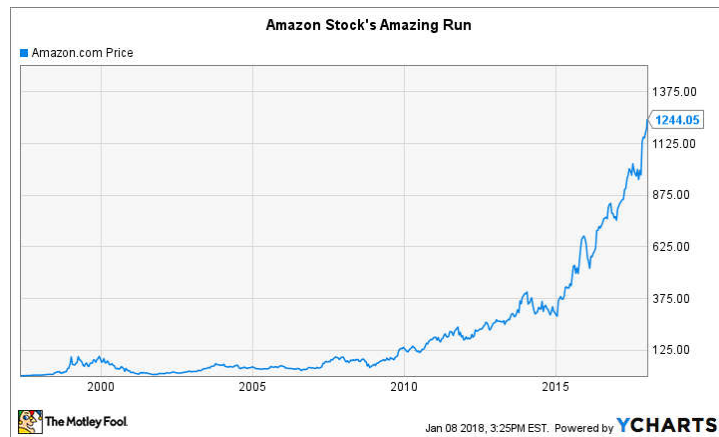


Dropout:

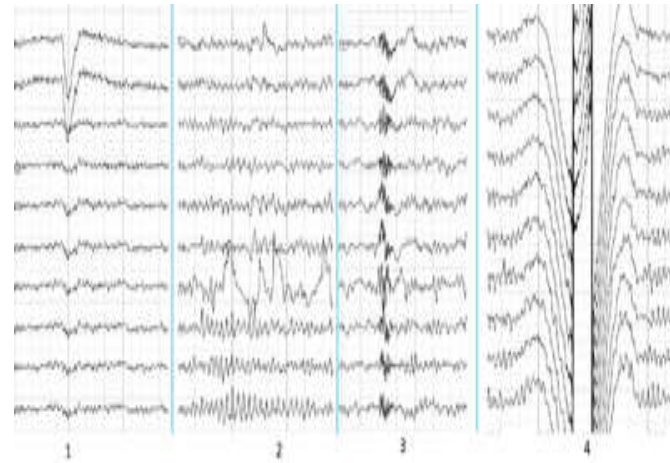


Time series and sequential data

A **time series** is a series of data points indexed in time order



Amazon stock price



EEG

NLP tasks

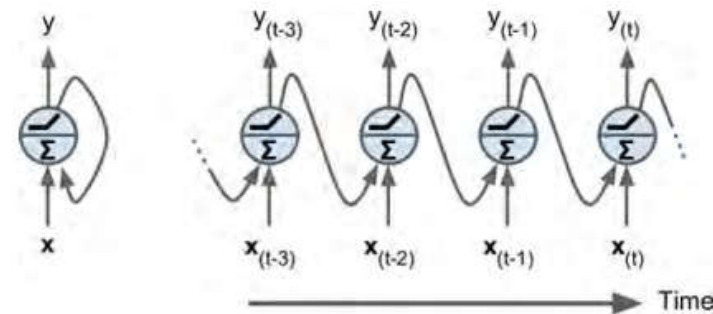
"You mean to imply that I have nothing to eat out of.... On the contrary, I can supply you with everything even if you want to give dinner parties," warmly replied Chichagov, who tried by every word he spoke to prove his own rectitude and therefore imagined Kutuzov to be animated by the same desire.

Kutuzov, shrugging his shoulders, replied with his subtle penetrating smile: "I meant merely to say what I said."

Recurrent neural network (RNN)

A **recurrent neural network (RNN)** is a class of ANNs where connections between nodes form a directed graph along a temporal sequence. This allows it to exhibit temporal dynamic behavior.

Recurrent neuron:



Basic structure

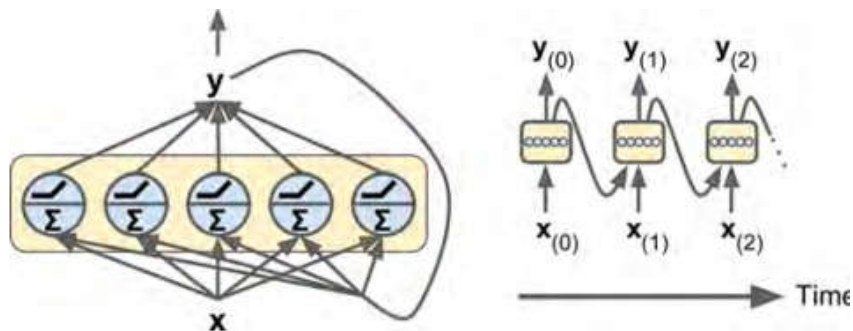
Unfolding in time

At each time step t this recurrent neuron receives the inputs x_n as well as its own output from the previous time step, $y_{(t-1)}$

$$y_{(t)} = \phi(x_{(t)}^T \cdot w_x + y_{(t-1)}^T \cdot w_y + b)$$

Each recurrent neuron has two sets of weights: one for the inputs and other the outputs

Layer of recurrent neurons



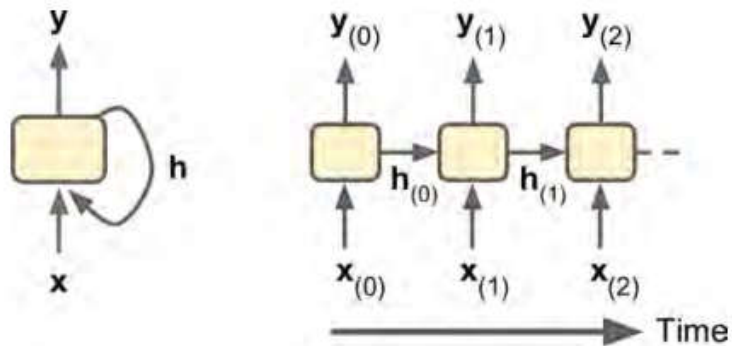
A layer of recurrent neurons Unfolding in time

- $\mathbf{Y}_{(t)}$ is an $m \times n_{\text{neurons}}$ matrix containing the layer's outputs at time step t for each instance in the mini-batch (m is the number of instances in the mini-batch and n_{neurons} is the number of neurons).
- $\mathbf{X}_{(t)}$ is an $m \times n_{\text{inputs}}$ matrix containing the inputs for all instances (n_{inputs} is the number of input features).
- $\mathbf{W}_x$ is an $n_{\text{inputs}} \times n_{\text{neurons}}$ matrix containing the connection weights for the inputs of the current time step.
- $\mathbf{W}_y$ is an $n_{\text{neurons}} \times n_{\text{neurons}}$ matrix containing the connection weights for the outputs of the previous time step.
- $\mathbf{b}$ is a vector of size n_{neurons} containing each neuron's bias term.

$$\begin{aligned} \mathbf{Y}_{(t)} &= \phi(\mathbf{X}_{(t)} \cdot \mathbf{W}_x + \mathbf{Y}_{(t-1)} \cdot \mathbf{W}_y + \mathbf{b}) \\ &= \phi\left(\begin{bmatrix} \mathbf{X}_{(t)} & \mathbf{Y}_{(t-1)} \end{bmatrix} \cdot \mathbf{W} + \mathbf{b}\right) \text{ with } \mathbf{W} = \begin{bmatrix} \mathbf{W}_x \\ \mathbf{W}_y \end{bmatrix} \end{aligned}$$

Memory cells

A cell's hidden state (h) and output (y) may be different



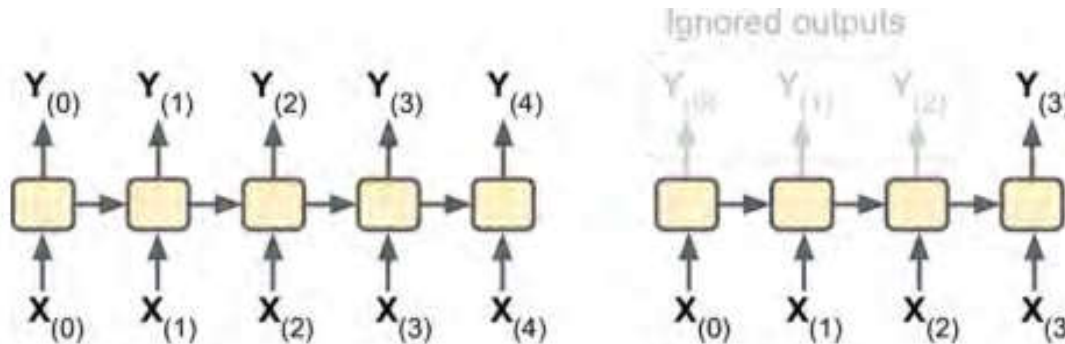
In general a cell's state at time step t , denoted $h(t)$, is a function of some inputs at that time step and its state at the previous time step

$$h_t = f_W(h_{t-1}, x_t)$$

The same function and the same set of parameters are used at every time step

Input and Output sequences

Time series forecasting



Sentiment analysis

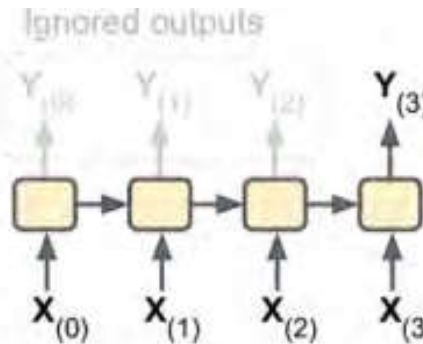
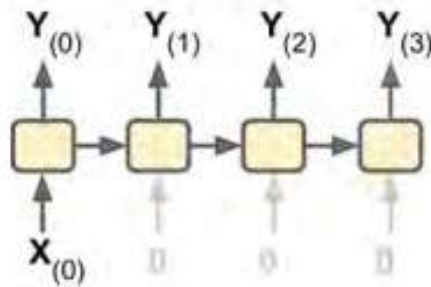
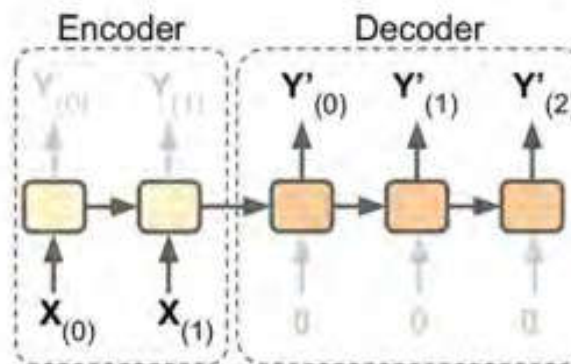


Image captioning

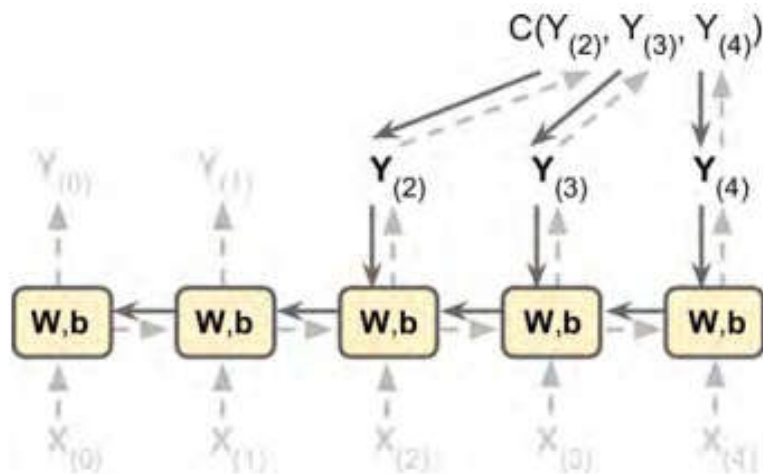


Machine translation



Seq to seq (top left), seq to vec (top right), vec to seq (bottom left), delayed seq to seq (bottom right)

Training RNNs

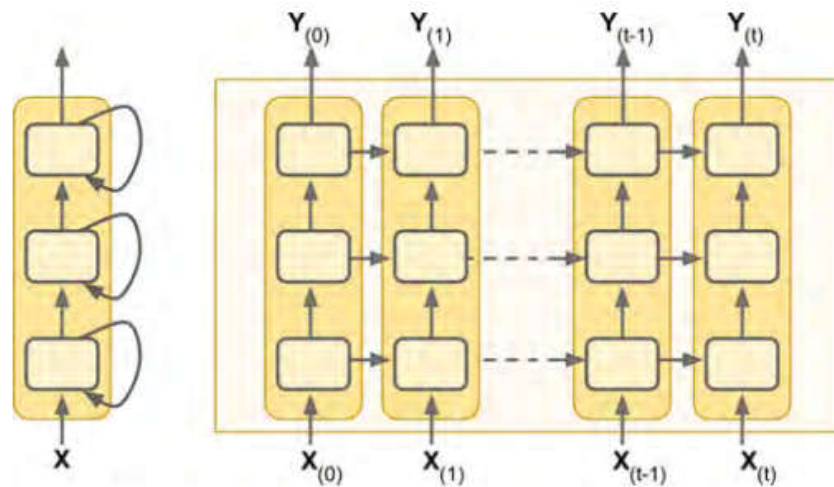


To train RNN we have to unroll it through time and then use regular backpropagation (BPTT, backpropagation through time)

Like in regular backpropagation there is a forward pass through unrolled RNN (dashed arrows); then the output sequence is evaluated using cost function $C(..)$, and the gradient of this function is propagated backward through unrolled network

Deep RNNs

It is quite common to stack multiple layers of cells. This gives you a *deep RNN*.



Deep RNN (left), unrolled through time (right)

The difficulty to train over many time steps

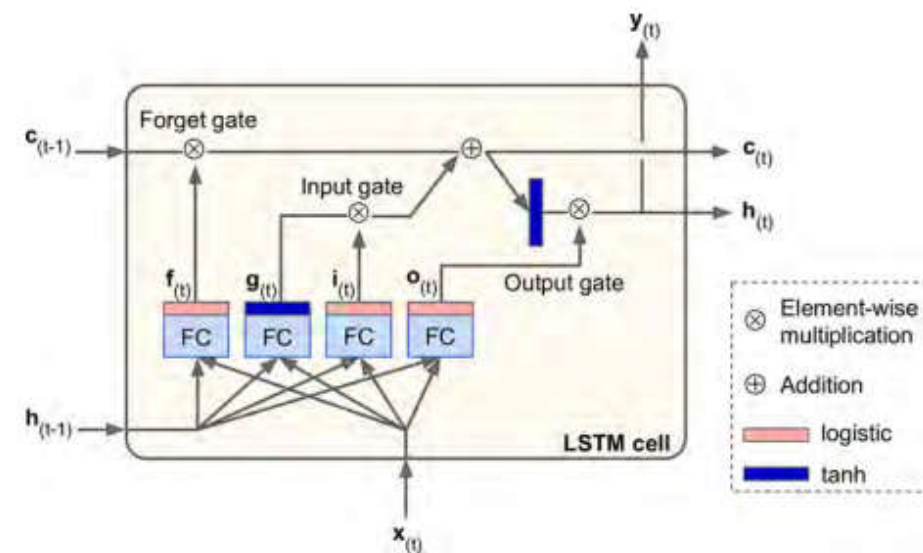
To train an RNN on long sequences, you will need to run it over many time steps, making the unrolled RNN a very deep network. Just like any deep neural network it may suffer from the vanishing/exploding gradients problem and training can be very slow.

Besides the long training time, a second problem faced by long-running RNNs is the fact that the memory of the first inputs gradually fades away.

Solution: use LSTM cell

LSTM (long Short-Term memory) cell

1997, S. Hochreiter and J. Schmidhuber



$$\mathbf{i}_{(t)} = \sigma(\mathbf{W}_{xi}^T \cdot \mathbf{x}_{(t)} + \mathbf{W}_{hi}^T \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_i)$$

$$\mathbf{f}_{(t)} = \sigma(\mathbf{W}_{xf}^T \cdot \mathbf{x}_{(t)} + \mathbf{W}_{hf}^T \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_f)$$

$$\mathbf{o}_{(t)} = \sigma(\mathbf{W}_{xo}^T \cdot \mathbf{x}_{(t)} + \mathbf{W}_{ho}^T \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_o)$$

$$\mathbf{g}_{(t)} = \tanh(\mathbf{W}_{xg}^T \cdot \mathbf{x}_{(t)} + \mathbf{W}_{hg}^T \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_g)$$

$$\mathbf{c}_{(t)} = \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \mathbf{g}_{(t)}$$

$$\mathbf{y}_{(t)} = \mathbf{h}_{(t)} = \mathbf{o}_{(t)} \otimes \tanh(\mathbf{c}_{(t)})$$

- $\mathbf{W}_{xi}$, $\mathbf{W}_{xf}$, $\mathbf{W}_{xo}$, $\mathbf{W}_{xg}$ are the weight matrices of each of the four layers for their connection to the input vector $\mathbf{x}_{(t)}$.
- $\mathbf{W}_{hi}$, $\mathbf{W}_{hf}$, $\mathbf{W}_{ho}$, and $\mathbf{W}_{hg}$ are the weight matrices of each of the four layers for their connection to the previous short-term state $\mathbf{h}_{(t-1)}$.
- $\mathbf{b}_i$, $\mathbf{b}_f$, $\mathbf{b}_o$, and $\mathbf{b}_g$ are the bias terms for each of the four layers. Note that TensorFlow initializes $\mathbf{b}_f$ to a vector full of 1s instead of 0s. This prevents forgetting everything at the beginning of training.

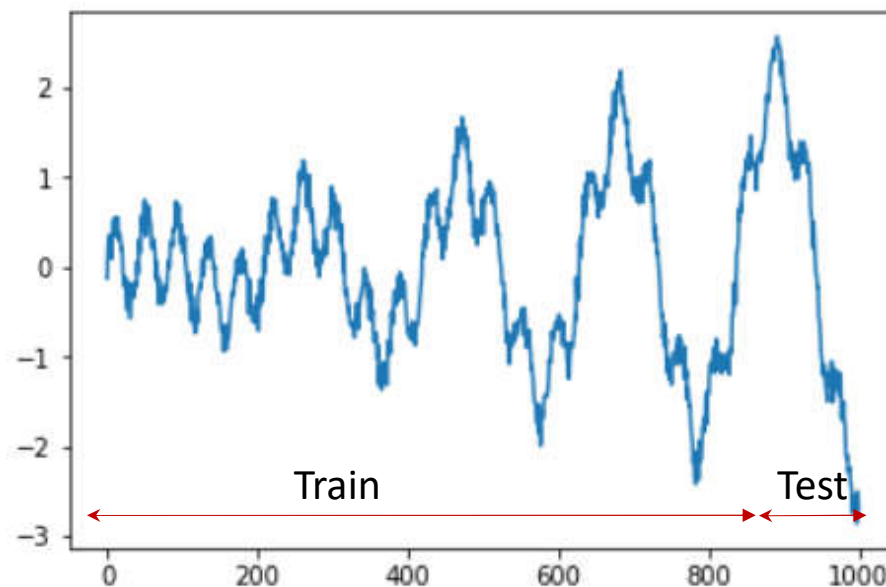
Example: time series forecasting

Generate synthetic time series

```
time_series_length = 1000
t = np.linspace(0, 30, time_series_length)
ts = t * np.sin(t) / 3 + 2 * np.sin(t*5) + 0.5 * np.random.standard_normal(size=time_series_length)
# normalize the dataset
ts_norm = (ts - np.mean(ts)) / np.std(ts)
```

Train/Test split

```
train_size = int(len(ts) * 0.9)
test_size = len(ts) - train_size
train, test = ts_norm[0:train_size],
ts_norm[train_size:len(ts)]
```



Example: time series forecasting

```
def create_dataset(dataset, look_back=1):  
    dataX, dataY = [], []  
    for i in range(len(dataset)-look_back-1):  
        a = dataset[i:(i+look_back)]  
        dataX.append(a)  
        dataY.append(dataset[i + look_back])  
    return np.array(dataX), np.array(dataY)
```

```
n_steps = 10  
n_inputs = 1  
  
look_back = n_steps  
trainX, trainY = create_dataset(train, look_back)  
testX, testY = create_dataset(test, look_back)
```

```
# reshape input to be [samples, n_steps, n_inputs]  
trainX = np.reshape(trainX, (trainX.shape[0], n_steps, n_inputs))  
testX = np.reshape(testX, (testX.shape[0], n_steps, n_inputs))
```

Example: time series forecasting

```
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM

n_neurons = 100

model = Sequential()
model.add(LSTM(n_neurons, input_shape=(n_steps, n_inputs), return_sequences=False,
activation = 'tanh'))
model.add(Dense(1))
model.compile(loss='mse', optimizer='adam')
model.summary()
```

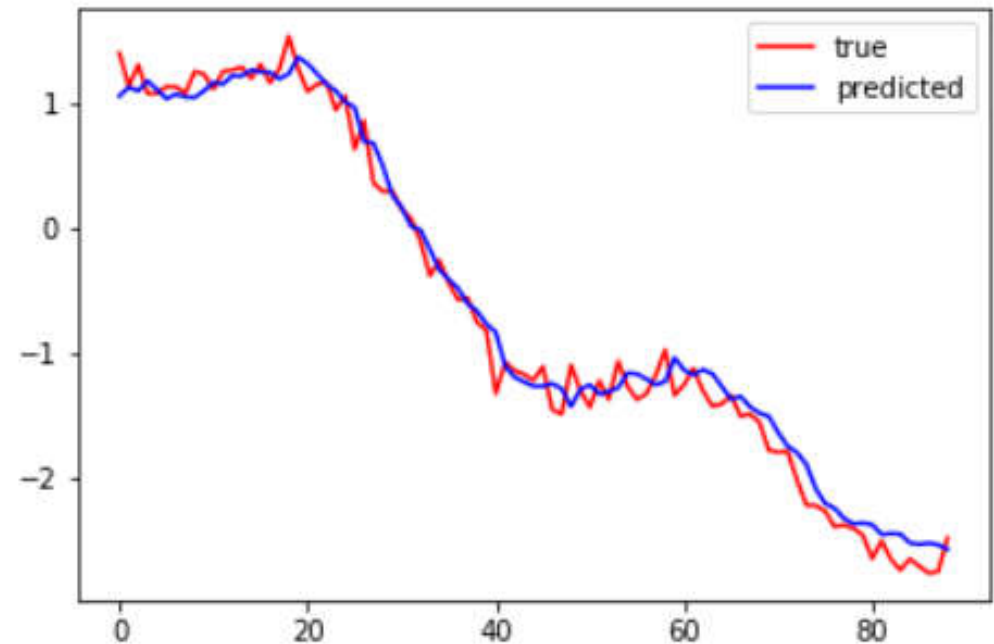
Layer (type)	Output Shape	Param #
lstm_4 (LSTM)	(None, 100)	40800
dense_4 (Dense)	(None, 1)	101

Total params: 40,901
Trainable params: 40,901
Non-trainable params: 0

Example: time series forecasting

```
model.fit(trainX, trainY, epochs=50, batch_size=20, verbose=1)  
testY_pred = model.predict(testX)
```

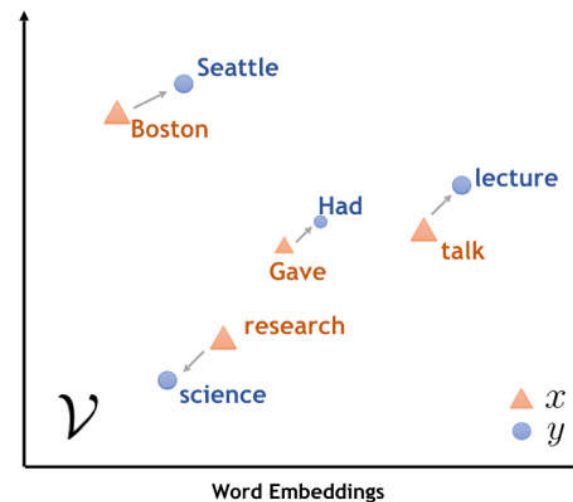
```
plt.plot(testY, 'r', label = 'true')  
plt.plot(testY_pred, 'b', label = 'predicted')  
plt.legend()
```



Natural Language Processing

Word Embeddings

Word embedding is the collective name for a set of feature learning techniques in NLP where words or phrases from the vocabulary are mapped to dense vectors of real numbers.



The most common solution is to represent each word in the vocabulary using a fairly small and dense vector called embedding.

Sentiment Analysis, IMDB movies



<http://ai.stanford.edu/~amaas/data/sentiment/>



- ✓ We'll be using a dataset of 50,000 movie reviews taken from IMDb.
- ✓ The data is split evenly with 25k reviews intended for training and 25k for testing your classifier.
- ✓ Each set has 12.5k positive and 12.5k negative reviews.
- ✓ IMDb lets users rate movies on a scale from 1 to 10. To label these reviews the curator of the data labeled anything with ≤ 4 stars as negative and anything with ≥ 7 stars as positive. Reviews with 5 or 6 stars were left out.

Sentiment Analysis, IMDB movies, LSTM

```
from keras.datasets import imdb
from keras.layers import Dense, Embedding
from keras.layers import LSTM
from keras.models import Sequential
from keras.preprocessing import sequence
```

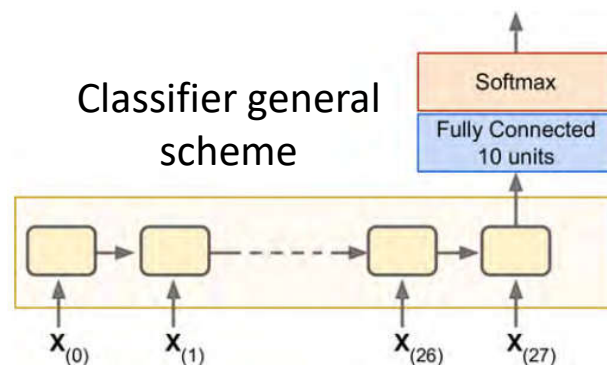
```
max_features = 20000
```

```
(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=max_features)
print(len(x_train), 'train sequences')
print(len(x_test), 'test sequences')
```

https://keras.io/examples/imdb_lstm/

Set the vocabulary size and load in training and test data
(among top max_features most common words)

25000 train sequences
25000 test sequences



Sentiment Analysis, LSTM

Padding

In order to feed data into our RNN, all input documents must have the same length. We will limit the maximum review length to maxlen by truncating longer reviews and padding shorter reviews with a null value (0)

```
maxlen = 80 # cut texts after this number of words
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)
print('x_train shape:', x_train.shape)
print('x_test shape:', x_test.shape)
```

Pad sequences

x_train shape: (25000, 80)

x_test shape: (25000, 80)

Sentiment Analysis, LSTM

Build model

```
batch_size = 32
```

```
model = Sequential()  
model.add(Embedding(max_features, 128))  
model.add(LSTM(128, activation = 'tanh'))  
model.add(Dense(1, activation='sigmoid'))
```

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
model.fit(x_train, y_train, batch_size=batch_size, epochs=15,  
validation_data=(x_test, y_test))  
score, acc = model.evaluate(x_test, y_test, batch_size=batch_size)
```

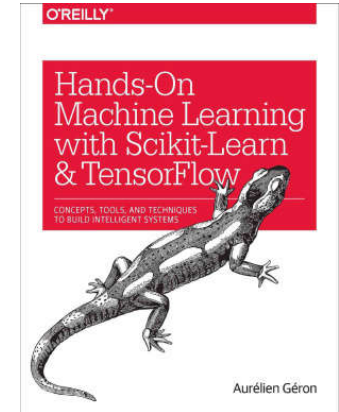
Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, None, 128)	2560000
lstm_1 (LSTM)	(None, 128)	131584
dense_1 (Dense)	(None, 1)	129
Total params: 2,691,713		
Trainable params: 2,691,713		
Non-trainable params: 0		

References

Books:

- A.Geron 'Hands-On Machine Learning with Scikit-Learn and Tensorflow'
- I.Goodfellow 'Deep Learning'

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

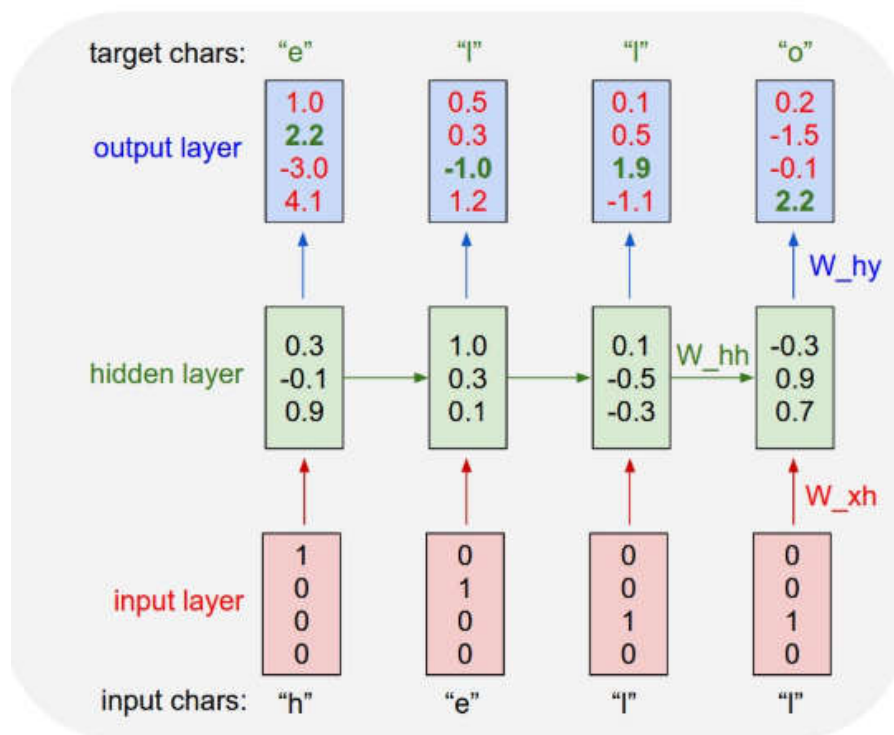


Appendix

The Unreasonable Effectiveness of Recurrent Neural Networks

May 21, 2015

Character
level
language
model



Fun with RNNs

Shakespeare

It looks like we can learn to spell English words. But how about if there is more structure and style in the data? To examine this I downloaded all the works of Shakespeare and concatenated them into a single (4.4MB) file. We can now afford to train a larger network, in this case lets try a 3-layer RNN with 512 hidden nodes on each layer. After we train the network for a few hours we obtain samples such as:

PANDARUS:

Alas, I think he shall be come approached and the day
When little strain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,
Breaking and strongly should be buried, when I perish
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and

Fun with RNNs

Algebra

For $\bigoplus_{n=1, \dots, m} \mathcal{L}_{m,n} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X , U is a closed immersion of S , then $U \rightarrow T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparicoly in the fibre product covering we have to prove the lemma generated by $\coprod Z \times_U U \rightarrow V$. Consider the maps M along the set of points Sch_{ppf} and $U \rightarrow U$ is the fibre category of S in U in Section, ?? and the fact that any U affine, see Morphisms, Lemma ?? . Hence we obtain a scheme S and any open subset $W \subset U$ in $\text{Sh}(G)$ such that $\text{Spec}(R') \rightarrow S$ is smooth or an

$$U = \bigcup U_i \times_S U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S . We claim that $\mathcal{O}_{X,x}$ is a scheme where $x, x', x'' \in S^*$ such that $\mathcal{O}_{X,x'} \rightarrow \mathcal{O}_{X',x'}$ is separated. By Algebra, Lemma ?? we can define a map of complexes $\text{GL}_S(x'/S'')$ and we win. $\square$

To prove study we see that $\mathcal{F}|_U$ is a covering of $\mathcal{X}'$, and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X/S}$ for $i > 0$ and $\mathcal{F}_p$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$ -modules on $\mathcal{C}$ as a $\mathcal{F}$ -module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\bar{M}^* = \mathcal{I}^* \otimes_{\text{Spec}(k)} \mathcal{O}_{S,s} - i_X^{-1} \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (\text{Sch}/S)_{\text{ppf}}^{\text{opp}}, (\text{Sch}/S)_{\text{ppf}}$$

and

$$V = \Gamma(S, \mathcal{O}) \rightarrow (U, \text{Spec}(A))$$

is an open subset of X . Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S .

Proof. See discussion of sheaves of sets. $\square$

The result for prove any open covering follows from the less of Example ?? . It may replace S by $X_{\text{spaces}, \text{étale}}$ which gives an open subspace of X and T equal to S_{Zar} , see Descent, Lemma ?? . Namely, by Lemma ?? we see that R is geometrically regular over S .

Lemma 0.1. Assume (3) and (3) by the construction in the description.

Suppose $X = \lim |X|$ (by the formal open covering X and a single map $\text{Proj}_X(\mathcal{A}) = \text{Spec}(B)$ over U compatible with the complex

$$\text{Set}(\mathcal{A}) = \Gamma(X, \mathcal{O}_X, \mathcal{O}_X).$$

When in this case of to show that $\mathcal{Q} \rightarrow \mathcal{C}_{Z/X}$ is stable under the following result in the second conditions of (1), and (3). This finishes the proof. By Definition ?? (without element is when the closed subschemes are catenary. If T is surjective we may assume that T is connected with residue fields of S . Moreover there exists a closed subspace $Z \subset X$ of X where U in X' is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem

(1) f is locally of finite type. Since $S = \text{Spec}(R)$ and $Y = \text{Spec}(R)$.

Proof. This is form all sheaves of sheaves on X . But given a scheme U and a surjective étale morphism $U \rightarrow X$. Let $U \cap U = \coprod_{i=1, \dots, n} U_i$ be the scheme X over S at the schemes $X_i \rightarrow X$ and $U = \lim_i X_i$. $\square$

The following lemma surjective restrocomposes of this implies that $\mathcal{F}_{x_0} = \mathcal{F}_{x_0} = \mathcal{F}_{x_0, \dots, 0}$.

Lemma 0.2. Let X be a locally Noetherian scheme over S , $E = \mathcal{F}_{X/S}$. Set $\mathcal{I} = \mathcal{I}_1 \subset \mathcal{I}_n$. Since $\mathcal{I}^n \subset \mathcal{I}^n$ are nonzero over $i_0 \leq \mathfrak{p}$ is a subset of $\mathcal{I}_{n,0} \circ \bar{A}_2$ works.

Lemma 0.3. In Situation ?? . Hence we may assume $\mathfrak{q}' = 0$.

Proof. We will use the property we see that $\mathfrak{p}$ is the next functor (??). On the other hand, by Lemma ?? we see that

$$D(\mathcal{O}_{X'}) = \mathcal{O}_X(D)$$

where K is an F -algebra where δ_{n+1} is a scheme over S . $\square$

Sampled (fake) algebraic geometry. [Here's the actual pdf.](#)

Fun with RNNs

```
static void action_new_function(struct s_stat_info *wb)
{
    unsigned long flags;
    int lel_idx_bit = e->edd, *sys & ~((unsigned long) *FIRST_COMPAT);
    buf[0] = 0xFFFFFFFF & (bit << 4);
    min(inc, slist->bytes);
    printk(KERN_WARNING "Memory allocated %02x/%02x, "
        "original MLL instead\n"),
        min(min(multi_run - s->len, max) * num_data_in),
        frame_pos, sz + first_seg);
    div_u64_w(val, inb_p);
    spin_unlock(&disk->queue_lock);
    mutex_unlock(&s->sock->mutex);
    mutex_unlock(&func->mutex);
    return disassemble(info->pending_bh);
}

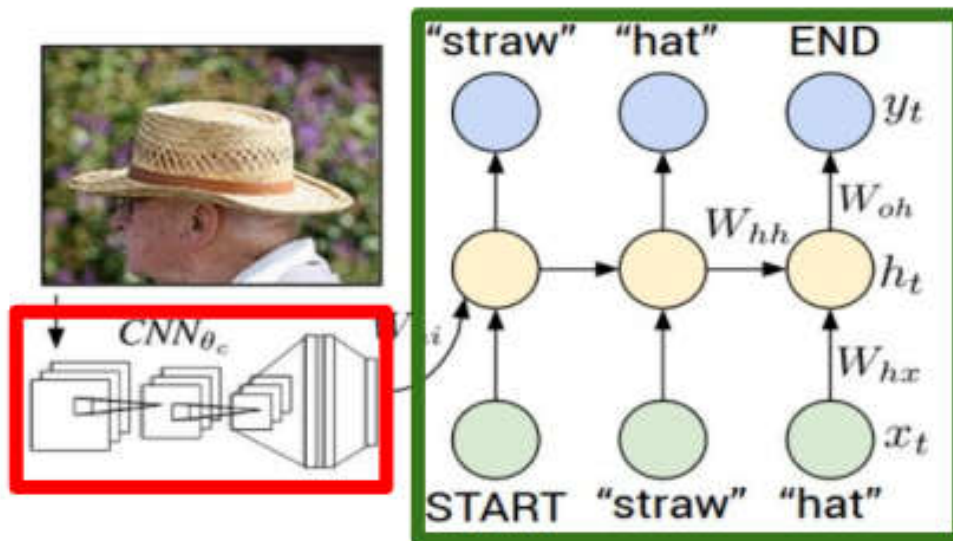
static void num_serial_settings(struct tty_struct *tty)
{
    if (tty == tty)
        disable_single_st_p(dev);
    pci_disable_spool(port);
    return 0;
}
```

The code looks really quite great overall. Of course, I don't think it compiles...

Image Captioning

CS231n, Lc 10

Recurrent Neural Network



Convolutional Neural Network

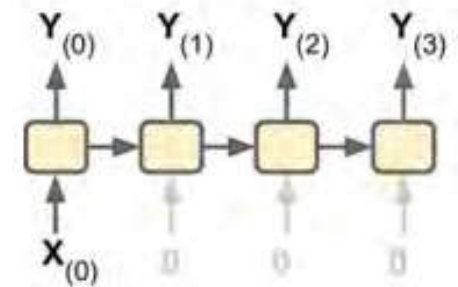


Image Captioning

CS231n, Lc 10

Image Captioning: Example Results

Captions generated using [pytorch](#)
All images are [CC0 Public Domain](#)
not suitable for use, show text,
surface, human, giraffe, motorcycle



*A cat sitting on a
suitcase on the floor*



*A cat is sitting on a tree
branch*



*A dog is running in the
grass with a frisbee*



*A white teddy bear sitting in
the grass*



*Two people walking on
the beach with surfboards*



*A tennis player in action
on the court*



*Two giraffes standing in a
grassy field*



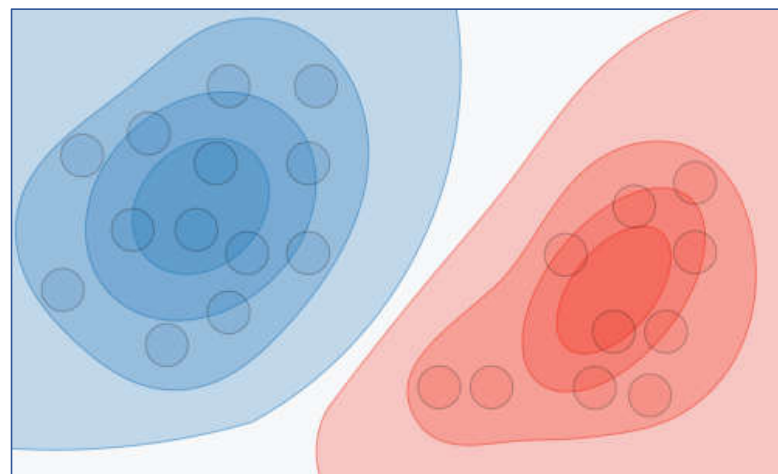
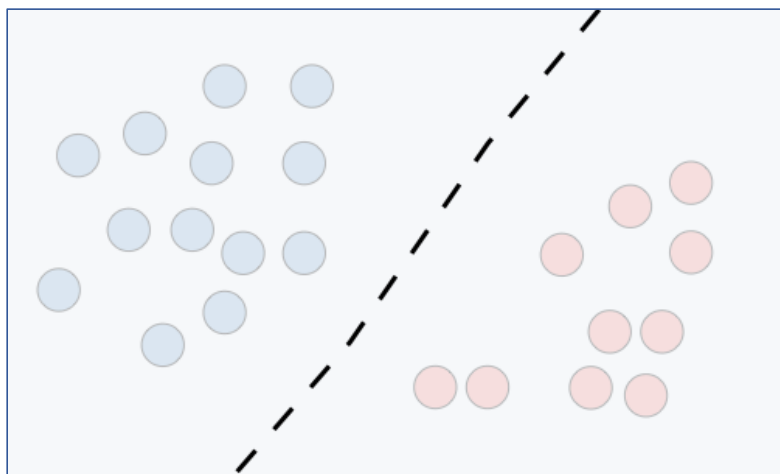
*A man riding a dirt bike on
a dirt track*

Data Analysis

Lecture 14: Generative Adversarial Networks

Dr. Nataliya K. Sakhnenko

Discriminative vs Generative Models



Discriminative models

Given an image X , predict a label Y ; estimates $P(Y|X)$

Discriminative models have several key limitations

Can't model $P(X)$; thus, can't sample from $P(X)$, i.e. **can't generate new images**

Generative models (in general) cope with all of above

Can model $P(X)$; Can generate new images

Generative Adversarial Networks

- Generative: Learn a generative model
- Adversarial: Trained in an adversarial setting
- Networks: Use Deep Neural Networks



I. Goodfellow, Generative Adversarial Nets, 2014
<https://arxiv.org/pdf/1406.2661.pdf>

“(GANs), and the variations that are now being proposed is the most interesting idea in the last 10 years in ML, in my opinion.”



--Yann LeCun, director of Facebook AI Research

Generative Adversarial Networks

4.5 years of GAN progress on face generation.

arxiv.org/abs/1406.2661

arxiv.org/abs/1511.06434

arxiv.org/abs/1606.07536

arxiv.org/abs/1710.10196

arxiv.org/abs/1812.04948



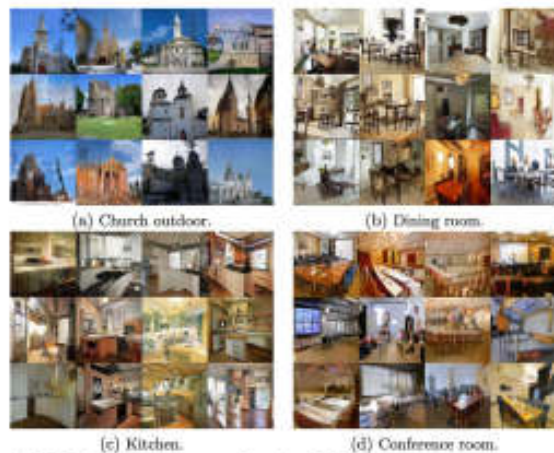
Generative Adversarial Networks



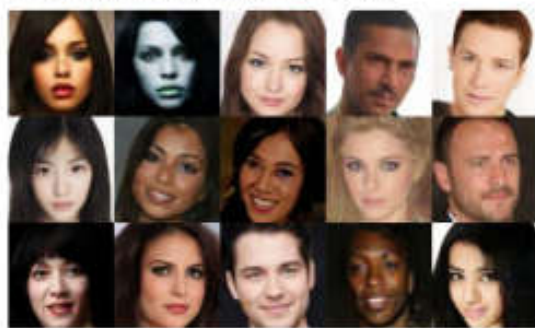
Radford et al,
ICLR 2016

2017: Year of the GAN

Better training and generation

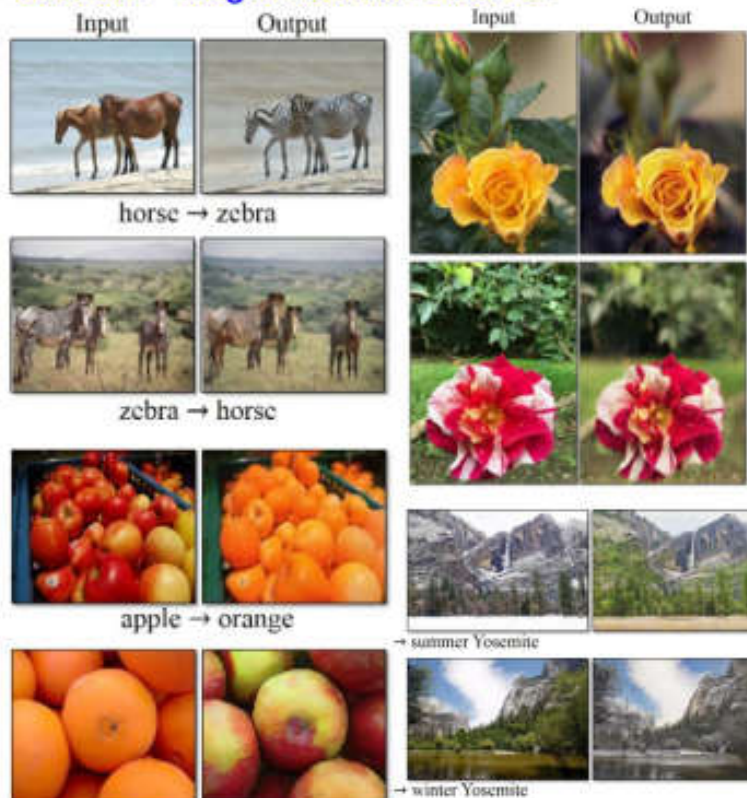


LSGAN. Mao et al. 2017.



REGAN Bertholet et al. 2017

Source->Target domain transfer



CycleGAN. Zhu et al. 2017.

Text -> Image Synthesis

this small bird has a pink breast and crown, and black primaries and secondaries. this magnificent fellow is almost all black with a red crest, and white cheek patch.



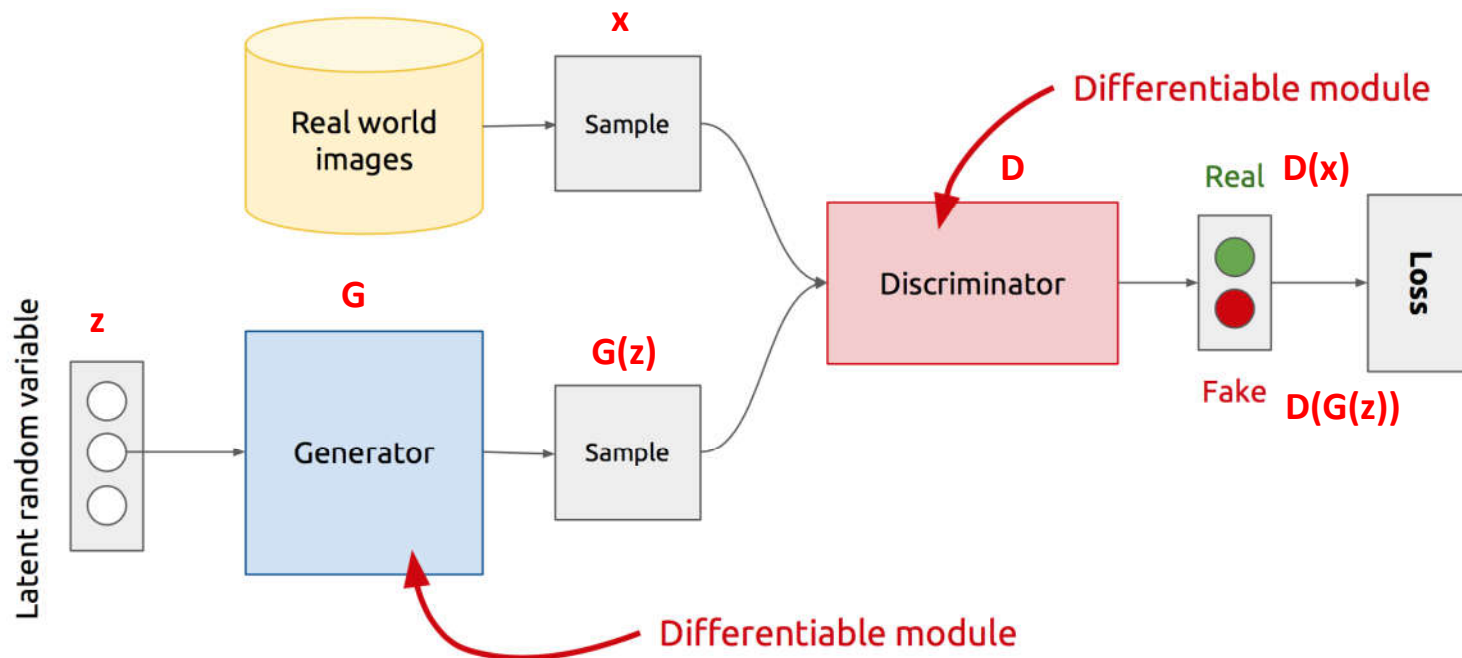
Reed et al. 2017.

Many GAN applications



Pix2pix. Isola 2017. Many examples at <https://phillipi.github.io/pix2pix/>

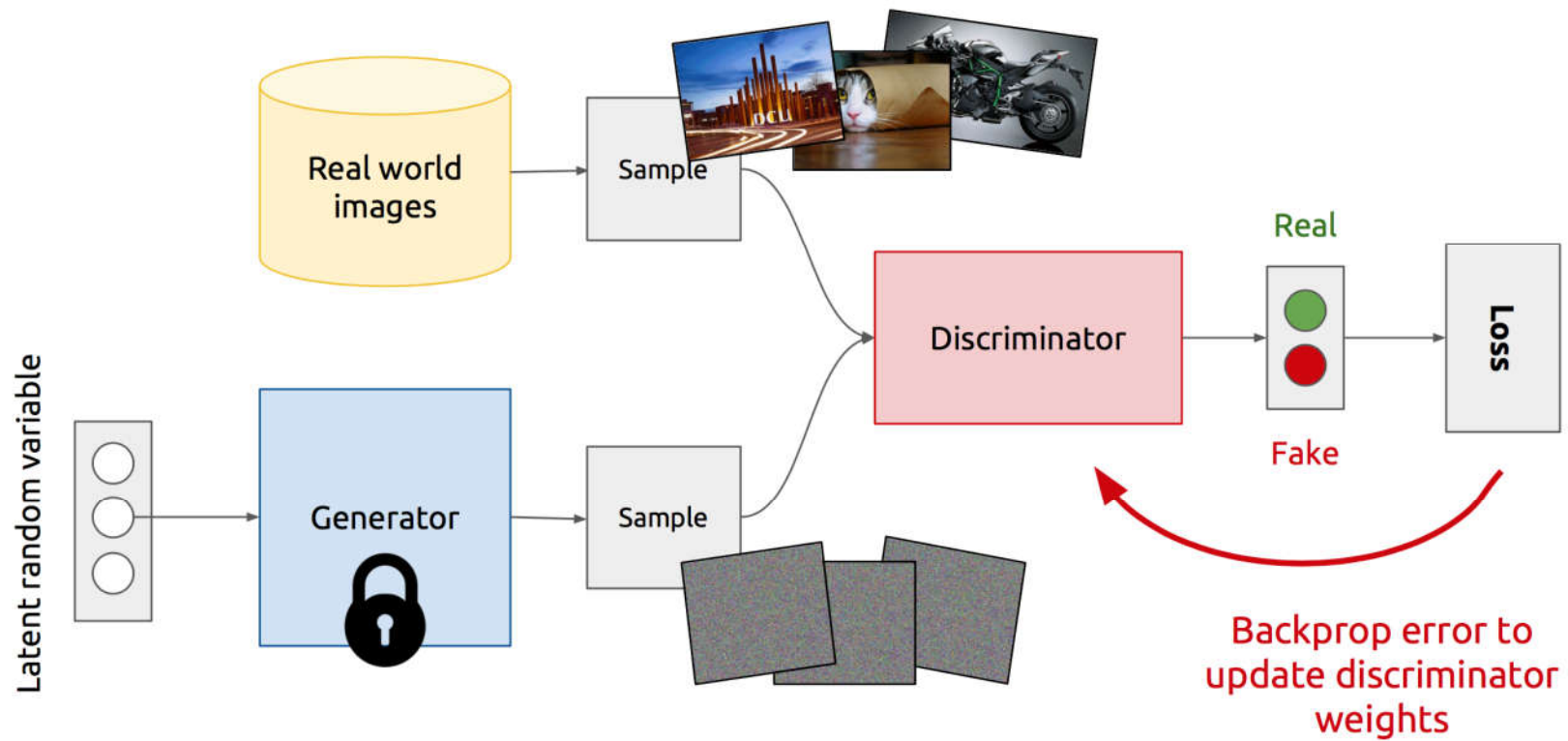
GAN's Architecture



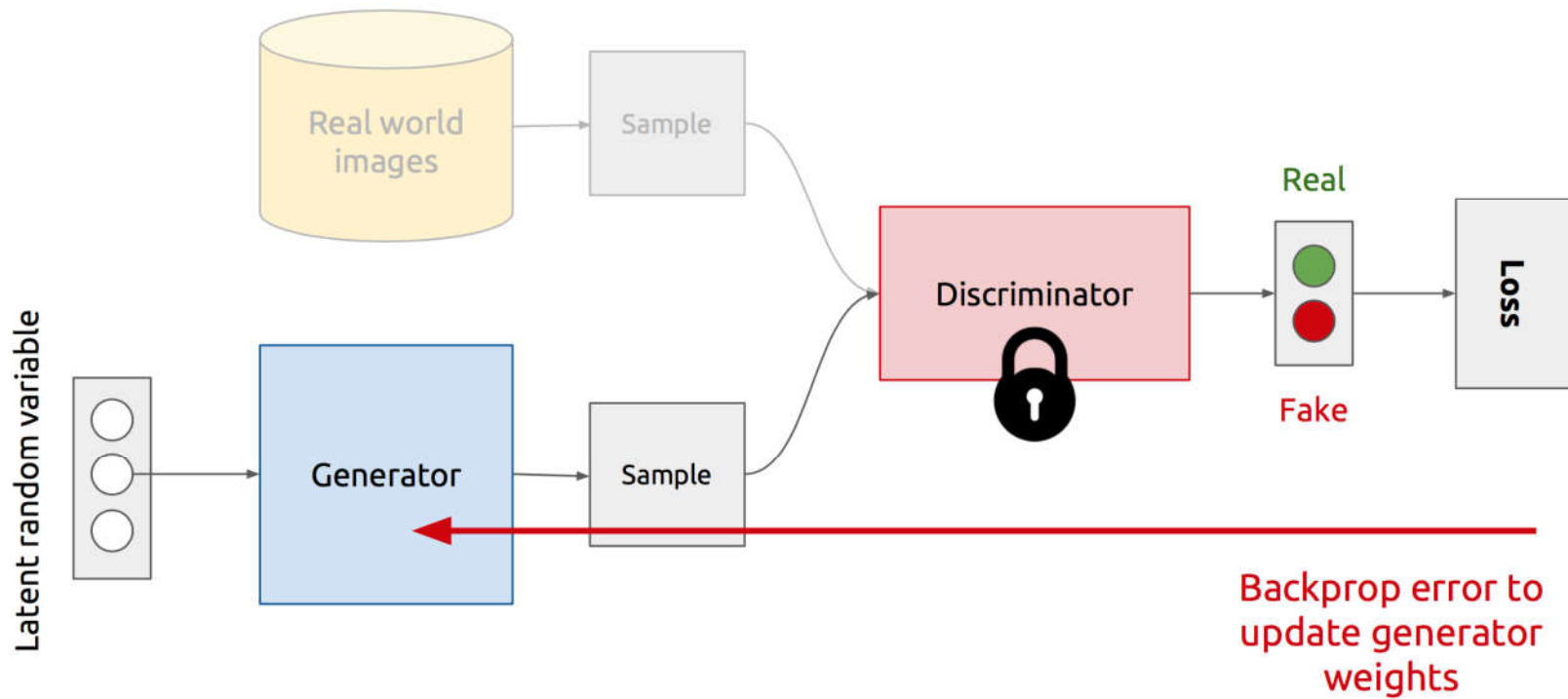
Generator: generate fake samples, tries to fool the Discriminator
Discriminator: tries to distinguish between real and fake samples
Train them against each other
Repeat this and we get better Generator and Discriminator

- **z** is some random noise (Gaussian/Uniform).
- **z** can be thought as the latent representation of the image.

Training Discriminator



Training Generator



Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images

Discriminator network: try to distinguish between real and fake images

Train jointly in **minimax game**

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- Discriminator (θ_d) wants to **maximize objective** such that $D(\mathbf{x})$ is close to 1 (real) and $D(G(\mathbf{z}))$ is close to 0 (fake)
- Generator (θ_g) wants to **minimize objective** such that $D(G(\mathbf{z}))$ is close to 1 (discriminator is fooled into thinking generated $G(\mathbf{z})$ is real)

The Nash equilibrium of this particular game is achieved at:

$$P_{\text{data}}(x) = P_{\text{gen}}(x) \quad \forall x$$
$$D(x) = \frac{1}{2} \quad \forall x$$

MNIST GAN example

```
from keras.datasets import mnist
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = (X_train.astype(np.float32) - 127.5) / 127.5
X_train = X_train.reshape(60000, 784)
```

Discriminator

```
def build_discriminator():
    discriminator = Sequential()
    discriminator.add(Dense(..., input_dim=784))
    discriminator.add(LeakyReLU(0.2))
    ....
    discriminator.add(Dense(1, activation='sigmoid'))
    discriminator.compile(loss='binary_crossentropy', optimizer='adam')
    return discriminator
```

Code: <https://pathmind.com/wiki/generative-adversarial-network-gan>

MNIST GAN example

Latent random variable: `random_dim = 100`

Generator

```
def build_generator():  
    generator = Sequential()  
    generator.add(Dense(..., input_dim=random_dim))  
    generator.add(LeakyReLU(0.2))  
    .....  
    generator.add(Dense(784, activation='tanh'))  
    generator.compile(loss='binary_crossentropy', optimizer=adam)  
    return generator
```

MNIST GAN example

GAN

```
generator = build_generator()
discriminator = build_discriminator()

# Combined network
discriminator.trainable = False
gan_input = Input(shape=(random_dim,))
x = generator(gan_input)
gan_output = discriminator(x)
gan = Model(inputs=gan_input, outputs=gan_output)
gan.compile(loss='binary_crossentropy', optimizer=adam)
```

MNIST GAN example

Train function (for 1 iteration)

```
noise = np.random.normal(0, 1, size=[batch_size, random_dim])
image_batch = X_train[np.random.randint(0, X_train.shape[0], size=batch_size)]

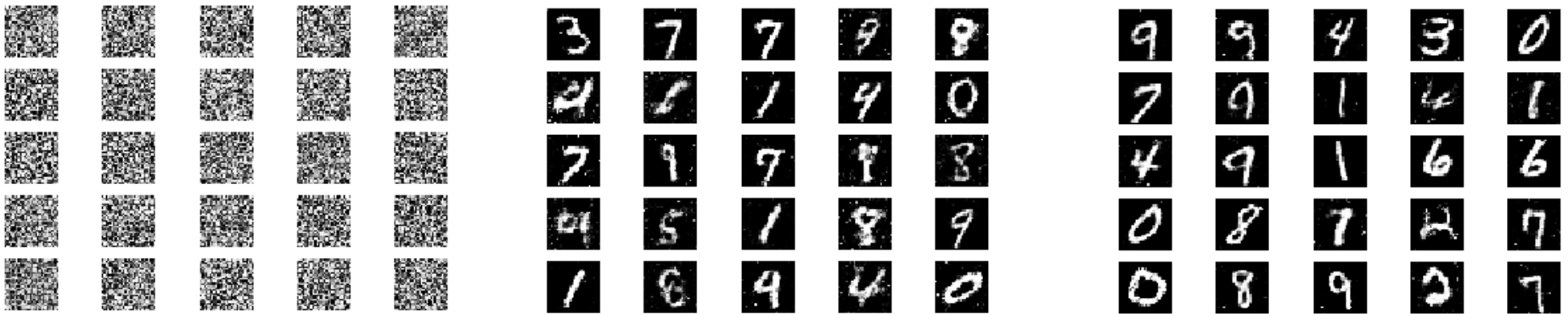
generated_images = generator.predict(noise)
X = np.concatenate([generated_images, image_batch])

# Labels for generated and real data
y_dis = np.zeros(2*batch_size)
y_dis[:batch_size] = 1.0
# Train discriminator
discriminator.trainable = True
discriminator.train_on_batch(X, y_dis)

# Train generator
noise = np.random.normal(0, 1, size=[batch_size, random_dim])
y_gen = np.ones(batch_size)
discriminator.trainable = False
gan.train_on_batch(noise, y_gen)
```

MNIST GAN example

```
noise = np.random.normal(0, 1, size=[100, random_dim])  
generated_images = generator.predict(noise)
```



Samples generated by GAN on different epochs

Conditional GAN, 2014

<https://arxiv.org/abs/1411.1784>

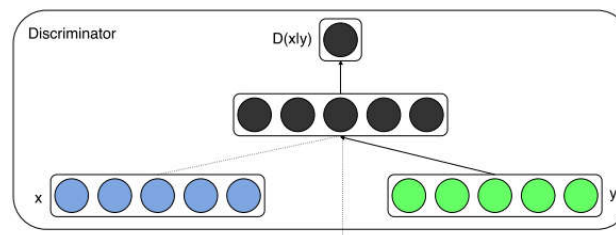
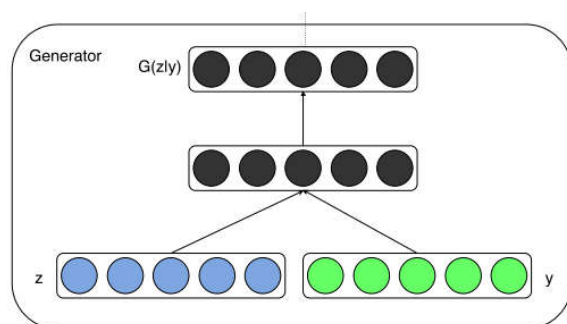
GANs can be extended to a conditional model if both the generator and discriminator are conditioned on some extra information (e.g label)

GAN objective function

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

transforms to

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x}|\mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z}|\mathbf{y})))]$$



Code: <https://github.com/eriklindernoren/Keras-GAN/blob/master/cgan/cgan.py>

Conditional GAN, 2014

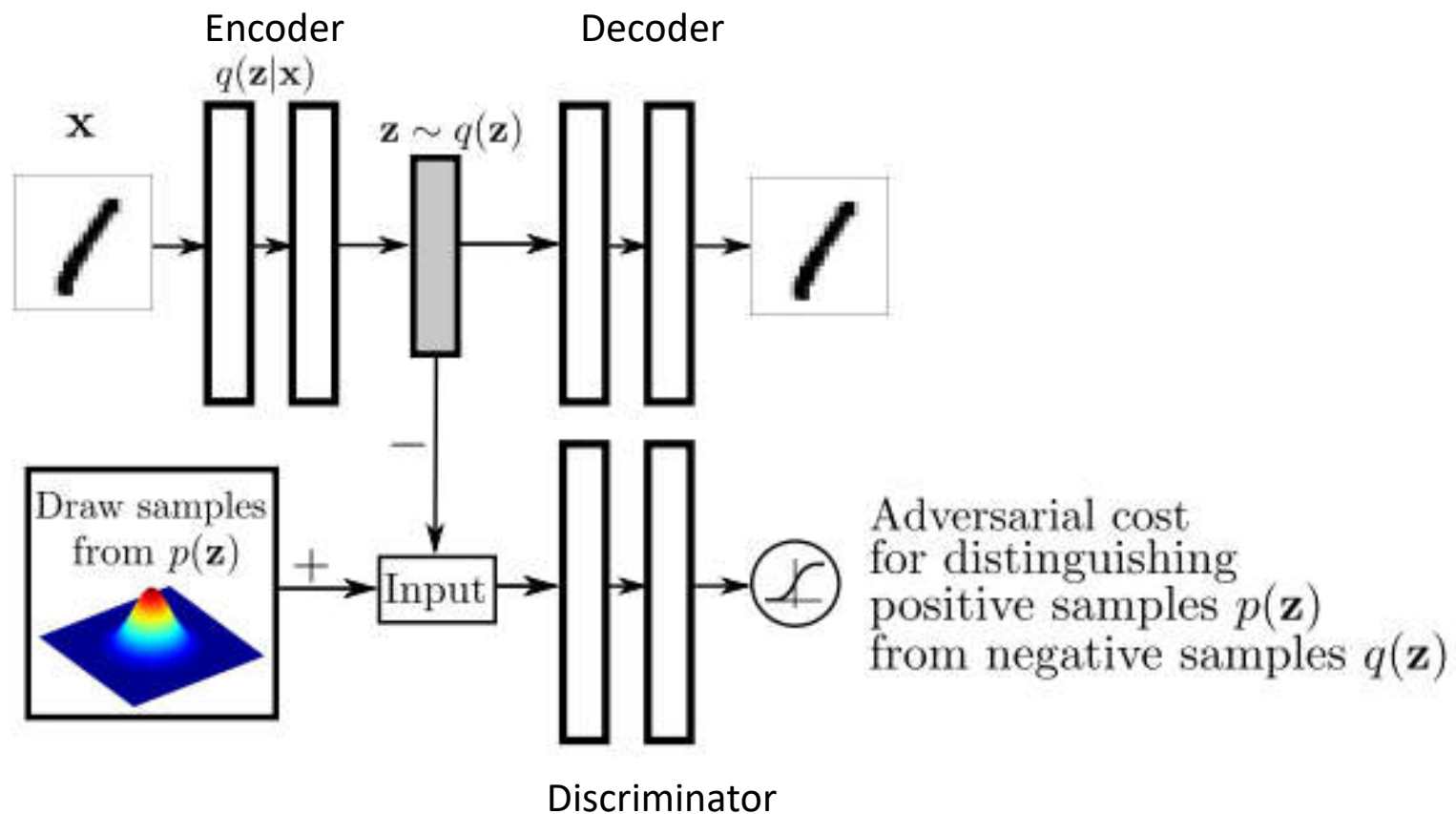


Generated MNIST digits, each row conditioned on one label

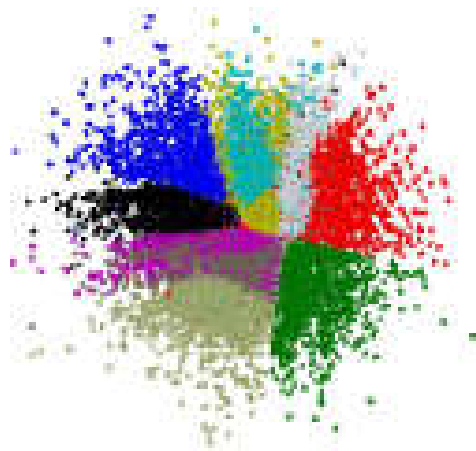
<https://arxiv.org/abs/1411.1784>

Adversarial Autoencoder (AAE), 2016

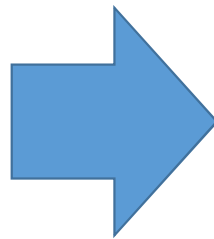
<https://arxiv.org/abs/1511.05644>



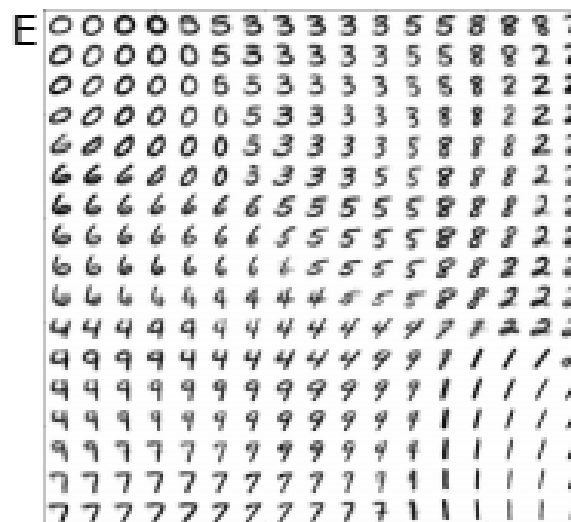
Latent space (2D)



Decoder



Manifold of
Adversarial Autoencoder



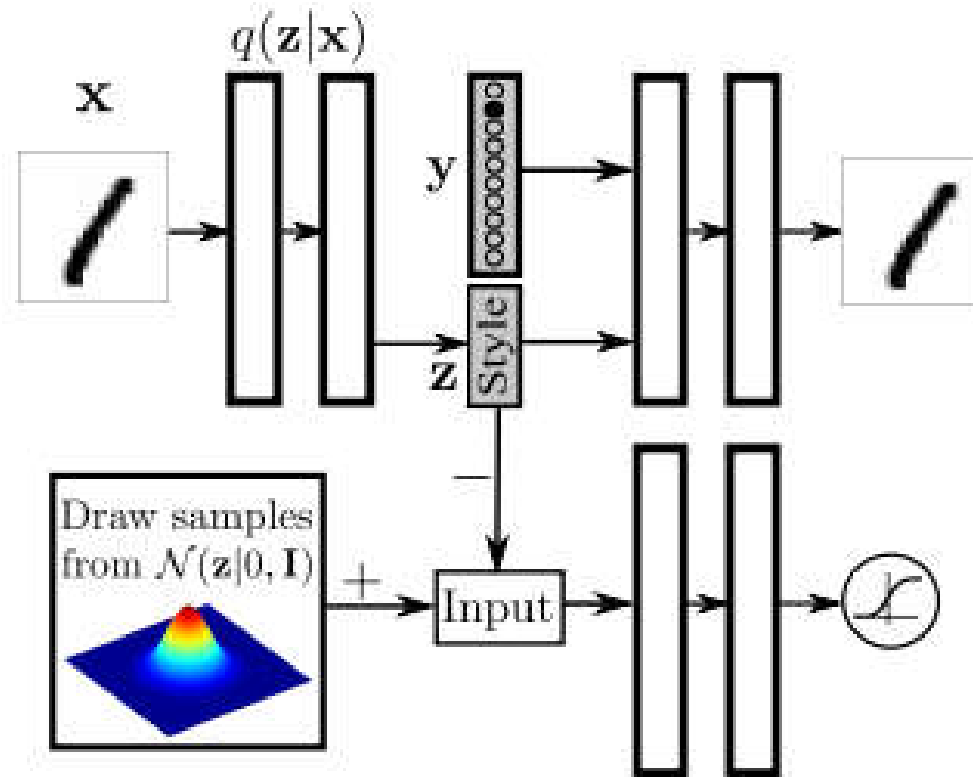
Training:

- Train Discriminator (like in basic GAN)
- Train Autoencoder (Encoder + Decoder) for Reconstruction Error
- Train Encoder + Discriminator: at this phase, we train encoder to generate latent codes the same as sampled ones (we freeze the discriminator weights and train encoder and discriminator together)

Code: <https://medium.com/@a.mirzaei69/adversarial-autoencoders-on-mnist-dataset-python-keras-implementation-5eeafd52ab21>

Conditional Adversarial Autoencoder

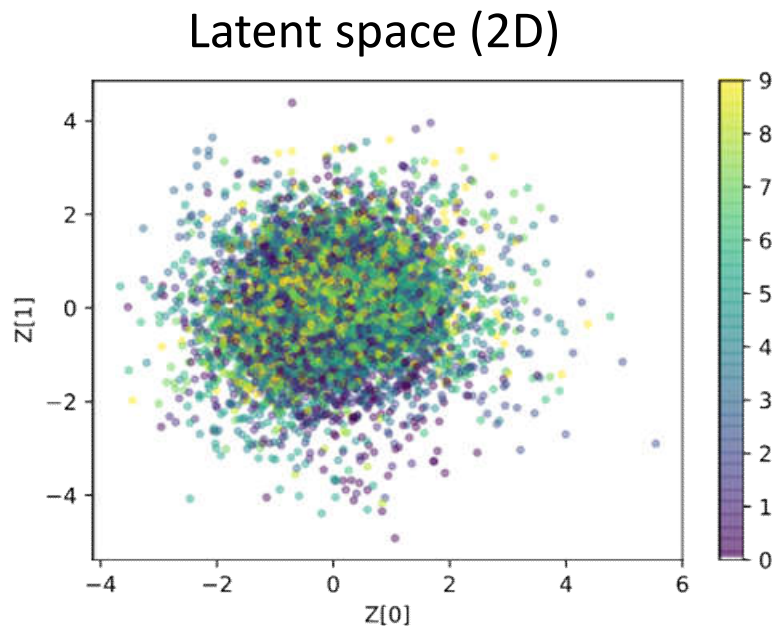
<https://arxiv.org/abs/1511.05644>



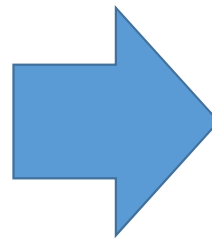
Disentangling the label information from the hidden code

Conditional Adversarial Autoencoder

<https://arxiv.org/abs/1511.05644>

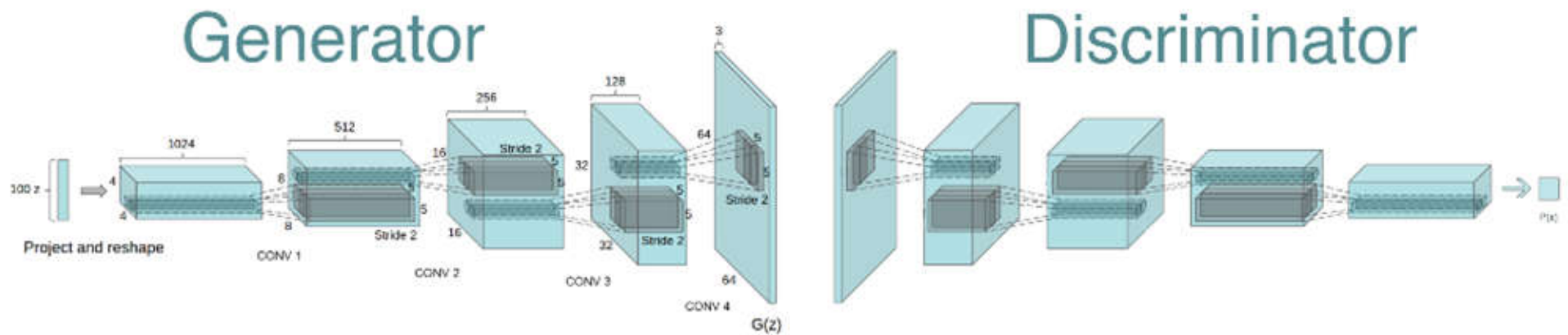


Decoder



Disentangling content and style on MNIST

Deep Convolutional GAN (DCGAN), 2015



A. Radford et al. Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks.

(DCGAN)

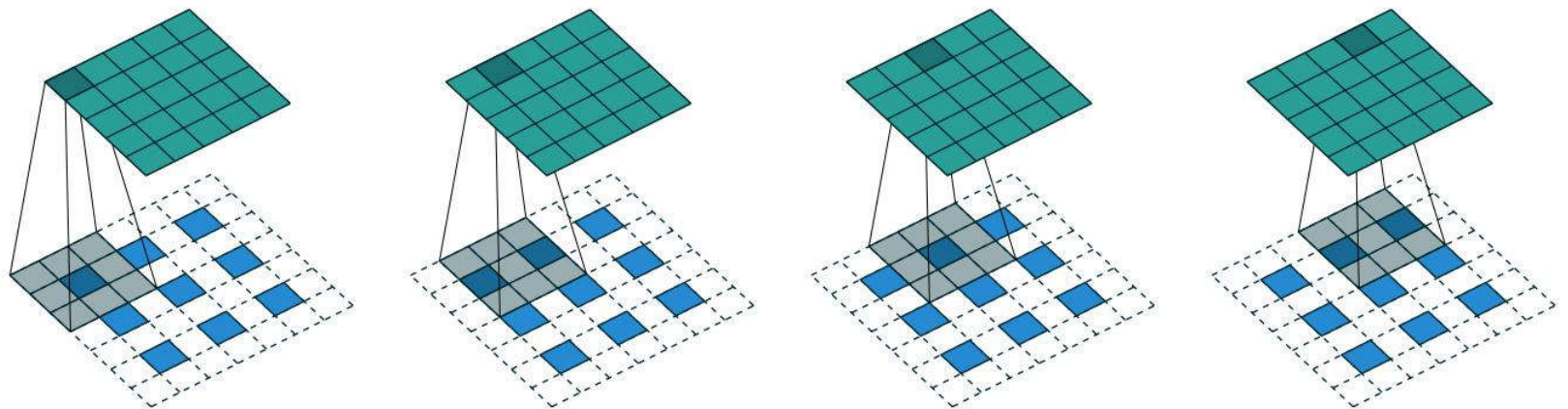
Generator:

- ✓ upsample layer (UpSampling2D) that simply doubles the dimensions of the input
- ✓ or the transpose convolutional layer (Conv2DTranspose)

```
model.add(UpSampling2D(...))
```

```
model.add(Conv2DTranspose(...))
```

Transpose Convolution, Fractionally Strided Convolution or Deconvolution



The GAN Zoo

- GAIN - [GAIN: Missing Data Imputation using Generative Adversarial Nets](#)
- GAMN - [Generative Adversarial Mapping Networks](#)
- GAN - [Generative Adversarial Networks \(github\)](#)
- GAN Lab - [GAN Lab: Understanding Complex Deep Generative Models using Interactive Visual Experimentation](#)
- GAN Q-learning - [GAN Q-learning](#)
- GAN-AD - [Anomaly Detection with Generative Adversarial Networks for Multivariate Time Series](#)
- GAN-ATV - [A Novel Approach to Artistic Textual Visualization via GAN](#)
- GAN-CLS - [Generative Adversarial Text to Image Synthesis \(github\)](#)
- GAN-RS - [Towards Qualitative Advancement of Underwater Machine Vision with Generative Adversarial Networks](#)
- GAN-SD - [Virtual-Taobao: Virtualizing Real-world Online Retail Environment for Reinforcement Learning](#)
- GAN-sep - [GANs for Biological Image Synthesis \(github\)](#)
- GAN-VES - [Generative Adversarial Network based Synthesis of Visible Faces from Polarimetric](#)

<https://github.com/hindupuravinash/the-gan-zoo>

References

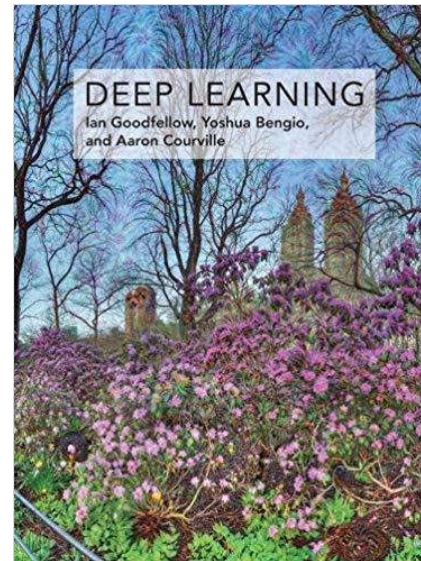
Original paper:

I. Goodfellow, Generative Adversarial Nets, 2014

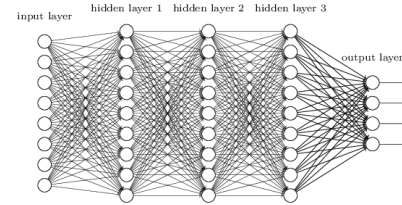
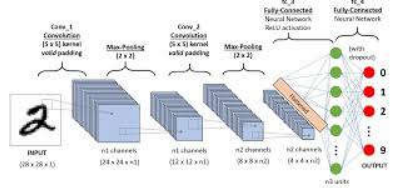
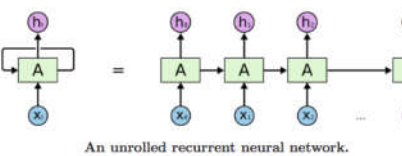
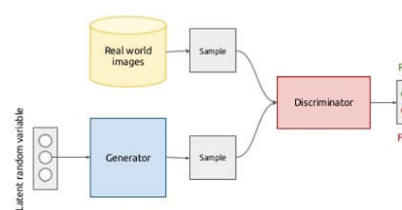
<https://arxiv.org/pdf/1406.2661.pdf>

Book:

I. Goodfellow 'Deep Learning'



Summary

Basic Machine Learning		Deep Learning	
Supervised	Unsupervised	☐ MLP  ☐ CNN  ☐ RNN  An unrolled recurrent neural network. ☐ GANs 	
☐ Regression (linear and polynomial) ☐ Classification ☒ Logistic regression, KNN, Decision Tree, SVM, NB ☒ Ensemble methods: Random Forest, AdaBoost, Gradient Boost	☐ Clustering (k-means, EM, Hierarchical clustering, DBSCAN) ☐ Dimensionality reduction (PCA, LDA, MDS, t-SNE) 		